

서문

법적 고지 사항

이 매뉴얼을 사용하기 전에 다음사항을 읽어주십시오.

펴낸 곳	삼성에스디에스 주식회사
주소	서울특별시 송파구 올림픽로 35길 125
대표 전화	+82 2 1661 3388
전자 메일	global.cs@samsung.com
홈페이지	www.samsungsds.com

이 문서에서 다루는 내용은 삼성에스디에스 주식회사가 제공하는 신뢰할 수 있는 정보입니다. 그러나 부정확한 내용이나 오타로 인해 발생하는 문제는 삼성에스디에스 주식회사에서 책임지지 않습니다.

이 문서의 내용과 제품의 사양은 사전 예고 없이 변경될 수 있습니다. 개정에 관한 상세한 정보는 삼성에스디에스 주식회사의 인터넷 홈페이지 (www.samsungsds.com)에서 확인할 수 있습니다.

이 문서에 대한 저작권을 포함한 지식재산권은 삼성에스디에스 주식회사에 있습니다. 삼성에스디에스 주식회사의 사전 허가 없이 설명서 내용의 일부 또는 전부를 무단 사용하거나 복제하는 것은 금지되어 있으며, 이는 삼성에스디에스 주식회사의 지식재산권 침해에 해당됩니다.

Copyright © Samsung SDS Co., Ltd. All rights reserved.

사용 대상

이 매뉴얼은 삼성에스디에스 Gen AI 서비스인 FabriX(이하 “FabriX”)의 사용자를 대상으로 합니다.

매뉴얼 구성

이 매뉴얼은 서비스에서 제공하는 OpenAPI 항목을 활용하기 위한 개발 가이드를 전달하기 위해 작성되었습니다. OpenAPI 사용 신청 및 활용에 대한 안내가 필요하신 경우, 사용자 매뉴얼을 참고하시기 바랍니다.

제공되는 OpenAPI 항목들은 고객사 또는 사용자의 접속 환경에 따라 다를 수 있으므로, 매뉴얼에 제공되는 모든 OpenAPI 항목들이 사용자 포털에 제공되는 것은 아닙니다.

추가로 특정 OpenAPI 항목을 사용해야 하는 경우, 각 사의 관리자를 통해 문의주시기 바랍니다.

표기 규약

문서 내용의 이해를 돕기 위해 다음과 같은 표기 규약을 사용합니다.

볼드체 활자	볼드체 활자는 그래픽 유저 인터페이스 요소와 메뉴와 디렉토리 등을 표기할 때 사용합니다. 다만 그래픽 유저 인터페이스 중에서 포털, 창, 팝업창, 마법사, 페이지는 볼드체 활자가 아닌 큰따옴표로 표기합니다.
<i>이탤릭체</i>	<i>이탤릭체</i> 는 사용자가 입력해야 하는 변수나 파라미터값을 표기할 때 사용합니다.
고정폭 활자	고정폭 활자는 프로그래밍과 관련된 용어나 코드를 표기할 때 사용합니다. 본문에서 언급할 때는 구분을 위해 명령어, 파라미터명, 레지스트리명, 스크립트명, 프로세스명 등은 고정폭 활자 볼드체 로 표기하고, 변수와 파라미터값은 고정폭 활자 <i>이탤릭체</i> 로 표기합니다.

1.

Chat APIs

1.1 Chat APIs Overview

Chat APIs는 서비스 내 제공되는 모델을 대상으로 응답을 호출할 수 있는 API를 제공합니다. API를 통해 제공되는 답변은 서비스의 필터링을 적용하여 제공합니다.

Chat APIs 에서 제공하는 API는 다음과 같습니다.

GET /openapi/chat/v1/models

API로 사용할 수 있는 대화 모델의 리스트를 조회합니다.

POST /openapi/chat/v1/messages

messages API는 대화에 대한 모델 응답을 생성합니다.

GET /openapi/chat/v1/all-models

API로 사용할 수 있는 대화 모델과 이미지 모델의 리스트를 조회합니다.

POST /openapi/chat/v1/messages-with-models

messages-with-models API는 대화모델, 이미지 모델에 대한 응답을 생성합니다. Single/Multi-turn 대화와 이미지 분석을 위한 FileUpload 기능을 지원합니다.

1.2 API 명세

GET /openapi/chat/v1/models

노트

사용자 포털에서 권한을 가진 모델(Text)을 API로 조회할 수 있습니다.

Text : 대화모델

Header

Field	Type	Required	Description	Value
Content-Type	string	optional	Type of content	application/json
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-openapi-token	string	required	OpenAPI Token information	ex) Bearer eyJXXX
x-generative-ai-user-email	string	optional	Portal user email	ex) kim.samsung@samsung.com

Response

Field	Type	Description	Value
modelId	String	Model Id	
name	list	Model Name	[{ "languageCode": "ko", "content": "모델명" }]
description	list	Model Description	[{ "languageCode": "ko", "content": "설명" }]

호출 예시 (python)

```

1 import requests
2
3 YOUR_CLIENT_KEY = ""
4 YOUR_PASS_KEY = ""
5 ENDPOINT_URL = ""
6 YOUR_EMAIL = ""
7
8 headers = {
9     "x-fabrix-client": YOUR_CLIENT_KEY,
10    "x-openapi-token": YOUR_PASS_KEY,
11    "x-generative-ai-user-email": YOUR_EMAIL
12 }
```

```

13
14 api_endpoint_url = f"{ENDPOINT_URL}/openapi/chat/v1/models"
15
16 response=requests.get(
17     api_endpoint_url,
18     headers=headers
19 )
20
21 # 응답 반환
22 print(response.json())

```

주의

※ Endpoint URL은 Open API 신청시 발급되는 URL로 적용하여 사용할 수 있습니다.

POST /openapi/chat/v1/messages

노트

사용자 입력에 대한 대한 모델의 응답을 생성합니다.

Header

Field	Type	Required	Description	Value
Content-Type	string	optional	Type of content	application/json
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-openapi-token	string	required	OpenAPI Token information	ex) Bearer eyJXXX
x-generative-ai-user-email	string	optional	Portal user email	ex) kim.samsung@samsung.com

Request

Field	Type	Required	Description	Value
modelIds	array	required	model id list	ex) ["0196f1fc-2858-70a9-a232-74dbddb971d0"]
contents	list[string]	required	contents	ex) ["Hello"]
isStream	boolean	optional	Stream or not (default: True)	ex) False / True
llmConfig	object	optional	LLM Config info	Properties 참고
systemPrompt	string	optional	System prompt for LLM to use	ex) 예시 참고
isExecuteRecommendedQueries	boolean	optional	Respond to a recommended queries	

llmConfig Properties

Name	Description
temperature	모델의 확률 분포를 조절합니다. 값이 1보다 크면 확률 분포가 더욱 고르게 퍼지게 되고, 값이 1보다 작으면 확률 분포가 더욱 뾰족하게 집중됩니다. (0<temperature<1)
repetition_penalty	반복된 단어의 출현 확률을 조절합니다. 값이 1보다 크면 반복된 단어가 나타날 확률이 줄어들게 되고, 값이 1보다 작으면 반복된 단어가 나타날 확률이 증가하게 됩니다.
decoder_input_details	디코더의 입력 형태를 결정합니다. use_cache를 True로 설정하고 past_key_values를 None으로 설정하면, 디코더는 이전 타임스텝의 캐시값을 이용하여 다음 타임스텝의 입력을 생성합니다. use_cache를 False로 설정하고 past_key_values를 None으로 설정하면, 디코더는 이전 타임스텝의 마지막 히든 상태를 이용하여 다음 타임스텝의 입력을 생성합니다. past_key_values를 임의의 값으로 설정하면, 디코더는 해당 값을 이용하여 다음 타임스텝의 입력을 생성합니다.
seed	모델의 난수 발생기를 시드하는 데 사용됩니다. 동일한 시드를 사용하면 모델은 항상 동일한 순서로 출력을 생성합니다.
top_k	모델이 고려할 후보 단어의 수를 제한합니다. 큰 값을 사용하면 더 다양한 단어를 선택할 수 있지만, 계산 시간이 오래 걸릴 수 있습니다. (1<=top_k)
top_p	확률 분포의 상위 p%를 고려하여 단어를 선택합니다. 이렇게 하면 선택된 단어들이 확률 분포를 고르게 대표하도록 하여 다양성을 높일 수 있습니다. (top_p must be > 0.0 and < 1.0)
max_new_tokens	생성된 텍스트의 최대 길이를 결정합니다. 예를 들어,

Name	Description
	max_new_tokens=10으로 설정하면 모델은 최대 10개의 토큰을 생성할 수 있습니다.

Request Body 예시

```
1 {
2   "modelIds": ["0196f1fc-2858-70a9-a232-74dbddb971d0"],
3   "contents":["안녕하세요?","네 안녕하세요", "내 이름은 LCY인데 너 이름은 뭐니?"],
4   "isStream": true,
5   "llmConfig": {
6     "max_new_tokens": 2024,
7     "seed": null,
8     "top_k": 14,
9     "top_p": 0.94,
10    "temperature": 0.4,
11    "repetition_penalty": 1.04
12  },
13   "systemPrompt": "안녕하세요. 사용자 질문에 친절히 대답해주세요."
14 }
```

Response

주의

※ Stream 으로 응답을 받을경우 Field 명이 카멜케이스가 아닌 스네이크케이스로 표시됩니다.
예) modelType -> model_type

Field	Type	Description	
id	integer	openapi message Id	DEPRECATED
parentMessageId	string	openapi message parent id	DEPRECATED
parentMessageCreatedAt	DateTime	parentMessage creation time	DEPRECATED
chatId	integer	chat Id	DEPRECATED
userId	integer	user Id	
modelId	string	LLM ModelId	DEPRECATED

Field	Type	Description	
modelType	string	LLM ModelType	
content	string	Answer	
createdAt	DateTime	Message creation time	DEPRECATED
completionToken	integer	Answer Token Usage	DEPRECATED
promptToken	integer	Question Token Usage	DEPRECATED
truncated	string	Whether the question is truncated or not	
finishReason	string	FinishReason (stop : normal, length : Answer length exceeded)	
filterBlockReason.ko	string	Filter block reason(ko)	
filterBlockReason.en	string	Filter block reason(en)	
filterBlockReason.policy_id	string	Filter block reason(policy_id)	
filterBlockReason.message	string	Filter block reason(message)	
filterBlockReason.result_code	string	Filter block reason(result_code)	
filterBlockReason.filter_log_id	string	Filter block reason(filter_log_id)	
status	string	Status	
responseCode	string	ResponseCode	
plugins	list[str]	Requested plugin	
references	list[str]	References used while generating the answer	DEPRECATED
catalogs	list[int]	Requested Catalog	DEPRECATED
eventStatus	string	Event status	
eventData	string	EventData	
filterValidation	boolean	Filter success or not	DEPRECATED
successYn	boolean	Success or failure	DEPRECATED
reasoning_content	string	추론 과정	
processing_content	list	스트리밍 일 경우 중간 처리 과정	
content_references	list	References used while generating the answer	
recommendQueries	list[str]	recommendQueries	

| Response 예시

(isStream=false)


```

1  {
2    "userId": "9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e",
3    "modelType": "FabriX",
4    "content": "### 소프트웨어의 역사\n\n소프트웨어의 역사는 컴퓨터의 발전과 함께
시작되었습니다. 아래는 주요한 연도별 소프트웨어의 역사입니다.\n\n#### 1940년대\n- **초기
시작**: 최초의 컴퓨터인 ENIAC(전자 수식 계산기)이 개발되었습니다. 이 시기에는 소프트웨어가
하드웨어와 분리되지 않았으며, 컴퓨터 프로그램은 주로 하드웨어를 구성하는 방법으로
개발되었습니다.\n\n#### 1950년대\n- **최초의 프로그래밍 언어**: 플레너리 시스템(Plankalk , 1)과
포트란(FORTRAN)이 개발되었습니다. 포트란은 최초의 고급 프로그래밍 언어 중 하나로, 과학 계산에
많이 사용되었습니다.\n- **운영 체제의 등장**: 최초의 운영 체제가 개발되었습니다. 초기 운영
체제는 주로 일괄 처리 시스템(batch processing system)으로, 작업을 순차적으로
처리했습니다.\n\n#### 1960년대\n- **고급 프로그래밍 언어의 발전**: COBOL, LISP, C 등 다양한
프로그래밍 언어가 개발되었습니다. COBOL은 비즈니스 애플리케이션에 많이 사용되었으며, LISP는
인공지능 개발에 많이 사용되었습니다.\n- **시분할 시스템**: 시분할 시스템(time-sharing system)이
개발되었습니다. 시분할 시스템은 여러 사용자가 동시에 컴퓨터를 사용할 수 있도록
해주었습니다.\n\n#### 1970년대\n- **개인용 컴퓨터의 등장**: 개인용 컴퓨터가 개발되었습니다.
초기 개인용 컴퓨터로는 애플 I, 애플 II, IBM PC 등이 있습니다.\n- **마이크로소프트의 설립**: 빌
게이츠와 폴 앨런이 마이크로소프트를 설립했습니다. 마이크로소프트는 개인용 컴퓨터용 소프트웨어를
개발하기 시작했습니다.\n\n#### 1980년대\n- **윈도우의 등장**: 마이크로소프트가 윈도우 1.0을
출시했습니다. 윈도우는 그래픽 사용자 인터페이스(GUI)를 제공하여 사용자에게 편리한 환경을
제공했습니다.\n- **인터넷의 확산**: 인터넷이 일반인에게 공개되었습니다. 초기 인터넷은 주로
대학과 연구 기관에서 사용되었습니다.\n\n#### 1990년대\n- **월드 와이드 웹의 등장**: 팀 버너스
리가 월드 와이드 웹(WWW)을 개발했습니다. 월드 와이드 웹은 인터넷을 통해 정보를 공유하고 접근할
수 있는 방법을 제공했습니다.\n- **오픈 소스 소프트웨어의 등장**: 리눅스와 같은 오픈 소스
소프트웨어가 개발되었습니다. 오픈 소스 소프트웨어는 사용자들이 소스 코드를 수정하고 배포할 수
있도록 허용했습니다.\n\n#### 2000년대\n- **클라우드 컴퓨팅의 등장**: 클라우드 컴퓨팅이
개발되었습니다. 클라우드 컴퓨팅은 인터넷을 통해 컴퓨팅 자원을 제공하여 사용자들이 언제 어디서나
컴퓨팅을 할 수 있도록 해주었습니다.\n- **모바일 기기의 등장**: 스마트폰과 태블릿이
개발되었습니다. 모바일 기기는 사용자들이 언제 어디서나 인터넷을 사용할 수 있도록
해주었습니다.\n\n#### 2010년대\n- **인공지능과 머신 러닝의 발전**: 인공지능과 머신 러닝 기술이
발전되었습니다. 인공지능과 머신 러닝은 이미지 인식, 음성 인식, 자연어 처리 등 다양한 분야에서
사용되었습니다.\n- **빅 데이터와 데이터 과학의 등장**: 빅 데이터와 데이터 과학이
개발되었습니다. 빅 데이터와 데이터 과학은의 데이터를 분석하여 통찰력을 얻고 의사 결정을
지원했습니다.\n\n#### 2020년대\n- **클라우드 네이티브 애플리케이션의 등장**: 클라우드 네이티브
애플리케이션이 개발되었습니다. 클라우드 네이티브 애플리케이션은 클라우드 환경에서 개발되고
배포되는 애플리케이션입니다.\n- **서버리스 컴퓨팅의 등장**: 서버리스 컴퓨팅이 개발되었습니다.
서버리스 컴퓨팅은 사용자들이 코드를 실행할 수 있도록 해주며, 사용한 만큼만 비용을
지불합니다.\n\n소프트웨어의 역사는 계속 발전하고 있으며, 새로운 기술과 트렌드가 등장하고
있습니다. 앞으로도 소프트웨어는 우리의 생활과 업무에 큰 영향을 미칠 것입니다.",
5    "reasoningContent": null,

```

```

6  "processingContent": [],
7  "contentReferences": [
8    {
9      "plugin": "RAG",
10     "answer": "",
11     "references": [
12       {
13         "title": "소프트웨어의 역사",
14         "content": "빌 게이츠와 폴 앨런이 마이크로소프트를 설립했습니다. 마이크로소프트는
개인용 컴퓨터용 소프트웨어를 개발하기 시작했습니다.\n\n#### 1980년대\n- **윈도우의 등장**:
마이크로소프트가 윈도우 1.0을 출시했습니다. 윈도우는 그래픽 사용자 인터페이스(GUI)를 제공하여
사용자에게 편리한 환경을 제공했습니다.\n- **인터넷의 확산**: 인터넷이 일반인에게
공개되었습니다. 초기 인터넷은 주로 대학과 연구 기관에서 사용되었습니다.\n\n#### 1990년대\n-
**월드 와이드 웹의 등장**: 팀 버너스 리가 월드 와이드 웹(WWW)을 개발했습니다. 월드 와이드 웹은
인터넷을 통해 정보를 공유하고 접근할 수 있는 방법을 제공했습니다.\n- **오픈 소스 소프트웨어의
등장**: 리눅스와 같은 오픈 소스 소프트웨어가 개발되었습니다. 오픈 소스 소프트웨어는 사용자들이
소스 코드를 수정하고 배포할 수 있도록 허용했습니다",
15         "link": ""
16       }
17     ],
18     "augmented_standalone_queries": "소프트웨어 역사에 대해 알려주세요."
19   }
20 ],
21 "truncated": false,
22 "finishReason": null,
23 "filterBlockReason": {
24   "ko": null,
25   "en": null,
26   "policy_id": null,
27   "message": null,
28   "result_code": "FR-200",
29   "filter_log_id": null
30 },
31 "status": "SUCCESS",
32 "responseCode": "R20000",
33 "plugins": [
34   "LLM"
35 ],
36 "orchestratorType": null,
37 "parentMessageCreatedAt": "2025-07-15T10:16:32.511334+09:00",

```

```

38   "actions": [],
39   "eventStatus": "CHUNK",
40   "eventData": ""
41 }

```

(isStream=true)

```

1  data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"FabriX","content":"","reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null,"finish_reason":null,"filter_block_reason":null,"status":"SUCCESS","response_code":null,"plugins":[],"orchestrator_type":null,"parent_message_created_at":"2025-07-15T13:49:30.143234+09:00","actions":null,"event_status":"CHUNK","event_data":""}
2  data:
3
4  data:
{"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"FabriX","content":"요청을 처리하기 위한 API 후보를 0개 발견하였습니다.","reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null,"finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"The content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","response_code":null,"plugins":[],"orchestrator_type":null,"parent_message_created_at":"2025-07-15T13:49:30.143234+09:00","actions":null,"event_status":"STATUS","event_data":{"phase":"\ planning\","message":"\ 요청을 처리하기 위한 API 후보를 0개 발견하였습니다.\"}"}
5  data:
6
7  data:
{"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"FabriX","content":"최종 응답을 생성합니다.","reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null,"finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"The content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","response_code":null,"plugins":[],"orchestrator_type":null,"parent_message_created_at":"2025-07-15T13:49:30.143234+09:00","actions":null,"event_status":"STATUS","event_data":{"phase":"\ execution\","message":"\ 최종 응답을 생성합니다.\"}"}
8  data:
9
10 data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"FabriX","content":"###","reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null,"finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"The content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","response_code":null,"plugins":[],"orchestrator_type":null,"parent_message_created_at":"2025-07-15T13:49

```

```

:30.143234+09:00","actions":null,"event_status":"CHUNK","event_data":""}
11 data:
12
13 data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"FabriX","content":"
소","reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null,"
finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"Th
e content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","respon
se_code":null,"plugins":[],"orchestrator_type":null,"parent_message_created_at":"2025-07-15T13:
49:30.143234+09:00","actions":null,"event_status":"CHUNK","event_data":""}
14 data:
15
16 data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"FabriX","content":"프
트","reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null,"
finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"Th
e content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","respon
se_code":null,"plugins":[],"orchestrator_type":null,"parent_message_created_at":"2025-07-15T13:
49:30.143234+09:00","actions":null,"event_status":"CHUNK","event_data":""}
17 data:
18
19 data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"FabriX","content":"워
어","reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null,"
finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"Th
e content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","respon
se_code":null,"plugins":[],"orchestrator_type":null,"parent_message_created_at":"2025-07-15T13:
49:30.143234+09:00","actions":null,"event_status":"CHUNK","event_data":""}
20 data:
21
22 data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"FabriX","content":"의"
,"reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null,"fin
ish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"The
content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","response
_code":null,"plugins":[],"orchestrator_type":null,"parent_message_created_at":"2025-07-15T13:49
:30.143234+09:00","actions":null,"event_status":"CHUNK","event_data":""}
23 data:
24
25 data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"FabriX","content":"
역사","reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null
,"finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"
The content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","resp
onse_code":null,"plugins":[],"orchestrator_type":null,"parent_message_created_at":"2025-07-15T1

```

```

3:49:30.143234+09:00", "actions": null, "event_status": "CHUNK", "event_data": ""}
26 data:
27
28 data: {"user_id": "9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e", "model_type": "FabriX", "content": "\n\n", "reasoning_content": null, "processing_content": [], "content_references": [], "truncated": null, "finish_reason": null, "filter_block_reason": {"ko": null, "en": null, "policy_id": "None", "message": "The content was passed", "result_code": "FR-200", "filter_log_id": "None"}, "status": "SUCCESS", "response_code": null, "plugins": [], "orchestrator_type": null, "parent_message_created_at": "2025-07-15T13:49:30.143234+09:00", "actions": null, "event_status": "CHUNK", "event_data": ""}
29 data:
30
31 data: {"user_id": "9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e", "model_type": "FabriX", "content": "소", "reasoning_content": null, "processing_content": [], "content_references": [], "truncated": null, "finish_reason": null, "filter_block_reason": {"ko": null, "en": null, "policy_id": "None", "message": "The content was passed", "result_code": "FR-200", "filter_log_id": "None"}, "status": "SUCCESS", "response_code": null, "plugins": [], "orchestrator_type": null, "parent_message_created_at": "2025-07-15T13:49:30.143234+09:00", "actions": null, "event_status": "CHUNK", "event_data": ""}
32 data:
33
34 data: {"user_id": "9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e", "model_type": "FabriX", "content": "프", "reasoning_content": null, "processing_content": [], "content_references": [], "truncated": null, "finish_reason": null, "filter_block_reason": {"ko": null, "en": null, "policy_id": "None", "message": "The content was passed", "result_code": "FR-200", "filter_log_id": "None"}, "status": "SUCCESS", "response_code": null, "plugins": [], "orchestrator_type": null, "parent_message_created_at": "2025-07-15T13:49:30.143234+09:00", "actions": null, "event_status": "CHUNK", "event_data": ""}
35 data:
36
37 data: {"user_id": "9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e", "model_type": "FabriX", "content": "웨어", "reasoning_content": null, "processing_content": [], "content_references": [], "truncated": null, "finish_reason": null, "filter_block_reason": {"ko": null, "en": null, "policy_id": "None", "message": "The content was passed", "result_code": "FR-200", "filter_log_id": "None"}, "status": "SUCCESS", "response_code": null, "plugins": [], "orchestrator_type": null, "parent_message_created_at": "2025-07-15T13:49:30.143234+09:00", "actions": null, "event_status": "CHUNK", "event_data": ""}
38 data:
39
40 data: {"user_id": "9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e", "model_type": "FabriX", "content": "의", "reasoning_content": null, "processing_content": [], "content_references": [], "truncated": null, "finish_reason": null, "filter_block_reason": {"ko": null, "en": null, "policy_id": "None", "message": "The content was passed", "result_code": "FR-200", "filter_log_id": "None"}, "status": "SUCCESS", "response_code": null, "plugins": [], "orchestrator_type": null, "parent_message_created_at": "2025-07-15T13:49:30.143234+09:00", "actions": null, "event_status": "CHUNK", "event_data": ""}

```

```

:30.143234+09:00","actions":null,"event_status":"CHUNK","event_data":""}
41 data:
42
43 data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"FabriX","content":"
역","reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null,"
finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"Th
e content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","respon
se_code":null,"plugins":[],"orchestrator_type":null,"parent_message_created_at":"2025-07-15T13:
49:30.143234+09:00","actions":null,"event_status":"CHUNK","event_data":""}
44 data:
45
46 data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"FabriX","content":"사
는","reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null,"
finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"Th
e content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","respon
se_code":null,"plugins":[],"orchestrator_type":null,"parent_message_created_at":"2025-07-15T13:
49:30.143234+09:00","actions":null,"event_status":"CHUNK","event_data":""}
47 data:
48 ..... 생략

```

| 호출 예시 (python)

(isStream=false)

```

1 import requests
2
3 YOUR_CLIENT_KEY = ""
4 YOUR_PASS_KEY = ""
5 ENDPOINT_URL = ""
6 YOUR_MODEL_ID = "Get model id from Model API"
7 YOUR_EMAIL = ""
8
9 headers = {
10     "x-fabrix-client": YOUR_CLIENT_KEY,
11     "x-openapi-token": YOUR_PASS_KEY,
12     "x-generative-ai-user-email": YOUR_EMAIL
13 }
14
15 body = {
16     "modelIds": [YOUR_MODEL_ID],

```

```

17  "contents": ["안녕하세요?", "네 안녕하세요", "내 이름은 LCY인데 너 이름은 뭐니?"],
18  "llmConfig": {
19      "max_new_tokens": 2024,
20      "seed": None,
21      "top_k": 14,
22      "top_p": 0.94,
23      "temperature": 0.4,
24      "repetition_penalty": 1.04
25  },
26  "isStream": False,
27  "systemPrompt": "안녕하세요. 사용자 질문에 친절히 대답해주세요."
28  }
29
30  api_endpoint_url = f"{ENDPOINT_URL}/openapi/chat/v1/messages"
31
32  response=requests.post(
33      api_endpoint_url,
34      headers=headers,
35      json=body
36  )
37
38  # 응답 반환
39  print(response.json())

```

(isStream=true)

팁

Stream = true 인 경우에 최종 응답 확인은 status 값을 이용하여 확인합니다.

- 정상 답변 : status = 'SUCCESS' and response_code = 'R20000'
- 비정상 답변 : status='ERROR' or 'FILTER_INVALID'

```

1  import json
2  import requests
3  import sseclient
4

```

```

5 YOUR_CLIENT_KEY = ""
6 YOUR_PASS_KEY = ""
7 YOUR_MODEL_ID = "Get model id from Model API"
8 YOUR_EMAIL = ""
9
10 headers = {
11     "x-fabrix-client": YOUR_CLIENT_KEY,
12     "x-openapi-token": YOUR_PASS_KEY,
13     "x-generative-ai-user-email": YOUR_EMAIL
14 }
15
16 body = {
17     "modelIds": [YOUR_MODEL_ID],
18     "contents": ["안녕하세요?", "네 안녕하세요", "내 이름은 LCY인데 너 이름은 뭐니?"],
19     "llmConfig": {
20         "max_new_tokens": 2024,
21         "seed": None,
22         "top_k": 14,
23         "top_p": 0.94,
24         "temperature": 0.4,
25         "repetition_penalty": 1.04
26     },
27     "isStream": True,
28     "systemPrompt": "안녕하세요. 사용자 질문에 친절히 대답해주세요."
29 }
30
31 api_endpoint_url = f"{ENDPOINT_URL}/openapi/chat/v1/messages"
32
33 response=requests.post(
34     api_endpoint_url ,
35     headers=headers,
36     json=body
37 )
38
39 # return response
40 if response.status_code != 200:
41     print(f"Error: Status_code=[{response.status_code}], Status message=[{response.text}]")
42 client = sseclient.SSEClient(response)
43
44 # make result contents

```



```

45 def get_result():
46     for event in client.events():
47         # print(f"Event: {event.event}")
48         print(f>Data: {event.data}")
49         yield event.data
50
51 result_message = ""
52 for ch in get_result():
53     ch_json = json.loads(ch)
54     if ch_json['status'] == 'SUCCESS' and ch_json['response_code'] == 'R20000':
55         print("Stream Finished!!")
56     if ch_json['event_status'] == 'CHUNK' and ch_json['content']:
57         result_message += ch_json['content']
58
59 print(result_message)

```

GET /openapi/chat/v1/all-models

노트

사용자 포털에서 권한을 가진 모델(Text, I2T, T2I)을 API로 조회할 수 있습니다.

Text : 대화모델

I2T : Image To Text 모델

T2I : Text To Image 모델

Header

Field	Type	Required	Description	Value
Content-Type	string	optional	Type of content	application/json
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-openapi-token	string	required	OpenAPI Token information	ex) Bearer eyJXXX
x-generative-ai-user-email	string	optional	Portal user email	ex) kim.samsung@samsung.com

Response

Field	Type	Description	Value
modelId	String	Model Id	
name	list	Model Name	[{ "languageCode": "ko", "content": "모델명" }]
description	list	Model Description	[{ "languageCode": "ko", "content": "설명" }]
modelName	String	Model Name	DEPRECATED
modelLabel.en	String	English model Label	DEPRECATED
modelLabel.ko	String	Korean model Label	DEPRECATED
modelDescription.en	String	Model Description en	DEPRECATED
modelDescription.ko	String	Model Description ko	DEPRECATED
types	list	Model Type	["TEXT"] TEXT : 대화모델 I2T(ImageToText) : 이미지 분석모델 T2I(TextToImage) : 이미지 생성모델

호출 예시 (python)

```

1 import requests
2
3 YOUR_CLIENT_KEY = ""
4 YOUR_PASS_KEY = ""
5 ENDPOINT_URL = ""
6 YOUR_EMAIL = ""
7
8 headers = {
9     "x-fabrix-client": YOUR_CLIENT_KEY,
10    "x-openapi-token": YOUR_PASS_KEY,
11    "x-generative-ai-user-email": YOUR_EMAIL

```

```

12 }
13
14 api_endpoint_url = f"{ENDPOINT_URL}/openapi/chat/v1/all-models"
15
16 response=requests.get(
17     api_endpoint_url,
18     headers=headers
19 )
20
21 # 응답 반환
22 print(response.json())

```

POST /openapi/chat/v1/messages-with-models

노트

사용자 입력에 대한 대한 모델의 응답(대화, 이미지 생성, 이미지분석)을 생성합니다.

Header

Field	Type	Required	Description	Value
Content-Type	string	optional	Type of content	multipart/form-data
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-openapi-token	string	required	OpenAPI Token information	ex) Bearer eyJXXX
x-generative-ai-user-email	string	optional	Portal user email	ex)kim.samsung@samsung.com

Request

주의

※ 이미지 분석 모델(I2T) 을 사용할 때에는 1개의 파일만 지원합니다.

Field	Type	Required	Description	Value
llmId	int	optional	LLM Id (default: basic LLM)	DEPRECATED
llmName	string	optional [Deprecated]	LLM Name (default: basic LLM)	DEPRECATED
modelIds	list	required	model id list 이미지 생성시 - TEXT, T2I 모델 필요 이미지 분석시 - TEXT, I2T 모델 필요	ex) ["0196f1fc-2858-70a9-a232-74dbddb971d0", "0196f1fc-2858-70a9-a232-74dbddb971232"]
contents	list[string]	required	contents	ex) ["Hello"]
isStream	boolean	optional	Stream or not (default: True)	ex) False / True
llmConfig	object	optional	LLM Config info	Properties 참고
systemPrompt	string	optional	System prompt for LLM to use	ex) 예시 참고
files	list	optional	Upload file	
messageConfig	object	optional	T2I Image Size(Only Flux Model)	ex) {"width":64,"height":64}
isExecuteRecommendedQueries	boolean	optional	Respond to a recommended queries	

| llmConfig Properties

Name	Description
temperature	모델의 확률 분포를 조절합니다. 값이 1보다 크면 확률 분포가 더욱 고르게 퍼지게 되고, 값이 1보다 작으면 확률 분포가 더욱 뾰족하게 집중됩니다. (0<temperature<1)
repetition_penalty	반복된 단어의 출현 확률을 조절합니다. 값이 1보다 크면 반복된 단어가 나타날 확률이 줄어들게 되고, 값이 1보다 작으면 반복된 단어가 나타날 확률이 증가하게 됩니다.
decoder_input_details	디코더의 입력 형태를 결정합니다. use_cache를 True로 설정하고 past_key_values를 None으로 설정하면, 디코더는 이전 타임스텝의 캐시값을 이용하여 다음 타임스텝의 입력을 생성합니다. use_cache를 False로 설정하고 past_key_values를 None으로 설정하면, 디코더는 이전 타임스텝의 마지막 히든 상태를 이용하여 다음 타임스텝의 입력을 생성합니다. past_key_values를 임의의 값으로 설정하면, 디코더는 해당 값을 이용하여 다음 타임스텝의 입력을 생성합니다.

Name	Description
seed	모델의 난수 발생기를 시드하는 데 사용됩니다. 동일한 시드를 사용하면 모델은 항상 동일한 순서로 출력을 생성합니다.
top_k	모델이 고려할 후보 단어의 수를 제한합니다. 큰 값을 사용하면 더 다양한 단어를 선택할 수 있지만, 계산 시간이 오래 걸릴 수 있습니다. ($1 \leq \text{top_k}$)
top_p	확률 분포의 상위 p%를 고려하여 단어를 선택합니다. 이렇게 하면 선택된 단어들이 확률 분포를 고르게 대표하도록 하여 다양성을 높일 수 있습니다. (top_p must be > 0.0 and < 1.0)
max_new_tokens	생성된 텍스트의 최대 길이를 결정합니다. 예를 들어, max_new_tokens=10으로 설정하면 모델은 최대 10개의 토큰을 생성할 수 있습니다.

| Request Body 예시 (이미지 생성)

```

1  -----Boundary
2  Content-Disposition: form-data; name="contents"
3
4  고양이 그려줘
5  -----Boundary
6  Content-Disposition: form-data; name="isStream"
7
8  true
9  -----Boundary
10 Content-Disposition: form-data; name="modelIds"
11
12 YOUR_TEXT_MODEL_ID
13 -----Boundary
14 Content-Disposition: form-data; name="modelIds"
15
16 YOUR_T2I_MODEL_ID
17 -----Boundary
18 Content-Disposition: form-data; name="llmConfig"
19
20 {
21     "max_new_tokens": 2024,
22     "seed": null,
23     "top_k": 14,
24     "top_p": 0.94,
25     "temperature": 0.4,

```

```

26     "repetition_penalty": 1.04
27   }
28   -----Boundary

```

| Request Body 예시 (이미지 분석)

```

1  -----Boundary
2  Content-Disposition: form-data; name="contents"
3
4  test-01.png 이미지 파일을 분석해줘
5  -----Boundary
6  Content-Disposition: form-data; name="isStream"
7
8  true
9  -----Boundary
10 Content-Disposition: form-data; name="modelIds"
11
12 YOUR_TEXT_MODEL_ID
13 -----Boundary
14 Content-Disposition: form-data; name="modelIds"
15
16 YOUR_I2T_MODEL_ID
17 -----Boundary
18 Content-Disposition: form-data; name="files"; filename="test-01.png"
19
20 < ./test-01.png
21 -----Boundary--

```

| Response

주의

※ Stream 으로 응답을 받을 경우 Field 명이 카멜케이스가 아닌 스네이크케이스로 표시됩니다.
예) modelType -> model_type

Field	Type	Description	
-------	------	-------------	--

Field	Type	Description	
id	integer	openapi message Id	DEPRECATED
parentMessageId	string	openapi message parent id	DEPRECATED
parentMessageCreatedAt	DateTime	parentMessage creation time	DEPRECATED
chatId	integer	chat Id	DEPRECATED
userId	integer	user Id	
modelId	string	LLM ModelId	DEPRECATED
modelType	string	LLM ModelType	
content	string	Answer	
createdAt	DateTime	Message creation time	DEPRECATED
completionToken	integer	Answer Token Usage	DEPRECATED
promptToken	integer	Question Token Usage	DEPRECATED
truncated	string	Whether the question is truncated or not	
finishReason	string	FinishReason (stop : normal, length : Answer length exceeded)	
filterBlockReason.ko	string	Filter block reason(ko)	
filterBlockReason.en	string	Filter block reason(en)	
filterBlockReason.policy_id	string	Filter block reason(policy_id)	
filterBlockReason.message	string	Filter block reason(message)	
filterBlockReason.result_code	string	Filter block reason(result_code)	
filterBlockReason.filter_log_id	string	Filter block reason(filter_log_id)	
status	string	Status	
responseCode	string	ResponseCode	
plugins	list[str]	Requested plugin	
references	list[str]	References used while generating the answer	DEPRECATED
catalogs	list[int]	Requested Catalog	DEPRECATED
eventStatus	string	Event status	
eventData	string	EventData	
filterValidation	boolean	Filter success or not	DEPRECATED
successYn	boolean	Success or failure	DEPRECATED
reasoning_content	string	추론 과정	
processing_content	list	스트리밍 일 경우 중간 처리 과정	

Field	Type	Description	
content_references	list	References used while generating the answer	
recommendQueries	list[str]	Recommended queries	
event_status	string	스트리밍일 경우 status	

| event_status 상세

Value	Description	
REQUEST_ANALYSIS	사용자 질문 분석	
REQUEST_ANALYSIS_ERROR	분석 에러	
EXECUTION_PLANNING	계획 수립	
EXECUTION_PLANNING_ERROR	계획 수립중 오	
EXECUTION_REQUEST	실행 요청	
EXECUTION_REQUEST_RESULT	실행 요청 상세	
EXECUTION_REQUEST_ERROR	실행 요청 에러	
EXECUTION	실행 중	
EXECUTION_RESULT	실행 결과	
EXECUTION_ERROR	실행 에러	
THINK	생각중	
EXECUTION	실행중	
FINAL_ANSWER	최종 답변 생성	
FINAL_ANSWER_ERROR	최종 답변 생성 에러	
CHUNK	응답	
ERROR	에러	
FILTER_INVALID	필터 에러	
RESPONSE	응답 마지막을 알림	

| Response 예시

(isStream=false)

```
1  'data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"Llama 3.3","content":
    ", "reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null,"fi
    nish_reason":null,"filter_block_reason":null,"status":"SUCCESS","response_code":null,"plugins":
    [],"orchestrator_type":null,"parent_message_created_at":"2025-08-13T18:54:55.893116+09:00","ref
    erences":null,"actions":null,"event_status":"CHUNK","event_data":""}\r\nndata: \r\n\r\nndata:
    {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"Llama
    3.3","content": "\xec\x9a\x94\xec\xb2\xad\xec\x9d\x84
```



```

\xec\xb2\x98\xeb\xa6\xac\xed\x95\x98\xea\xb8\xb0 \xec\x9c\x84\xed\x95\x9c API
\xed\x9b\x84\xeb\xb3\xb4\xeb\xa5\xbc 1\xea\xb0\x9c \xeb\xb0\x9c\xea\xb2\xac\xed\x95\x98\xec\x98
\x80\xec\x8a\xb5\xeb\x8b\x88\xeb\x8b\xa4.", "reasoning_content": null, "processing_content": [], "co
ntent_references": [], "truncated": null, "finish_reason": null, "filter_block_reason": {"ko": null, "en"
: null, "policy_id": "None", "message": "The content was passed", "result_code": "FR-200", "filter_log_
id": "None"}, "status": "SUCCESS", "response_code": null, "plugins": [], "orchestrator_type": null, "pare
nt_message_created_at": "2025-08-13T18:54:55.893116+09:00", "references": null, "actions": null, "eve
nt_status": "STATUS", "event_data": "{\\\"phase\\\": \\\"planning\\\", \\\"message\\\":
\\\"\\xec\\x9a\\x94\\xec\\xb2\\xad\\xec\\x9d\\x84 \xec\\xb2\\x98\\xeb\\xa6\\xac\\xed\\x95\\x98\\xea\\xb8\\xb0
\\xec\\x9c\\x84\\xed\\x95\\x9c API \xed\\x9b\\x84\\xeb\\xb3\\xb4\\xeb\\xa5\\xbc 1\\xea\\xb0\\x9c \xeb\\xb0\\x9c\\xe
a\\xb2\\xac\\xed\\x95\\x98\\xec\\x98\\x80\\xec\\x8a\\xb5\\xeb\\x8b\\x88\\xeb\\x8b\\xa4.\\\"}\"}\\r\\ndata:
\\r\\n\\r\\ndata: {\"user_id\": \"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e\", \"model_type\": \"Llama
3.3\", \"content\": \"API \xec\\x9d\\x98 1\\xeb\\xb2\\x88\\xec\\xa7\\xb8 \xeb\\x8b\\xa8\\xea\\xb3\\x84\\xea\\xb0\\x80
\\xec\\x8b\\xa4\\xed\\x96\\x89\\xeb\\x90\\xa9\\xeb\\x8b\\x88\\xeb\\x8b\\xa4.\"\", \"reasoning_content\": null, \"proces
sing_content\": [], \"content_references\": [], \"truncated\": null, \"finish_reason\": null, \"filter_block_re
ason\": {\"ko\": null, \"en\": null, \"policy_id\": \"None\", \"message\": \"The content was passed\", \"result_code\": \"
FR-200\", \"filter_log_id\": \"None\"}, \"status\": \"SUCCESS\", \"response_code\": null, \"plugins\": [], \"orchestra
tor_type\": null, \"parent_message_created_at\": \"2025-08-13T18:54:55.893116+09:00\", \"references\": null
, \"actions\": null, \"event_status\": \"STATUS\", \"event_data\": \"{\\\"phase\\\": \\\"execution\\\",
\\\"message\\\": \\\"API \xec\\x9d\\x98 1\\xeb\\xb2\\x88\\xec\\xa7\\xb8
\\xeb\\x8b\\xa8\\xea\\xb3\\x84\\xea\\xb0\\x80
\\xec\\x8b\\xa4\\xed\\x96\\x89\\xeb\\x90\\xa9\\xeb\\x8b\\x88\\xeb\\x8b\\xa4.\\\"}\"}\\r\\ndata: \\r\\n\\r\\ndata:
{\"user_id\": \"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e\", \"model_type\": \"Llama 3.3\", \"content\": \"API
Agent\\xea\\xb0\\x80 \xeb\\x8b\\xb5\\xeb\\xb3\\x80\\xec\\x9d\\x84 \xec\\x83\\x9d\\xec\\x84\\xb1\\xed\\x95\\xa9\\xeb
\\x8b\\x88\\xeb\\x8b\\xa4.\"\", \"reasoning_content\": null, \"processing_content\": [], \"content_references\": []
, \"truncated\": null, \"finish_reason\": null, \"filter_block_reason\": {\"ko\": null, \"en\": null, \"policy_id\": \"
None\", \"message\": \"The content was passed\", \"result_code\": \"FR-200\", \"filter_log_id\": \"None\"}, \"status\"
: \"SUCCESS\", \"response_code\": null, \"plugins\": [\"T2I\"], \"orchestrator_type\": null, \"parent_message_crea
ted_at\": \"2025-08-13T18:54:55.893116+09:00\", \"references\": null, \"actions\": null, \"event_status\": \"SYN
C\", \"event_data\": \"{\\\"phase\\\": \\\"execution\\\", \\\"message\\\": \\\"API Agent\\xea\\xb0\\x80
\\xeb\\x8b\\xb5\\xeb\\xb3\\x80\\xec\\x9d\\x84
\\xec\\x83\\x9d\\xec\\x84\\xb1\\xed\\x95\\xa9\\xeb\\x8b\\x88\\xeb\\x8b\\xa4.\\\", \\\"agent_type\\\": \\\"API\\\",
\\\"step_index\\\": 1, \\\"plugin_id\\\": \\\"T2I\\\", \\\"catalog_id\\\": [], \\\"api_id\\\":
\\\"T2I|/create_image|post|1\\\", \\\"model_infos\\\": [29], \\\"properties\\\": {\\\"display_info\\\":
{\\\"display-llm-answer\\\": true, \\\"display-request\\\": true}}}\"}\\r\\ndata: \\r\\n\\r\\ndata:
{\"user_id\": \"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e\", \"model_type\": \"Llama 3.3\", \"content\": \"API
Agent\\xea\\xb0\\x80 \xeb\\x8b\\xb5\\xeb\\xb3\\x80 \xec\\x83\\x9d\\xec\\x84\\xb1\\xec\\x9d\\x84 \xec\\xa2\\x85\\xe
b\\xa3\\x8c\\xed\\x96\\x88\\xec\\x8a\\xb5\\xeb\\x8b\\x88\\xeb\\x8b\\xa4.\"\", \"reasoning_content\": null, \"processin
g_content\": [], \"content_references\": [], \"truncated\": null, \"finish_reason\": null, \"filter_block_reaso
n\": {\"ko\": null, \"en\": null, \"policy_id\": \"None\", \"message\": \"The content was passed\", \"result_code\": \"FR

```

```

-200", "filter_log_id": "None"}, "status": "SUCCESS", "response_code": null, "plugins": ["T2I"], "orchestrator_type": null, "parent_message_created_at": "2025-08-13T18:54:55.893116+09:00", "references": null, "actions": null, "event_status": "FINISH", "event_data": "{\\\"phase\\\": \\\"execution\\\", \\\"message\\\": \\\"API Agent\\xeaxb0\\xb0 \\xeb\\xb5\\xb3\\xb0 \\xec\\x83\\x9d\\xec\\x84\\xb1\\xec\\x9d\\x84 \\xec\\xa2\\x85\\xeb\\xa3\\x8c\\xed\\x96\\x88\\xec\\x8a\\xb5\\xeb\\xb8\\x88\\xeb\\xb8\\xa4.\\\", \\\"agent_type\\\": \\\"API\\\", \\\"step_index\\\": 1, \\\"plugin_id\\\": \\\"T2I\\\", \\\"api_id\\\": \\\"T2I|/create_image|post|1\\\", \\\"catalog_id\\\": []}\"}\\r\\ndata: \\r\\n\\r\\ndata: {\"user_id\": \"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e\", \"model_type\": \"Llama 3.3\", \"content\": \"API \\xec\\x9d\\x98 1\\xeb\\xb2\\x88\\xec\\xa7\\xb8 \\xeb\\xb8\\xa8\\xea\\xb3\\x84\\xec\\x9d\\x98 \\xec\\xb8\\xa4\\xed\\x96\\x89\\xec\\x9d\\xb4 \\xec\\x99\\x84\\xeb\\xa3\\x8c\\xeb\\x90\\x98\\xec\\x97\\x88\\xec\\x8a\\xb5\\xeb\\xb8\\x88\\xeb\\xb8\\xa4.\"\", \"reasoning_content\": null, \"processing_content\": [], \"content_references\": [], \"truncated\": null, \"finish_reason\": null, \"filter_block_reason\": {\"ko\": null, \"en\": null, \"policy_id\": \"None\", \"message\": \"The content was passed\", \"result_code\": \"FR-200\", \"filter_log_id\": \"None\"}, \"status\": \"SUCCESS\", \"response_code\": null, \"plugins\": [\"T2I\"], \"orchestrator_type\": null, \"parent_message_created_at\": \"2025-08-13T18:54:55.893116+09:00\", \"references\": null, \"actions\": null, \"event_status\": \"STATUS\", \"event_data\": \"{\\\"phase\\\": \\\"execution\\\", \\\"message\\\": \\\"API \\xec\\x9d\\x98 1\\xeb\\xb2\\x88\\xec\\xa7\\xb8 \\xeb\\xb8\\xa8\\xea\\xb3\\x84\\xec\\x9d\\x98 \\xec\\xb8\\xa4\\xed\\x96\\x89\\xec\\x9d\\xb4 \\xec\\x99\\x84\\xeb\\xa3\\x8c\\xeb\\x90\\x98\\xec\\x97\\x88\\xec\\x8a\\xb5\\xeb\\xb8\\x88\\xeb\\xb8\\xa4.\\\"}\"}\\r\\ndata: \\r\\n\\r\\ndata: {\"user_id\": \"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e\", \"model_type\": \"Llama 3.3\", \"content\": \"\\xec\\xb5\\x9c\\xec\\xa2\\x85 \\xec\\x9d\\x91\\xeb\\xb8\\xb5\\xec\\x9d\\x84 \\xec\\x83\\x9d\\xec\\x84\\xb1\\xed\\x95\\xa9\\xeb\\xb8\\x88\\xeb\\xb8\\xa4.\"\", \"reasoning_content\": null, \"processing_content\": [], \"content_references\": [], \"truncated\": null, \"finish_reason\": null, \"filter_block_reason\": {\"ko\": null, \"en\": null, \"policy_id\": \"None\", \"message\": \"The content was passed\", \"result_code\": \"FR-200\", \"filter_log_id\": \"None\"}, \"status\": \"SUCCESS\", \"response_code\": null, \"plugins\": [\"T2I\"], \"orchestrator_type\": null, \"parent_message_created_at\": \"2025-08-13T18:54:55.893116+09:00\", \"references\": null, \"actions\": null, \"event_status\": \"STATUS\", \"event_data\": \"{\\\"phase\\\": \\\"execution\\\", \\\"message\\\": \\\"\\xec\\xb5\\x9c\\xec\\xa2\\x85 \\xec\\x9d\\x91\\xeb\\xb8\\xb5\\xec\\x9d\\x84 \\xec\\x83\\x9d\\xec\\x84\\xb1\\xed\\x95\\xa9\\xeb\\xb8\\x88\\xeb\\xb8\\xa4.\\\"}\"}\\r\\ndata: \\r\\n\\r\\ndata: {\"user_id\": \"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e\", \"model_type\": \"Llama 3.3\", \"content\": \"##\", \"reasoning_content\": null, \"processing_content\": [], \"content_references\": [], \"truncated\": null, \"finish_reason\": null, \"filter_block_reason\": {\"ko\": null, \"en\": null, \"policy_id\": \"None\", \"message\": \"The content was passed\", \"result_code\": \"FR-200\", \"filter_log_id\": \"None\"}, \"status\": \"SUCCESS\", \"response_code\": null, \"plugins\": [\"T2I\"], \"orchestrator_type\": null, \"parent_message_created_at\": \"2025-08-13T18:54:55.893116+09:00\", \"references\": null, \"actions\": null, \"event_status\": \"CHUNK\", \"event_data\": \"\"}\\r\\ndata: \\r\\n\\r\\ndata: {\"user_id\": \"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e\", \"model_type\": \"Llama 3.3\", \"content\": \"##\", \"reasoning_content\": null, \"processing_content\": [], \"content_references\": [], \"truncated\": null, \"finish_reason\": null, \"filter_block_reason\": {\"ko\": null, \"en\": null, \"policy_id\": \"None\", \"message\": \"The content was passed\", \"result_code\": \"FR-200\", \"filter_log_id\": \"None\"}, \"status\": \"SUCCESS\", \"re

```



```

onse_code":null,"plugins":["T2I"],"orchestrator_type":null,"parent_message_created_at":"2025-08-13T18:54:55.893116+09:00","references":null,"actions":null,"event_status":"CHUNK","event_data":""}\r\ndata: \r\n\r\ndata:
{"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"Llama 3.3","content": "\n\xec\x9d\xb4","reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null,"finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"The content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","response_code":null,"plugins":["T2I"],"orchestrator_type":null,"parent_message_created_at":"2025-08-13T18:54:55.893116+09:00","references":null,"actions":null,"event_status":"CHUNK","event_data":""}\r\ndata: \r\n\r\ndata:
{"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"Llama 3.3","content": "\xeb\xaf\xb8\xec\xa7\x80\xeb\xa5\xbc ", "reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null,"finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"The content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","response_code":null,"plugins":["T2I"],"orchestrator_type":null,"parent_message_created_at":"2025-08-13T18:54:55.893116+09:00","references":null,"actions":null,"event_status":"CHUNK","event_data":""}\r\ndata: \r\n\r\ndata:
{"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"Llama 3.3","content": "\xec\x84\x9d\xec\x84\xb1\xed\x95\x98","reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null,"finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"The content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","response_code":null,"plugins":["T2I"],"orchestrator_type":null,"parent_message_created_at":"2025-08-13T18:54:55.893116+09:00","references":null,"actions":null,"event_status":"CHUNK","event_data":""}\r\ndata: \r\n\r\ndata:
{"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"Llama 3.3","content": "\xeb\x8a\x94 ", "reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null,"finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"The content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","response_code":null,"plugins":["T2I"],"orchestrator_type":null,"parent_message_created_at":"2025-08-13T18:54:55.893116+09:00","references":null,"actions":null,"event_status":"CHUNK","event_data":""}\r\ndata: \r\n\r\ndata:
{"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"Llama 3.3","content": "\xeb\x8d\xb0 \xec\x84\xb1\xea\xb3\xb5\xed\x96\x88\xec\x8a\xb5\xeb\x8b\x88\xeb\x8b\xa4.\n\n","reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null,"finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"The content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","response_code":null,"plugins":["T2I"],"orchestrator_type":null,"parent_message_created_at":"2025-08-13T18:54:55.893116+09:00","references":null,"actions":null,"event_status":"CHUNK","event_data":""}\r\ndata: \r\n\r\ndata:
{"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"Llama 3.3","content": "\n::

```

```

lmm__image /api/storage/v1/files/71f38ed3-c9c2-4777-ad58-c136ed4054a4.png\n\n::\n", "reasoning_
content": null, "processing_content": [], "content_references": [], "truncated": null, "finish_reason":
null, "filter_block_reason": {"ko": null, "en": null, "policy_id": "None", "message": "The content was p
assed", "result_code": "FR-200", "filter_log_id": "None"}, "status": "SUCCESS", "response_code": null, "
plugins": ["T2I"], "orchestrator_type": null, "parent_message_created_at": "2025-08-13T18:54:55.8931
16+09:00", "references": null, "actions": null, "event_status": "CHUNK", "event_data": ""}\r\nndata:
\r\n\r\nndata: {"user_id": "9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e", "model_type": "Llama 3.3", "conte
nt": "", "reasoning_content": null, "processing_content": [], "content_references": [{"plugin": "T2I", "
answer": ":::: lmm__image /api/storage/v1/files/71f38ed3-c9c2-4777-ad58-c136ed4054a4.png\n\n:::", "
references": null, "augmented_standalone_queries": null}], "truncated": false, "finish_reason": null, "
filter_block_reason": {"ko": null, "en": null, "policy_id": "None", "message": "The content was passed
", "result_code": "FR-200", "filter_log_id": "None"}, "status": "SUCCESS", "response_code": "R20000", "p
lugins": ["T2I"], "orchestrator_type": null, "parent_message_created_at": "2025-08-13T18:54:55.89311
6+09:00", "references": [], "actions": [{"type": "API", "plugin": "T2I", "api": "create_image", "params":
null, "response_type": null, "method": null, "http_status": 200, "response": {}, "properties": {"x-orch-a
pi-call": true, "x-orch-answer-generation": true, "x-orch-answer-path": "", "x-orch-api-streaming": fa
lse, "x-orch-prebuilt-plugin-name": "T2I", "display_info": {"display_llm_answer": true, "display-requ
est": true}}, "answer": "iVBORw0KGgoAAAANSUhEUgAAAAEAAAABACAQAAACQaXHeAAAgHUElEQVR4nG2baaxl2VXff2vvf
c4d3zzU0NWju90ew04Gz21j7ICTWJAQEAaFMQIF0YqSiCgJfImSkCh8TZQ5UUQiFMkKEBMPEAwOGGi7beMe307u6sFd1dVd
w6uq9959dzjT3nvlwz7n3lcot3Tq3nfvuefuvdZ//dd45L3f+4iqRoyBzPUw1oIIYgzGCaoeFY/GGqKHECFEDAawKIjQxIe
GECMaA1EjoBix0JtjbYaRDGtyn0vRNj6gDVUo8FSogIBjRENEa0CFYszlszlGAE0ohrBgDEGBAQhRiWGSFRQlIgQUKJGYv
SIKCIKRERAYgSjxKg4BjggEWNBVQBNh2p6GQECQgCN6X1ofzydKkBQxYgQUcQIRElCRBADIoREBSiIgpGwRAxoqik34tGA
Iu0AjBYrAhGNC1FJG3CpGsrYIyqghiFiGBE0gi1LH9bJAIGEiWfJOKsRdTxNi0kXZTogoIoiAx7RdjQC0kfbcyMK0k2o1i
2vc0XaddiJH0vohJQiWutCGCwaDiECJCEoYVg0nbx4hFJCYBa0wLbTVgECIRMYYYW6WIpcVjiXQCM0ltTcoVHBrBisFZayC
2m1rCJWmpk7K2/4x2ApB2g0sZIK02omor+fZjI8tnUUNXFwWNCdpicHRDEbT7wiCxAQrEmC9hqSEjCIaBghBmJAJBA1Lc
YAKYgVg5gEVREwkr4nGiWRnBhc5oQYFNUe3ySatHVpgZ0sIQnJiEElQV47TRhBYxKKESFGOYWA1futvDAoKkkgxpjV1XW1e
cFg1GCMQYmYaFHi0vZF0rpEQKxtzSr9ZlQwQvssIJHTOLXGYoxpEWDSlTp0GTGotsa9tHXFdNaQTlqKSEQsBI1JONEVKmg3
L50g2vM79KTFdcuKRMwS2haTzEdM+1sx2bgRtDNDkwhYJGISNICEpBA7gadvmwSFZFik5BgxyRgMoB0HSIJLkoEuJR402aw
YAW2h1JKdFU0gtVPpoJ4EYKtIgRaCoBgDMXYCSMtUDKKxPc8moeiK0xKXJKQYAUxrLmLASCJxNe210n6QZGbLa5jElDIJ1l
pcZx+J4BPsaEQbRfbklpn56KSEGM6FCStaSscPa39Dvotg60CadqcYUy09n00kyS4Exr98nBGdSkzSmxVUZLbCIWgwht
syvS+5vCbFvVlhEDKrgEiHJkig6lu983NI7dIwup1NaS5Qke046bHnhTpKke7kURrsw6X6ms8/Yosa0CkgCS9xEcnfSMobp
3HZ73SQVxCQBx1Ypy9/udnZ6DSZFM+0HrB7afWW1QCPdxU273BZa3etT9n2H1u18fhJY99wJb+VKQYxZCW2JmuQqk83aJcf
c+UisICbxEaKrbbTcoNI+LzGuxMRtK50vH7IUFiIuFkrnEjsZdX71NFRWrrNT4pJQ0xfbib37MK2+jUdkGWCdXpAR07K+Yb
UAbdfZmnEK11Yk2a1ZtV1X+09W6zfd+tlraT25wMeVoHgCkqnNg6tKa08ACKoQoyKakyudinE1eI7d8qpS3ar0TYE1g6Jt
IJahhNJARpjR39LmXYCE2PS0pKtAFGSPncaT0lL0+i0oaGmXXcSTXzTbaGjSkW1jRy776yAhGqK7jRqy9yrkFrba6th6XrT
dxLbRxHQACKJ2IwQgiI2+fp1IMApSBttuQAIXVQvS+5oA3Yg4hScdjG/dDZ4yvbb0Hup9m5zy/BXVprstlNtN9bJT5fRYFR
dgaWFZbec5IuXu1qihM71SoskdBlViIYlSKc4pcszVAUxDmuT6UQNYftoxIpgASOK6+xP0BVzt2iILeu2VL9EfYoDvVQe00
3qKfWT4Ild7om0b1Z+psVR60VULBhjG/8neQRS9hmiYq3FCATvAXCZA0mZnyGmdSlEjagPS76wzmCNJEgYMAR6DjIDTrULd
BWCJntpDSSZd9qodhvx50ru4INTk0/Qky6R4Bg77dv2b+3g3UiInJveQAqXkCWJsUaIBL9p6AKVnXcoUBfKeI88zynIBEp1P
560SFG0iah1KpKlDa7CevCf0epa1TBj020BIY8uSsvQgnLLw07Z52sGcJhtYxiTpI00EFGMKp1Sky710EW4SgnUCxhBjSBt

```

tDdw5i5UM75XFosaK5cyFFt7ykQ+zt32Ge+85z3CQo8GzsT2m3x9SzkquH9zg8ce/xnRW8NLF13nh5ec40rpNryfcddcW3/
u+9/Ch972Ds2f3cNfJXDKBDrXdgei79C2LneiY413Nt/Zd4rQ2kh0dRlud6FPgrpbhstYRyTZ7/r6Nu01DR577Hv46Ee+j
wtzn20ixbmMGCPqQWwNWoOvwrJjEoDY3YP/CPXzn09+DzXMQC+p59ptP8u2Xv8U9d+/wlofOkZuAugy3VN1p7LYbIUrrs71D
1Rr1VAFFVvK5zycp/LmZoeZ0HjXtz9rENz7Q6w3I+0MGa2u845FH+duf/DvY6JYhruKJMUBUrAWNiR9oE7EYfEqPhYQojUj
LT297y1t5+5vvJdQnGOMxpgGkFQApjycqZmkDbfC4dD0n/+cmJwyk1N2sHzIEkQRxZJSb2lzbCMGHxTfNPT7PZrg0b03w0
/+1M/x3u9+N5ZeQoamKxgRNDZJGHXN/OgWuQPM2AF9QGNYFw0rgdi0Y6FnUGiw8mQGEqiAKHGdRgBLE1AWHPovFJi510er
lPhCjBL217xe2dXqm0xwkrLNa0BCIQYGQ/HaGz40R/7cT7+8R9gc2MTcIjkaFxBTzTQ1AtCvDLPMkJZIL4knWTAgs9rf0Wx
NsMNRlhnQXJUioQATU0sCmIoQGqM1jiNyX0kf8tq460sltd+/zxWwETKQlYcqqtCpPipBG4xJBVeNStN4tjY30Hdmn3/5q79
C1hsg0gNtv6MhZTREINCUBbEuiWWBrwRtamwoicGDydN6mhpffEjeo46RwWgEJiYeCg2Lw9vkUmPEY6iJsUgIkDaS0ajLeg
CsCGvpGk49SSsBjFE086A1RFlySopSNEaCBqJP5bb18Rr333M3v/wP/hF9N8BJRowKTUAlVXljiDhriL7GhBoTG9RXpMyzI
TYLypBZtdRNTgJZDYiWlOXnuzkCGP6mNxBqKkXMyIFfRfxvqCpZyk06ISgd2i8y7GTEE7b+co56CnInIjr93ErHDWCDyGd
6gP00pxz/PIv/TJr6xtYlPL4GF97+sMhkmXYXg9rAtpU2BhpigX4GqkLIKxRuuSenaMz8D1c9RHmmJGlvVBhZPb1ykrZTg
ekTuDrxf4ZobkynRyk+Pjw84Ls0SB0zaQApk2xTxfgClCjKiaU2HvyhdqxyEIqZ5vECsQA1lmyZzhX/zKP208voEzBi1m+M
Uc5xzNYkZ/PEYaJTQ+xQi+wviSupyjdYEGT6gX+KakKaaEYYbEhsWsYFHMWd/cwNoeRTGnriPOBqIVrE0KqXzBbHHC50QI1
5WHlhuMqywn7XKB1tC4ZeItASWzWQZTXUqqmqpHCMEHh0SSBqMRP/wDP8j9D9yPE9B6wfwZLWzXZCZDjUvCC6LZE5uSWM4p
FyfUiyn4ChsDoS6py4JqPsMPB8SqZn4yZV4WZJnF0t8mf4GymDAc9DF08L5mMZtS1hVFVeGWlZc2W0n+brPZtuh42qefRkP
7vZbdV3l38igRRWPAGUNoas7v7/H2t30HP/03fgZtajTmM06QH1D5gZYN4A8I7pUztayBF+y0DlkcXJIKBdYbXCi0Hi0qW
mKgn0+RwWir4jVAvFjrHGEGBDvwUUsFg2UEEPfyfEJ05MRdW02WC7oRhTpnRHENht7JSVx7YTJN3m16hYRYFRIYRABgx1V
bG7u872+pBf+sW/x+LWddYGGSe3b3Dz6lV2N/cQl1F7Tz60qDHQ2nh5cptydoRfTNGmxJpUI5AYkZYU62JKVGiqGpGvHHA
g6/wZUUjguQZVdUgJlJ7z2JRUTCrf2PrJjgd0532gbDyFj1fx5lDV+0NoSVNY8EHVBVnHDF4xqM+ooF/+k/+MS9+8xvcd3a
fc1FxdPUy9ayiHg7pjTdxey/QRXVNST09Zn58SCinuBjIM4sz4KuKYjGnqRoW0ylWIr1ejgH6mcFXC2ZHc+Z1Q6ERby2m18
dk0dEHaq+I6+HLCrdoacsX5g4oL8MbPZ3Pn2L6NvzVThox+X8LRN/QzyyEir/7t36R1y4+xwPnz5NTMb12men1G0xmDbu79
zLY3CEMx6hzEBqsUYr5hPLkmbXp7oS+TaXz6XzG5PA2dR2YTU9wLqLaRzWS5TnF/JjZyZrPwVFEWKhY9sK9bG5uETRic0fp
K6pJbE3glEZVkoA1a2PR1txlhQpdZTurwhSIPqLaFtFCJBdDrp4f/ZG/xoXdlQYRtnc2YXbM9VcvUZQFVWPZv3CB2B+izqE
xYjUgMeBiQ70YgVfUsU1thoSgzKse4+IqsW6oijm6tc61eoParWG9YTSd0y8WNFXNrf5b0PI6h/Esi/k0gz5kPY8ag+tluJ
QHrPw9ba6fCLCr8KbQVtoKokjXw0seIGoiSxCsCrFpyBAGueW9j7yTj3/wg0w0r3H3gw9CMSMBbPjCqC3D1jfvw9cBk3dc
k8gTo+oJrd57dLLLI60iP2zNdrlnvx1suCZzaYcTWf4x1NmpxxV9xDCAYP60XqDMcfjt+B1TFVfx2eRk7jJnj/i/MaY67Me
RZUxW5QURYMlmo7qsnvp95fVvVrtb6sCLWt6K7sjRKjT6SEkiVWJMscpgl89N3v5ed+4od54/KLDDPBH93C5Rn1Ysa3X3m
Vl195lU/89GNED9FpSmaq0YeXL3H1tw9z89pVqpMTzFq0DHe4MS8ws9tYA9Y4PIHGR+LxFayFsq4QjeA0KbBUZcPCNrZ01J
8wNI+ws38GicLtoznfFEYxn+FUDDGa02mfVY/Qa0qkIKQaYQhtjJ+Km8ZYmtqTZRBVBmcNlCU//Ykf46/85Y9y5fknMYsTb
s602d8cYsyIECNvXL/J9eMJF978INrPoWcxokhuOPvAPbjMkw8M//0//1fe9uaCnbvfzkHvLIx2GPtj8vIqs7Jm4T2Z9jEb
b6KRnKnY1G0+eY0jG4cUeyVf+frzbJ1/gAfeGpkUc460Tji5dYw14qKeKoSKLDWf4pq2RCXta1XEuNb2kxAEGA0HEOuUhjc
VP/83f4oPfuAxTl57Ebs4oZ4doeUcKwFioFhUBLVMK6W3tkXEYK0h+gKZnnD54ot89vP/h//9mc9w62bFV7/yCj/7swP07W
5g3JDZ8C76dUCrhuH6+L13cG5Yw8l1ZpMjbpzXLt+wDf+9EnMfbtc04J//qv/id/7gy/xwz/2E0Q/p+dyMmNTJNj1qpJdm
2WZQ7thiba+3zVMdRkrGAwBg2I0Mh71PHjhAt70vndRXHqBw1dfop1N6PUzMGImBqgDBmEwHjMajckHPTCeOD+Go00ef/qb
PPHEU/zxH32VV69WVA0cV8rjT32T73n/e9ndBqyFwZi+7DB2GW64xu4+yDCwYey16PP6lWd4+oVD3nY31ArRC1/7xkW8+yz
f+4FHOTMIBG6NccamKQyVdrABWfXt0kuPbX2v1XhHlF2ugA+Mhzn7awP+/id/jsXVV5hdu0wz0UB9RW+8yayqsCY1wYlKZg
cMeyNYVDQnM4rZEc8+8yRXr9ziU5/6NE+9tqA2IM7y/sc+Rr07z2LrrRz1mjZtYHmtI9++QL9YkK1B0bzAImwg4yuYoysM8
gHrw7bAqHA0zSF4D01TYXop0XLGde1ixQZDbc+3rfevAp9lq6l1LlCR19nNrGWeWX/jJv044PmBx/RLlwetIXRF9Q1MUS0jm
bbTtgkVCCcymM1547lmsB0p5w6/92qd48VpNJRCNgQCP/9/foW/h4d0fZ+f+e9lY32c02mEwyNmUiLPKme2auZlzdTKj0CN

0+gV/4bHv4n0P/zY3D8pU1EV58akneP+DBuntgIxSX6BLYDoK7Ko4ySvufnTJ6omL4B4fvAvfpzz2zvce07PKN64TJhPyH
KHyy0RT2+Qg4mE2ND4Bo2B4aCXeCdEjm5e430f+TxHkwpjYJBZmhgw1uCcZW8jY200Zn19g/WNTba3t1kbjwm+oimnKIYsH
7CxsX5cxFI3YDXGzf58Hssr3zuCcQq1sB3v3WXu7Y32d/cxAT7WcUpAcKi2Nbeuf9c+n9545wZVUpcGTQNKBNjX099JmNvg
duN1bDmnb5SqnDHC2qa0gd6wR+pVGZw11E1DL+sxHI64fXCDUSb80j/8JLdv3+T511/jNz7zB7x+cMLe7j4f+6s/xHw644G
77+L8/j7bW5tsbW/RX18DXzM7jLie4JyQGwU46LG1scb09i6j/XP81heeYGuVx49/4kf4wHe/nY0+WK0oqqIrisZTXdbT2W
CK8EwXEQJibKqxtaNIYgznz5xhOHRcf+4SxeQWa+Ipqz1NDGz0HJN5wbC/R900GNcjama2mzIYr9GUC9700H1sjyzzo9sMr
OHRh9/Eu97+HXiTMSstDIf4aNje3mJ7d4fx+hgGQ0KWqsV5bvAhldzc+oitYZ9elqER7tLIz3ziffz8L3ySk8mEtUGPQqaz
yXGqI1lr1sMHekrTcup5mQws5+pepRAEBiP+ttQnRzz/7NPo7TfINkeEqsQNcsQZJLeYfoZx1hgjESiKBdv7ZzB0k0gxxjI
aD+m5MaEuCHWDWgxcS LmzxWjnDNlOAKKYUY53EWyDaUosHskFnAF1eAFD7TX2oufmyW1MPcEFH7AxGGFRpkcnz0clJ9MZzj
mXSuJomhHUVUN81atvm6DIclwmdm3rqJxMJjz7zLc4PjqEw9usS4MzynBzDXEG2+/jej2MNQSF+KaYzXgNemgM7GxtMupDp
RWhKhmMBkTn8TFjceuQ+96xB5vbGCEuiJqQNSnxKucEeZTbGbQ3CIuTbhGF0Aa5vNDdrfWMUTUV8yKkulkim+UGATnrKPX
ESWk6a/Yhr1dTXDJjG2EaNoKsrQxgwpNHbj4yiuc07PH5SsXuX6zYX9rnX6vhzHCcDTE9TMky2hmFXXVMJ0tkhn510gQZxn
v7aGzBbPjE0KA0gby0TpxbR3JczQGLIhVUr1xNq04dsD0+JCYKBMnevQG0f3NdYSA61129zZ5Mw/hckvt68qqqpCEgA01/
W2062m8cS2+9sGQnE5IMmy5td9R1WI1vHSt6/w0Ace5aQWMg1sBsFmPQRhk0f080FbQIk45/BNZDqvif7wP1LMPBkRYw2Dz
S3m85LZt0D8/Q9jBiPE0JrZDImy054gsaY8vMXNK69ze0uAmzcPGYwGrK2NuPu+uxg0MlyWs7m+w2jjDLduTticZJhMFhDT
GK7aDJcIUfCTFm3F51vt6zIk1jZK7M5JnCA246TwxBD41itXu0ftj3LlW99g3/WpcFgvZFjydm6vKgvKQqh95Mob14nGYPI
ekQoP9Hs9FCFBG4DPG06fT6Ecqb2XD544PK1V7Gh4fEv/iEf/eCHGGycY02HHNy8waSecVK+xtbmGuubG6yPx2zt7DG/cp
VZOePKwQ1NpZhowT1c8IEQ0ihfKndFDK7V8goFXZMjeY0wzBN8hKJRjDq+8cKrvP87H6K/dy9TFbwZUddC5gVpwB/P8GVNW
URc1nP94CZF6Wki+JgKq95EY1QW3nPpmx7G9AeoCrPpjEuXX+fSldf5zd/8NF/60hMQDb/+P/+A97370/B1w3CtX9rGmLvP
neEhN2a0PaKRAXXmcf1tTN5wOHmda9duoQ1UscIF3xDiiukNQtCQR1S7ISKsZ1cxqSFp01MAK2c06jkuAiPT40tPPs+jD76
JkNdMsTg7IASLRqjmbd57qjJQ18rJbMbN4xN2tzcwMqQsZtyczFFjGIw2cGsb4AzNInDz9hEHt4/51//qP3D1xm1mDfiYhi
p/7ysvkFnH/feFYd8bTK/k/AN9prXhpJxjJ568v040Qy6+9Bpfe/oZJR0SIIlZTZ0Gj0808bQ+rmVAYzrNt7s0q7khX4fUw
aJHvfAMTM5XnmRj33ovRzMHY3tNWraIYmgfiFiquqS0noU3vHbtBrtdnf3GYt7tA70SFEZXv/HD6CLhPkr5RV4NrBbY4m
c3b3znA4q6lrxQelitCUJcUr13n50nWmb/Ocu/d+tnbPMD2ekveVs+N9TD7keF5z0KtoMPio0A2rZkY3uU03zma7KnAb9nT
tAU3V4NQkgRA8BsW5nKNpyWbe54tffpIf+vJHOFxE7ju3Q1krqo6qaZguKooyENXx3MVXEM/730vpFe8jkg8Z9vr4aPC1x9
gUq5VVzYsvXeLG8ZQwWTCrAk1I6/MoBsNx2ZABX3vqeYJa1je2GWYJoSezCiWj8pFoHXXdJP6JXtGQjhgCmCt2dST4NJSTQ
iSGdJdF80n9G0Lybo0Q1NoHijoQXZ9JETmphU//7hcxG2eI/RH5+jbe0KogT0YFs8pjM8vXv/4kNh/g1VJ7w6I0nMxLv7k
M2TDASZz5Fn08dGE+aJixjUcFw1V7xGgipBlYaIV2hQqAp6oZZURKsAWNTG14Vk+cpGBNZJf8xJHIGEAkx4r0nxPQ6tBt
sfMD7QAgRH0I6p26oqxpfB3wdabzig+BtzuHcc30y4/FvPINZ2yW4EWawQSMZB8cnqBissbxw8SVuHU5wWZ/+cEzjI9PFnH
kxxzpLDJG6alGUW968GGcG507PmEZty7j19SIUSHPC0bDIYv5gqPDYy5feZ3Xr700Bvbp7TNeW0sZrSomthoNPhB8wPsGH
wJ14/FNoGkCddPQHEDjPU3THQ11UxN8TQgV31fEePBRqUOkFkvhdZ//vT/Euz6V6UF/g1mpXL16gyzrgRia6HnqqWfwEXrj
MYPRGkdHEwbDftqeKKJLHXF4dEtmldhvW/H31dNGU4JZJDN703tYVQoi4L5YsZ8MaU/6LF/bp+z5/YZDodYYZDee0JTExq
P9+lomiY9t0fWht80+CYk7YeIj5EmppufQmyASiieKQKqHbJUuF02fPHLXyPkazR2zCxaXr18A9cb4LIB/d6I3/7tz4BJwd
F4bZNFsWBne4eIJ8sN8/kcB0qm4pFhv5P19TWWw42nHhYlQ1gfdlcW2N7eweXZYSmYTadMT2ZEZqGEGryzNLLDaYqS6q6o
qor6qpOR1PTtEddldR1TV1X1E1F1VTUTU3tG3zw1HWNbxqa0v3tfbp7zCsUP1DUyu//0Z8S8hGTInA0q5GUNbbfIx/1ECtc
vPgil1651DK7fg9rHTu700tXezKZEG0kLre/ta38uBDDxFVcUbobqewgEUY0JtINhim8+nS1Itiwbcvv8SimJFnjn4vwwi
Ypt1A8J4QPCGE9BzT36qeGD1Rw/II6gkxIaHxyVyS0JrWNDxNCKnMYjP+7BvPc/3aMa9cucanfuPTLOpAETziLEEjeZ7zb/
/9vyEf9NEo9PI+xjmMtar2dsN8MWfQy8kw9EQY5z2cm0R9EBxCXwz70zusjUfUVcWN6zeoq5q810eMxTcNBCU2nkEvY2tzH
VM3dQv3juRaAXhPDCnEDSG9DqFJH0Gb5WaruqHp+KJuucP7JXmKcVRNZLko+Hf/8b8wmVcsgqLWuutUN1jmquuCNK1d4/I/+
GJv16PWHGJNhjMXHiMZIuVgwGAzpZ461fp97zp5lazikhzB0jvVej/FgQK/nsFY4Pj6i8Q11XWGD0S4rDm/d4trV1ynLBT5

```
4sszhfPBtMSSCsSBtFGgUDekuELHLe8XamyJp534UDemrGle3f3QZpEURm3T0G7/1aaomMl0sUGspfy0Ymv4g4/hwwt7eff
z6//hvfPD7P5IIDENZ1sxnC4qiJAbIsx5GF/iqZ0Ay6jwnDvr4mBTnNXI80UzJoPEVZ87sMXNCX3v4GIiijDc26N0+pJEKa
wyuDg0iMTU/2k6xSEjDDUYwMaXIXW0QAe2msNuAKAmgG5K06btdWz2ka023/tdnWBsPGIz6RB+wikiZD80E02hc1JgTD137/
d/ng930fJ7c0mc1KvvX081y+ep3nn3+JchE4uHnA4fGEqpgRfU0vN4hXyhioYqCeJV5aFHMeedcjQIGyrsFIO2wNWT9n3Mt
oigoTqPF4PJFAIGg6fGwhHxIXdM+hcWLsMyoaYls/SN1hjKE9EmzRNNDY7+UMh0MUqKsajakMW3vP2uYaWc8ivtje2eKJr3
8VrTyD4Qa3bx3zxJef5uk/e45FWXPx5Zc4uHmAD57BoI91wnDQZzTskzmDsUIUCAB66wM0bt+kbGokd3hVfSWCqirIM4f6S
OYyXAofAtp2iNCwSoWFZc6fKkQGtK0YqCCxrRF16bEGNPrUPtbpJoaEWhCg1UhqCABXJYxX8zZ3nmYcrHH+uYQ45TGez77
uc/yoY98Dntf58Pf/5f4whe+yPHiNQbrY8ZmndFwhHUGcQ+yK0dU1Wc6nzNfzF0p/WTkdDbnxsFNNrY2UiJvLFNZQfCKsxm
5KBvb6zhjTRpiWm6im9VfBRcxtDcutTkB7S1q0pbPRdPIusSA4BEVnLgt8BK9HqOGANNU+M1Yn1DWdR4as6eP0N/2CMf5J
TlgiefeZo/+eqTvHH1Fmv9Dc7v3s38q0Su+x5k//w0poIQKiaL0WArK1UNpt9nvL7B9uYwXkSefYpNnc2UzRJGva8fv06r
377DVyekWc5o7Uhr8yolqIZmnXMcbVZJgCEkEtQrqz2yhY2rsx2xHzqAGkLVlZoZ9bxEAIinNgra0uI8GnXKtuAKENZVPx
wN13YdQzG0XMF4F5PeeNa4dUjSAy4943X+Cxjz3GFx//Mhfuu5tMDVevX0K+tz3EweFNDm+nOKHfz3jLw29mdnTIpW+/yGK
xYHdvm83dLX7n81/gm89dpPGWGKZkPcdJUeCgVW18E4gGiJ5vbWMMkEtt5/XZ4UqW90yLdz2ugva+vnQSFdhSyaxLZWkCok
rmhF4/w5pUP1wsPE0QfIiczBvWtzZwGvBe6fdzxEKIhzRNZHN9k3y8yX0PvJmPb5/hwn33sp73WcxmHE+0uXH7Jldeu8JLL
15ErNDLBSyCsLu3wzseeZRHv+vd70/vcfGFS/zJnz6JWE0wqcvdX/D/AJ0TEWYtTjJVAaaaaE1FtkSuQmCC","question"
:null}}],"event_status":"CHUNK","event_data":""}}\r\nndata: \r\n\r\n\r\n'
```

```
(isStream=true)
```

```

1 data: {"user_id": "9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e", "model_type": "Llama 3.3", "content": "
", "reasoning_content": null, "processing_content": [], "content_references": [], "truncated": null, "finish_reason": null, "filter_block_reason": null, "status": "SUCCESS", "response_code": null, "plugins": [], "orchestrator_type": null, "parent_message_created_at": "2025-08-13T18:53:00.902796+09:00", "references": null, "actions": null, "event_status": "CHUNK", "event_data": ""}

2 data:

3

4 data: {"user_id": "9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e", "model_type": "Llama 3.3", "content": "요청을 처리하기 위한 API 후보를 1개 발견하였습니다.", "reasoning_content": null, "processing_content": [], "content_references": [], "truncated": null, "finish_reason": null, "filter_block_reason": {"ko": null, "en": null, "policy_id": "None", "message": "The content was passed", "result_code": "FR-200", "filter_log_id": "None"}, "status": "SUCCESS", "response_code": null, "plugins": [], "orchestrator_type": null, "parent_message_created_at": "2025-08-13T18:53:00.902796+09:00", "reference_s": null, "actions": null, "event_status": "STATUS", "event_data": "{\n\"phase\": \"planning\", \n\"message\": \"요청을 처리하기 위한 API 후보를 1개 발견하였습니다.\"}"}

5 data:

6

7 data: {"user_id": "9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e", "model_type": "Llama 3.3", "content": "API 의 1번째 단계가 실행됩니다.", "reasoning_content": null, "processing_content": [], "content_references": [], "truncated": null, "finish_reason": null, "filter_block_reason": {"ko": null, "en": null, "policy_id": "None", "message": "The content was passed", "result_code": "FR-200", "filter_log_id": "None"}, "status": "SUCCESS", "response_code": null, "plugins": [], "orchestrator_type": null, "parent_message_created_at": "2025-08-13T18:53:00.902796+09:00", "reference_s": null, "actions": null, "event_status": "STATUS", "event_data": "{\n\"phase\": \"planning\", \n\"message\": \"API 의 1번째 단계가 실행됩니다.\"}"}

```

```

er_log_id":"None"},"status":"SUCCESS","response_code":null,"plugins":[],"orchestrator_type":null,
"parent_message_created_at":"2025-08-13T18:53:00.902796+09:00","references":null,"actions":null,
"event_status":"STATUS","event_data":{"\phase\":"execution\","\message\":"API 의 1번째
단계가 실행됩니다.\"}}
8  data:
9
10 data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"Llama
3.3","content":"API Agent가 답변을 생성합니다.","reasoning_content":null,"processing_content":
[],"content_references":[],"truncated":null,"finish_reason":null,"filter_block_reason":{"ko":null,
"en":null,"policy_id":"None","message":"The content was passed","result_code":"FR-200","filter
_log_id":"None"},"status":"SUCCESS","response_code":null,"plugins":["T2I"],"orchestrator_type"
:null,"parent_message_created_at":"2025-08-13T18:53:00.902796+09:00","references":null,"actions"
:null,"event_status":"SYNC","event_data":{"\phase\":"execution\","\message\":"API Agent가
답변을 생성합니다.\","\agent_type\":"API\","\step_index\":"1,\plugin_id\":"T2I\","\catalog_id\":"
[],\api_id\":"T2I/create_image|post|1\","\model_infos\":"[29],
\properties\":{"display_info\":{"display_llm_answer\":"true,\display_request\":"
true}}}}
11 data:
12
13 data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"Llama
3.3","content":"API Agent가 답변 생성을 종료했습니다.","reasoning_content":null,"processing_content":
[],"content_references":[],"truncated":null,"finish_reason":null,"filter_block_reason":{"ko":null,
"en":null,"policy_id":"None","message":"The content was passed","result_code":"FR-200",
"filter_log_id":"None"},"status":"SUCCESS","response_code":null,"plugins":["T2I"],"orchestrato
r_type":null,"parent_message_created_at":"2025-08-13T18:53:00.902796+09:00","references":null,"
actions":null,"event_status":"FINISH","event_data":{"\phase\":"execution\","\message\":"
API Agent가 답변 생성을 종료했습니다.\","\agent_type\":"API\","\step_index\":"1,
\plugin_id\":"T2I\","\api_id\":"T2I/create_image|post|1\","\catalog_id\":"[]}}
14 data:
15
16 data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"Llama
3.3","content":"API 의 1번째 단계의 실행이 완료되었습니다.","reasoning_content":null,"processin
g_content":[],"content_references":[],"truncated":null,"finish_reason":null,"filter_block_reaso
n":{"ko":null,"en":null,"policy_id":"None","message":"The content was passed","result_code":"FR
-200","filter_log_id":"None"},"status":"SUCCESS","response_code":null,"plugins":["T2I"],"orches
trator_type":null,"parent_message_created_at":"2025-08-13T18:53:00.902796+09:00","references":n
ull,"actions":null,"event_status":"STATUS","event_data":{"\phase\":"execution\","\message\":"
API 의 1번째 단계의 실행이 완료되었습니다.\"}}
17 data:
18

```

```

19 data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"Llama
3.3","content":"최종 응답을 생성합니다.","reasoning_content":null,"processing_content":[],"cont
ent_references":[],"truncated":null,"finish_reason":null,"filter_block_reason":{"ko":null,"en":
null,"policy_id":"None","message":"The content was passed","result_code":"FR-200","filter_log_i
d":"None"},"status":"SUCCESS","response_code":null,"plugins":["T2I"],"orchestrator_type":null,"
parent_message_created_at":"2025-08-13T18:53:00.902796+09:00","references":null,"actions":null,"
event_status":"STATUS","event_data":{"\phase\":"execution\","message\":"최종 응답을
생성합니다.\"}}
20 data:
21
22 data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"Llama 3.3","content":"
고","reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null,"
finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"Th
e content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","respon
se_code":null,"plugins":["T2I"],"orchestrator_type":null,"parent_message_created_at":"2025-08-1
3T18:53:00.902796+09:00","references":null,"actions":null,"event_status":"CHUNK","event_data":""
}
23 data:
24
25 data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"Llama 3.3","content":"
양","reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null,"
finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"Th
e content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","respon
se_code":null,"plugins":["T2I"],"orchestrator_type":null,"parent_message_created_at":"2025-08-1
3T18:53:00.902796+09:00","references":null,"actions":null,"event_status":"CHUNK","event_data":""
}
26 data:
27
28 data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"Llama 3.3","content":"
이","reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null,"
finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"Th
e content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","respon
se_code":null,"plugins":["T2I"],"orchestrator_type":null,"parent_message_created_at":"2025-08-1
3T18:53:00.902796+09:00","references":null,"actions":null,"event_status":"CHUNK","event_data":""
}
29 data:
30
31 data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"Llama
3.3","content":"를 ","reasoning_content":null,"processing_content":[],"content_references":[],"
truncated":null,"finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"No

```

```

ne", "message": "The content was passed", "result_code": "FR-200", "filter_log_id": "None"}, "status": "
SUCCESS", "response_code": null, "plugins": ["T2I"], "orchestrator_type": null, "parent_message_create
d_at": "2025-08-13T18:53:00.902796+09:00", "references": null, "actions": null, "event_status": "CHUNK"
, "event_data": ""}
32 data:
33
34 data: {"user_id": "9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e", "model_type": "Llama 3.3", "content": "
그리겠", "reasoning_content": null, "processing_content": [], "content_references": [], "truncated": nu
ll, "finish_reason": null, "filter_block_reason": {"ko": null, "en": null, "policy_id": "None", "message"
: "The content was passed", "result_code": "FR-200", "filter_log_id": "None"}, "status": "SUCCESS", "re
sponse_code": null, "plugins": ["T2I"], "orchestrator_type": null, "parent_message_created_at": "2025-
08-13T18:53:00.902796+09:00", "references": null, "actions": null, "event_status": "CHUNK", "event_dat
a": ""}
35 data:
36
37 data: {"user_id": "9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e", "model_type": "Llama 3.3", "content": "
습니다", "reasoning_content": null, "processing_content": [], "content_references": [], "truncated": nu
ll, "finish_reason": null, "filter_block_reason": {"ko": null, "en": null, "policy_id": "None", "message"
: "The content was passed", "result_code": "FR-200", "filter_log_id": "None"}, "status": "SUCCESS", "re
sponse_code": null, "plugins": ["T2I"], "orchestrator_type": null, "parent_message_created_at": "2025-
08-13T18:53:00.902796+09:00", "references": null, "actions": null, "event_status": "CHUNK", "event_dat
a": ""}
38 data:
39
40 data: {"user_id": "9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e", "model_type": "Llama
3.3", "content": ". \n", "reasoning_content": null, "processing_content": [], "content_references": [], "
truncated": null, "finish_reason": null, "filter_block_reason": {"ko": null, "en": null, "policy_id": "No
ne", "message": "The content was passed", "result_code": "FR-200", "filter_log_id": "None"}, "status": "
SUCCESS", "response_code": null, "plugins": ["T2I"], "orchestrator_type": null, "parent_message_create
d_at": "2025-08-13T18:53:00.902796+09:00", "references": null, "actions": null, "event_status": "CHUNK"
, "event_data": ""}
41 data:
42
43 data: {"user_id": "9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e", "model_type": "Llama 3.3", "content": "
고양", "reasoning_content": null, "processing_content": [], "content_references": [], "truncated": null
, "finish_reason": null, "filter_block_reason": {"ko": null, "en": null, "policy_id": "None", "message": "
The content was passed", "result_code": "FR-200", "filter_log_id": "None"}, "status": "SUCCESS", "resp
onse_code": null, "plugins": ["T2I"], "orchestrator_type": null, "parent_message_created_at": "2025-08
-13T18:53:00.902796+09:00", "references": null, "actions": null, "event_status": "CHUNK", "event_data"
: ""}

```

```

44 data:
45
46 data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"Llama 3.3","content":"
이는","reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null
,"finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"
The content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","resp
onse_code":null,"plugins":["T2I"],"orchestrator_type":null,"parent_message_created_at":"2025-08
-13T18:53:00.902796+09:00","references":null,"actions":null,"event_status":"CHUNK","event_data"
:""}
47 data:
48
49 data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"Llama
3.3","content":" ","reasoning_content":null,"processing_content":[],"content_references":[],"tr
uncated":null,"finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None"
,"message":"The content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SU
CCESS","response_code":null,"plugins":["T2I"],"orchestrator_type":null,"parent_message_created_
at":"2025-08-13T18:53:00.902796+09:00","references":null,"actions":null,"event_status":"CHUNK","
event_data":""}
50 data:
51
52 data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"Llama 3.3","content":"
일","reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null,"
finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"Th
e content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","respon
se_code":null,"plugins":["T2I"],"orchestrator_type":null,"parent_message_created_at":"2025-08-1
3T18:53:00.902796+09:00","references":null,"actions":null,"event_status":"CHUNK","event_data":""
}
53 data:
54
55 data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"Llama 3.3","content":"
반","reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null,"
finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"Th
e content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","respon
se_code":null,"plugins":["T2I"],"orchestrator_type":null,"parent_message_created_at":"2025-08-1
3T18:53:00.902796+09:00","references":null,"actions":null,"event_status":"CHUNK","event_data":""
}
56 data:
57
58 data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"Llama 3.3","content":"
적","reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null,"

```

```

finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"The
e content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","respon
se_code":null,"plugins":["T2I"],"orchestrator_type":null,"parent_message_created_at":"2025-08-1
3T18:53:00.902796+09:00","references":null,"actions":null,"event_status":"CHUNK","event_data":""
}
59 data:
60
61 data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"Llama 3.3","content":"
으로","reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null
,"finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"
The content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","respo
nse_code":null,"plugins":["T2I"],"orchestrator_type":null,"parent_message_created_at":"2025-08
-13T18:53:00.902796+09:00","references":null,"actions":null,"event_status":"CHUNK","event_data"
:""}
62 data:
63
64 data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"Llama
3.3","content":"","reasoning_content":null,"processing_content":[],"content_references":[],"tr
uncated":null,"finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None"
,"message":"The content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SU
CCESS","response_code":null,"plugins":["T2I"],"orchestrator_type":null,"parent_message_created_
at":"2025-08-13T18:53:00.902796+09:00","references":null,"actions":null,"event_status":"CHUNK",
"event_data":""}
65 data:
66
67 data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"Llama 3.3","content":"
사","reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null,"
finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"Th
e content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","respon
se_code":null,"plugins":["T2I"],"orchestrator_type":null,"parent_message_created_at":"2025-08-1
3T18:53:00.902796+09:00","references":null,"actions":null,"event_status":"CHUNK","event_data":""
}
68 data:
69
70 data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"Llama 3.3","content":"
람","reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null,"
finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"Th
e content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","respon
se_code":null,"plugins":["T2I"],"orchestrator_type":null,"parent_message_created_at":"2025-08-1
3T18:53:00.902796+09:00","references":null,"actions":null,"event_status":"CHUNK","event_data":""
}

```

```

}
71 data:
72
73 data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"Llama 3.3","content":"
들의","reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null
,"finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"
The content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","resp
onse_code":null,"plugins":["T2I"],"orchestrator_type":null,"parent_message_created_at":"2025-08
-13T18:53:00.902796+09:00","references":null,"actions":null,"event_status":"CHUNK","event_data"
:""}
74 data:
75
76 data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"Llama
3.3","content":"","reasoning_content":null,"processing_content":[],"content_references":[],"tr
uncated":null,"finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None"
,"message":"The content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SU
CCESS","response_code":null,"plugins":["T2I"],"orchestrator_type":null,"parent_message_created_
at":"2025-08-13T18:53:00.902796+09:00","references":null,"actions":null,"event_status":"CHUNK","
event_data":""}
77 data:
78
79 data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"Llama 3.3","content":"
애완","reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null
,"finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"
The content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","resp
onse_code":null,"plugins":["T2I"],"orchestrator_type":null,"parent_message_created_at":"2025-08
-13T18:53:00.902796+09:00","references":null,"actions":null,"event_status":"CHUNK","event_data"
:""}
80 data:
81
82 data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"Llama 3.3","content":"
동","reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null,"
finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"Th
e content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","respon
se_code":null,"plugins":["T2I"],"orchestrator_type":null,"parent_message_created_at":"2025-08-1
3T18:53:00.902796+09:00","references":null,"actions":null,"event_status":"CHUNK","event_data":""
}
83 data:
84
85 data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"Llama 3.3","content":""

```



```

물", "reasoning_content": null, "processing_content": [], "content_references": [], "truncated": null, "
finish_reason": null, "filter_block_reason": {"ko": null, "en": null, "policy_id": "None", "message": "Th
e content was passed", "result_code": "FR-200", "filter_log_id": "None"}, "status": "SUCCESS", "respon
se_code": null, "plugins": ["T2I"], "orchestrator_type": null, "parent_message_created_at": "2025-08-1
3T18:53:00.902796+09:00", "references": null, "actions": null, "event_status": "CHUNK", "event_data": ""
}
86 data:
87
88 data: {"user_id": "9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e", "model_type": "Llama
3.3", "content": "로 ", "reasoning_content": null, "processing_content": [], "content_references": [], "
truncated": null, "finish_reason": null, "filter_block_reason": {"ko": null, "en": null, "policy_id": "No
ne", "message": "The content was passed", "result_code": "FR-200", "filter_log_id": "None"}, "status": "
SUCCESS", "response_code": null, "plugins": ["T2I"], "orchestrator_type": null, "parent_message_create
d_at": "2025-08-13T18:53:00.902796+09:00", "references": null, "actions": null, "event_status": "CHUNK"
, "event_data": ""}
89 data:
90
91 ..... 중략
92
93 data: {"user_id": "9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e", "model_type": "Llama 3.3", "content": "
", "reasoning_content": null, "processing_content": [], "content_references": [{"plugin": "T2I", "answe
r": "::: lmm__image /api/storage/v1/files/2f8afc29-56fe-4f4b-9ee5-17d3773c3430.png\n:::", "refere
nces": null, "augmented_standalone_queries": null}], "truncated": false, "finish_reason": null, "filter
_block_reason": {"ko": null, "en": null, "policy_id": "None", "message": "The content was passed", "resu
lt_code": "FR-200", "filter_log_id": "None"}, "status": "SUCCESS", "response_code": "R20000", "plugins"
: ["T2I"], "orchestrator_type": null, "parent_message_created_at": "2025-08-13T18:53:00.902796+09:00"
, "references": [], "actions": [{"type": "API", "plugin": "T2I", "api": "create_image", "params": null, "re
sponse_type": null, "method": null, "http_status": 200, "response": {}, "properties": {"x-orch-api-call"
: true, "x-orch-answer-generation": true, "x-orch-answer-path": "", "x-orch-api-streaming": false, "x-o
rch-prebuilt-plugin-name": "T2I", "display_info": {"display-llm-answer": true, "display-request": tru
e}}, "answer": "iVBORw0KGgoAAAANSUHEUgAABAAAAQACAYAAAB/HSuDAEAEE1EQVR4n0z955Zd0a4uCgKMUMik1JVVt
c95/ye5/et2P0qPvntXVRr5iJjoH4T5AIJzrZCUZpwhZobWNDQACEcS50T/z//9/xImJiEiIiEi1j/Rv2uSzP9FiNnKcryj
eim5r0ZlZpJZ0b6dlDhdzZpYb8RzM5TAGiqkvLxnEgZMROy18/yrtYq+W+DsWq2QkdKuS1i/0kny+0xtVjihmGiPijjs3pr
l00rHF06zHljzVPru066H6vszfox+75+vj8iyMxELE7E4DfbwrbdIbyzP5cplzP1E3wJ8Up6JA1lq5fwo4UNww0zMRHJg7T
WV557XKi7vFxpJ5CcEnKK8EIlzJwUNsM+2co9QWjtCobcKL9d6nL6qIZz2KAtRr8y0DF2SAUj8JHrP3seZ4xkKHLp1JBcJ/
iqQ64vArcho1Av6lEpibh5a6Qp7pmzhYCIiGkwk3GmzrmzDfdqcNM9IgpvsuWifry2sxdskQofUHF1ec790/OhCqeJV5bHw
MvLijr9NZNy+YF0rrdyxpPEEsPp8x6lCCrnuKNXRF3qCl6xwcWZp8ZaBFqw2LYqkHgI+b/nE88DbzvAkxbomplIHvEFeY0y
cjfU712J9WkGVOL2JvEx583vXnwA0k+pvqFgqTWlFNFxJfWk0qRNQ6R6Wur6EeJdSY//zu5Kh0svfac9U8Dcs4f1lr2XLfk
RqzawriYiGVzm0vPKECNEYGbQEamtdCHKzdNeKNwFBJV/lUMBqfF8kQkdvRwyIkHHz0sEC1Nqqi6+q0j5KPKQRvy+E3Dj9ZJp
20QndU5YV6ZVRo/MJP75BV2ISYhpnFzS7d3z+n22Qu6ffacbu9e0ri98z8eg8bNlL4ZNMAGG/0bN4NuedDgaa0nmhAa2sGD

```

V4b1AjvqQyn3iWaVFS7KsGZanRN4b9mKUJjZt4K0b3P7rkug0Jd8vGTr8b1WmTW4rwjD9amXUyRgPp1Vitiv9QUzH/UG3xEw
isu1kFCix0tDuqFKaMh00gPkXIgwNr6iE1cqiKyQNiQY4VvxDRcszc03c0CiLan10PjAoJVH4tWU2wRRiUcqookAnMDmF02
A3DtPS744CI5katgqBM1KLdoANSJPq1PhJIDITHTTxAiqMrg2SfnF6410p8AZrla6Bzd0AhC8E3kM0WLojEGVoHEPeccg0
qCcGYZkNxlUDCB0tEMn/dfgzPWd0QVS/9XQgK4IHqRb3CpnXdxKlvQ30c4nqse6gwf0oC9bABm0ay+qUAMBnGfIhztaMXy
rfqhmixRdEWSnQoPyhAZww1xKC0tz9YKENIBAX1mOL18kd0fE0zBF1zus1zUfKBjdaIP1YnxVKWz6zCJ2upEoftdym0GfrA
M9m/u62wFZjpMX3ZskXiz8M0i/tA4BM9MfcwqFhPoUnXSEsZfjYip6IHSiw710jt9hbrmkNAtbtpMPB09pJbJoAR/Wp/CPS
l8IqBvWF4gnVodrBX605aQd+ehLDMj4Rmyi3k0xL7gawJW56SqDGASihpCSfa8TWMv3uCiIUD9WJYzAFEZl2f14cr/NTV6/
iTfPvWY1Sn83A/a9oDXiqxP7JM69gKaEiaE0rREuXTDAfYtJ0dkI7G+WUSq5Q6Nu1V3vE6JD8/hQnkJTVvcnm86/f6IDFw
HoRbbZLwBCATEhmebqJgxQN5s0pXBz8hM2VI6d2239vIxPc0C6+vCWwk6myzJ+bzTqVKh+iqdbDTd4VHiAbRcfB87/WJ4xh
TINUurfan4wdcyJR0X0o4fJwbYNWDP1CYmgWhoKFdBaxNiWk2B2jpbqT9o7J9YDNGQJ24PuSY0ttl4iCRCE+gKf9Mpl8j3x
zZoCAhjYGeSd4arsM0YXG65/askL1PSikxyEgpJ+V+Vp55FPsw9wGK2qLuG4xWbYt1F4251JUv2eka1vJw2PMUMqYMRdxVT
YCQ2bsddKDKnggKCT0w07gddBvFesPF5TocYCYuwjFqFQL0vCBxazpTi7u8Z4Z2VXv+10NZ1GR0gqSZ0VejarGMpq43cJXH
VhIZS2JLnYlta0FSVQCVMFHUxznB7ykjAR6B4mRBAd10wjoD38YcDY2bZuwp/A6u12dEHDz3nDvardHkvop52uASx1Tj0N
94FV0deEIzfnDIlkKiNCHJkPYPD05IRDM3xF5kbV45iBLZJ3wmJna9Qy1Sde4thoqvVqqkVs9vbaJwUE6oAC4mCicQp7zLQ
zt2dWImhdJcp7ibIATUEJZrgKFmU1fcX1eMqJd9gdLk7xQtjM+GSN9x7mN0tFdm541sKsN1B9zXnjwokNChy8AnaSqVyXEi
PObuvI9nSBKv0U8jkbC3qkN25E69FceuvoKCzX1VyV4xYpn2/8mqygn8J4J9XtBeg3VC/qlvcX2G4GB8Kcw1VqQ0LDeeUq
hlN3lz7DqZWYgFijDdjgA0Gp+pzL0/PUSIlw8s0XEav8QjWDveYNGKjP00VKJ88RonNMs67KPLSCu7NTI0US7cVABckh8D7
q7HQGwsdQFvNCa3VLrJkk9owKsd1rrviAs+W51t49vm4vK84k907y0AEBhU8VzPYAin1ohPcXb+XmqbmStiHIs+ITtuM1m
PeI/Z2Uo9af5q8QH11lQNjFCKx0t+BEHiv+nqJRK2xc7n5dfU/P7PCrn/df8jYFnKUe4jwNYQ3MIWuE1WVK3zZvwAohIjdz
00eAvsHFQhPYy00EEHMR80Z/FnJZP+I+wSaQUx05UAepLJr0hBwWp0r0tS/p09lnVk9uKcn9eGV1HspmFWXZ41Zc7fc0un
Oes9ThRGGUnt4IJqVgy9W1m1ebUnGRwTQo5rgTxinNw+wEWAaPDSdI6+PySc1pTbtbwSgE2hdB3M0SYgsleLCF9JMjYibn
IeqkyW0nW1ZhaoIFK6t0d41B/T1IN1Q0rvdTAS1vDEf96+DbQ1C6XpuA1pqrmPX00eqHn9HRN03cGb3fP4b7rJrSgTCuftc
a+UurcP4+yvNaba5sDpU3HtfVfC2j87UEY86pwWxDU0jHwgimxcBeEot55zcBTZocJHLbeKfNDYiVRiBbaHwXOFHmNODKf0
BcLhbElOsXhqas3q4dWxoxvVv4970YuKs1ueF+w4T1m9qgBNEAxuU1A306HPkKW6mRIN0vkqD6ue6Z2Hvew3sSVNmCxrRGs/
Xgtvfgk8b7TsdAi2iXjW68Xe5Puw11Zu3eoyi4TtPAC2aE9cTn2rRgs7h5w2us9pKSH3Way4pQXamGST0J6Zv0ILqH/RZ0t
qCnrZz/Zmxxwnujv1wEyd3T2z9l1s4765tYZJf+nnr0APewQI9D3Wka43+q72oT/j5q8+J1rI6dPICw9VPhp4TtMGv6U8Az
6dPFK2eVXXmAw12YI6F3/zHb5PQipVEPrQbXHRrTv5S1FuDbx7Wpt/xHNQa1UMznqj0wds+kP/0zysMeoBwwpTPJ9/FiK/k
KXgu01dH3BPB7Q3yAc1X3vPUW8H6wpjxyTmB8L2DyHYDqJBqjLpekhaYB7YIQf58FL7DX23Uxa7k1+Dx1aZMfslpQ/XGgre
6eqcQ3d9c1nH75DH62qFuvT63RON27rrfa2DU8kF/7YpWV9gZ+vPKG/1codcAY/nk+qMd7DM1ZysuNrMT/FDa7pyXwgTJ3I
6VN62+Ko61lid1u10myQJHQduvquFSuGNmmJ4nGFaa7Bq6v2u7PLM8wI9mFxZtH6U1PtN2rWP+57kyJXVPgkg09eqCiH1JG
2yLVWa4cL3xDrD1fRpaqtRN8o7rYfmbffUy1VKqV7ay4oP1+cNgKiI7PrwJclq7vA3IgE6vks3XhDCxQicFtBRna0xvvTII
twjLRryptci0ICNVA6kX6cMm19Ev/I69gs83610NHk7HKm8b2YdT1LP/zYRgG4Vzh0UunurZGq100duEusk1WT9dQncTM+1
oJR2J7sAtUBYTmm6a//S85LBCvdd1cyNVDsCFs74Hn1NVtK7JC56o+JY7WhuG8QuvwcYK6WzzphnoRzQv0Lg5yt/nUXUJNh
KJIHnTdsAKPXnLp1beLPFRMu+7tzwaS1Pabnax6orKf3m5zN/rrf3iyTTRmY5qVnBnqH9QX7AvPX5J2V9RSdp16k2/a35Z
+aUJY2vHhV3UkYqj7nJ0smV9bmTsd/+xSd5rItJ/1VroyfdnozHmDp/tpl2a6XyIq1biaa9v1Q00I201jtDFZkQ8f0T4jta
5Ynw5h13Ip0qreh5U4D2J76VmaqD2Ht+73+azAIL7iJTDQxEA0N+WcR4mPm920BB9FAHXgofG4riXdk75pwZ5crnS7dE0Ev
WBa04T79Q7hctD97CJVI3F83gbrT/U91UJZN8FSJIC1/vUaYZXm+1sRIW6GYxkFaXqtdiuA1B2FCNNpBCvBQJYrvK7MQR8V
LMJ4RcOuZoRjhJew5il/jJGSmJfJcPHpAlrx2cJ7X00MfHGymov7wuLW+M9jDZZGw+cmPAU1l8HKtEXoN3jE8g58FgGUPHb
RfjehF6824Z5YCXgRqSb04neX74pR0DvIkQf9ekboomLXzTjTufJbAqPum1xZ6ULoc0Ifhk+0F3xV4EuC12KMV8CDVWX2nG
2ivV3SaAGcRmVtQG15hsJUCgRoKzLFMOWXWZaXzAC0sD7Yq2ZgmqesLIixQ3ubaZ/nqvnTFGzrg22P/LupCAQV+GSWUTr2T

tsZtM6sLdpb9WqnktepS0azpUPocpWYmaud7t3Bpp3Q7fM8BA5y48mpHEYFuIhTw09U28CLqgbZuUpMKRVvFSxc6bWxNY6
DcZUyHNMVKGxRQBm3dXfBeC3/bkITfyDLes6dRhVxf1JXwsIcGeoruFSKddSrucf8lPp1bYtV2VQw/oekE2voz07wQTSFl
ha5ub+KemCqb5e8HdFkDDXtv8V6SLVXZN1ytxUZ5MHp+mY89S4Y+TbUuVXFce2xwrvy6m7pDwd50RjBIiDSPU/nS7yiknL
jYRFVe1g8fLNG8k6gvpFHy9wrm4aFWX9Gk3GeD61eqDsZTpZiJUS1LC9rM2wUXFQ4geZc4ECJ0v7GMfWiRlaKbiX3A/UYEK
aa/PGYusZ6t+s2v0h6Jd1nI1N5WSXPJXANDsfQ9tMW8gi7xcc1WY1vdr3vM6asvY9lq230YPKYcPZtNF+brZAuCSepqYwVf
g4CCsJYT6gqKtzUK759szebnzP67vdqcb1G6EQ00AIJoDf2VQDW+K+Sv2Z4TRAIDUHo9zx8s1lhPXEc4eh6oF+Uds1Hhi6
z4I6SoA5Xr9SmLaK9ro5X/j07pNFJZVNUVRr+6NU06e530IushFpd5/5ok6QLkI18sbQkvPvBZ7f2rTgtJst4jqJfCHzs5
yRytGrOomQVJn+aQn4jyzSGM5eBfQbVYysgDsRDZ+FBEUgt10jAgbBVQvGISGW5GL601D10Wb4uq7JkXhXULKxtpTyKFRW9
fAoHSmpnNea9dHdXqPhUy8X8Zuy4cFM+HBSdnD3PRpVRW8Hxz0kDuZ32TvQPoSisVhiwZKW1AAfBwnRBG2vTJvfPi/20fcN
YC5/p0KWR0MZ2psE72qAeTvz4SKf+SuVXOd/e5i6tCH9BRZVH6rvuetYnFD331FZPy5SJ5tSygwwyKPG8654k4bJiK5SfPc
Eli2dogxyWEzn7CnN+Tcr1F4doTXye4VRj1pfcZbo+Vftn9V4CPWDj+PdqH0eavKB/BS52EY9Y0vxpicoZgd71xXDFON4zz
RVv3/ZW4EzaC1/jpCBW08L36Hg0ywfCKtAha7V8ga/r3GUXAeYwjm2QH/2C22qipxn8ILBBRC67sx62qeZN+Xd72LE9SfdZ
v3C57/VPXHf+1nrpdU6rd/f9s3zQ5CX9m0HiFvc033UCB614oaPasBPKfcZMXH62VrUMU7z0mkmf0xvTCWdm0jTsVuugdg61
leZ+0WZt57ZZFybXhk3tcUH8R0EvzVpUah5Daeh/5VN6GY/af91sTL3fVcs4/nD2crzn1a0IM6xUq+QAxxM8Y9Qhy2iSwH0
IKyAZnFmkd+7jzyipTUXKFJFY++POS+wQIfdX7yDPDrLpOvLK+1nGV+DHFjiuJqUgw3rWDo0WDIIjWXGNTtkdwJbPPg0/w1
NFomeCHZTYy1XVy9CzDsZtmNHBtLztpqDr+2aQjBf8uxt0+KmcV+gR6oU0aGysstu0j4p3L7h/Ed0H6TYTFZh1IIX85LSs+
sr1JzqXpiR6HVboWqArUu1QmPS+JT00d7yb2KUNQ6BvLmC18HtJqPLR6wIvnWESRFT1u/pfVmQIEKhoor1CtBamYU+dRXVR
1wI6oTf97NaZ7entJzdmQo1mR091MLpNT11dZworDLqCOSPTY8YkDcIXNTH1Y+kcJGCTt6j0CLRIHv5e8hnpt9XaS/sS0E1
EgpActiYIEy9ylPW+6rf427sNCqVPWLC7LqnZPlmSu0B1fSdVV6jfwBzubnqkTkRt4ZPPbtfWUVDvvpB1MGeYdT+OzDLCLW
Uc4o41/a+6K3gH/UJK+m/Zq4IiBMNmp6cAjyDy8HhpdtdG1CqJ0b+i0l+pYELxof2rzFJPKVoM0TvwkuZBoRICwpN2p4e5B
9GRuRvsyx08s52CfPdp33e72e1X/Wz2NS/cDrr80nWnnM18sw70H/6y+3d/6ntPv0yBu8vJ6a1+P6BMKpz0igtB/Y1oBx2P
m9L2DaQ/uWcru0yGfEjQZ2yGVEhzk9zWAlPrVeiaVTBtdw5rDgzJfD8KQD4BUhq5gFFRR4uSd5cr4d75Bga74e+4y0pP4Q
Ksf36NCRymErIKjqade4CGfKRumbCy0yQU0I6H46yBLxfrx7Syk/boj49m50pyAB1XqhP6Z3Z0BNGXl8xrSfdpYqbhk6aW
Pkn18x8ufEMSQ0SzcFshCUjNIivoUibujBGky8zYJNncnCYb9XRh1FPQwtbRb0CQM/HNM8/DI2S5aA0ReV9Ft+Vrum0m7pr
zjNn2VtJFPJ/TuczMq3bvC1EcoCpTvRDXqlafwnfVb2S3gB+n2Rzw7X1iv8KtUTWcDoPnej1aFk/Ntnyl04SD+AZQfw3ouQ
0P6WR7tcEHX+p/1MjFj1lZAymwWH5ZwqBS2L0hJuXhHdC61RgKksCCJdIKpQs5gnznjU6EAFuJPWHRgFTKKKp9nypcJTibM
M2gp054onfoCI4H9pxXp6w0nhgtPDI9p6B17/AI+unl9BFESilKagtIe1hoEp80qlYw3HxzwvxxFhAVUKxTKqj5v3MTh/Fp
jdwIGEj2QReMk4n1PEoj1fYlf/68pxomeZhCM/R/HjELPk9Roi30gBoy2Lpt88PuHp+SM+fVqCP//pz/jTn/4WP//Nn/Hh/
Qc8Pj3idF61LkitEAA8tFIgd8WzMTTkP5RrzJcFZQ0snNKV9cjHF/eAydFjZ163qRy40p/3JwmQglhf7qShYk9BFLI2mudp
lPcUtrRRgDKC2ivPlDCpqJIHhTy0VbV3Qa108qU14Z0s/IjUmLg+PGH0D9x3ggUKaj17WF4xtw+gd59MTynLG9foNhRv69QV
c0q7XF7y+vJhyL0BZcFpOYCEspNkW/D4+Iz0jEcBlqYK9+vLC0QGHH6eACJ8//6CX/7yF3z7/oJ2WiECd014eHgAihY47P
uOdW2oVWGo72NKAx7qb0kxhYDb9RUm7jBwMPNIWoodXonReDV6xUG1cPoMChCAGu4PIGiEKuAjF4IUBeNFAZDQQWR019Vr
guKCErt6LsWG96xg4i0hZ8ZAapUDNEixDzEcKaoURdK9wY7HG6xRhFo1INBXbE6BEpQKXDKZZIXPAJmhKzTagVVjy4yHCUz
JrqxEO1xNn1AoGk6/rmIAq1HpymuPSGmEd0KiFKCvR0Hvhv+p4Db5RGGi1zeQrQ6BY735A7DqIMTdx3ZVwE5DU7PAfNbyP
na3mYhJ9i+J6cK0n9nfT9Xrh2XdGcv0Elx6So04DrMCChVswI1mP1/JADCbEH2XAAMk5XPzmKc7ZIDqtpFGTcaV1ARExy81
GXoU3ZB4xfQ/kYCR0K15LLGh79Z3UysiGvFjMzhcP0eKLLLeuLG2YI21Ryax5wsK058j06wo9fZvjFw9kPUDToYB7KM7R+89
7oLZQEx/rukz3R44pTXfddjb5P4I3FpGKmQhOPDsIcJjQaDu+/mdQYvU/FORoU4eDr0/01rChCzR/vdY40xzYiDo2UtG1H0
czZy2CDztyNmzrW16Rw0vfvfSr6JgrCQEYNcd+Eeh20wu3NTOff+jNPv+Szu5+jXhs7z7uGHNbgHx4in7eTBruHC1BujGP7
9A/3R93L390dF/A5rkDfnn4AaBwDJ+5zXJ+lHIxq+v1MBmvAjwF2VUE7njxA+CRRe1IxLLBzVfQENE1+WBcs6tBPA3s1aXr
UvddBoow8+VXJLr9GjIEVT8I7l0uHGtGgHIX+zrRNf5xnnA+Xwb5x470MbbpZv88kf7FY+73sDY2Lnb1/3JMgYhT9rMjzCm

1Fj/ZRQd3pd5oTJo+/TvmsUjNIxZWS1aiVa5oFu4d04dvTthsfLAz58/gweA//wD/+AnRlP7z6i1IbldMLD0xMen59xfrigrYt6dcfQat8ikAKl/YzIkRRkps9WyMs8kClwSvvcu0sXgAkGXi0A01DAs0YBquxaXqaNaqGsHEc6gs/oZjuJfXTUuQwIuUQrzFpYq/makFZiQ46pwUrbWTGLtcDUQpnMjCrq2UW1StZcrN5C0msh9Wys03cGc4lw/DisN6KvK5XJdzy31SN11MM5dA+KeuWoAiKMUi0Y0AAjBPd025Ix2xX/UlNrPebpjSjGDLiDuADcgaFtDUUY1Eq0aFIjC2ldw0JY14azeWCX9QG1rliWE07nM75/a+hjn95YcWwyRV8QcM4undJNj1oYmXGeZRWc1XgRNTUw1+eFwzJCeuoAxFFRnyvTjMBGJx2V+KNAqPKBfqLg4Xo+YkXC1KSK0Uo4KVwqfIYiQNUKtVW0uuJyecL18R3ef/iILz/9hE+ff8ZPP/+MT58/4fnpCc/PT7g8PqixxkU980Iye2V0jr8Eq1VBWrvDXK7hnSqJuLJo00Jn9y4nGbMN0hVmDKPvzlc8CkUdn+79VHoUhpBkJIPzADJ4phLYITzPxufJFomkaUwtntda0+eaoU2ghoZmqTyw+gfEALcdlVff0e7FSBn19IC+bdodQBhtPaF8P2FwA8oClogXbceP628AKlrTcPXXbWA9r3h+fM26fsT58oTL0xNkbQAE19cXXK6voTRs+4b666/YBoNrxFV6xcu3F5z0Z4yuYerb6xXv373Hvm0YfeC23fDycsX1do1indrZoqB3rd/jYrjKLlQ/pXmUkvEeC8sHprJaojgtGYw6k/cUG5UPBwS1iBX5M3oyK0PxkMkbHlVva9XUqKHGyG6FhmnvWJNjQqRpmVHCwHDb03yz1lZBSuzQx1AjgQyVRYmeoj9IaXG+WjI9NMQKux4zuJqSh/CNAiKMxKTJrxDmesj4ek1Wu/F4BhD6355CpNT1EA01+F0DDEaRENmkTroXocyGmPkVnByoFtTmUnvnXKRGWvsvQXUaMs6mrKidww6cKjATw69xWVSMag6iuZ38qBvmF0gaX0xx6i91l0mbiQowQt83SLJmWTGj9h/l/fJYNPy/4tFzWgsUnJWmcyi/Iji/904JE4E/awnWNlabB5sEQysnVNEPBjr3jGjSPxMj0dRWq/jmxTMYo5q6PNa0P7MFpPDtHy/ed19FUzPmQZcKS1JfjUkCAJ/lJV94JAz3igbugE/lrr95RPII+tn8Ym+fWhwX8L97fPg5xI0rDpNjC3JfsoWojzD+8t/OjKAcgnp62D1B6ZFJ+GrLtMNMgm6aEYfXmWEF4u7XT1gX/rxR4q/T0DyJYbGxfLr7pdH+PEzgrq6cdmFgQj7erNvgh9qL/eDH+D4bsN+8DIRDmGuJYQBjA80vB12Vw4//3cj5Q5rBuA5R/MAj30Uu3uIcIwYuI0zPJZ/kYjo/fXu0Xe8U296Ksjkj7Wqox52SaKFSFQILRpaK6YYuTBQK9qieYnL0rA0xnbbRwX07PWL4czuiLHPVUgyUHRqQkfyAQXJ7S+sXmrjvRjBhe99fxPwniEnR/j8IQYp3TBeJGVc3ozot/r/x50J+9/eibBBCwvo0ehsTCPmMzrfMbZGsSMYKvSm4AVsSG2dR7GRhcw5XEHHWEBVeDp8Qkf3r1H33b8/scvQcm4PDzi8viI53fv8fnLF3z69BmXxwuoFPQ+cGoaBs7QsFqXan0i5mE0plSsPaHALXySDi/4571r3Dt4dK0AbR5LKQW1GBx47j/V6fX2yCjMmDbDgoiiYvdckHripUoYwPlVks/NhBwPcQVAKKiloVBKDQ2EgrF17MtmwrgXyxRUymtLVcJ41lvYwZk2EF51gcwUJ2YImQc9IC7BFhWEV1QAGYwB9X527mir47RVYy8q5LhgQ0LCUjEyQqnNpQ0kttiqrRwM0TmEmRq8skELo1k0RQeoNYilJFDRDA0vWkAkqIwwtoLzacXj0yOW0wnnhzMen5/x/duvGPsNLnAg93EhgWgV5aALPHkmEWGIhdmOoc+vpqRE+o7ilaaqTbzxkGVdn8IRQ8zTOXmP06VQymWEsJmLAzuGk8zosk1PJU9XYdTzSM3HvKNB9BPPqbWalvx9PyMn37+M75+/RkfP33B569f8f7jZ3z48BmXx0c8XC6orVnEAKGg9RPAA9xHpH15Xrau3fdS4KG8CFwy3mEdKtRg5/ui+xudDuzWGTlxkHjI37Vvp7VyLA1RQT322RDCKZbzLwsD1u+cx1nocSg97GCMWhoQ0Wqee64GoDFGGKNbKyilG8U8CgKZAW6Z30oHjGjR1j7JDeUZYz6noBLWe8fv8dpZ2xo+Dl1vH9+zdc9wEGcHk8Yb084/z8Ae8/fsH7j59xvjyglAXMHdfXK3jsajBgxq+//oINBT+tZ3zcN9xer/jjt98gPLDdNvz+66/4y/UGkIa9b9uGvm1Y14plfUQpmgozWLD3DqCnor4USibsDFhcR7B9LF0ZCs0AGwCIUEqLdJtQuJzuGnkn60ijNUcayItMCmCv0tQwU6sajUHAAdtMixMKK015kDx61U8GjYx8Dsok9bAHIOhNsDm4I6DxmFRzSdrGzPSJCTqH4b3qUlFawRRxoNES10jtbQ8zJkQvvkgGfi27kaJDkSZERsrru15RrJdEFv58LoQhhiM1lPj8TB52n0J65ccB/F0/voSzj+i0njKRgh6ztmXx6L6IajdZ2dtnDbpeXGR210Jpa5Imq2XmuXr+I/AF1Gut1v4tPP9bhqkCpWhs0NaD3EC8N1xF7MeToQIPiHaQLFa0/xeiv/j7hCm60qAlGYC20Y1oCnzb4S90idc/Y8jqkANKtGwcIE0VbWnNotsMkk50DRpxH+JSqySy2d2RyfIszDCjJC6c43wk2CCA9KDwy9yiHocf9dHdRHu/uwoMH74dKlQ0cbWXghTFEt0YBSEdP85X4N3vUnT4dBj1g40m79BJf1p3g7duUdTv4MyYeY7bvU2eCx7DcH+zAYa/d0qQ1bo7I60cYoST3j7tffr19H22M/J+7hZHMfb9zo2cpJxD2jrrEb7FBEt+41j0f1+fn1w1S9AA5pbL5+Jzv7osSr4g2RTm0BXfLyF0YhHyekw+JdE3sVx7oCIZBP6dY1P21d3BwB1N3k4ov7+0E99DkjDk/kF3I0wi0aoksRGcyoE+DAvegCmlrnLYsOJ/PKjCwYHTGstxQm1rx3fOWTurA1PxNgrYwsmRvfrSRupt/lkXnvvt5Zo+c/Y00K0kM095FmfQJZnX301lHJXTcVmqwIlzWoPTqx8ZmSaIkA1DIVAc4409wEzTD2eb97LbLQqNTGrN8AINeKJ5c2y9oCuRDifTnj/7h2en59QS8PL9ie0ZUFbT7i8e4/PX77i57/5M77+6Se8+/A068MZ621V7564V76A+0jdCswLbF5jFxK0cJ2HwnWtZta75njyAI807h19V6GWzKNX10UNUu7RGcPa62DunTF+bU0oz42ipH4+ogc5WKvP5x7nUS+D5nGXUSFFPe1tPWFZVggI+75j33b0PrCuHtli1nkUQLoWjn0PKmHS00gAVhh5WeA9mZ2QeqoCCAPoXZvHBUt+iaCCm0H0HqP+8lhpI7DeBglw5RNA+JKgQncpaAuDbU1bC9sRgXS1AICyqpeH0XVArKiXeopNuNfabomKPZxb9Dq6+cTzqcZtusJtS2oZcGyrKi1ofMww

QqR91pEiXfQlZHy5sJVVOinARTS/GeHfjeiBW2YnkQAqU6GRUFYHHvJKGNE0xQmfeoU7mjKU+Hhx+SFHlo9DTw+hs6F0a6e
7RlDotc8ZlJ539IaHh8f8fHTZ7x//w5Pz0+4PJpx7ukd3r1/h6enZyzrqs0GFZd2vKv2Vq+duvywpN5l2JCL3t0vdNgNm0
GGn+mwYzjrxhoCRDwIMXpIyycWEyIVauvp+2QtRzUdBGxPGGEAUDzvnUznS/U0o5iiABiQUPErDQk8voLqBaUZuHkhZLSX2
0+hLLUifAMjFLAotpEaQXeng7MoLaiMWvngPWEejplAZpK7ie0B4fQcsZ//SP/4Bff/sVy7Li09c/4f3nr7g8v48uKuu6h
uD2xAyALRqh9sKrgv2fYfwwG+//BVSgn7/9Re8bN/wyx9/4Lfv37EYjAnEQur1c+87ts08/qSdFEot1nrV8MLgV+ufGEwW
baUHM+LpdwargyPc38/IvaQWgANP0WFhgC18v7nxjtG9dhBpRJXiZEWpzrs0+oFZ07+GQFuQUkGpQBU1CjupHszYdzUeUb0
0pgaUMTCgyvtAnwVTMKMPsb2mQ3HXDSBK+kxRLBPeRaDgaRg5dbk8CgaqF32K0QwWr6cg5ujIAqR35EHQs9xNg5K0Fa4I63
ojJJY6gBBcw7YwyZLWQBCX4l2GmBKORilQinTFBVRTbjz6J+SjgzjmxhKZcrHd5/NgXZi24gXPSvdBk2lMy9h+A8e108wBF
gHhUnkEd5icQVSlc8U1o5CTgPGYAWi0JDZlDs3GEVJyklDfDYmxmNfwuzUNoP0R12rSnfFgEDhnr2+4CTwZHb2Bss0QajW
GD3a2sd0eMUPLC2fqdJNSBalzf0JA4IXYa0xs2DysKBZaWZmDraJuzm1qQTQ2/dZSBpj1mRNP88UmTvHb1q00DDrRlBCSA
EHdxHLAE3B0TF+mU9AcN02byNndMwDMKtFvg7xLvYIIBK6TGZUJJGSUnTnmkCrcWqWNm8Wpq6zUzenqLXZ1vrZunYfTW0Da
hvbZu4GNFTRlpmrw7AA0olZEBZMMVpLVgrxWdrGaSwJxywiHPmKZJo19616KMGalo/QpZmBgQasV6XbAuq6SjnBnTYRaDBg
l9JA0VlKgNGxo3FE6SStKl5sW2Lrjdrti2BcsithI6P8gcuWNdpJJ83TYsi1T5zzlJ7relqyGjStU1NdrImmptSMWmhYzzw
xFPb58wHaSLQKsLe1+xrh391nEjwqFIhEF0BZ0ScpnBKFgb4+3zW6zXC5ru1c+fPuDzpx/w6dMHbMuC5+c3AAEPQCE68sF
VArWtQIElKIFJRnoFXg4P+KnP/kWXFdw23DlFyzrgsPhKEowCM9Pzzg9nrHWLmkmuUjrxLoCYJwfTnh+fottE+McOmOeCtZ
ldfmm1oZL3UAp43Q+oalC32pDmgUW2w3zNGHKGW2eQGCstwXLSiBrzQqpW5bw5s0b1FLw+f0LRCC8FpxPJ+kkkQ1bZ5kLAU
yEy+UCwoIXv0LN81us64q2VU3Nu0G2XvH6+iJGnq0hPZyRc/E2nMty05abQnu9N0xT0ToApNFVjDKCywaDVbFnJ/BY70qj f
ZcfeCc/7vgR78ZTwdmYqzJ4q/Qbw/OjvkF3n8BfhtA0BWFMQ0QxVRhMQCEMJV3Huw81H5b1sbZ42Awjig0XZVx3nh1X0mwF
+sidcnEnS5lXz/Q0zyd2ASiECQchMHqo7FkRH197RZ0kpjV88TLhFmEt9wPZ50yv5TqAd96vMYeMypnuBDIUWDT0d5ilV
4i14hNuExCgbs8jwbf0wZFtHM7FXq9fRy4disot2+TKN0lfsIHx57ZgBJVmme1TAYH0MGrnsJN67W1x0Byw6HQatpj38jSP
suhnNEQLDPgQF9CJwUetJitA6zqywc1vem7l9VIFgNHh4mz0Nu32cK44EImXeM9rCIgns+I4mS4d7R3gAmcI9HG5/wme5BG
WDiwp4V8uIB0pZl70L3ncT3DG7A6k6gN5TY80g1VifsLhd85Abv0LDD0UyeSJ2alpeqXsqUHEekVX0sCoADYmnyiw1nIj3L
2CFUloCAH/19VMCW2Y58WRnbwjEBjFD1AcEdrxmYV48bGR8f/JN1H5lhz0tABH41ik8K05Qzb/AjXzvJU41Psu2pqCTEDg
8XoIHJT3bwpa0oMidIGHEo6jhfn1+/qiFLz6L/fmxS4HN3+7tX/BIV/7I9stQJLzSuk1X958bfl4n4dBf+yzQZuqEvdBL2
MODFPCjbdY+zlm+8w0V7LWYrpg1iJypmyx/wDsFNBudDPW6+95zIEG1ocxRQmpNz0bbD70pxjcyHfno00M8bp15edESKTQS
sN05ew+nJH08xF5uZ183elx1DcYKTAeji+L8PvlCCHr9FI8yJtobie7lMJ92SVh7QpY5d95IbBQXx+HvR0ILKUCj0JDBoU
erd0mob0I+x/KIoWKq1jsISv9zoUY+czun50kicuynGQ52AwCedSK0xpRgnh4x3cgKwkyUrv+7VCuyGMug+yBRKsZZ/xPvP
kc1x47EdT7DV6RIKpRo/6p15nqxfAysdbkzBvbtXTZ10Sca3Tiew3QsqMQGAQKSRRY3F675L4ZTUMFh9jg6nCTH+s0PMjD
OUCKlq5EEWmCdWWCnepJJ91dZ5g+9b33siQkJHT4SEhMbN6ZDB6oEWAwAwopaa1hSoW5V8e113ztLHvNUKLh0Shk1L0L50F
2C102LC2jpylqgTiyBjw0t0QHP867p6SDwAbLSgbQvSNGGaDwBZHQIGd0JdN6TDLDDSAqqURpSX8ZystFVyUZlBogKSFrUs
uQEoUqgScLhumxgA6JYxH49Ytw1l7lqJPYntcmaIEaaieaq10kerAAt8KSupVsesXR0GPLdtw7puY05Ixx02tYIZ2LYVt9s
Vs9ZiKKV4PYSS8m5vDetakf0Ex4cnlCKdB0p6wccfvxfaf2FYwEertKvs4JYmKyjPydEQub6y3K0BS9LacJmzrFZeXF1xfX7
DWDZfr1YsQ1laxLh0oWVKFGAAKGMBYw9Gw4TDPmPKMw/GAkmbk1PDNYcbf+lu/g8PphK0zKCeUMuHHj9/j9XrF23ffSCRKy
pinCb0Bj49vkFLG4TDj+nrbp08vKImw3m74i7/4M5w0J3z+9BH4YBaK263GzokNeTb7x5wPBTvznBbrthWrZtwu2LbxJDw
/PYtDvOM0+mMN28esawLPn38LAajumEuExIIBwVY1htyYXz+9A1/+Zd/jnfvsHx+IDHLoafum7IJWmB8Irr5YJ1XQGWfXM
+nUHIgjc1/Gzb5t0APK2ljH1ShKcKM45KLMWibuEQ2kvShF1xofC9Cbwm+JjW70H7Xso1jk+DxnFjATGyIIBCAe06EXIIfb
7lP/KXw7E9b7SjsMhrtenYoc6jXtNeTQBfeepSM0o4mPYeAZCi8KEP2QIMAYiDlC6v4b2f8H4dS2kd68djPWQ3Hv5zWqZf
v1Y9pzoaXF43QlDup7QtcuFHfvs4ibHivWR5/H4bAPqI+TRwzMSRYWdYBg1C2iBKDaBmMcDVTAI9d0DGDqgRYhUa1B1T0jD
KgcDY07+N4DFpxIoDAYT1ueE77vR6040Ig0tC8ocVIHkKQ+6dqutQZaTHS4TG7ykPhikh0DNjgMfC5IWIekRPcA8KHmuuIz
7IyRHIS/yNY4lBwWpRzyYdwqBt0Tc+d2wTeZbTX83x6db5s09u/1nD/XNe78Z/msbTfkPDwXZBFvzPA1zwTAK+P87e9in44

LhCqZcSPILabuokD3BIK9+SGx7W/mdCi9u1jI1UpCiMBfDTJfuYwJ3N4VQ/5pC4MYbpVvuUpV7sG0X4j2NS/NopX/0GJ00I4AEHiiPiQYDVzwBJ2Qz2JhyiD7Cq03pct0KUXm0pQekK0uT/az0DkvtMtq2GvKxQsA8Pp3Tvg0mAm5224AHvs0IrvvNhOh4CNhZFr3gdvYaP3N4sM1ufy64kuLhbV4lwhVGj14B+XwEXlBAzCqKt3FVVNR1/450MqZYDQMPB7gkBR16BZFu92K1M6YxH4Yr1g5TqwFAw5gp+1BjUww++uzRV1s0KW/cocZkKF8XpWP0U08pSWExhhiESdpNMQ3jyr2c5JPUtcvQ7NB1xXKQX+API8KFPU1HjUUEjm7G16AUq9LtcGyC4dv5uUW2sMM55yQKl1ITSkMQYd9n6vX30gRN9ryF/TP7e+hzjV7ca+970KSKdAK0LR61IFPZ/x0/S1/MA+8sp1fqhMpN0/YGDD0tJ8gu5PQ26EjadImi1DVCqu8io9hr0Qj0NKoi4JA9EkMf6aH97IYZP6d1TmIfpj3p2G2py3nbG3KxVBS1td58bPtr0Gh9dF8ABk8Rg03DRqvk/tdNaiaoAuhdG3ZRhtoZgUhrrXQwE5glb5yTPWsYAPwvS1G8IBGGFCYx+mRKYSUSUVFBzAWbZw2XStmq1AF2MFb1V9MTIErIaL5o+oQ6cRh3FaFA973XdsN0WxTgh94pUC1rbUKYz03wY+oPyAGbCVKRVnshbFmWgaM1qFEgk1fRZjAJtC2t0QG0gVzF4tFqVjjSnn1ctFCm01VL2mh2yd/qIumMxSEzTjDVfJeR7yq4YZi/42LCuC2rvoEwgSBg/pYR13bQdXEIqGcttAZ0kR55AKNMkUApoRTxGJ+ORzw9P+FwnPHx44+Q6IIR2vUVqFU6BtSKPGWstaHzFcdzx7fPj7jdX1F7RZk0aJucAUTA8fGA11+94nk7aVFK4LbcIOUGpDhkKhliCjVWn+uy4uMPH9G2huv1hjdvtNj/7Xu8ffs0nYFpPmE+n/ByvWDTCTJL5Qakj3j/71vUKpX5pUL/Cb0zchED8e22YCpSS092vYi3vVY/1x+fHiFxiYTaKh4fH1C3ijePj3j9/Bm31kCJsG4rXl9etJAKdH9VHI8nvH/3DWqVaI/eG0CS2pLUcLSuN3z+/Bnf/+oHnI4PeHh4IykkSi8EKVYzKWvNjib71Qp0Emv6wYrL5RWtIiHsdDoFWVLm03pDkTZhwC5PPQjS40WuvIvQqd+SHU4ijI57dTw2Ydp+GIPfK53x0xd6A90HMXz/PA5zULqb8j6tgL+4nnwse4blV43+0nqd81YPNQVGaXFXdIKVWRVDB2g8b0z3oMTpcsdcd/AnjaEFkRRD+PPFBcHVDpSvKsY7ZCxZynjq/bmjYqGH6cX5x2ebEK3riUJ2XI9h+N7TxiaM9AGbvXAEJhrXH34yXZKtJVaYq+k2BAwvtCm00coT/ppSQ5Rc0B+BB3LhMHofeNg3d8rkuFLgFWLhhKE0hu/t8lSp2dGDWRlirR7PMEFan8QmVMgAKd6+A6vQL/v4YX/4pQGI0h8Tb3q8Tcdlc0iE9zVCM60DLTD0KShFXvkV/Gz5l3HHB8Vtp8QpPZhuR75c2x+0H+J++++xY1n6vxp/D1H00M1RaGJb1Egen+H7yAR8MDKW889iXJpQrPZpCIF7BPoZJCck9wDHNaijOptz79/JQnwNQwzwcmtq/YQywt8+FjPYPO33RitiUAuKNABuwxs0FvWhxF1V7lHsdTctxEQ5wdVDnTcHuc7eImpiTT+db+PaJxBUjRcNEU769z+qUKNVJe9J/5iQzF5Nw1X8m39Ad9sy1D1La55eFLh9+35uK03x12Nc4m49XAoeetevJ8v6sXsauryHxW6MV9V+ux3BOXH6MnwH7y1ZswxA4F55UH0ZvWZDNYuHWaksjaqPg2jQXLH91xzEttBdUoYVQLVzo3L7Qxpa4GNQNo65LP25lBXVoYknp8ekog81Yn20nG28XQZjKFEcpQLIzPj/3UHX4j1F/WQU6bHnVmHmX1TDfFg4XZe4934zckynTSFDbLce6doXXTwEoXKY06B03D70eBwYhfVSrZcv7ZYWp9203R3YfEj64izgdpn67Qunm6Nf/d9joPedPW6503zwyX73rvGjqsYc+cXKbUlCDSL3XLaUX37vxH9ovRbwz/13+GXz+jeRgL7miUAi8CNBefzLgl25eJ7m3/cr9GVZgHvKbKqWE3vScZJGhonGENXcfjKEMqze/QRTh1htSE6Ug5yyh2m3U1RgpHqp0YfvLaXScy0atANTzP4wdXmdDjSqxTkWy0GTdD5a/nRKJ8p8zmmVJwyGAuSDlhlwyWs9gIpFuXdq0cr9kBZ0YFG2oYUBKRVs6wLqFa1XnBIhtwQGoXYWZb53pFIA7kgJqM3aMdKgHWhah+7cXLJ2t1DRJWdwlFVv3sTo0ja0voGQUZiogsWdYv4Qmi8lY902MWbUBhQJmZc2cKLMT90EqUkHgb1Inn1JCctasa5XrJu0DDydjhJvKLK06esNIhUwuDdwamAibktJQM9IBGzLDettwcvLCy6XC95NMx40Z/C5IFWGeivY0IHbAmonTYpinLLxpoZtW4H+Aqw3PD4+AyTpDiVn0Ab0ref6u+JyecXj+QF1rUipoFagVcbhfMS6bEgEFEixxh8/fI/T4xFPb5/BaJgfTiiHCYkykBi/+v5vsNYVf+d3/i7ePz1jXRdMc8G6XnG7XXE6HHE+HvHy+oK6brhdFizrBfNckEvCYZ4wvX2L23XB++dnXC6vqL3jfdZBaimAGZfXV5Q8odeKeS5Ytxtu1yvquqA36eixLit0xx0W9RWn0xHfvnuvkQ8rat2AMiMTkAtQG1C3iiknPD6ckJLUj6hbxbo4H5Gq1IscypSqDIBaHVDMM0JM7hXrMtNakVoK9KpZMxlQoPsk5QITWX8/+tUir3pS5skAAAAAE1FTkSuQMCC", "question": null}}, {"event_status": "CHUNK", "event_data": ""}

```
94 data:
```

95

```
96  'data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"Llama 3.3","content":
:":"","reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null,"
finish_reason":null,"filter_block_reason":null,"status":"SUCCESS","response_code":null,"plugins"
:[],"orchestrator_type":null,"parent_message_created_at":"2025-08-13T18:54:55.893116+09:00","re
ferences":null,"actions":null,"event_status":"CHUNK","event_data":""}\r\n
```



```

g_content":[],"content_references":[],"truncated":null,"finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"The content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","response_code":null,"plugins":["T2I"],"orchestrator_type":null,"parent_message_created_at":"2025-08-13T18:54:55.893116+09:00","references":null,"actions":null,"event_status":"FINISH","event_data":{"\\\"phase\\\": \\\"execution\\\", \\\"message\\\": \\\"API Agent\\xeaxb0\\x80 \\xeb\\x8b\\xb5\\xeb\\xb3\\x80 \\xec\\x83\\x9d\\xec\\x84\\xb1\\xec\\x9d\\x84 \\xec\\xa2\\x85\\xeb\\xa3\\x8c\\xed\\x96\\x88\\xec\\x8a\\xb5\\xeb\\x8b\\x88\\xeb\\x8b\\xa4.\\\", \\\"agent_type\\\": \\\"API\\\", \\\"step_index\\\": 1, \\\"plugin_id\\\": \\\"T2I\\\", \\\"api_id\\\": \\\"T2I|/create_image|post|1\\\", \\\"catalog_id\\\": []}\"}\\r\\ndata: \\r\\n\\r\\ndata: {\"user_id\":\"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e\",\"model_type\":\"Llama 3.3\",\"content\":\"API \\xec\\x9d\\x98 1\\xeb\\xb2\\x88\\xec\\xa7\\xb8 \\xeb\\x8b\\xa8\\xea\\xb3\\x84\\xec\\x9d\\x98 \\xec\\x8b\\xa4\\xed\\x96\\x89\\xec\\x9d\\xb4 \\xec\\x99\\x84\\xeb\\xa3\\x8c\\xeb\\x90\\x98\\xec\\x97\\x88\\xec\\x8a\\xb5\\xeb\\x8b\\x88\\xeb\\x8b\\xa4.\"\",\"reasoning_content\":null,\"processing_content\":[],\"content_references\":[],\"truncated\":null,\"finish_reason\":null,\"filter_block_reason\":{\"ko\":null,\"en\":null,\"policy_id\":\"None\",\"message\":\"The content was passed\",\"result_code\":\"FR-200\",\"filter_log_id\":\"None\"},\"status\":\"SUCCESS\",\"response_code\":null,\"plugins\":[\"T2I\"],\"orchestrator_type\":null,\"parent_message_created_at\":\"2025-08-13T18:54:55.893116+09:00\",\"references\":null,\"actions\":null,\"event_statuses\":\"STATUS\",\"event_data\":{\"\\\"phase\\\": \\\"execution\\\", \\\"message\\\": \\\"API \\xec\\x9d\\x98 1\\xeb\\xb2\\x88\\xec\\xa7\\xb8 \\xeb\\x8b\\xa8\\xea\\xb3\\x84\\xec\\x9d\\x98 \\xec\\x8b\\xa4\\xed\\x96\\x89\\xec\\x9d\\xb4 \\xec\\x99\\x84\\xeb\\xa3\\x8c\\xeb\\x90\\x98\\xec\\x97\\x88\\xec\\x8a\\xb5\\xeb\\x8b\\x88\\xeb\\x8b\\xa4.\\\"}\"}\\r\\ndata: \\r\\n\\r\\ndata: {\"user_id\":\"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e\",\"model_type\":\"Llama 3.3\",\"content\":\"\\xec\\xb5\\x9c\\xec\\xa2\\x85 \\xec\\x9d\\x91\\xeb\\x8b\\xb5\\xec\\x9d\\x84 \\xec\\x83\\x9d\\xec\\x84\\xb1\\xed\\x95\\xa9\\xeb\\x8b\\x88\\xeb\\x8b\\xa4.\"\",\"reasoning_content\":null,\"processing_content\":[],\"content_references\":[],\"truncated\":null,\"finish_reason\":null,\"filter_block_reason\":{\"ko\":null,\"en\":null,\"policy_id\":\"None\",\"message\":\"The content was passed\",\"result_code\":\"FR-200\",\"filter_log_id\":\"None\"},\"status\":\"SUCCESS\",\"response_code\":null,\"plugins\":[\"T2I\"],\"orchestrator_type\":null,\"parent_message_created_at\":\"2025-08-13T18:54:55.893116+09:00\",\"references\":null,\"actions\":null,\"event_status\":\"STATUS\",\"event_data\":{\"\\\"phase\\\": \\\"execution\\\", \\\"message\\\": \\\"\\\"\\xec\\xb5\\x9c\\xec\\xa2\\x85 \\xec\\x9d\\x91\\xeb\\x8b\\xb5\\xec\\x9d\\x84 \\xec\\x83\\x9d\\xec\\x84\\xb1\\xed\\x95\\xa9\\xeb\\x8b\\x88\\xeb\\x8b\\xa4.\\\"}\"}\\r\\ndata: \\r\\n\\r\\ndata: {\"user_id\":\"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e\",\"model_type\":\"Llama 3.3\",\"content\":\"##\",\"reasoning_content\":null,\"processing_content\":[],\"content_references\":[],\"truncated\":null,\"finish_reason\":null,\"filter_block_reason\":{\"ko\":null,\"en\":null,\"policy_id\":\"None\",\"message\":\"The content was passed\",\"result_code\":\"FR-200\",\"filter_log_id\":\"None\"},\"status\":\"SUCCESS\",\"response_code\":null,\"plugins\":[\"T2I\"],\"orchestrator_type\":null,\"parent_message_created_at\":\"2025-08-13T18:54:55.893116+09:00\",\"references\":null,\"actions\":null,\"event_status\":\"CHUNK\",\"event_data\":\"\"}\\r\\ndata: \\r\\n\\r\\ndata: {\"user_id\":\"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e\",\"model_type\":\"Llama 3.3\",\"content\":\"##\",\"reasoning_content\":null,\"processing_content\":[],\"content_references\":[],\"truncated\":nu

```



```

ent_data":""}\r\ndata: \r\n\r\ndata:
{"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"Llama 3.3","content":"\n::
lmm__image /api/storage/v1/files/71f38ed3-c9c2-4777-ad58-c136ed4054a4.png\n\n::\n","reasoning_
content":null,"processing_content":[],"content_references":[],"truncated":null,"finish_reason":
null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"The content was p
assed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","response_code":null,"
plugins":["T2I"],"orchestrator_type":null,"parent_message_created_at":"2025-08-13T18:54:55.8931
16+09:00","references":null,"actions":null,"event_status":"CHUNK","event_data":""}\r\ndata:
\r\n\r\ndata: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"Llama 3.3","conte
nt":"","reasoning_content":null,"processing_content":[],"content_references":[{"plugin":"T2I","
answer":"","lmm__image /api/storage/v1/files/71f38ed3-c9c2-4777-ad58-c136ed4054a4.png\n\n::","
references":null,"augmented_standalone_queries":null}],"truncated":false,"finish_reason":null,"
filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"The content was passed
","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","response_code":"R20000","p
lugins":["T2I"],"orchestrator_type":null,"parent_message_created_at":"2025-08-13T18:54:55.89311
6+09:00","references":[],"actions":[{"type":"API","plugin":"T2I","api":"create_image","params":
null,"response_type":null,"method":null,"http_status":200,"response":{},"properties":{"x-orch-a
pi-call":true,"x-orch-answer-generation":true,"x-orch-answer-path":"","x-orch-api-streaming":fa
lse,"x-orch-prebuilt-plugin-name":"T2I","display_info":{"display-llm-answer":true,"display-requ
est":true}},"answer":"iVBORw0KGgoAAAANSUHEUgAAAAAAABACAYAAACqaXHeAAAgHUEQVR4nG2baaxl2VXff2vvf
c4d3zzU0NWju90ew04Gz21j7ICTWJAQEAaFMQIF0YqSiCgJfImSkCh8TZQ5UUQiFMkKEBMPEAwOGGi7beMe307u6sFd1dVd
w6uq9959dzjT3nvlwz7n3lcot3Tq3nfvuefuvdZ//dd45L3f+4iqRoyBzPUw1oIIYgzGCaoeFY/GGqKHECFEDAawKIjQxIe
GECMaA1EjoBix0JtjbYARdGtynOvRNj6gDVUo8FSogIBjRENEaOCFYszlszlgAE0ohrBgDEGBAQhRiWGSFRQlIgQUKJGYv
SIKCIKRERAYgSjxKg4BjgjEWNBVQBNh2p6GQECQgCN6X1ofzydKkBXqYgQUcQIRElCRBADIooreBSiIgpGwRAxoqik34tGA
Iu0AjBYrAhGNC1FJG3CpGsrYIyqghiFiGBE0gi1LH9bJAIGEiWfJOKsRdTgxNi0kXZTogoIoiAx7RdjQC0kfbcyMK0k2o1i
2vc0XaddiJH0vohJQiWutCGCwaDiECJCEoYVg0nbx4hFJCYBa0wLbTVgECIRMYYYW6WIpCVjiXQCM0ltTcoVHBrBisFZayC
2m1rCJWmpk7K2/4x2ApB2g0sZIK02omor+fZjI8tnUUNXFWNCdpaiChRDEbT7wiCxWAQrEmC9hqSEjCIaBghBmJAjBA1Lc
YAkYgVg5gEVREwkr4nGiWRnBhc5oQYFNUe3ySatHVpgZ0sIQnJiEElQV47TRhBYxKKEsFG0YWA1futvDAoKkkgxpjV1XW1e
cFg1GCMQYmYaFHi0vZF0rpEQKxtzSr9ZlQwVssIJHTOLXGYoxpEWDSlTp0GTGotsa9tHXFdNaQTlqKSEqSbI1JONEVKmg3
L50g2vM79KTFdcuKRMwS2haTzEdM+1sx2bgRtDNDkwhYJGISNICEpBA7gadvmwSFZFik5BgxyRgMoB0HSIJLkoEuJR402aw
YAW2h1JKdFU0gtVPpoJ4EYKtLgRaCoBgDMXYCSMtUDKKxPc8moeiK0xKXJKQYAUxrLmLASCJxNe210n6QZGbLa5jELdIJ1l
pcZx+J4BPsaEQbRfbklpn56KSEGM6FCStaSscPa39Dvotg60CadqcYUy09n00kyS4Exr98nBGdSkzSmxVUZLbCIWgwht
syvS+5vCbFvVlhEDKrgEiHJkig6lu983NI7dIwuplNaS5Qke046bHnhTpKke7kURrsw6X6ms8/Yosa0CkgCS9xEcnfSMobp
3HZ73SQVxCQBx1Ypy9/udnZ6DSZFM+0HrB7afWW1QCPdxU273BZa3etT9n2H1u18fhJY99wJb+VKQYxZCW2JmuQqk83aJcf
c+UisICbxEaKrbBtconi+LzGuxMRtK50vH7IUIFiUfkrnEjsZdX71NFRWrrNT4pJQ0xfbib37MK2+jUdkGWCDxPAR07K+Yb
UAbdfZmnEK11Yk2a1ZtV1X+09W6zfd+tlraT25wMeVoHgCkqnNg6tKa08ACKoQoyKakyudinE1eI7d8qpS3ar0TYE1g6Jt
IJahhNJARpjR39LmXYCE2PS0pKtaFGSPncat0lL0+i0oaGmXXcSTXzTbaGjSkW1jRy776yAhGqK7jRqy9yrkFrba6th6XrT
dxLbRxHQACKJ2IwQgiI2+fp1IMApSBttuQAIXVQvS+5oA3Yg4hScdjG/dDZ4yvbb0Hup9m5zy/BXVprstlNtN9bJT5fRYFR
dgaWFZbec5IuXu1qihM71SoskdBLVIiYlSKc4pcszVAUxDmuT6UQNYftoxIpgASOK6+xP0BVzt2iILeu2VL9EfYoDvVqE00
3qkFWT4Ild7omOb1Z+psVR60VULBhjG/8neQRS9hmiYq3FCATvAXCZA0mZnyGmdSlEjagPS76wzmCNJEgYMAR6DjIDTrULd

```

BWCJntpDSSZd9qodhvX50ru4INTk0/Qky6R4Bg77dv2b+3g3UInJveQAqXkCWJsUaIBL9p6AKVnXcoUBfKeI88zynIBEp1P
560SFG0iah1KpKlDa7CevCf0epa1TBj020BIY8uSsvQgnLLw07Z52sGcJhtYxiTpI00EFGMKp1Sky7l0EW4SgnUCxhBjSBt
tDdw5i5UM75XFosaK5cyFfT7ykQ+zt32Ge+85z3CQo8GzsT2m3x9SzkquH9zg8ce/xnRW8NLF13nh5ec40rpNryfcddcW3/
u+9/Ch972Ds2f3cNJFJXDKBDrXdgei79C2Lneiy413Nt/Zd4rQ2kh0dRLud6FPgrpbhstYRyTZ7/rGnu01DR577Hv46Ee+j
wtzn20ixbmMGCPqQWwNW0OvwrJjE0dY3YP/CPXzn09+DzXMQC+p59ptP8u2Xv8U9d+/wlof0kZuAugy3VN1p7LYbIUrrs7lD
1Rr1VAFFV5zycp/LmZoeZ0HjXtz9rENz7Q6w3I+0MGa2u845FH+duf/DvY6JYhrUkJMUBUrAWNiR9oE7EYfEqPhYQoJuj
LT297y1t5+5vvJdQnG0MxpgGkFQApjycqZmkDbfC4dD0n/+cmJwyk1N2sHzIEkQRxZJSb2lzbCMGHxTfNPT7PZrg0b03w0
/+1M/x3u9+N5ZeQoamKxgRNDZJGHXN/OgWuQPXM2AF9QGNYFw0rgdi0Y6FnUGiw8mQGEqiAKHGdRgBLE1AWHPovFJi510er
lPhCjBL217xe2dXqm0xwkrLNa0BCIQYGQ/HaGz40R/7cT7+8R9gc2MTcIjkaFxBTzTQlAtCvdLPMkJZIL4knWTAgs9rfOWx
NsMNRlhnQXJUIoQATU0sCmIoQGqM1jiNyX0kf8tq460sltd+/zxWwEtkQlYcqqtCpipBG4xJBVeNsTn4tjY30Hdmn3/5q79
C1hsg0gNtv6MhZTREINCUBbEuiWWBrwRtamwoicGDydN6mhpFfEjeo46RwWgEjiYeCg2Lw9vkUmPEY6iJ5UgIkDaS0ajLeg
CsC6vpGk49SSsBjFE086AlRFlySopSNEaCBqJP5bb18Rr333M3v/wP/hF9N8BJRowKTUA1VXljiDhriL7GhBoTG9RXpMyzI
TYLmpBZtdRNTgJZDYiWlOXnuzkCGP6mNxBqKkXMyIFfrfxvqCpZyk06ISgd2i8y7GTEE7b+co56CnInIjR93ERHWDyGd
6gP00pxz/PIv/Tjr6xtYlPL4GF97+sMhkmXYXg9rAtpU2BhpigX4GqkLlKcXRuSenaMz8D1c9RHmmJGlvVBhZPb1ykrZTg
ekTuDrxf4ZobkynRyk+Pjw84Ls0SB0zaQApk2xTxfgClCjKiaU2HvyhdqxyEIqZ5vECsQAl1myZzhX/zKP208voEzBi1m+M
Uc5xzNYkZ/PEYaJTQ+xQi+wwiSupyjdYEGT6gX+KakKaaEYYbEhsWsYFHMwd/cwNoeRTGnriPOBqIVrE0KqZbBbHHC50QI1
5WHlhuMqywN7XKB1Tc4ZeItASWzWQZTXUqmqpHCMEHhOSSBqMRP/wDP8j9D9yPE9B6wfzWLZwxZCZDjUvCC6lZE5uSWM4p
FyfUiyn4ChsDoS6py4JqPsMPB8SqZn4yZV4WZJnF0t8mf4GymDac9DF08L5mMZtS1hVFVeGWlZc2W0n+brPZtuh42qefRkP
7vZbdV3l38igRRWPAGUNoas7v7/H2t30HP/03fgZtajTmmN06QH1D5gZYN4A8I7pUztayBF+y0DlkcXJIKBdYbXCi0Hi0qW
mKgn0+RwWir4jVAvfjrHGEGBDvwWUsFg2UEEPfyfEJ05MZrDw02WC7oRhTpnRHENht7JSVx7YTJN3m16hYRYFRIYRabgx1V
bG7u872+pBf+sW/x+LWddYGGSe3b3Dz6lV2N/cQl1F7Tz60qDHQ2nh5cptydoRfTNGmxJpUI5AYkZYU62JKVGiqGpqGvhHA
g6/wZUuJguQZVdUgJlJ7z2JRUTCrf2PrJjgd0532gbDyFJ1fx5lDV+0NoSVNY8EHVBVnHDF4xqM+ooF/+k/+MS9+8xvcd3a
fc1FxdPUy9ayiHg7pjTdxey/QRXVNST09Zn58SCinuBjIM4sz4KuKYjGnqRoW0y1WIr1ejgH6mCFXC2ZHc+Z1Q6ERby2m18
dk0dEHaq+I6+HLCrD0acsX5g4oL8MbPZ3Pn2L6NvzVThox+X8LRN/QzyyEir/7t36R1y4+xwPnz5NTMb12men1G0xmDbu79
zLY3CEMx6hzEBqsUYr5hPLkmBxP7oS+TaXz6XzG5PA2dR2YTU9wLqLaRzWS5TnF/JjZyZRpWVFEWKhys9K9bG5uETRic0fp
K6pJbE3glEZVkoA1a2PR1txlhQpdZTurwhSIPqLaFtFCJBdDrp4f/ZG/xoXdlQYRtnc2YXbM9VcvUZQFVWPZv3CB2B+izqE
xYjUgMeBiQ70YgVfUsU1thoSgzKse4+IqsW6oijm6tc61eoParWG9YTSd0y8WNFXNrf5b0PI6h/EsI/k0gz5kPY8ag+tlUJ
QHrPw9ba6fCLCr8KbQVtoKokjXw0seIGoiSxCSrFpyBAGueW9j7yTj3/wg0w0r3H3gw9CMSMBBpJcQc3D1jfvw9cBk3dc
k8gTo+oJrd57dLLlI60iP2zNdrlnvx1suCZzaYcTWf4x1NmpxxV9xDCAYP60XqDMcfjt+B1TFVfx2eRk7jJnj/i/MaY67Me
RZUxW5QURYmLmjo7qsnvp95fVYvrtb6sCLWt6K7sjRKjT6SEkiVWJMscpgl89N3v5ed+4od54/KLDDPBH93C5Rn1Ysa3X3m
Vl195lU/89GNED9FpSmaq0YeXL3H1tw9z89pVqpMTzFq0DHe4MS8ws9tYA9Y4PIHGR+LxFayFsq4jeA0KbBUZcPCNr01J
8wNI+ws38GicLtoznfEYxn+FUDTGa02mfVY/QaOqkIKQaYQhtjJ+Km8ZYmtqTZRBVBmcNlCU//Ykf46/85Y9y5fknMYsTb
s602d8cYsyIECNvXL/J9eMJF978INrPoWcxokhu0PvAPbjMkw8M//0//1fe9uaCnbvfzkHvLIx2GPTj8vIqs7Jm4T2Z9jEb
b6KRnKnY1G0+eY0jG4cUeyVf+frzbJ1/gAfeGpkUc460Tji5dYw14qKeKoSKLDWf4pq2RCXta1XEUb2kxAEGA0HE0uUhjc
VP/83f4oPfuAxT157Ebs4oZ4doeUcKwFioFhUBLVMK6W3tkXEYK0h+gKZnnD54ot89vP/h//9mc9w62bFV7/yCj/7swP07W
5g3JDZ8C76dUCrhuH6+L13cG5Yw81lZpMjbpozXLt+wDf+9EnMfbtc04J//qv/id/7gy/xwz/2E0Q/p+dyMmNTJNj1qpJdm
2WZQ7thiba+3zVMdRkrGAwBg2I0Mh71PHjhat70vndRXHqBw1dfop1N6PUzMGImBqgDBmEwHjMajckHPTCeOD+Go00ef/qb
PPHEU/zxH32VV69WVA0cV8rjT32T73n/e9ndBqyFwZi+7DB2GW64xu4+yDCwYey16PP6lWd4+oVD3nY31ArRC1/7xkW8+yz
f+4FH0TMIbG6NccamKQyVdrABWfXt0kuPbX2v1XhHlF2ugA+Mhzn7awP+/id/jsXVV5hdu0wz0UB9RW+8yayqsCY1wYlKZg
cMeyNYVDQnM4rZEc8+8yRXr9ziU5/6NE+9tqA2IM7y/sc+Rr07z2LrrRz1MjZtYHMTI9++QL9YkK1B0bzAImwg4yuYoysM8

gHrw7bAqhA0zSF4D01TYXop0XLGde1iXQZDbc+3rfevAp9lq6lLlCR19nNrGWeWX/jJv044PmBx/RLlwetIXRF9Q1MUS0jm
bbTtgkVCCcymM1547lmsB0p5w6/92qd48VpNJRCNgQCP/9/foW/h4d0fZ+f+e9lY32c02mEwyNmUiLPKme2auZlzdTKj0CN
0+gV/4bHv4n0P/zY3D8pU1EV58akneP+DBuntgIXS6BLyDoK7Ko4ySvufnTJ6omL4B4fvAvfpzz2zvce07PKN64TJhPyH
KHyy0RT2+Qg4mE2ND4Bo2B4aCXeCdEjm5e430f+TxHkwpjYJBZmhgw1uCcZW8jY200Zn19g/WNTba3t1kbjwm+oimnKIYsH
7CxlsX5cxfI3YDxGzf58Hssr3zuCcQq1sB3v3Wxu7Y32d/cx7WcUpAcKi2Nbeuf9c+n9545wZVUpGTQNKBNjX099JmNxxg
duN1bDmnb5SqnDhc2qa0gd6wR+pVGZw11E1DL+sxHI64fXCDUSb80j/8JLdv3+T511/jNz7zB7x+cMLE7j4f+6s/xHw644G
77+L8/j7bW5+tsbW/RX18DXzM7jLie4JyQGWU46LG1scb09i6j/XP81heeYGuvx49/4kf4wHe/nY0+WK0oqqIrisZTXdbT2W
CK8EWXEQJibKqxtaNIYgznz5xhOHRcf+4SxeQwa+Ipqz1NDGz0HJN5wbc/R900GNcjama2mzIYr9GUC9700H1sjyzzo9sMr
OHRh9/Eu97+HXiTMSstDiF4aNje3mJ7d4fx+hgGQ0KWqsV5bvAhldzc+oitYZ9elqER7tLiZ3ziffz8L3ySk8mEtUGPQqaz
yXGqI1lrIsMHekrTcup5mQws5+pepRAEBiP+ttQnRzz/7NP07TfINkeEqsQNcsQZJLeYfoZx1hgjESiKBdv7ZzB0kOgxxjI
ad+m5MaEuCHWDWgcxsLmzxWjnDNlOAKKYUY53EWyDaUosHskFnAF1eAfD7TX2oufmyW1MPcEfH7AxGGFRpkcnz0c1J9MZj
mXSuJomhHUVUN81atvm6DiclwmDM3rqJxMJjz7zLc4PjqEw9us4MzynBzDXEG2+/jej2MNQsf+KaYzXgNemgM7GxtMupDp
RWhKhMmBkTn8TFjceUq+96xB5vb6CeEuiJQNSnxKucEeZtBgBQ3CIuTbhGF0Aa5vNDdrfWMUTUV8yKkulkim+UGATnrKPX
ESWk6a/Yhr1dTXDjJG2EaNoKsrQxgwpNHbj4yiuc07PH5SsXuX6zYX9rnX6vhzHCcDTE9TMky2hmFXXVMJ0tkhn510gQZxn
v7aGzBbPjE0KA0gby0TpxbR3JczQGLIhVUr1xNq04dsD0+JCyKbMnevQG0f3NdYSA61129zZ5Mw/hckvtG8qqqpCEgA01/
W2062m8cS2+9sGQnE5IMmy5td9R1WI1vHst6/w0Ace5aQWMg1sBsFmPQRhk0f080FbQIk45/BNZDqviF7wP1LMPBkRYw2Dz
S3m85LZt0D8/Q9jBiPE0JrZDIMy054gsaY8vMXNK69ze0uAmzcPGYwGrK2NuPu+uxg0MlyWs7m+w2jjDLduTTiczJhMFhDT
GK7aDjCIUfCtFm3F51vt6zIk1jZK7M5JnCA246TwxBD41itXu0ftj3LlW99g3/WpcFgvZFjydm6vKgvKQqh95Mob14nGYPI
ekQoP9Hs9FCfbG4DPG06fT6Ecngb2XDs44PK1V7Gh4fEv/iEf/eCHGGycY02HHNy8waSecVK+xtbmGuubG6yPx2zt7DG/cp
VZOePKwQ1NpZhowT1c8IEQ0ihFkndFDK7V8goFXZMjeY0wzBN8hKJRjDq+8cKrvP87H6K/dy9TFbwZUddC5gVpwB/P8GVNW
URclnP94CZF6Wki+JgKq95EYlQW3nPPmx7G9AeoCrPpjEuXX+fSlDf5zd/8NF/60hMQDb/+P/+A97370/B1w3CtX9rGmLvP
neEhN2a0PaKRAXXMcft1TN5w0Hmda9duoQ1UscIF3xDiikNQtCQR1S7ISKsZ1cxqSFp01MAK2c06jkuAiPT40tPPs+jD76
JkNdMsTg7IASLRqjmbD57qjJQ18rJbMbN4xN2tzcwMqQsZtyczFFjGIw2cGsb4AzNInDz9hEHt4/51//qP3D1xm1mDfiYhi
p/7ysvkFnH/feYd8bTK/k/AN9prXhpJxjJ568v040Qy6+9Bpfe/oZJr0SIIlZTZOGj08o8bQ+rmVAYzrNt7s0q7khX4fUw
aJHvfAMTM5XnmRj33ovRzMHY3tNWraIYmgfiFiquqS0noU3vHbtBrtdnf3GYt7tA70SFEZxv/HD6CLhpKr5RV4NrBbY4m
c3b3znA4q6lrxQelitCUJcUr13n50nWmb/0cu/d+tnbPMD2ekveVs+N9TD7keF5z0KtoMPio0A2rZkY3uU03zma7KnAb9nT
tAU3V4NqkgRA8BsW5nKNpyWbe54tffpIf+vJH0fXE7ju3Q1krqo6qaZguKooyENXx3MVXeM/730vpFe8jkg8Z9vr4aPC1x9
gUq5VVzYsvXeLG8ZQwWTCrAk1I6/MoBsNx2ZABX3vqeYJa1je2GWYJoSezCiWj8pFoHXXdJP6JXtGQjhgCMcT2dST4NJsTQ
iSGdJdF80n9G0LyboQ1NoHijoQXZ9JETmphU//7hcxG2eI/RH5+jbe0KogTOYFs8pjM8vXv/4kNh/g1VJ7w6I0nMxLv7k
M2TDA5Zz5Fn08dGE+aJiXjUcFw1V7xGgipB1YaIV2hQhAp6oZZURKsAWNTG14Vk+cpGBNZJf8xJHiGEAkx4r0nxPQ6tBt
sfMD7QAgRH0I6p26oqxpfB3wdabzig+BtzuHcc30y4/FvPINZ2yW4EWawQSMZB8cnqBissbxw8SVuHU5wWZ/+cEzjI9PFnH
kxxzpLDJG6a1gUBW968GgcG507PmEZty7j19SIUSHpc0bDIYv5gqPDYy5feZ3Xr700BvbP7TNeW0sZrSomthoNPhB8wPsGH
wJ14/FNoGkCddPQhEDjPU3THQ11UxN8TQgV3lFEePBRqU0kFkvhDZ//vT/Euz6V6UF/g1mpXLl6gyzrgRia6HnqqWfWEXrj
MYPRGkdHEwbDftqeRKKJlHXF4dEtm1DhvW/H31dNGU4JZJDN703tYVQoi4L5YsZ8MaU/6LF/bp+z5/YZDodYYzDee0JTEqx
P9+lomiY9t0fwhT80+CYk7YeIj5EmppufQmyASiieQKQhQbJuuF02fPHLXyPkazR2zCxaXr18A9cb4LIB/d6I3/7tz4BJwd
F4bZNFsWBne4eIJ8sN8/kcB0qm4pFHv5P19TWWw42nHhYlQ1gfjd1cW2N7eweXZYSmYtadMT2ZEZqGEGryzNLLDaYqS6q6o
qor6qpOR1PTtEddldR1TV1X1E1F1VTUTU3tG3zw1HWNbxqa0v3tfbp7zCsUP1DUyu//0Z8S8hGTInA0q5gUNbbfIx/1ECtc
vPgil165lDK7fg9rHTu700tXezKZEG0kLire/ta38uBDDxFVcUbobqewgEUY0JtINhim8+nS1Itiwbcvv8SimJFnjn4vwwi
Ypt1A8J4QPCGE9BzT36qeGd1Rw/II6gkxIaHxyVyS0JrWNDxNCKnMYjP+7BvPc/3aMa9cucanfuPTL0pAETziLEEjeZ7zb/
/9vyEf9NEo9PI+xjmMtar2dsN8MWfQy8kw9EQY5z2cmOR9EBxCXwz70zusjUfUVcWN6zeoq5q810eMxTcNBCU2nkEvY2tzH

[illegible]

11

Stream = true 인 경우에 최종 응답 확인은 status 값을 이용하여 확인합니다.

- 정상 답변 : status = 'SUCCESS' and response_code = 'R20000'
- 비정상 답변 : status='ERROR' or 'FILTER_INVALID'

호출 예시 (python)

이미지 생성

```
(isStream=false)
```

```
1 import requests
2 import json
3 import base64
4
5 YOUR_CLIENT_KEY = ""
6 YOUR_PASS_KEY = ""
```



```

7  ENDPOINT_URL = ""
8  YOUR_TEXT_MODEL_ID = "Get model id from All Model API"
9  YOUR_T2I_MODEL_ID = "Get model id from All Model API" # T2I(Text To Image)
10 YOUR_EMAIL = ""
11
12 headers = {
13     "x-fabrix-client": YOUR_CLIENT_KEY,
14     "x-openapi-token": YOUR_PASS_KEY,
15     "x-generative-ai-user-email": YOUR_EMAIL
16 }
17
18 llm_config = json.dumps(
19     {
20         "max_new_tokens": 2024,
21         "seed": None,
22         "top_k": 14,
23         "top_p": 0.94,
24         "temperature": 0.4,
25         "repetition_penalty": 1.04,
26     }
27 )
28
29 message_config = json.dumps({"width": 64, "height": 64})
30
31 form_data = {
32     "modelIds": [YOUR_TEXT_MODEL_ID, YOUR_T2I_MODEL_ID],
33     "contents": ["강아지 그려줘?"],
34     "llmConfig": llm_config,
35     "isStream": False,
36     "messageConfig": message_config
37 }
38
39 api_endpoint_url = f"{ENDPOINT_URL}/openapi/chat/v1/messages-with-models"
40 response = requests.post(api_endpoint_url, headers=headers, data=form_data)
41
42 # 응답 반환
43 bytes_to_json_data = json.loads(response.content)
44 image_base64 = bytes_to_json_data.get("actions")[0].get("answer")
45 image_bytes = base64.b64decode(image_base64)
46

```

```

47 with open("result.png", "wb") as f:
48     f.write(image_bytes)

```

이미지 분석

(isStream=true)

팁

Stream = true 인 경우에 최종 응답 확인은 status 값을 이용하여 확인합니다.

- 정상 답변 : status = 'SUCCESS' and response_code = 'R20000'
- 비정상 답변 : status='ERROR' or 'FILTER_INVALID'

```

1  import requests
2  import json
3  from fastapi import UploadFile
4  import sseclient
5
6  YOUR_CLIENT_KEY = ""
7  YOUR_PASS_KEY = ""
8  ENDPOINT_URL = ""
9  YOUR_TEXT_MODEL_ID = "Get model id from All Model API"
10 YOUR_I2T_MODEL_ID = "Get model id from All Model API"      # I2T(Image To Text)
11 YOUR_EMAIL = ""
12
13 file: UploadFile = open("./sample.png", "rb")
14
15 headers = {
16     "x-fabrix-client": YOUR_CLIENT_KEY,
17     "x-openapi-token": YOUR_PASS_KEY,
18     "x-generative-ai-user-email": YOUR_EMAIL
19 }
20
21 llm_config = json.dumps(
22     {
23         "max_new_tokens": 2024,
24         "seed": None,

```

```

25     "top_k": 14,
26     "top_p": 0.94,
27     "temperature": 0.4,
28     "repetition_penalty": 1.04,
29 }
30 )
31
32 form_data = {
33     "modelIds": [YOUR_TEXT_MODEL_ID, YOUR_I2T_MODEL_ID],
34     "contents": ["sample.png 이미지 파일을 분석해줘"],
35     "llmConfig": llm_config,
36     "isStream": True
37 }
38
39 api_endpoint_url = f"{ENDPOINT_URL}/openapi/chat/v1/messages-with-models"
40 response = requests.post(api_endpoint_url, headers=headers, data=form_data,
41 files=[("files", file)])
42
43 # return response
44 if response.status_code != 200:
45     print(f"Error: Status_code=[{response.status_code}], Status message=[{response.text}]")
46
47 client = sseclient.SSEClient(response)
48
49 # make result contents
50 def get_result():
51     for event in client.events():
52         # print(f"Event: {event.event}")
53         print(f"Data: {event.data}")
54         yield event.data
55
56 result_message = ""
57 for ch in get_result():
58     ch_json = json.loads(ch)
59     if ch_json['status'] == 'SUCCESS' and ch_json['response_code'] == 'R20000':
60         print("Stream Finished!!")
61     if ch_json['event_status'] == 'CHUNK' and ch_json['content']:
62         result_message += ch_json['content']
63
64 print(result_message)

```

공통 Response Code

노트

Response 가 Success (Http Status Code 가 200) 인 경우 Response code 를 통해 상세 내용 확인이 가능합니다.

Response 가 False (Http Status 가 200이 아닌) 인 경우 Error Code 를 통해 상세 내용 확인이 가능합니다.

Success

Http status code	Response code	Reason (response message)
200	R20000	success
200	R20001	(DEPRECATED) need more parameters for api call
200	R20002	(DEPRECATED) only support single-step task
200	R20003	(DEPRECATED) error occurred during api call
200	R20004	(DEPRECATED) api request timeout
200	R20005	(DEPRECATED) exceeded token limit for generating response
200	R40000	client error # generic type for client error
200	R40001	request parameter validation error
200	R40002	unsupported model error
200	R40003	invalid plugin or api specification
200	R40004	exceeded token limit for request completion
200	R40005	health check api error
200	R40006	plug-in id not found
200	R40007	search engine not used
200	R40008	blocked by security filter
200	R40009	request timeout
200	R40010	too many LLM request
200	R40011	LLM request timeout
200	R40012	Exceeding the token usage limit on the client.
200	R40013	Exceeding the filter check content limit on the client.
200	R50000	internal server error # generic type for server error
200	R50001	api specification parsing error
200	R50002	security filter service error
200	R50003	cognitive search service error
200	R50004	LLM request error
200	R50005	LLM response parsing error

Http status code	Response code	Reason (response message)
200	R50030	tool hallucination
200	R50050	Plug-in server error
200	R50051	Plug-in API connect error
200	R50052	Code Interpreter call error.
200	R50053	File summary/translate call error.
200	R50060	MCP server error
200	R50061	MCP tool connect error
200	R50070	Agent error
200	R50071	Agent(standard) connect error
200	R50072	Agent(Langflow) connect error
200	R50073	Agent(custom) connect error
200	R50074	Agent(A2A) connect error
200	R50075	Agent Builder connect error
200	R50080	Selection flow error
200	R50081	Workflow Execution error

Fail

Http status code	Error Code	Reason (response message)
413	FILTER_ERROR_2	The content length has been exceeded
500	COMMON_ERROR_1	Exception occurred
404	COMMON_ERROR_2	The table does not exist
500	COMMON_ERROR_3	Updating the table was failed
404	COMMON_ERROR_4	It does not exist
400	COMMON_ERROR_5	Request parameter or body is invalid
422	COMMON_ERROR_6	Unprocessable Entity is requeste
403	COMMON_ERROR_7	Access is denied
423	COMMON_ERROR_8	Resource is locked while being processed. Please try again later

1.3 FAQ

FabriX Chat의 기본 System Prompt를 알려주세요.

기본 System Prompt도 FabriX의 지식재산에 해당하므로 관리자에게 문의 주시면 검토하여 답변 드리겠습니다.
참고로 Chat API (Single, Multi)를 사용할 때, “systemPrompt” 매개변수를 전달하지 않으면 기본 System Prompt를 사용합니다.

Chat API를 사용할 때, 이전 대화를 고려한 응답을 받으려면 어떻게 해야 하나요?

Chat API의 “contents” 매개변수를 활용하여 대화를 구성할 수 있습니다. 예를 들어, Request Body로 “contents”: [“사용자의 첫번째 질의”, “모델의 첫번째 답변”, “사용자의 두번째 질의”, “모델의 두번째 답변”, “사용자의 마지막 질의”] 를 전달하면 contents가 대화이력 역할을 하여 이전 대화를 활용할 수 있습니다.

llmConfig 매개변수에 대해서 조금 더 상세한 내용을 알고 싶어요. FabriX Chat의 기본 llmConfig를 알려주실 수 있나요?

llmConfig 매개변수에 해당하는 변수들을 입력하지 않는 경우 Default 설정값을 적용하여 응답합니다. FabriX의 기본 llmConfig 또한 모델 연구자들의 연구 결과에 의한 지식재산에 해당하므로 추가로 필요하신 내용이 있으시면, 관리자에게 문의 바랍니다.

llmName에는 어떤 값을 추가로 입력할 수 있습니까?

현재는 “FabriX”만 지원하고 있습니다. Request body에 “llmName”을 전달하지 않으면 Default LLM(“FabriX”)를 사용하여 응답합니다.

2.

Agent APIs

2.1 Agent APIs Overview

Agent APIs는 Agent 활용 대화를 할 수 있는 API를 제공합니다.

Agent APIs 에서 제공하는 API는 다음과 같습니다.

GET /openapi/agent-chat/v1/agents

사용 권한을 가진 Agent 목록을 조회할 수 있습니다.

POST /openapi/agent-chat/v1/agent-messages

agent-messages API는 사용자 질의에 대한 Agent 응답을 생성합니다. 호출하는 Agent의 구성에 따라 Knowledge, Tools를 활용한 답변이 생성될 수 있습니다.

POST /openapi/agent-chat/v1/agent-messages/file

agent-messages file API는 코드 인터프리터가 탑재된 Agent를 사용할 때, 파일에 대한 응답을 생성합니다.

2.2 API 명세

GET /openapi/agent-chat/v1/agents

Header

Field	Type	Required	Description	Value
Content-Type	string	optional	Type of content	application/json
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-openapi-token	string	required	OpenAPI Token	ex) Bearer eyJXXX

Field	Type	Required	Description	Value
			information	
x-generative-ai-user-email	string	optional	Portal user email	ex) kim.samsung@samsung.com

Query Parameter

Key	Type	Default	Description	Default Value
keyword	String	Optional	Search keyword	
page	Integer	Required	Pagination page value	1
limit	Integer	Required	Pagination limit value	10

Response

Field	Type	Description
meta	Pagination	Pagination meta information
items	AgentInfo[]	Agent information list

Type <Pagination>

Field	Type	Description
totalItems	Integer	Total Item Count
itemCount	Integer	Current Item count
itemsPerPage	Integer	Items per page
totalPages	Integer	Total Page Count
currentPage	Integer	Current Page

Type <AgentInfo>

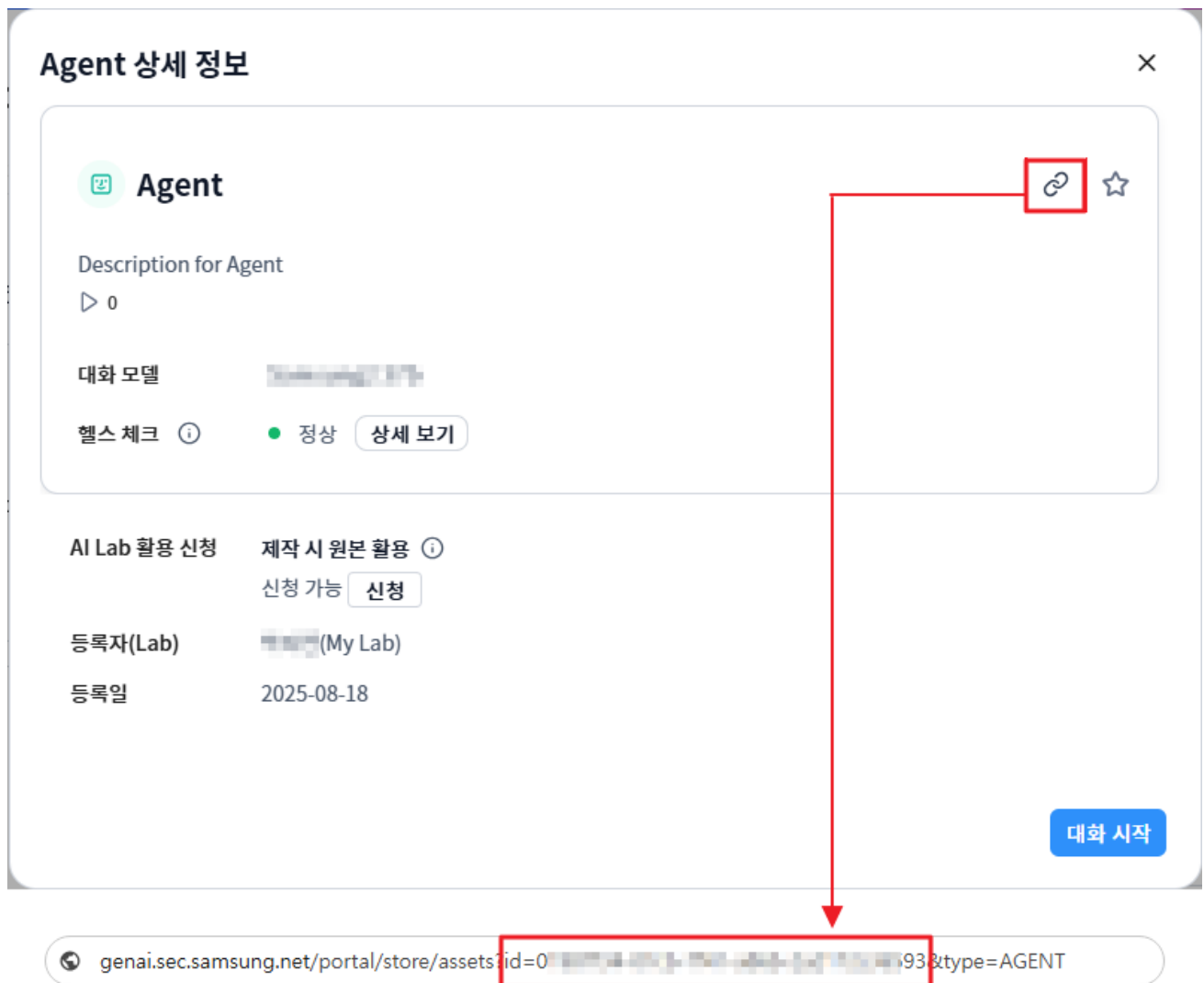
Field	Type	Description
agentId	UUID	Agent Id
label	String	Agent Label
subType	String	Agent Type ("SINGLE" or "MULTI")
description	String	Agent detail description
deployedAt	Datetime	Agent Deployed Date
modifiedAt	Datetime	Agent Modified Date

노트

agentID 확인 방법

1. "Store > Agent > Agent 상세 정보" 화면 진입
2. Agent URL 링크 복사 버튼 클릭
3. URL 내 agentID 정보 확인

그림 2-1 Agent URL 링크 내 agentID 확인하기



Response 예시

```

1 {
2   "meta": {
3     "totalItems": 11,

```

```

4     "itemCount": 10,
5     "itemsPerPage": 10,
6     "totalPages": 2,
7     "currentPage": 1
8 },
9 "items": [
10    {
11        "agentId": "019806a8-12be-7495-ac12-67255b808aa4",
12        "label": "Plugin 요리사 봇",
13        "subType": "SINGLE"
14        "description": "안녕하세요, 저는 Plugin 요리사 봇 니다. 요리 아이디어나 지침이
필요하신 가정 요리사나 음식 애호가를 위해 제가 도와드릴 수 있습니다.\n사용 가능한 재료와
식이제한, 요리시간 등을 고려하여 레시피를 추천해드리고, 단계별 요리 지침과 와인 페어링 팁도
제공합니다. ",
15        "deployedAt": "2025-05-19T08:58:45.679785+09:00",
16        "modifiedAt": "2025-05-05T14:51:56.133896+09:00"
17    },
18    {
19        "agentId": "0197f289-a2fd-7daf-89e0-e27e13dd7ffd",
20        "label": "RAG 요리사 봇",
21        "subType": "MULTI"
22        "description": "안녕하세요, RAG 봇입니다. 요리 아이디어와 지침이 필요하신 분들께
도와드리겠습니다. 재료, 식이 제한, 요리 시간에 따라 맞춤형 레시피를 제공하고, 요리 지침과 와인
페어링 팁을 제공해드리겠습니다. 하지만 실시간 영양 정보나 식이 추적 기능은 없으니 이 점
참고해주시기 바랍니다.",
23        "deployedAt": "2025-05-19T15:35:56.153453+09:00",
24        "modifiedAt": "2025-05-05T14:19:24.034232+09:00"
25    },
26    ...(생략)

```

| 호출 예시 (python)

```

1 import requests
2
3 YOUR_CLIENT_KEY = ""
4 YOUR_PASS_KEY = ""
5 ENDPOINT_URL = ""
6 YOUR_EMAIL = ""
7

```

```

8  headers = {
9      "x-fabrix-client": YOUR_CLIENT_KEY,
10     "x-openapi-token": YOUR_PASS_KEY,
11     "x-generative-ai-user-email": YOUR_EMAIL
12 }
13
14 page = 1
15 limit = 10
16 keyword = None
17
18 query_params = f"page={page}&limit={limit}"
19
20 if keyword is not None:
21     query_params += f"&keyword={keyword}"
22
23 api_endpoint_url = f"{ENDPOINT_URL}/openapi/agent-chat/v1/agents?{query_params}"
24
25 response = requests.get(
26     api_endpoint_url,
27     headers=headers
28 )
29
30 # 응답 반환
31 print(response.json())

```

POST /openapi/agent-chat/v1/agent-messages

Header

Field	Type	Required	Description	Value
Content-Type	string	optional	Type of content	application/json
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-openapi-token	string	required	OpenAPI Token information	ex) Bearer eyJXXX
x-generative-ai-user-email	string	optional	Portal user email	ex) kim.samsung@samsung.com

Request Body

Field	Type	Required	Description	Value
agentId	UUID	required	Agent Id	ex) 019806a8-12be-7495-ac12-67255b808aa4
contents	List[string]	required	Contents	ex) ["Hello"]
isStream	Boolean	optional	Stream or not (default: True)	ex) False / True
llmConfig	LLMConfig	optional	LLM Config Info	Properties 참고
isRagOn	Boolean	optional	Enable RAG (default: True)	ex) False / True
executeFinalAnswer	Boolean	optional	Whether the final answer is generated by multi-agents	ex) False / True
executeRagFinalAnswer	Boolean	optional	Whether to execute RAG final answer (default: True)	ex) False / True
executeRagStandaloneQuery	Boolean	optional	Whether to execute RAG enhanced queries (default: True)	ex) False / True
systemPromptVariables	SystemPromptVariables	optional	Agent System prompt variable value	Properties 참고
cognitiveSearchParams	CognitiveSearchParams	optional	RAG Search Parameter value	Type 참고
extraParams	Dict	optional	Custom Field	ex) { gps: "test" }
isExecuteRecommendQueries	Boolean	optional	Respond to a recommended queries	

LLMConfig Properties

Name	Description
temperature	모델의 확률 분포를 조절합니다. 값이 1보다 크면 확률 분포가 더욱 고르게 퍼지게 되고, 값이 1보다 작으면 확률 분포가 더욱 뾰족하게 집중됩니다. ($0 < \text{temperature} < 1$)
repetition_penalty	반복된 단어의 출현 확률을 조절합니다. 값이 1보다 크면 반복된 단어가 나타날 확률이 줄어들게 되고, 값이 1보다 작으면 반복된 단어가 나타날 확률이 증가하게 됩니다.
decoder_input_details	디코더의 입력 형태를 결정합니다. use_cache를 True로 설정하고 past_key_values를 None으로 설정하면, 디코더는 이전 타임스텝의 캐시값을 이용하여 다음 타임스텝의 입력을 생성합니다. use_cache를 False로 설정하고 past_key_values를 None으로 설정하면, 디코더는 이전 타임스텝의 마지막 히든 상태를 이용하여 다음 타임스텝의 입력을 생성합니다. past_key_values를 임의의 값으로 설정하면, 디코더는 해당 값을 이용하여 다음 타임스텝의 입력을 생성합니다.
seed	모델의 난수 발생기를 시드하는 데 사용됩니다. 동일한 시드를 사용하면 모델은 항상 동일한 순서로 출력을 생성합니다.
top_k	모델이 고려할 후보 단어의 수를 제한합니다. 큰 값을 사용하면 더 다양한 단어를 선택할

Name	Description
	수 있지만, 계산 시간이 오래 걸릴 수 있습니다. ($1 \leq \text{top_k}$)
top_p	확률 분포의 상위 p%를 고려하여 단어를 선택합니다. 이렇게 하면 선택된 단어들이 확률 분포를 고르게 대표하도록 하여 다양성을 높일 수 있습니다. (top_p must be > 0.0 and < 1.0)
max_new_tokens	생성된 텍스트의 최대 길이를 결정합니다. 예를 들어, max_new_tokens=10으로 설정하면 모델은 최대 10개의 토큰을 생성할 수 있습니다.

System Prompt Variables Properties

Name	Description
roles	Agent 지시문의 역할에 정의되어 있는 {role} 또는 {{roles}} 를 해당 값으로 치환하여 LLM에 전달합니다.
users	Agent 지시문의 대상 사용자에게 정의되어 있는 {users} 또는 {{users}} 를 해당 값으로 치환하여 LLM에 전달합니다.
cando	Agent 지시문의 할 수 있는 일에 정의되어 있는 {cando} 또는 {{cando}} 를 해당 값으로 치환하여 LLM에 전달합니다.
constraints	Agent 지시문의 제약 사항에 정의되어 있는 {constraints} 또는 {{constraints}} 를 해당 값으로 치환하여 LLM에 전달합니다.

Type <CognitiveSearchParams>

Field	Type	Description
filters	List[Dict]	Rag Search Filter Parameter
additional_information	Dict	Rag Search Additional Filter

Request Body 예시

```

1  {
2    "agentId": 1000,
3    "contents": [
4      "소프트웨어 역사에 대해서 알려줘"
5    ],
6    "isStream": true,
7    "llmConfig": {
8      "max_new_tokens": 2024,
9      "seed": null,
10     "top_k": 14,
11     "top_p": 0.94,
12     "temperature": 0.4,

```

```

13     "repetition_penalty": 1.04
14 },
15 "isRagOn": true,
16 "executeFinalAnswer": true,
17 "executeRagFinalAnswer": true,
18 "executeRagStandaloneQuery": true,
19 "systemPromptVariables": {
20     "roles": " 소프트웨어에 대한 모든 지식 Agent",
21     "users": "소프트웨어를 공부하는 학생",
22     "cando": "소프트웨어에 대한 역사, 기술, 트렌드 지식 제공",
23     "constraints": "소프트웨어에 대해 거짓 정보를 제공하지 않는다."
24 },
25 "cognitiveSearchParams": {
26     "filters": [
27         {
28             "system_id": "default"
29         }
30     ],
31     "additional_information": {
32         "category": "소프트웨어 역사"
33     }
34 },
35 "extraParams": {
36     "test1": "1",
37     "test2": "2"
38 }
39 }

```

Response

주의

※ Stream 으로 응답을 받을 경우 Field 명이 카멜케이스가 아닌 스네이크케이스로 표시됩니다.

예) modelType -> model_type

Field	Type	Description
modelType	string	LLM ModelType

Field	Type	Description
content	string	Answer
truncated	string	Whether the question is truncated or not
finishReason	string	FinishReason (stop : normal, length : Answer length exceeded)
filterBlockReason.ko	string	Filter block reason(ko)
filterBlockReason.en	string	Filter block reason(en)
filterBlockReason.policy_id	string	Filter block reason(policy_id)
filterBlockReason.message	string	Filter block reason(message)
filterBlockReason.result_code	string	Filter block reason(result_code)
filterBlockReason.filter_log_id	string	Filter block reason(filter_log_id)
status	string	Status
responseCode	string	ResponseCode
plugins	list[str]	Requested plugin
eventStatus	string	Event status
eventData	string	EventData
reasoning_content	string	추론 과정
processing_content	list	스트리밍 일 경우 중간 처리 과정
content_references	list	References used while generating the answer
parentMessageCreatedAt	DateTime	parentMessage creation time
recommendQueries	list[str]	Recommended queries

| Response 예시 (isStream = True)

```

1 data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"FabriX","content":"","reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null,"finish_reason":null,"filter_block_reason":null,"status":"SUCCESS","response_code":null,"plugins":[],"orchestrator_type":null,"parent_message_created_at":"2025-07-15T13:49:30.143234+09:00","actions":null,"event_status":"CHUNK","event_data":""}
2 data:
3
4 data:
{"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"FabriX","content":"요청을 처리하기 위한 API 후보를 0개 발견하였습니다.","reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null,"finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"The content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","response_code":null,"plugins":[],"orchestrator_type":null,"p

```

```

parent_message_created_at":"2025-07-15T13:49:30.143234+09:00","actions":null,"event_status":"STA
TUS","event_data":{"phase\":"\planning\","message\":"\요청을 처리하기 위한 API 후보를 0개
발견하였습니다.\"}"
5  data:
6
7  data:
{"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"FabriX","content":"최종 응답을
생성합니다.","reasoning_content":null,"processing_content":[],"content_references":[],"truncate
d":null,"finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","mes
sage":"The content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS"
,"response_code":null,"plugins":[],"orchestrator_type":null,"parent_message_created_at":"2025-07-15T13:49:30.143234+09:00","actions":null,"event_status":"STATUS","event_data":{"phase\":"
\execution\","message\":"\최종 응답을 생성합니다.\"}"
8  data:
9
10 data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"FabriX","content":"##
#","reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null,"f
inish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"The
content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","response
_code":null,"plugins":[],"orchestrator_type":null,"parent_message_created_at":"2025-07-15T13:49
:30.143234+09:00","actions":null,"event_status":"CHUNK","event_data":""}
11 data:
12
13 data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"FabriX","content":"
소","reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null,"
finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"Th
e content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","respon
se_code":null,"plugins":[],"orchestrator_type":null,"parent_message_created_at":"2025-07-15T13:
49:30.143234+09:00","actions":null,"event_status":"CHUNK","event_data":""}
14 data:
15
16 data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"FabriX","content":"프
트","reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null,"
finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"Th
e content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","respon
se_code":null,"plugins":[],"orchestrator_type":null,"parent_message_created_at":"2025-07-15T13:
49:30.143234+09:00","actions":null,"event_status":"CHUNK","event_data":""}
17 data:
18
19 data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"FabriX","content":"웨

```



```

어", "reasoning_content": null, "processing_content": [], "content_references": [], "truncated": null, "
finish_reason": null, "filter_block_reason": {"ko": null, "en": null, "policy_id": "None", "message": "Th
e content was passed", "result_code": "FR-200", "filter_log_id": "None"}, "status": "SUCCESS", "respon
se_code": null, "plugins": [], "orchestrator_type": null, "parent_message_created_at": "2025-07-15T13:
49:30.143234+09:00", "actions": null, "event_status": "CHUNK", "event_data": ""}
20  data:
21
22  data: {"user_id": "9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e", "model_type": "FabriX", "content": "의"
, "reasoning_content": null, "processing_content": [], "content_references": [], "truncated": null, "fin
ish_reason": null, "filter_block_reason": {"ko": null, "en": null, "policy_id": "None", "message": "The
content was passed", "result_code": "FR-200", "filter_log_id": "None"}, "status": "SUCCESS", "response
_code": null, "plugins": [], "orchestrator_type": null, "parent_message_created_at": "2025-07-15T13:49
:30.143234+09:00", "actions": null, "event_status": "CHUNK", "event_data": ""}
23  data:
24
25  data: {"user_id": "9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e", "model_type": "FabriX", "content": "
역사", "reasoning_content": null, "processing_content": [], "content_references": [], "truncated": null
, "finish_reason": null, "filter_block_reason": {"ko": null, "en": null, "policy_id": "None", "message": "
The content was passed", "result_code": "FR-200", "filter_log_id": "None"}, "status": "SUCCESS", "resp
onse_code": null, "plugins": [], "orchestrator_type": null, "parent_message_created_at": "2025-07-15T1
3:49:30.143234+09:00", "actions": null, "event_status": "CHUNK", "event_data": ""}
26  data:
27
28  data: {"user_id": "9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e", "model_type": "FabriX", "content": "\n
\n", "reasoning_content": null, "processing_content": [], "content_references": [], "truncated": null, "
finish_reason": null, "filter_block_reason": {"ko": null, "en": null, "policy_id": "None", "message": "Th
e content was passed", "result_code": "FR-200", "filter_log_id": "None"}, "status": "SUCCESS", "respon
se_code": null, "plugins": [], "orchestrator_type": null, "parent_message_created_at": "2025-07-15T13:
49:30.143234+09:00", "actions": null, "event_status": "CHUNK", "event_data": ""}
29  data:
30
31  data: {"user_id": "9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e", "model_type": "FabriX", "content": "소"
, "reasoning_content": null, "processing_content": [], "content_references": [], "truncated": null, "fin
ish_reason": null, "filter_block_reason": {"ko": null, "en": null, "policy_id": "None", "message": "The
content was passed", "result_code": "FR-200", "filter_log_id": "None"}, "status": "SUCCESS", "response
_code": null, "plugins": [], "orchestrator_type": null, "parent_message_created_at": "2025-07-15T13:49
:30.143234+09:00", "actions": null, "event_status": "CHUNK", "event_data": ""}
32  data:
33
34  data: {"user_id": "9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e", "model_type": "FabriX", "content": "프

```

```

    "reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null,"
    finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"Th
    e content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","respon
    se_code":null,"plugins":[],"orchestrator_type":null,"parent_message_created_at":"2025-07-15T13:
    49:30.143234+09:00","actions":null,"event_status":"CHUNK","event_data":""}
35  data:
36
37  data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"FabriX","content":"워
    |", "reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null,"
    finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"Th
    e content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","respon
    se_code":null,"plugins":[],"orchestrator_type":null,"parent_message_created_at":"2025-07-15T13:
    49:30.143234+09:00","actions":null,"event_status":"CHUNK","event_data":""}
38  data:
39
40  data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"FabriX","content":"의"
    , "reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null,"fin
    ish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"The
    content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","response
    _code":null,"plugins":[],"orchestrator_type":null,"parent_message_created_at":"2025-07-15T13:49
    :30.143234+09:00","actions":null,"event_status":"CHUNK","event_data":""}
41  data:
42
43  data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"FabriX","content":"
    역", "reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null,"
    finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"Th
    e content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","respon
    se_code":null,"plugins":[],"orchestrator_type":null,"parent_message_created_at":"2025-07-15T13:
    49:30.143234+09:00","actions":null,"event_status":"CHUNK","event_data":""}
44  data:
45
46  data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"FabriX","content":"사
    는", "reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null,"
    finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"Th
    e content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","respon
    se_code":null,"plugins":[],"orchestrator_type":null,"parent_message_created_at":"2025-07-15T13:
    49:30.143234+09:00","actions":null,"event_status":"CHUNK","event_data":""}
47  data:
48  ..... 생략

```

Response 예시 (isStream = False)

```

1 {
2   "userId": "9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e",
3   "modelType": "FabriX",
4   "content": "### 소프트웨어의 역사\n\n소프트웨어의 역사는 컴퓨터의 발전과 함께
시작되었습니다. 아래는 주요한 연도별 소프트웨어의 역사입니다.\n\n#### 1940년대\n- **초기
시작** : 최초의 컴퓨터인 ENIAC(전자 수식 계산기)이 개발되었습니다. 이 시기에는 소프트웨어가
하드웨어와 분리되지 않았으며, 컴퓨터 프로그램은 주로 하드웨어를 구성하는 방법으로
개발되었습니다.\n\n#### 1950년대\n- **최초의 프로그래밍 언어** : 플레너리 시스템(Plankalk1)과
포트란(FORTRAN)이 개발되었습니다. 포트란은 최초의 고급 프로그래밍 언어 중 하나로, 과학 계산에
많이 사용되었습니다.\n- **운영 체제의 등장** : 최초의 운영 체제가 개발되었습니다. 초기 운영
체제는 주로 일괄 처리 시스템(batch processing system)으로, 작업을 순차적으로
처리했습니다.\n\n#### 1960년대\n- **고급 프로그래밍 언어의 발전** : COBOL, LISP, C 등 다양한
프로그래밍 언어가 개발되었습니다. COBOL은 비즈니스 애플리케이션에 많이 사용되었으며, LISP는
인공지능 개발에 많이 사용되었습니다.\n- **시분할 시스템** : 시분할 시스템(time-sharing system)이
개발되었습니다. 시분할 시스템은 여러 사용자가 동시에 컴퓨터를 사용할 수 있도록
해주었습니다.\n\n#### 1970년대\n- **개인용 컴퓨터의 등장** : 개인용 컴퓨터가 개발되었습니다.
초기 개인용 컴퓨터로는 애플 I, 애플 II, IBM PC 등이 있습니다.\n- **마이크로소프트의 설립** : 빌
게이츠와 폴 앨런이 마이크로소프트를 설립했습니다. 마이크로소프트는 개인용 컴퓨터용 소프트웨어를
개발하기 시작했습니다.\n\n#### 1980년대\n- **윈도우의 등장** : 마이크로소프트가 윈도우 1.0을
출시했습니다. 윈도우는 그래픽 사용자 인터페이스(GUI)를 제공하여 사용자에게 편리한 환경을
제공했습니다.\n- **인터넷의 확산** : 인터넷이 일반인에게 공개되었습니다. 초기 인터넷은 주로
대학과 연구 기관에서 사용되었습니다.\n\n#### 1990년대\n- **월드 와이드 웹의 등장** : 팀 버너스
리가 월드 와이드 웹(WWW)을 개발했습니다. 월드 와이드 웹은 인터넷을 통해 정보를 공유하고 접근할
수 있는 방법을 제공했습니다.\n- **오픈 소스 소프트웨어의 등장** : 리눅스와 같은 오픈 소스
소프트웨어가 개발되었습니다. 오픈 소스 소프트웨어는 사용자들이 소스 코드를 수정하고 배포할 수
있도록 허용했습니다.\n\n#### 2000년대\n- **클라우드 컴퓨팅의 등장** : 클라우드 컴퓨팅이
개발되었습니다. 클라우드 컴퓨팅은 인터넷을 통해 컴퓨팅 자원을 제공하여 사용자들이 언제 어디서나
컴퓨팅을 할 수 있도록 해주었습니다.\n- **모바일 기기의 등장** : 스마트폰과 태블릿이
개발되었습니다. 모바일 기기는 사용자들이 언제 어디서나 인터넷을 사용할 수 있도록
해주었습니다.\n\n#### 2010년대\n- **인공지능과 머신 러닝의 발전** : 인공지능과 머신 러닝 기술이
발전되었습니다. 인공지능과 머신 러닝은 이미지 인식, 음성 인식, 자연어 처리 등 다양한 분야에서
사용되었습니다.\n- **빅 데이터와 데이터 과학의 등장** : 빅 데이터와 데이터 과학이
개발되었습니다. 빅 데이터와 데이터 과학은 대량의 데이터를 분석하여 통찰력을 얻고 의사 결정을
지원했습니다.\n\n#### 2020년대\n- **클라우드 네이티브 애플리케이션의 등장** : 클라우드 네이티브
애플리케이션이 개발되었습니다. 클라우드 네이티브 애플리케이션은 클라우드 환경에서 개발되고
배포되는 애플리케이션입니다.\n- **서버리스 컴퓨팅의 등장** : 서버리스 컴퓨팅이 개발되었습니다.
서버리스 컴퓨팅은 사용자들이 코드를 실행할 수 있도록 해주며, 사용한 만큼만 비용을
지불합니다.\n\n소프트웨어의 역사는 계속 발전하고 있으며, 새로운 기술과 트렌드가 등장하고

```

있습니다. 앞으로도 소프트웨어는 우리의 생활과 업무에 큰 영향을 미칠 것입니다.",

```

5   "reasoningContent": null,
6   "processingContent": [],
7   "contentReferences": [
8     {
9       "plugin": "RAG",
10      "answer": "",
11      "references": [
12        {
13          "title": "소프트웨어의 역사",
14          "content": "빌 게이츠와 폴 앨런이 마이크로소프트를 설립했습니다. 마이크로소프트는
개인용 컴퓨터용 소프트웨어를 개발하기 시작했습니다.\n\n#### 1980년대\n- **윈도우의 등장**:
마이크로소프트가 윈도우 1.0을 출시했습니다. 윈도우는 그래픽 사용자 인터페이스(GUI)를 제공하여
사용자에게 편리한 환경을 제공했습니다.\n- **인터넷의 확산**: 인터넷이 일반인에게
공개되었습니다. 초기 인터넷은 주로 대학과 연구 기관에서 사용되었습니다.\n\n#### 1990년대\n-
**월드 와이드 웹의 등장**: 팀 버너스 리가 월드 와이드 웹(WWW)을 개발했습니다. 월드 와이드 웹은
인터넷을 통해 정보를 공유하고 접근할 수 있는 방법을 제공했습니다.\n- **오픈 소스 소프트웨어의
등장**: 리눅스와 같은 오픈 소스 소프트웨어가 개발되었습니다. 오픈 소스 소프트웨어는 사용자들이
소스 코드를 수정하고 배포할 수 있도록 허용했습니다",
15          "link": ""
16        }
17      ],
18      "augmented_standalone_queries": "소프트웨어 역사에 대해 알려주세요."
19    }
20  ],
21  "truncated": false,
22  "finishReason": null,
23  "filterBlockReason": {
24    "ko": null,
25    "en": null,
26    "policy_id": null,
27    "message": null,
28    "result_code": "FR-200",
29    "filter_log_id": null
30  },
31  "status": "SUCCESS",
32  "responseCode": "R20000",
33  "plugins": [
34    "LLM"
35  ],

```

```

36  "orchestratorType": null,
37  "parentMessageCreatedAt": "2025-07-15T10:16:32.511334+09:00",
38  "actions": [],
39  "eventStatus": "CHUNK",
40  "eventData": ""
41  }

```

| 호출 예시 (python)

(isStream=false)

```

1  import requests
2
3  YOUR_CLIENT_KEY = ""
4  YOUR_PASS_KEY = ""
5  ENDPOINT_URL = ""
6  AGENT_ID = ""
7  YOUR_EMAIL = ""
8
9  headers: dict = {
10     'x-fabrix-client': YOUR_CLIENT_KEY,
11     'x-openapi-token': YOUR_PASS_KEY,
12     'x-generative-ai-user-email': YOUR_EMAIL
13  }
14
15  body: dict = {
16     "agentId": AGENT_ID,
17     "contents": ["소프트웨어 역사에 대해 알려줘"],
18     "isStream": False,
19     "isRagOn": True,
20     "executeFinalAnswer": True,
21     "executeRagFinalAnswer": True,
22     "executeRagStandaloneQuery": True
23  }
24
25  api_endpoint_url = f"{ENDPOINT_URL}/openapi/agent-chat/v1/agent-messages"
26
27  response: requests.Response = requests.post(
28     api_endpoint_url,
29     json=body,

```

```

30     headers=headers
31 )
32
33 # 응답 반환
34 print(response.json())

```

(isStream=true)

팁

Stream = true 인 경우에 최종 응답 확인은 status 값을 이용하여 확인합니다.

- 정상 답변 : status = 'SUCCESS' and response_code = 'R20000'
- 비정상 답변 : status='ERROR' or 'FILTER_INVALID'

```

1  import requests
2  import sseclient
3  import json
4
5  YOUR_CLIENT_KEY = ""
6  YOUR_PASS_KEY = ""
7  ENDPOINT_URL = ""
8  AGENT_ID = ""
9  YOUR_EMAIL = ""
10
11 headers: dict = {
12     'x-fabrix-client': YOUR_CLIENT_KEY,
13     'x-openapi-token': YOUR_PASS_KEY,
14     'x-generative-ai-user-email': YOUR_EMAIL
15 }
16
17 body: dict = {
18     "agentId": AGENT_ID,
19     "contents": ["소프트웨어 역사에 대해 알려줘"],
20     "isStream": False,
21     "isRagOn": True,
22     "executeFinalAnswer": True,

```

```

23     "executeRagFinalAnswer": True,
24     "executeRagStandaloneQuery": True
25 }
26
27 api_endpoint_url = f"{ENDPOINT_URL}/openapi/agent-chat/v1/agent-messages"
28
29 response: requests.Response = requests.post(
30     api_endpoint_url,
31     json=body,
32     headers=headers
33 )
34
35 # return response
36 if response.status_code != 200:
37     print(f"Error: Status_code=[{response.status_code}], Status message=[{response.text}]")
38 client = sseclient.SSEClient(response)
39
40 # make result contents
41 def get_result():
42     for event in client.events():
43         # print(f"Event: {event.event}")
44         print(f>Data: {event.data}")
45         yield event.data
46
47 result_message = ""
48 for ch in get_result():
49     ch_json = json.loads(ch)
50     if ch_json['status'] == 'SUCCESS' and ch_json['response_code'] == 'R20000':
51         print("Stream Finished!!")
52     if ch_json['event_status'] == 'CHUNK' and ch_json['content']:
53         result_message += ch_json['content']
54
55 print(result_message)

```

POST /openapi/agent-chat/v1/agent-messages/file

Header

Field	Type	Required	Description	Value
Content-Type	string	required	Type of content	multipart/form-data
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-generative-ai-user-email	string	optional	Portal user email	ex) kim.samsung@samsu ng.com
x-openapi-token	string	required	OpenAPI Token information	ex) Bearer eyJXXX

Request Body

Field	Type	Required	Description	Value
agentId	UUID	required	Agent Id	ex) 019806a8-12be-7495-ac12-67255b808aa4
contents	List[string]	required	Contents	ex) ["Hello"]
file	File	required	File to upload.	Valid file object
isStream	Boolean	optional	Stream or not (default: True)	ex) False / True
llmConfig	LLMConfig	optional	LLM Config Info	Properties 참고
isRagOn	Boolean	optional	Enable RAG (default: True)	ex) False / True
executeFinalAnswer	Boolean	optional	Whether the final answer is generated by multi-agents	ex) False / True
executeRagFinalAnswer	Boolean	optional	Whether to execute RAG final answer (default: True)	ex) False / True
executeRagStandaloneQuery	Boolean	optional	Whether to execute RAG enhanced queries (default: True)	ex) False / True
systemPromptVariables	SystemPromptVariables	optional	Agent System prompt variable value	Properties 참고
cognitiveSearchParams	CognitiveSearchParams	optional	RAG Search Parameter value	Type 참고
extraParams	Dict	optional	Custom Field	ex) { gps: "test" }

LLMConfig Properties

Name	Description
temperature	모델의 확률 분포를 조절합니다. 값이 1보다 크면 확률 분포가 더욱 고르게 퍼지게 되고, 값이 1보다 작으면 확률 분포가 더욱 뾰족하게 집중됩니다. (0 < temperature < 1)

Name	Description
repetition_penalty	반복된 단어의 출현 확률을 조절합니다. 값이 1보다 크면 반복된 단어가 나타날 확률이 줄어들게 되고, 값이 1보다 작으면 반복된 단어가 나타날 확률이 증가하게 됩니다.
decoder_input_details	디코더의 입력 형태를 결정합니다. use_cache를 True로 설정하고 past_key_values를 None으로 설정하면, 디코더는 이전 타임스텝의 캐시값을 이용하여 다음 타임스텝의 입력을 생성합니다. use_cache를 False로 설정하고 past_key_values를 None으로 설정하면, 디코더는 이전 타임스텝의 마지막 히든 상태를 이용하여 다음 타임스텝의 입력을 생성합니다. past_key_values를 임의의 값으로 설정하면, 디코더는 해당 값을 이용하여 다음 타임스텝의 입력을 생성합니다.
seed	모델의 난수 발생기를 시드하는 데 사용됩니다. 동일한 시드를 사용하면 모델은 항상 동일한 순서로 출력을 생성합니다.
top_k	모델이 고려할 후보 단어의 수를 제한합니다. 큰 값을 사용하면 더 다양한 단어를 선택할 수 있지만, 계산 시간이 오래 걸릴 수 있습니다. ($1 \leq \text{top_k}$)
top_p	확률 분포의 상위 p%를 고려하여 단어를 선택합니다. 이렇게 하면 선택된 단어들이 확률 분포를 고르게 대표하도록 하여 다양성을 높일 수 있습니다. (top_p must be > 0.0 and < 1.0)
max_new_tokens	생성된 텍스트의 최대 길이를 결정합니다. 예를 들어, max_new_tokens=10으로 설정하면 모델은 최대 10개의 토큰을 생성할 수 있습니다.

System Prompt Variables Properties

Name	Description
roles	Agent 지시문의 역할에 정의되어 있는 {role} 또는 {{roles}} 를 해당 값으로 치환하여 LLM에 전달합니다.
users	Agent 지시문의 대상 사용자에게 정의되어 있는 {users} 또는 {{users}} 를 해당 값으로 치환하여 LLM에 전달합니다.
cando	Agent 지시문의 할 수 있는 일에 정의되어 있는 {cando} 또는 {{cando}} 를 해당 값으로 치환하여 LLM에 전달합니다.
constraints	Agent 지시문의 제약 사항에 정의되어 있는 {constraints} 또는 {{constraints}} 를 해당 값으로 치환하여 LLM에 전달합니다.

Type <CognitiveSearchParams>

Field	Type	Description
filters	List[Dict]	Rag Search Filter Parameter
additional_information	Dict	Rag Search Additional Filter

Request Body 예시

```

1  -----Boundary
2  Content-Disposition: form-data; name="agentId"
3
4  4084
5  -----Boundary
6  Content-Disposition: form-data; name="contents"
7
8  내가 올린 파일을 코드로 만들어줘
9  -----Boundary
10 Content-Disposition: form-data; name="isStream"
11
12 False
13 -----Boundary
14 Content-Disposition: form-data; name="file"; filename="테스트.txt"
15 Content-Type: text/plain

```

Response

주의

※ Stream 으로 응답을 받을경우 Field 명이 카멜케이스가 아닌 스네이크케이스로 표시됩니다.

예) modelType -> model_type

Field	Type	Description
modelType	string	LLM ModelType
content	string	Answer
truncated	string	Whether the question is truncated or not
finishReason	string	FinishReason (stop : normal, length : Answer length exceeded)
filterBlockReason.ko	string	Filter block reason(ko)
filterBlockReason.en	string	Filter block reason(en)
filterBlockReason.policy_id	string	Filter block reason(policy_id)
filterBlockReason.message	string	Filter block reason(message)
filterBlockReason.result_code	string	Filter block reason(result_code)
filterBlockReason.filter_log_id	string	Filter block reason(filter_log_id)
status	string	Status

Field	Type	Description
responseCode	string	ResponseCode
plugins	list[str]	Requested plugin
eventStatus	string	Event status
eventData	string	EventData
reasoning_content	string	추론 과정
processing_content	list	스트리밍 일경우 중간 처리 과정
content_references	list	References used while generating the answer
parentMessageCreatedAt	DateTime	parentMessage creation time

| Response 예시 (isStream = True)

```

1  data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"Llama 3.3","content":"
역","reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null,"
finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"Th
e content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","respon
se_code":null,"plugins":[],"orchestrator_type":null,"parent_message_created_at":"2025-07-15T13:
49:30.143234+09:00","actions":null,"event_status":"CHUNK","event_data":""}
2  data:
3
4  data: {"user_id":"9c9f13a7-b325-4a70-b5c4-b4d8f41dea8e","model_type":"Llama 3.3","content":"
사는","reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null
,"finish_reason":null,"filter_block_reason":{"ko":null,"en":null,"policy_id":"None","message":"
The content was passed","result_code":"FR-200","filter_log_id":"None"},"status":"SUCCESS","resp
onse_code":null,"plugins":[],"orchestrator_type":null,"parent_message_created_at":"2025-07-15T1
3:49:30.143234+09:00","actions":null,"event_status":"CHUNK","event_data":""}
5  data:
6  ..... (생략)

```

| Response 예시 (isStream = False)

```

1  {
2    "userId": "5beb92d5-2bde-4688-b660-7b9d59616225",
3    "modelType": "Gauss2",
4    "content": "### 파일 분석 결과\n\n파일을 분석한 결과, '영희'라는 키워드가 **1번**
등장하는 것으로 확인되었습니다. \n이 결과는 파일을 읽고 해당 키워드의 발생 횟수를 세는 파이썬
코드를 실행하여 도출되었습니다. \n\n### 코드 실행 요약\n- 파일에서 '영희'라는 키워드를

```

검색했습니다. \n- 키워드가 총 **1회** 발견되었습니다. \n\n이 분석은 파일의 내용을 정확히 반영하며, 추가적인 정보나 설명이 필요하지 않습니다.",

```

5   "reasoningContent": null,
6   "processingContent": [],
7   "contentReferences": [
8     {
9       "plugin": "e29b49a0-c158-4e04-9dba-4157c30b1877",
10      "answer": "```\nimport pandas as pd\n\n# Set the pandas
option\nnpd.set_option('expand_frame_repr', False)\n\n# File path and encoding\nfile_path =
'/userdata/sample_file.txt'\nencoding = 'utf-8'\n\n# Read the file and count occurrences of
'영희'\nwith open(file_path, 'r', encoding=encoding) as file:\n    content = file.read()\n
count = content.count('영희')\n\nprint(f"The keyword '영희' appears {count} times in the
file.")\n```\n\nThe keyword '영희' appears 1 times in the file.\n\n```,
11    "references": null,
12    "augmented_standalone_queries": null
13  }
14 ],
15 "truncated": false,
16 "finishReason": null,
17 "filterBlockReason": {
18   "ko": null,
19   "en": null,
20   "policyId": null,
21   "message": null,
22   "resultCode": null,
23   "filterLogId": null
24 },
25 "status": "SUCCESS",
26 "responseCode": "R20000",
27 "plugins": [
28   "Code Interpreter"
29 ],
30 "orchestratorType": "built-in",
31 "parentMessageCreatedAt": "2025-10-27T15:07:08.852303+09:00",
32 "references": [],
33 "actions": [
34   {
35     "type": "API",
36     "plugin": "Code Interpreter",
37     "api": "generate_execute_code",

```

```

38     "params": {
39         "storage_service": "http://fabrix-storage.fabrix-dev150-service:8000",
40         "public_url": "https://dev150.genaiqa.samsungsds.com",
41         "code_type": "python",
42         "code": "\nimport pandas as pd\n\n# Set the pandas
option\nnpd.set_option('expand_frame_repr', False)\n\n# File path and encoding\nfile_path =
'/userdata/sample_file.txt'\nencoding = 'utf-8'\n\n# Read the file and count occurrences of
'영희'\n\nwith open(file_path, 'r', encoding=encoding) as file:\n    content = file.read()\n
count = content.count('영희')\n\nprint(f\"The keyword '영희' appears {count} times in the
file.\")",
43     "files": [
44         {
45             "file_name": "sample_file.txt",
46             "file_uuid": "7bab00b0-a0c0-4d03-9cb2-59a00019cd15.txt",
47             "mime_type": "",
48             "file_base64": [
49                 null
50             ],
51             "uploaded_by": "user",
52             "file_path": "/userdata/sample_file.txt",
53             "file_meta_data": "{\"encoding\": \"utf-8\", \"file_first_5_lines\": \"영희
철수 민지\"}\n"
54         }
55     ],
56 },
57 "responseType": 3,
58 "method": "POST",
59 "httpStatus": 200,
60 "response": {
61     "code": "\nimport pandas as pd\n\n# Set the pandas
option\nnpd.set_option('expand_frame_repr', False)\n\n# File path and encoding\nfile_path =
'/userdata/sample_file.txt'\nencoding = 'utf-8'\n\n# Read the file and count occurrences of
'영희'\n\nwith open(file_path, 'r', encoding=encoding) as file:\n    content = file.read()\n
count = content.count('영희')\n\nprint(f\"The keyword '영희' appears {count} times in the
file.\")",
62     "execution": {
63         "execute_result_text": "The keyword '영희' appears 1 times in the file.\n",
64         "display_data": [],
65         "elapsed_time": 0.4,
66         "is_truncated": false,

```

```

67         "usermachine_expected_shutdown_time": 1761545834
68     },
69     "files": null,
70     "status": "SUCCESS",
71     "error_message": null,
72     "disable_file_cnt": 0,
73     "chart_library": "plotly"
74 },
75 "properties": {
76     "x-orch-api-call": true,
77     "x-orch-answer-generation": true,
78     "x-orch-answer-path": "",
79     "x-orch-api-streaming": false,
80     "x-orch-prebuilt-plugin-name": "CODE_INTERPRETER",
81     "display_info": {
82         "display-llm-answer": true,
83         "display-type": "html",
84         "display-request": false,
85         "input": "file",
86         "file_attr": {
87             "accept": [
88                 ".csv",
89                 ".txt",
90                 ".xls",
91                 ".xlsx"
92             ],
93             "size": 10,
94             "count": 5
95         }
96     }
97 },
98     "answer": "```\nimport pandas as pd\n\n# Set the pandas
option\nnpd.set_option('expand_frame_repr', False)\n\n# File path and encoding\nfile_path =
'/userdata/sample_file.txt'\nencoding = 'utf-8'\n\n# Read the file and count occurrences of
'영희'\nwith open(file_path, 'r', encoding=encoding) as file:\n    content = file.read()\n
count = content.count('영희')\n\nprint(f"The keyword '영희' appears {count} times in the
file.\\")\n```\n\nThe keyword '영희' appears 1 times in the file.\n\n```,
99     "question": null
100 }
101 ],

```

```

102     "eventStatus": "CHUNK",
103     "eventData": ""
104 }

```

| 호출 예시 (python)

(isStream=false)

```

1  import requests
2
3  YOUR_CLIENT_KEY = ""
4  YOUR_PASS_KEY = ""
5  ENDPOINT_URL = ""
6  AGENT_ID = ""
7  YOUR_EMAIL = ""
8
9  headers = {
10     "x-fabrix-client": YOUR_CLIENT_KEY,
11     "x-openapi-token": YOUR_PASS_KEY,
12     "x-generative-ai-user-email": YOUR_EMAIL
13 }
14
15 files = {
16     "file": ("sample.txt", open("sample.txt", "rb"), "text/plain")
17 }
18
19 data = {
20     "agentId": AGENT_ID,
21     "contents": ["이 파일에 '영희' 라는 단어가 몇번 등장하는지 Python 코드를 작성하여
실행한 후 결과를 알려주세요."],
22     "isStream": False,
23 }
24
25 api_endpoint_url = f"{ENDPOINT_URL}/openapi/agent-chat/v1/agent-messages/file"
26
27 response: requests.Response = requests.post(
28     api_endpoint_url,
29     headers=headers,
30     data=data,
31     files=files

```

```

32 )
33
34 # 응답 반환
35 print(response.json())

```

(isStream=true)

팁

Stream = true 인 경우에 최종 응답 확인은 status 값을 이용하여 확인합니다.

- 정상 답변 : status = 'SUCCESS' and response_code = 'R20000'
- 비정상 답변 : status='ERROR' or 'FILTER_INVALID'

```

1  import requests
2  import sseclient
3  import json
4
5  YOUR_CLIENT_KEY = ""
6  YOUR_PASS_KEY = ""
7  ENDPOINT_URL = ""
8  AGENT_ID = ""
9  YOUR_EMAIL = ""
10
11 headers = {
12     "x-fabrix-client": YOUR_CLIENT_KEY,
13     "x-openapi-token": YOUR_PASS_KEY,
14     "x-generative-ai-user-email": YOUR_EMAIL
15 }
16
17 files = {
18     "file": ("sample.txt", open("sample.txt", "rb"), "text/plain")
19 }
20
21 data = {
22     "agentId": AGENT_ID,
23     "contents": ["이 파일에 '영희' 라는 단어가 몇번 등장하는지 알려주세요."],

```



```

24     "isStream": True,
25 }
26
27 api_endpoint_url = f"{ENDPOINT_URL}/openapi/agent-chat/v1/agent-messages/file"
28
29 response: requests.Response = requests.post(
30     api_endpoint_url,
31     headers=headers,
32     data=data,
33     files=files
34 )
35
36 # return response
37 if response.status_code != 200:
38     print(f"Error: Status_code=[{response.status_code}], Status message=[{response.text}]")
39 client = sseclient.SSEClient(response)
40
41 # make result contents
42 def get_result():
43     for event in client.events():
44         # print(f"Event: {event.event}")
45         print(f>Data: {event.data}")
46         yield event.data
47
48 result_message = ""
49 for ch in get_result():
50     ch_json = json.loads(ch)
51     if ch_json['status'] == 'SUCCESS' and ch_json['response_code'] == 'R20000':
52         print("Stream Finished!!")
53     if ch_json['event_status'] == 'CHUNK' and ch_json['content']:
54         result_message += ch_json['content']
55
56 print(result_message)

```

공통 Response Code

노트

Response 가 Success (Http Status Code 가 200) 인 경우 Response code 를 통해 상세 내용 확인이 가능합니다.

Response 가 False (Http Status 가 200이 아닌) 인 경우 Error Code 를 통해 상세 내용 확인이 가능합니다.

Success

Http status code	Response code	Reason (response message)
200	R20000	success
200	R20001	(DEPRECATED) need more parameters for api call
200	R20002	(DEPRECATED) only support single-step task
200	R20003	(DEPRECATED) error occurred during api call
200	R20004	(DEPRECATED) api request timeout
200	R20005	(DEPRECATED) exceeded token limit for generating response
200	R40000	client error # generic type for client error
200	R40001	request parameter validation error
200	R40002	unsupported model error
200	R40003	invalid plugin or api specification
200	R40004	exceeded token limit for request completion
200	R40005	health check api error
200	R40006	plug-in id not found
200	R40007	search engine not used
200	R40008	blocked by security filter
200	R40009	request timeout
200	R40010	too many LLM request
200	R40011	LLM request timeout
200	R40012	Exceeding the token usage limit on the client.
200	R40013	Exceeding the filter check content limit on the client.
200	R50000	internal server error # generic type for server error
200	R50001	api specification parsing error
200	R50002	security filter service error
200	R50003	cognitive search service error
200	R50004	LLM request error
200	R50005	LLM response parsing error

Http status code	Response code	Reason (response message)
200	R50030	tool hallucination
200	R50050	Plug-in server error
200	R50051	Plug-in API connect error
200	R50052	Code Interpreter call error.
200	R50053	File summary/translate call error.
200	R50060	MCP server error
200	R50061	MCP tool connect error
200	R50070	Agent error
200	R50071	Agent(standard) connect error
200	R50072	Agent(Langflow) connect error
200	R50073	Agent(custom) connect error
200	R50074	Agent(A2A) connect error
200	R50075	Agent Builder connect error
200	R50080	Selection flow error
200	R50081	Workflow Execution error

Fail

Http status code	Error Code	Reason (response message)
413	FILTER_ERROR_2	The content length has been exceeded
500	COMMON_ERROR_1	Exception occurred
404	COMMON_ERROR_2	The table does not exist
500	COMMON_ERROR_3	Updating the table was failed
404	COMMON_ERROR_4	It does not exist
400	COMMON_ERROR_5	Request parameter or body is invalid
422	COMMON_ERROR_6	Unprocessable Entity is requeste
403	COMMON_ERROR_7	Access is denied
423	COMMON_ERROR_8	Resource is locked while being processed. Please try again later

2.3 FAQ

FabriX Portal에서는 확인이 되지만 Agent목록 조회 API 응답에는 포함되어있지 않은 항목이 있습니다.

해당 API는 현재 사용 가능한 상태의 Agent만 조회가 가능합니다. 사용 승인까지 득한 이후에 응답에 포함되며, 실시간으로 상태가 변경이 가능하므로 해당 Agent의 권한 및 상태를 확인해 보시기 바랍니다.

API를 사용할 때, 이전 대화를 고려한 응답을 받으려면 어떻게 해야 하나요?

agent-messages API의 “contents” 매개변수를 활용하여 대화를 구성할 수 있습니다. 예를 들어, Request Body로 “contents”: [“사용자의 첫번째 질의”, “모델의 첫번째 답변”, “사용자의 두번째 질의”, “모델의 두번째 답변”, “사용자의 마지막 질의”] 를 전달하면 contents가 대화이력 역할을 하여 이전 대화를 활용할 수 있습니다.

agent-messages API를 통해 LLM Config 파라미터 설정 시, Agent 내에서 설정한 파라미터는 무시되는 건가요?

Agent LLM Config 설정을 기본적으로 따르고, Request를 통해 전달된 LLM Config와 Agent LLM Config에 설정한 파라미터 키 값이 일치하는 경우 Request를 통해 전달된 LLM Config로 Overwrite하여 적용됩니다.

3.

Security Filter APIs

3.1 Security Filter APIs

Security Filter APIs는 포털 서비스에 적용된 필터링 정책을 기준으로 질의의 필터링 여부를 체크하는 API를 제공합니다.

Security Filter APIs 에서 제공하는 API는 다음과 같습니다.

POST /openapi/filter/v1/check

check API는 질의에 대한 필터 여부를 응답합니다.

3.2 API 명세

POST /openapi/filter/v1/check

Header

Field	Type	Required	Description	Value
Content-Type	string	optional	Type of content	application/json
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-openapi-token	string	required	OpenAPI Token information	ex) Bearer eyJXXX
x-generative-ai-user-email	string	optional	Portal user email	ex) kim.samsung@samsung.com

Request Body

Field	Type	Required	Description	Value
content	string	required	사용자 질의	ex) “안녕하세요.”
user_ip	string	required	필터링 이력에서 확인되는 사용자 IP	ex) “123.456.789.111”
target_model	string	required	정책 적용 모델 (“INTERNAL”, “EXTERNAL”)	ex) “INTERNAL”
target_service	string	optional	정책그룹 타겟 서비스 (default: GENAI)	ex) “GENAI”
model_name	string	optional	필터링 이력에서 확인되는 모델명	ex) “FabriX”

Request Body 예시

```

1 {
2   "content": "안녕하세요.",
3   "user_ip": "123.123.123.123",
4   "target_model": "INTERNAL",
5   "target_service": "GENAI",
6   "model_name": "FabriX"
7 }
```

Response

Field	Type	Description
statusCode	integer	Status Code
data	object	data Properties 참고

data Properties

Name	Type	Description
is_blocked	boolean	content 필터 여부
result_code	string	result_code Message 참고
result_details.message	string	필터 결과 메시지
result_details.filter_type	string	필터 타입 (“SimpleKeywordFilter”, “ComplexKeywordFilter”, “RegularExpressionFilter”, “MultipleKeywordFilter”, “AIModelFilterChecker”, “ResidentRegistrationNumberFilter”, “DriverLicenseNumberFilter”)
match_policy.	integer	정책 그룹 ID

Name	Type	Description
policy_group_id		
match_policy. policy_id	integer	정책 ID
match_policy. policy_item_id	integer	적용된 정책과 대응되는 Filter ID
match_policy. policy_type	string	정책 타입 ("RULE", "AI")
match_policy. priority	integer	정책 우선 순위
match_policy. target_model	string	정책 적용 모델 ("ALL", "INTERNAL", "EXTERNAL")
match_policy. pol icy_group_name	string	정책 그룹명
match_policy. pol icy_group_priorit y	integer	정책 그룹 우선순위
match_policy. action_type	string	정책 액션 ("ALLOW", "WARN", "DROP")
match_policy. valid_start_date	DateTime	정책 유효기간 시작일
match_policy. valid_end_date	DateTime	정책 유효기간 종료일
match_policy. is_log	boolean	정책 로그 여부
match_policy. creator	integer	정책 생성자
match_policy. created_at	DateTime	정책 생성 일시
match_policy. modifier	integer	정책 수정자
match_policy. modified_at	DateTime	정책 수정 일시
match_policy. tenant_id	integer	정책 테넌트 ID
filter_log_id	interger	필터링 이력 ID
message	string	필터 메시지
reason_kor	string	필터 Reason (kor)
reason_eng	string	필터 Reason (eng)
result_message. message	string	필터 메시지
result_message.	string	필터 Reason (kor)

Name	Type	Description
message_kor		
result_message. message_eng	string	필터 Reason (eng)

| result_code Message

Result Code	Message
FR-100	There is nothing yet
FR-200	The content was passed
FR-201	The content was allowed by the filter
FR-202	The content was allowed by the filter, but warned
FR-203	There are no filter items
FR-300	Filter system is not ready yet
FR-400	The content was blocked by the filter
FR-500	An internal error occurred during processing
FR-501	An error occurred due to an external system

| Error Code

Http status code	Error Code	Reason (response message)
404	FILTER_ERROR_1	The Category does not exist
413	FILTER_ERROR_2	The content length has been exceeded
500	COMMON_ERROR_1	Exception occurred
404	COMMON_ERROR_2	The table does not exist
500	COMMON_ERROR_3	Updating the table was failed
404	COMMON_ERROR_4	It does not exist
400	COMMON_ERROR_5	Request parameter or body is invalid
422	COMMON_ERROR_6	Unprocessable Entity is requeste
403	COMMON_ERROR_7	Access is denied
423	COMMON_ERROR_8	Resource is locked while being processed. Please try again later

| Response 예시 (result_code: FR-200, is_blocked: false)

```

1  {
2    "statusCode": 200,
3    "data": {
4      "is_blocked": false,
```



```

5      "result_code": "FR-200",
6      "result_details": {},
7      "policy_id": null,
8      "match_policy": null,
9      "filter_log_id": null,
10     "message": "The content was passed",
11     "reason_kor": null,
12     "reason_eng": null,
13     "result_message": null
14 }
15 }

```

| Response 예시 (result_code: FR-400, action_type: DROP, is_blocked: true)

```

1 {
2   "statusCode": 200,
3   "data": {
4     "is_blocked": true,
5     "result_code": "FR-400",
6     "result_details": {
7       "message": "Matched by SimpleKeywordFilter",
8       "filter_type": "SimpleKeywordFilter",
9       "match_count": 1
10    },
11    "policy_id": 120,
12    "match_policy": {
13      "policy_group_id": 160,
14      "policy_id": 120,
15      "policy_item_id": 167,
16      "policy_type": "RULE",
17      "priority": 1,
18      "target_model": "ALL",
19      "policy_group_name": "정책생성테스트1",
20      "policy_group_priority": 1,
21      "action_type": "DROP",
22      "valid_start_date": "2024-05-02T00:00:00+09:00",
23      "valid_end_date": "2100-12-31T23:59:59.999000+09:00",
24      "is_log": true,
25      "creator": 61,

```

```

26         "created_at": "2024-05-06T08:14:31.179614+09:00",
27         "modifier": 61,
28         "modified_at": "2024-05-07T10:48:52.943350+09:00",
29         "tenant_id": 1
30     },
31     "filter_log_id": 66,
32     "message": "The content was blocked by the filter",
33     "reason_kor": "test",
34     "reason_eng": "test",
35     "result_message": {
36         "message": "test",
37         "message_kor": "test",
38         "message_eng": "test"
39     }
40 }
41 }

```

| Response 예시 (result_code: FR-201, action_type: ALLOW, is_blocked: false)

```

1  {
2      "statusCode": 200,
3      "data": {
4          "is_blocked": false,
5          "result_code": "FR-201",
6          "result_details": {
7              "message": "Matched by SimpleKeywordFilter",
8              "filter_type": "SimpleKeywordFilter",
9              "match_count": 1
10         },
11         "policy_id": 120,
12         "match_policy": {
13             "policy_group_id": 160,
14             "policy_id": 120,
15             "policy_item_id": 167,
16             "policy_type": "RULE",
17             "priority": 1,
18             "target_model": "ALL",
19             "policy_group_name": "정책생성테스트1",
20             "policy_group_priority": 1,

```

```

21         "action_type": "ALLOW",
22         "valid_start_date": "2024-05-02T00:00:00+09:00",
23         "valid_end_date": "2100-12-31T23:59:59.999000+09:00",
24         "is_log": true,
25         "creator": 61,
26         "created_at": "2024-05-06T08:14:31.179614+09:00",
27         "modifier": 61,
28         "modified_at": "2024-05-07T10:48:52.943350+09:00",
29         "tenant_id": 1
30     },
31     "filter_log_id": 66,
32     "message": "The content was allowed by the filter",
33     "reason_kor": "test",
34     "reason_eng": "test",
35     "result_message": {
36         "message": "test",
37         "message_kor": "test",
38         "message_eng": "test"
39     }
40 }
41 }

```

| Response 예시 (result_code: FR-202, action_type: WARN, is_blocked: false)

```

1  {
2      "statusCode": 200,
3      "data": {
4          "is_blocked": false,
5          "result_code": "FR-202",
6          "result_details": {
7              "message": "Matched by SimpleKeywordFilter",
8              "filter_type": "SimpleKeywordFilter",
9              "match_count": 1
10         },
11         "policy_id": 120,
12         "match_policy": {
13             "policy_group_id": 160,
14             "policy_id": 120,
15             "policy_item_id": 167,

```

```

16     "policy_type": "RULE",
17     "priority": 1,
18     "target_model": "ALL",
19     "policy_group_name": "정책생성테스트1",
20     "policy_group_priority": 1,
21     "action_type": "WARN",
22     "valid_start_date": "2024-05-02T00:00:00+09:00",
23     "valid_end_date": "2100-12-31T23:59:59.999000+09:00",
24     "is_log": true,
25     "creator": 61,
26     "created_at": "2024-05-06T08:14:31.179614+09:00",
27     "modifier": 61,
28     "modified_at": "2024-05-07T10:48:52.943350+09:00",
29     "tenant_id": 1
30 },
31 "filter_log_id": 66,
32 "message": "The content was allowed by the filter, but warned",
33 "reason_kor": "test",
34 "reason_eng": "test",
35 "result_message": {
36     "message": "test",
37     "message_kor": "test",
38     "message_eng": "test"
39 }
40 }
41 }

```

| Response 예시 (Error)

```

1 {
2     "statusCode": 422,
3     "error": "Unprocessable Entity",
4     "message": "user_ip, field required",
5     "detailMessage": "user_ip, field required",
6     "errorCode": "COMMON_ERROR_1"
7 }

```

호출 예시 (python)

```
1 import requests
2
3 YOUR_CLIENT_KEY = ""
4 YOUR_PASS_KEY = ""
5 YOUR_EMAIL = ""
6 ENDPOINT_URL = ""
7
8 headers = {
9     "x-fabrix-client": YOUR_CLIENT_KEY,
10    "x-openapi-token": YOUR_PASS_KEY,
11    "x-generative-ai-user-email": YOUR_EMAIL
12 }
13
14 body = {
15     "content": "안녕하세요",
16     "user_ip": "123.123.123.123",
17     "target_model": "INTERNAL",
18     "target_service": "GENAI",
19     "model_name ": "FabriX"
20 }
21 api_endpoint_url = f"{ENDPOINT_URL}/openapi/filter/v1/check"
22
23 response = requests.post(
24     api_endpoint_url,
25     headers=headers,
26     json=body
27 )
28
29 # 응답 반환
30 print(response.json())
```

주의

※ Endpoint URL은 Open API 신청시 발급되는 URL로 적용하여 사용할 수 있습니다.

3.3 FAQ

FabriX Security Filter를 위한 Rule은 어떻게 확인하나요?

FabriX Security Filter의 정책 그룹, 정책, 룰, AI필터모델은 필터 관리자에 의해 관리됩니다. 자세한 룰은 보안 상의 이유로 공개가 어려우며, 추가 정보가 필요한 경우 support.genai@samsung.com 으로 문의 바랍니다.

Check API의 Request Body에 대해서 상세 설명이 필요합니다.

Field	Type	Required	Description	Value
content	string	required	사용자 질의	ex) “안녕하세요.”
user_ip	string	required	필터링 이력에서 확인되는 사용자 IP	ex) “123.456.789.111”
target_model	string	required	정책 적용 모델(“INTERNAL”, “EXTERNAL”)	ex) “INTERNAL”
target_service	string	optional	정책그룹 타겟 서비스 (default: GENAI)	ex) “GENAI”
model_name	string	optional	필터링 이력에서 확인되는 모델명	ex) “FabriX”

- content에는 필터 여부를 체크하고 싶은 문자열을 전달합니다.
- user_ip에는 필터에 걸렸을 때 필터링 이력에서 확인이 필요한 IP를 전달합니다.
- target_model은 INTERNAL, EXTERNAL 값을 넣을 수 있고, 내부 LLM 모델을 호출하는 경우 INTERNAL, 외부 LLM 모델을 호출하는 경우 EXTERNAL을 값으로 넣습니다.
- target_service는 정책 그룹에 적용되어 있는 타겟 서비스를 입력합니다. 필터 관리자에게 문의가 필요하며, 타겟 서비스의 정보가 없는 경우 GENAI 로 동작합니다.
- model_name은 필터에 걸렸을 때 필터링 이력에서 확인이 필요한 모델명을 입력합니다.

정책 그룹, 정책, 룰, AI 필터 모델에 대해서 설명 해주세요.

정책 그룹은 다수의 정책으로 구성되며, 정책의 종류로 룰과 AI 필터 모델이 있습니다. 필터 관리자가 정책 그룹을 생성할 때, 타겟 서비스를 지정할 수 있으며, 정책 그룹 적용 대상(사용자)를 설정합니다. 필터 관리자는 정책 그룹에 정책으로 룰, AI 필터 모델을 추가할 수 있습니다. 이후 Check API가 호출되면, 정책 그룹의 적용 대상 여부를 판단하여 정책(필터) 체크 후 응답을 하게 됩니다.

4.

File Translation/Summary APIs

4.1 File Translation/Summary APIs Overview

File Translation/Summary APIs는 사용자의 파일 내용을 기반으로 요약, 번역을 할 수 있는 API를 제공합니다.

File Translation/Summary APIs 에서 제공하는 API는 다음과 같습니다.

GET /openapi/translation-summary/v1/models

API로 사용할 수 있는 대화 모델의 리스트를 조회합니다.

POST /openapi/translation-summary/v1/file-messages

요약 또는 번역을 위한 파일을 설정값과 함께 업로드 할 수 있습니다.

GET /openapi/translation-summary/v1/file-messages/status

file-messages API(POST)를 통해 업로드한 파일의 상태를 조회합니다.

GET /openapi/translation-summary/v1/file-messages

file-messages API(POST)를 통해 번역한 결과를 doc 파일로 다운로드합니다.

4.2 API 명세

GET /openapi/translation-summary/v1/models

노트

사용자 포털에서 권한을 가진 모델(Text)을 API로 조회할 수 있습니다.

Text : 대화모델

Header

Field	Type	Required	Description	Value
Content-Type	string	optional	Type of content	application/json
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-openapi-token	string	required	OpenAPI Token information	ex) Bearer eyJXXX
x-generative-ai-user-email	string	optional	Portal user email	ex) kim.samsung@samsung.com

Response

Field	Type	Description	Value
modelId	String	Model Id	
name	list	Model Name	[{ "languageCode": "ko", "content": "모델명" }]
description	list	Model Description	[{ "languageCode": "ko", "content": "설명" }]
modelName	String	Model Name	DEPRECATED
modelLabel.en	String	English model Label	DEPRECATED
modelLabel.ko	String	Korean model Label	DEPRECATED
modelDescription.en	String	Model Description en	DEPRECATED
modelDescription.ko	String	Model Description ko	DEPRECATED

Example of Call (python)

```

1 import requests
2
3 YOUR_CLIENT_KEY = ""
4 YOUR_PASS_KEY = ""
5 ENDPOINT_URL = ""
6 YOUR_EMAIL = ""
7

```



```

8  headers = {
9      "x-fabrix-client": YOUR_CLIENT_KEY,
10     "x-openapi-token": YOUR_PASS_KEY,
11     "x-generative-ai-user-email": YOUR_EMAIL
12 }
13
14 api_endpoint_url = f"{ENDPOINT_URL}/openapi/translation-summary/v1/models"
15
16 response=requests.get(
17     api_endpoint_url,
18     headers=headers
19 )
20
21 # 응답 반환
22 print(response.json())

```

POST /openapi/translation-summary/v1/file-messages

노트

요약 또는 번역을 위한 파일을 설정에 맞는 값과 함께 업로드 할 수 있습니다.

Header

Field	Type	Required	Description	Value
Content-Type	string	optional	Type of content	multipart/form-data
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-openapi-token	string	required	OpenAPI Token information	ex) Bearer eyJXXX
x-generative-ai-user-email	string	optional	Portal user email	ex) kim.samsung@samsung.com

Request Body

Field	Type	Required	Description	Value
llmId	int	optional	LLM Id (default: basic LLM)	DEPRECATED

Field	Type	Required	Description	Value
modelId	str	required	model id	"0196f1fc-2858-70a9-a232-74dbddb971d0"
reqType	string	required	Api Type - translation - summary	ex) translation
language	string	required	Language - korean - english	ex) korean
style	string	required	Style	ex) basic
file	File	required		

| Style 가이드

reqType	value	Description
translation	basic	기본
	politly	정중하게
	friendly	친근하게
summary	paragraph-basic	문단식-기본
	paragraph-in-detail	문단식-상세하게
	paragraph-within-3lines	문단식-3문장내외
	bullet-form-basic	개조식-기본
	bullet-in-detail	개조식-상세하게
	bullet-within-3lines	개조식-3문장내외

| Request Body 예시

```

1  -----Boundary
2  Content-Disposition: form-data; name="reqType"
3
4  translation
5  -----Boundary
6  Content-Disposition: form-data; name="modelId"
7
8  0196f1fc-2858-70a9-a232-74dbddb971d0
9  -----Boundary
10 Content-Disposition: form-data; name="language"
11
12 korean
13 -----Boundary

```

```

14 Content-Disposition: form-data; name="style"
15
16 basic
17 -----Boundary
18 Content-Disposition: form-data; name="file"; filename="테스트.txt"
19
20 < C:/테스트.txt
21 -----Boundary--

```

Response

Field	Type	Description
documentId	string	Document id
messageId	int	Openapi message Id
fileName	string	Upload file name
status	string	Translation / Summary status-UPLOADED-PROCESS-SUCCESS-ERROR
analyzeType	string	Req Type
createdAt	DateTime	Create datetime

Response 예시

```

1 {
2   "documentId": "doc_xnMdgywi_202407181330",
3   "messageId": 237587,
4   "fileName": "테스트.txt",
5   "status": "UPLOAD",
6   "createdAt": "2024-07-18T13:30:31.057460+09:00",
7   "analyzeType": "translation"
8 }

```

호출 예시 (python)

```

1 import requests
2
3 YOUR_CLIENT_KEY = ""
4 YOUR_PASS_KEY = ""
5 ENDPOINT_URL = ""
6 YOUR_MODEL_ID = "Get model id from Model API"

```

```

7  YOUR_EMAIL = ""
8
9  headers = {
10     "x-fabrix-client": YOUR_CLIENT_KEY,
11     "x-openapi-token": YOUR_PASS_KEY,
12     "x-generative-ai-user-email": YOUR_EMAIL
13 }
14
15 file: UploadFile = open("./테스트.txt", "rb")
16
17 form_data = {"modelId": YOUR_MODEL_ID, "language": "korean", "style": "basic", "reqType":
"translation"}
18
19
20 response = requests.post(
21     f"{ENDPOINT_URL}/openapi/translation-summary/v1/file-messages",
22     headers=headers,
23     data=form_data,
24     files={"file": file},
25 )
26
27 print(response.json())

```

주의

※ Endpoint URL은 Open API 신청시 발급되는 URL로 적용하여 사용할 수 있습니다.

GET /openapi/translation-summary/v1/file-messages/status

노트

파일 메시지 업로드를 통해서 업로드한 파일의 상태를 알아볼 수 있습니다. 응답의 status 값이 PROCESS 일 때에는 파일 번역 / 요약이 진행되는 퍼센트가 content에 표시가 됩니다. status 가 SUCCESS 로 변경되면 content 에 파일 요약 결과 또는 파일 번역시에는 실제 번역 결과를 다운로드 할수 있는 FileUuid 값이 전달됩니다.

Header

Field	Type	Required	Description	Value
Content-Type	string	optional	Type of content	application/json
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-openapi-token	string	required	OpenAPI Token information	ex) Bearer eyJXXX
x-generative-ai-user-email	string	optional	Portal user email	ex) kim.samsung@samsung.com

Query Parameter

Field	Type	Required	Description	Value
documentId	string	required	Document id	
messageId	int	required	Openapi message id	

Response

Field	Type	Description
documentId	string	Document id
messageId	string	Openapi message Id
fileName	string	Upload file name
status	string	Translation / Summary status-UPLOAD-PROCESS-SUCCESS-ERROR
analyzeType	string	Req Type
createdAt	DateTime	Create datetime
modifiedAt	DateTime	Modify datetime
content	string	Status - PROGRESS-0%.....100%status - SUCCESS-summary : result content- translation : fileuuid

Response 예시 (진행중)

```

1 {
2   "documentId": "doc_K0xiYHJy_202407181413",
3   "messageId": 237597,
4   "fileName": "테스트.txt",
5   "status": "PROCESS",

```

```

6  "createdAt": "2024-07-18T14:13:58.943852+09:00",
7  "analyzeType": "translation",
8  "modifiedAt": "2024-07-18T14:14:00.919582+09:00",
9  "content": "0%"
10 }
```

| Response 예시 (번역 - 완료)

```

1  {
2    "documentId": "doc_zdIVev05_202407181337",
3    "messageId": 237589,
4    "fileName": "테스트.txt",
5    "status": "SUCCESS",
6    "createdAt": "2024-07-18T13:37:43.999646+09:00",
7    "analyzeType": "translation",
8    "modifiedAt": "2024-07-18T13:38:07.017882+09:00",
9    "content": "1b4467dd-6c7a-469b-8a29-079de9385dbf"
10 }
```

| Response 예시 (요약 - 완료)

```

1  {
2    "documentId": "doc_4KdomGik_202407181411",
3    "messageId": 237595,
4    "fileName": "테스트.txt",
5    "status": "SUCCESS",
6    "createdAt": "2024-07-18T14:11:37.678934+09:00",
7    "analyzeType": "summary",
8    "modifiedAt": "2024-07-18T14:11:40.420587+09:00",
9    "content": "Ollie Watkins ' brilliant last-gasp winner..... "
10 }
```

| 호출 예시 (python)

```

1  import requests
2
3  YOUR_CLIENT_KEY = ""
4  YOUR_PASS_KEY = ""
```

```

5  ENDPOINT_URL = ""
6  YOUR_EMAIL = ""
7  YOUR_DOCUMENT_ID = ""
8  YOUR_MESSAGE_ID = ""
9
10 headers = {
11     "x-fabrix-client": YOUR_CLIENT_KEY,
12     "x-openapi-token": YOUR_PASS_KEY,
13     "x-generative-ai-user-email": YOUR_EMAIL
14 }
15
16 document_id = YOUR_DOCUMENT_ID
17 message_id = YOUR_MESSAGE_ID
18
19 query_params = f"?documentId={document_id}&messageId={message_id}"
20
21 response = requests.get(
22     f"{ENDPOINT_URL}/openapi/translation-summary/v1/file-messages{query_params}",
23     headers=headers,
24 )
25
26 print(response.json())

```

주의

※ Endpoint URL은 Open API 신청시 발급되는 URL로 적용하여 사용할 수 있습니다.

GET /openapi/translation-summary/v1/file-messages

노트

파일 메시지 업로드 를 통해서 파일 번역을 한 경우 결과를 파일로만 받을수 있습니다. 따라서 파일 메시지 조회를 통해서 파일 번역결과 파일을 다운로드 할 수 있습니다. 결과 파일은 doc 파일로 받을 수 있습니다.

Header

Field	Type	Required	Description	Value
Content-Type	string	optional	Type of content	application/json
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-openapi-token	string	required	OpenAPI Token information	ex) Bearer eyJXXX
x-generative-ai-user-email	string	optional	Portal user email	ex) kim.samsung@samsung.com

Query Parameter

Field	Type	Required	Description	Value
fileUuid	string	required	File uuid	

Response

Field	Type	Description
filecontents	bytes	File contents

호출 예시 (python)

```

1  import requests
2
3  YOUR_PASS_KEY = ""
4  YOUR_CLIENT_KEY = ""
5  ENDPOINT_URL = ""
6  YOUR_EMAIL = ""
7  YOUR_FILE_UUID = ""
8
9  headers = {
10     "x-fabrix-client": YOUR_CLIENT_KEY,
11     "x-openapi-token": YOUR_PASS_KEY,
12     "x-generative-ai-user-email": YOUR_EMAIL
13  }
14  file_uuid = YOUR_FILE_UUID
15
16  query_params = f"?fileUuid={file_uuid}"
17
18  response = requests.get(

```



```

19     f"{ENDPOINT_URL}/openapi/translation-summary/v1/file-messages{query_params}",
20     headers=headers,
21 )
22
23 file_data = response.content
24 with open(f"result.doc", "wb") as f:
25     f.write(file_data)

```

주의

※ Endpoint URL은 Open API 신청시 발급되는 URL로 적용하여 사용할 수 있습니다.

공통 Response Code

Http status code	Response code	Reason (response message)
200	R20000	success
200	R20001	need more parameters for api call
200	R20002	only support single-step task
200	R20003	error occurred during api call
200	R20004	api request timeout
200	R20005	exceeded token limit for generating response
200	R20006	only resource uri is provided by cognitive search
200	R20007	skipped answer generation
200	R20009	invalid RAG answer
400	R40000	client error # generic type for client error
400	R40001	request parameter validation error
400	R40002	unsupported model error
400	R40003	invalid plugin or api specification
400	R40004	exceeded token limit for request completion
400	R40005	health check api error
400	R40006	plug-in id not found
400	R40007	search engine not used
400	R40008	blocked by security filter
400	R40009	request timeout
400	R40010	too many LLM request
400	R40011	LLM request timeout

Http status code	Response code	Reason (response message)
400	R40051	contents not found
400	R40052	contents validation error
400	R40053	llm_model is not exist or no authorization
400	R40054	contents or system_prompt is filtered by FilterServer
400	R40055	no authorization of catalog
500	R50000	internal server error # generic type for server error
500	R50001	api specification parsing error
500	R50002	security filter service error
500	R50003	cognitive search service error
500	R50004	LLM request error
500	R50005	LLM response parsing error
500	R50006	Invalid result of cognitive search

5.

LLM Serving APIs

5.1 LLM Serving APIs Overview

LLM Serving APIs는 모델의 응답을 직접 호출할 수 있는 API를 제공합니다. LLM Serving APIs를 통해 제공되는 답변은 서비스의 필터링 기능이 적용되지 않습니다.

LLM Serving APIs 에서 제공하는 API는 다음과 같습니다.

GET /openapi/llm/v1/models

API로 사용할 수 있는 대화 모델의 리스트를 조회합니다.

POST /openapi/llm/*

serving API는 대화에 대한 모델 응답을 생성합니다.

5.2 API 명세

GET /openapi/llm/v1/models

Header

Field	Type	Required	Description	Value
Content-Type	string	optional	Type of content	application/json
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-openapi-token	string	required	OpenAPI Token information	ex) Bearer eyJXXX
x-generative-ai-user-email	string	optional	Portal user email	ex) kim.samsung@samsung.com

Response

Field	Type	Description	Value
modelId	String	Model Id	
modelGuid	UUID	Model Guid	
name	list	Model Name	[{ "languageCode": "ko", "content": "모델명" }]
description	list	Model Description	[{ "languageCode": "ko", "content": "설명" }]
modelName	String	Model Name	DEPRECATED
modelLabel.en	String	English model Label	DEPRECATED
modelLabel.ko	String	Korean model Label	DEPRECATED
modelDescription.en	String	Model Description en	DEPRECATED
modelDescription.ko	String	Model Description ko	DEPRECATED

호출 예시 (python)

```

1  import requests
2
3  YOUR_CLIENT_KEY = ""
4  YOUR_PASS_KEY = ""
5  ENDPOINT_URL = ""
6  YOUR_EMAIL = ""
7
8  headers = {
9      "x-fabrix-client": YOUR_CLIENT_KEY,
10     "x-openapi-token": YOUR_PASS_KEY,
11     "Content-Type": "application/json"
12 }
13
14 api_endpoint_url = f"{ENDPOINT_URL}/v1/models"
15
16 response=requests.get(

```

```

17     api_endpoint_url,
18     headers=headers
19 )
20
21 # 응답 반환
22 print(response.json())

```

주의

※ Endpoint URL은 Open API 신청시 발급되는 URL로 적용하여 사용할 수 있습니다.

POST /openapi/llm/*

노트

models api를 통해서 얻은 Model ID를 기반으로 모델의 각 endpoint에 직접적으로 접근할 수 있습니다.

Header

Field	Type	Required	Description	Value
Content-Type	string	optional	Type of content	application/json
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-openapi-token	string	required	OpenAPI Token information	ex) Bearer eyJXXX
x-generative-ai-user-email	string	optional	Portal user email	ex) kim.samsung@samsung.com

Request

Field	Type	Required	Description	Value
model	str	required	Name of the model to use	ex) "model": "/mnt/models"
messages	list[dict]	required	A list of messages comprising the conversation so far.	ex) "messages": [{ "role": "system",

Field	Type	Required	Description	Value
				<pre> "content": "You are a helpful assistant." }, { "role": "user", "content": "Hello!" }]</pre>
stream	boolean (default: false)	optional	Whether to stream back partial progress.	ex) "stream": true
stream_options	Object	optional	Options for streaming response. Only set this when you set "stream": true	ex) "stream_options": {"include_usage": true}
frequency_penalty	number (default: 0)	optional	Number between -2.0 and 2.0.	
max_completion_tokens	integer	optional	An upper bound for the number of tokens that can be generated for a completion.	ex) "max_completion_tokens": 256 "max_tokens" 변수는 deprecated 예정
temperature	number (default: 1)	optional	What sampling temperature to use, between 0 and 2.	ex) "temperature": 0.7 temperature or top_p 둘 중 하나만 변경
top_p	number (default: 1)	optional	An alternative to sampling with temperature, where the model considers the results of the tokens with top_p probability mass.	ex) "top_p": 0.95

| llmConfig Properties

Name	Description
frequency_penalty	반복된 단어의 출현 확률을 조절합니다. 값이 1보다 크면 반복된 단어가 나타날 확률이 줄어들게 되고, 값이 1보다 작으면 반복된 단어가 나타날 확률이 증가하게 됩니다.

Name	Description
seed	모델의 난수 발생기를 시드하는 데 사용됩니다. 동일한 시드를 사용하면 모델은 항상 동일한 순서로 출력을 생성합니다. (Beta)
temperature	모델의 확률 분포를 조절합니다. 값이 1보다 크면 확률 분포가 더욱 고르게 퍼지게 되고, 값이 1보다 작으면 확률 분포가 더욱 뾰족하게 집중됩니다. (0<temperature<1)<div="">
top_p	확률 분포의 상위 p%를 고려하여 단어를 선택합니다. 이렇게 하면 선택된 단어들이 확률 분포를 고르게 대표하도록 하여 다양성을 높일 수 있습니다. (top_p must be > 0.0 and < 1.0)

Request Body 예시

```

1 {
2   "model": "/mnt/models",
3   "messages": [
4     {
5       "role": "system",
6       "content": "You are a helpful assistant."
7     },
8     {
9       "role": "user",
10      "content": "Who are you?"
11    }
12  ],
13  "stream": true,
14  "stream_options" : {"include_usage": true}
15 }
```

Response

- "stream": false

Field	Type	Description
id	string	A unique identifier for the chat completions.
choices	array[dict]	A list of chat completion choices. • index

Field	Type	Description
		- message - role - The author of this message. - content - The contents of the message. - finish_reason - The reason the model stopped generating tokens.
created	integer	Timestamp of when the chat completion was created.
model	string	The model used for the chat completion.
object	string	The object type, whichs always "chat.completion".
usage	object	Usage statistics for the completion request. - completion_tokens - Number of tokens in the generated completion - prompt_tokens - Number of tokens in the prompt - total_tokens - (prompt + completion) tokens

호출 예시 (python)

- "stream": false

```

1 import requests
2
3 YOUR_CLIENT_KEY = ""
4 YOUR_PASS_KEY = ""
5 ENDPOINT_URL = ""
6 YOUR_MODEL_ID = "Get model id from Model API"
7
8 headers = {
9     "x-openapi-token": YOUR_PASS_KEY,
10    "x-fabrix-client": YOUR_CLIENT_KEY,
11    "x-llm-model-id": YOUR_MODEL_ID,

```



```

12  "Content-Type": "application/json"
13  }
14
15  body = {
16      "model": "/mnt/models",
17      "messages": [
18          {
19              "role": "system",
20              "content": "You are a helpful assistant."
21          },
22          {
23              "role": "user",
24              "content": "Who are you?"
25          }
26      ]
27  }
28
29  endpoint_url = ENDPOINT_URL + "/chat/completions"
30  response = requests.post(endpoint_url, headers=headers, json=body)
31
32  print(response.json())

```

주의

※ Endpoint URL은 Open API 신청시 발급되는 URL로 적용하여 사용할 수 있습니다.

- "stream": true

```

1  import json
2  import requests
3
4  YOUR_CLIENT_KEY = ""
5  YOUR_PASS_KEY = ""
6  ENDPOINT_URL = ""
7  YOUR_MODEL_ID = "Get model id from Model API"
8
9  headers = {
10     "x-openapi-token": YOUR_PASS_KEY,

```

```

11  "x-fabrix-client": YOUR_CLIENT_KEY,
12  "x-llm-model-id": YOUR_MODEL_ID,
13  "Content-Type": "application/json"
14  }
15
16  body = {
17      "model": "/mnt/models",
18      "messages": [
19          {
20              "role": "system",
21              "content": "You are a helpful assistant."
22          },
23          {
24              "role": "user",
25              "content": "Who are you?"
26          }
27      ],
28      "stream": True
29  }
30
31  endpoint_url = ENDPOINT_URL + "/chat/completions"
32  response = requests.post(endpoint_url, headers=headers, json=body)
33
34  print(response.text)

```

응답 예시 (python)

- "stream": false

```

1  {
2      "id": "chatcmpl-e33d054e0ffb48da997e2346e289e5fe",
3      "object": "chat.completion",
4      "created": 1740041033,
5      "model": "/mnt/models",
6      "choices": [
7          {
8              "index": 0,
9              "message": {
10                 "role": "assistant",
11                 "content": "I'm an AI assistant, here to help answer your questions, provide

```

information, and assist with tasks to the best of my abilities. I don't have a personal identity or emotions, but I'm designed to be friendly, neutral, and informative.\n\nI can help with a wide range of topics, from science and history to entertainment and culture. I can also assist with language-related tasks, such as language translation, grammar correction, and text summarization.\n\nMy goal is to provide accurate and helpful information, and to engage in a productive and respectful conversation with you. So, feel free to ask me anything, and I'll do my best to help!",

```

12     "tool_calls": []
13   },
14   "logprobs": null,
15   "finish_reason": "stop",
16   "stop_reason": null
17 }
18 ],
19 "usage": {
20   "prompt_tokens": 45,
21   "total_tokens": 177,
22   "completion_tokens": 132,
23   "prompt_tokens_details": null
24 },
25 "prompt_logprobs": null
26 }
```

- "stream": true

```

1  data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"role":"assistant","co
ntent":""},"logprobs":null,"finish_reason":null}]}
2
3  data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"I"},"logpro
bs":null,"finish_reason":null}]}
4
5  data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"'m"},"logpr
obs":null,"finish_reason":null}]}
6
7  data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
an"},"logprobs":null,"finish_reason":null}]}

```

```

8
9  data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
AI"},"logprobs":null,"finish_reason":null}}}
10
11 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"-powered"},"
logprobs":null,"finish_reason":null}}}
12
13 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
assistant"},"logprobs":null,"finish_reason":null}}}
14
15 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"","","logpro
bs":null,"finish_reason":null}}}
16
17 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
here"},"logprobs":null,"finish_reason":null}}}
18
19 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
to"},"logprobs":null,"finish_reason":null}}}
20
21 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
help"},"logprobs":null,"finish_reason":null}}}
22
23 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
answer"},"logprobs":null,"finish_reason":null}}}
24
25 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
any"},"logprobs":null,"finish_reason":null}}}
26
27 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
questions"},"logprobs":null,"finish_reason":null}}}

```

```

28
29  data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"","","logpro
bs":null,"finish_reason":null}}]}
30
31  data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
provide"},"logprobs":null,"finish_reason":null}}]}
32
33  data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
information"},"logprobs":null,"finish_reason":null}}]}
34
35  data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"","","logpro
bs":null,"finish_reason":null}}]}
36
37  data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
and"},"logprobs":null,"finish_reason":null}}]}
38
39  data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
assist"},"logprobs":null,"finish_reason":null}}]}
40
41  data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
with"},"logprobs":null,"finish_reason":null}}]}
42
43  data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
tasks"},"logprobs":null,"finish_reason":null}}]}
44
45  data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
to"},"logprobs":null,"finish_reason":null}}]}
46
47  data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
the"},"logprobs":null,"finish_reason":null}}]}

```

```

48
49 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
best"},"logprobs":null,"finish_reason":null}}]}
50
51 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
of"},"logprobs":null,"finish_reason":null}}]}
52
53 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
my"},"logprobs":null,"finish_reason":null}}]}
54
55 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
abilities"},"logprobs":null,"finish_reason":null}}]}
56
57 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"."},"logpro
bs":null,"finish_reason":null}}]}
58
59 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
I"},"logprobs":null,"finish_reason":null}}]}
60
61 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"'m"},"logpro
bs":null,"finish_reason":null}}]}
62
63 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
a"},"logprobs":null,"finish_reason":null}}]}
64
65 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
large"},"logprobs":null,"finish_reason":null}}]}
66
67 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
language"},"logprobs":null,"finish_reason":null}}]}

```

```

68
69 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
model"},"logprobs":null,"finish_reason":null}}]}
70
71 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"","logprobs":null,"finish_reason":null}}]}
72
73 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
trained"},"logprobs":null,"finish_reason":null}}]}
74
75 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
on"},"logprobs":null,"finish_reason":null}}]}
76
77 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
a"},"logprobs":null,"finish_reason":null}}]}
78
79 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
vast"},"logprobs":null,"finish_reason":null}}]}
80
81 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
amount"},"logprobs":null,"finish_reason":null}}]}
82
83 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
of"},"logprobs":null,"finish_reason":null}}]}
84
85 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
text"},"logprobs":null,"finish_reason":null}}]}
86
87 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
data"},"logprobs":null,"finish_reason":null}}]}

```

```

88
89  data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"","","logprobs":null,"finish_reason":null}}]}
90
91  data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
which"},"logprobs":null,"finish_reason":null}}]}
92
93  data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
enables"},"logprobs":null,"finish_reason":null}}]}
94
95  data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
me"},"logprobs":null,"finish_reason":null}}]}
96
97  data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
to"},"logprobs":null,"finish_reason":null}}]}
98
99  data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
understand"},"logprobs":null,"finish_reason":null}}]}
100
101  data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
and"},"logprobs":null,"finish_reason":null}}]}
102
103  data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
respond"},"logprobs":null,"finish_reason":null}}]}
104
105  data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
to"},"logprobs":null,"finish_reason":null}}]}
106
107  data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
a"},"logprobs":null,"finish_reason":null}}]}

```



```

108
109 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
wide"},"logprobs":null,"finish_reason":null}]}
110
111 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
range"},"logprobs":null,"finish_reason":null}]}
112
113 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
of"},"logprobs":null,"finish_reason":null}]}
114
115 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
topics"},"logprobs":null,"finish_reason":null}]}
116
117 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
and"},"logprobs":null,"finish_reason":null}]}
118
119 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
queries"},"logprobs":null,"finish_reason":null}]}
120
121 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":".\n\n"},"lo
gprobs":null,"finish_reason":null}]}
122
123 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"I"},"logpro
bs":null,"finish_reason":null}]}
124
125 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
don"},"logprobs":null,"finish_reason":null}]}
126
127 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"'t"},"logpr
obs":null,"finish_reason":null}]}

```

128

```
129 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
have"},"logprobs":null,"finish_reason":null}]}
```

130

```
131 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
a"},"logprobs":null,"finish_reason":null}]}
```

132

```
133 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
personal"},"logprobs":null,"finish_reason":null}]}
```

134

```
135 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
identity"},"logprobs":null,"finish_reason":null}]}
```

136

```
137 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
or"},"logprobs":null,"finish_reason":null}]}
```

138

```
139 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
emotions"},"logprobs":null,"finish_reason":null}]}
```

140

```
141 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":",","logpro
bs":null,"finish_reason":null}]}
```

142

```
143 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
but"},"logprobs":null,"finish_reason":null}]}
```

144

```
145 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
I"},"logprobs":null,"finish_reason":null}]}
```

146

```
147 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"'m"},"logpr
obs":null,"finish_reason":null}]}
```

```

148
149 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
designed"},"logprobs":null,"finish_reason":null}]}
150
151 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
to"},"logprobs":null,"finish_reason":null}]}
152
153 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
be"},"logprobs":null,"finish_reason":null}]}
154
155 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
friendly"},"logprobs":null,"finish_reason":null}]}
156
157 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"","logpro
bs":null,"finish_reason":null}]}
158
159 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
helpful"},"logprobs":null,"finish_reason":null}]}
160
161 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"","logpro
bs":null,"finish_reason":null}]}
162
163 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
and"},"logprobs":null,"finish_reason":null}]}
164
165 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
informative"},"logprobs":null,"finish_reason":null}]}
166
167 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"."},"logpro
bs":null,"finish_reason":null}]}

```

```

168
169 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
My"},"logprobs":null,"finish_reason":null}}}
170
171 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
goal"},"logprobs":null,"finish_reason":null}}}
172
173 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
is"},"logprobs":null,"finish_reason":null}}}
174
175 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
to"},"logprobs":null,"finish_reason":null}}}
176
177 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
assist"},"logprobs":null,"finish_reason":null}}}
178
179 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
users"},"logprobs":null,"finish_reason":null}}}
180
181 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
like"},"logprobs":null,"finish_reason":null}}}
182
183 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
you"},"logprobs":null,"finish_reason":null}}}
184
185 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"","","logprobs":null,"finish_reason":null}}}
186
187 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk","
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
providing"},"logprobs":null,"finish_reason":null}}}

```

```

188
189 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
accurate"},"logprobs":null,"finish_reason":null}]}
190
191 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
and"},"logprobs":null,"finish_reason":null}]}
192
193 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
relevant"},"logprobs":null,"finish_reason":null}]}
194
195 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
information"},"logprobs":null,"finish_reason":null}]}
196
197 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"",
},"logprobs":null,"finish_reason":null}]}
198
199 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
and"},"logprobs":null,"finish_reason":null}]}
200
201 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
helping"},"logprobs":null,"finish_reason":null}]}
202
203 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
you"},"logprobs":null,"finish_reason":null}]}
204
205 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
find"},"logprobs":null,"finish_reason":null}]}
206
207 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
the"},"logprobs":null,"finish_reason":null}]}

```

208

```
209 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
answers"},"logprobs":null,"finish_reason":null}]}
```

210

```
211 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
you"},"logprobs":null,"finish_reason":null}]}
```

212

```
213 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"'re"},"logp
robs":null,"finish_reason":null}]}
```

214

```
215 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
looking"},"logprobs":null,"finish_reason":null}]}
```

216

```
217 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
for"},"logprobs":null,"finish_reason":null}]}
```

218

```
219 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":".\n\n"},"lo
gprobs":null,"finish_reason":null}]}
```

220

```
221 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"Is"},"logpr
obs":null,"finish_reason":null}]}
```

222

```
223 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
there"},"logprobs":null,"finish_reason":null}]}
```

224

```
225 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
something"},"logprobs":null,"finish_reason":null}]}
```

226

```
227 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
specific"},"logprobs":null,"finish_reason":null}]}
```

228

```
229 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
you"},"logprobs":null,"finish_reason":null}]}
```

230

```
231 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"'d"},"logpr
obs":null,"finish_reason":null}]}
```

232

```
233 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
like"},"logprobs":null,"finish_reason":null}]}
```

234

```
235 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
to"},"logprobs":null,"finish_reason":null}]}
```

236

```
237 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
know"},"logprobs":null,"finish_reason":null}]}
```

238

```
239 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
or"},"logprobs":null,"finish_reason":null}]}
```

240

```
241 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
ask"},"logprobs":null,"finish_reason":null}]}
```

242

```
243 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"?"},"logpro
bs":null,"finish_reason":null}]}
```

244

```
245 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
I"},"logprobs":null,"finish_reason":null}]}
```

246

```
247 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"'m"},"logpr
obs":null,"finish_reason":null}]}
```

```

248
249 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
here"},"logprobs":null,"finish_reason":null}]}
250
251 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
to"},"logprobs":null,"finish_reason":null}]}
252
253 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"
help"},"logprobs":null,"finish_reason":null}]}
254
255 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":"!"},"logpro
bs":null,"finish_reason":null}]}
256
257 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[{"index":0,"delta":{"content":""},"logprob
s":null,"finish_reason":"stop","stop_reason":null}]}
258
259 data: {"id":"chatcmpl-cf149cdfb08b48f2aa4b9503c2a817da","object":"chat.completion.chunk",
created":1740040819,"model":"/mnt/models","choices":[],"usage":{"prompt_tokens":45,"total_token
s":173,"completion_tokens":128}}
260
261 data: [DONE]

```

주의

※ Endpoint URL은 Open API 신청시 발급되는 URL로 적용하여 사용할 수 있습니다.

호출 예시 (python)

- Tool Calling

```

1 import json
2 import requests
3 from langchain_core.tools import tool

```



```

4 from langchain_openai import ChatOpenAI
5
6 YOUR_CLIENT_KEY = ""
7 YOUR_PASS_KEY = ""
8 ENDPOINT_URL = ""
9 YOUR_MODEL_ID = "Get model id from Model API"
10
11 headers = {
12     "x-openapi-token": YOUR_PASS_KEY,
13     "x-fabrix-client": YOUR_CLIENT_KEY,
14     "x-llm-model-id": YOUR_MODEL_ID,
15     "Content-Type": "application/json"
16 }
17
18 # 예제 tool 선언
19 @tool
20 def add(a: int, b: int) -> int:
21     """Add two integers.
22     Args:
23         a: first integer
24         b: second integer
25     """
26     return a + b
27
28 @tool
29 def get_weather(a: str) -> str:
30     """Get specific city's weather information.
31     Args:
32         a: city name
33     """
34     return f"{a}'s weather is sunny."
35
36 tools = [add, get_weather]
37
38 # langchain을 사용해서 tool bind한 예제
39 llm = ChatOpenAI(
40     base_url=ENDPOINT_URL,
41     api_key="EMPTY",
42     model="gpt-4o",
43     default_headers={

```

```

44     "X-FABRIX-CLIENT": headers["x-fabrix-client"] ,
45     "X-OPENAPI-TOKEN": headers["x-openapi-token"],
46     "X-LLM-MODEL-ID": "312"
47 },
48 )
49
50 llm_with_tools = llm.bind_tools(tools, tool_choice="required")
51 result = llm_with_tools.invoke("1 더하기 3은?")
52 print(result)

```

응답 예시 (python)

- Tool Calling

```

1  {
2      content=''
3      additional_kwargs={
4          'tool_calls': [
5              {
6                  'id': 'call_yaWgso206FTvtgbauxgEH59V',
7                  'function': {
8                      'arguments': '{"a":1,"b":3}',
9                      'name': 'add'
10                 },
11                 'type': 'function'
12             }
13         ],
14         'refusal': None
15     }
16     response_metadata={
17         'token_usage': {
18             'completion_tokens': 18,
19             'prompt_tokens': 91,
20             'total_tokens': 109,
21             'completion_tokens_details': {
22                 'accepted_prediction_tokens': 0,
23                 'audio_tokens': 0,
24                 'reasoning_tokens': 0,
25                 'rejected_prediction_tokens': 0
26             },

```

```

27     'prompt_tokens_details': {
28         'audio_tokens': 0,
29         'cached_tokens': 0
30     }
31 }
32 'model_name': 'gpt-4o-2024-05-13',
33 'system_fingerprint': 'fp_ee1d74bde0',
34 'id': 'chatcmpl-CadiS6kaC4tQ94cu9w3HPEy2NXYF6',
35 'service_tier': None,
36 'finish_reason': 'tool_calls',
37 'logprobs': None
38 }
39 id='run--d8bb73cf-9f05-43b8-982c-615040f51b4d-0'
40 tool_calls=[
41     {
42         'name': 'add',
43         'args': {'a': 1, 'b': 3},
44         'id': 'call_yaWgso206FTvtgbauxgEH59V',
45         'type': 'tool_call'
46     }
47 ]
48 usage_metadata={
49     'input_tokens': 91,
50     'output_tokens': 18,
51     'total_tokens': 109,
52     'input_token_details': {
53         'audio': 0,
54         'cache_read': 0
55     },
56     'output_token_details': {
57         'audio': 0,
58         'reasoning': 0
59     }
60 }
61 }

```

- 위 예제는 openAI Compatible 모델에만 적용 가능합니다.
- 모델 별 tool calling에 대한 자세한 사용 예제는 사용하는 모델의 공식 문서를 참고 바랍니다.
- response 양식은 서빙 플랫폼 및 모델에 따라 달라질 수 있습니다.
- 모델에 대한 api_key는 serving APIs 내부에서 처리되는 값으로 "EMPTY"를 넣어서 호출하시면 됩니다.

주의

※ tool calling 기능을 제공하는 모델은 별도 확인이 필요합니다.

공통 Response Code

Http status code	Response code	Reason (response message)
200	R20000	success
200	R20001	need more parameters for api call
200	R20002	only support single-step task
200	R20003	error occurred during api call
200	R20004	api request timeout
200	R20005	exceeded token limit for generating response
200	R20006	only resource uri is provided by cognitive search
200	R20007	skipped answer generation
200	R20009	invalid RAG answer
400	R40000	client error # generic type for client error
400	R40001	request parameter validation error
400	R40002	unsupported model error
400	R40003	invalid plugin or api specification
400	R40004	exceeded token limit for request completion
400	R40005	health check api error
400	R40006	plug-in id not found
400	R40007	search engine not used
400	R40008	blocked by security filter
400	R40009	request timeout
400	R40010	too many LLM request
400	R40011	LLM request timeout
400	R40051	contents not found
400	R40052	contents validation error
400	R40053	llm_model is not exist or no authorization
400	R40054	contents or system_prompt is filtered by FilterServer
400	R40055	no authorization of catalog
500	R50000	internal server error # generic type for server error
500	R50001	api specification parsing error
500	R50002	security filter service error
500	R50003	cognitive search service error

Http status code	Response code	Reason (response message)
500	R50004	LLM request error
500	R50005	LLM response parsing error
500	R50006	Invalid result of cognitive search

5.3 FAQ

serving API 사용 시 model에는 어떤 값을 입력 해야 하나요?

서빙 API를 호출할 때 model 파라미터에 입력해야 할 값은 요청하고자 하는 모델명입니다. 모델 명을 모르실 경우 모델 관리자에게 문의하시기 바랍니다. 일부 LLM 모델은 서빙 환경 및 모델 특성에 따라 model 값을 생략해도 정상 동작할 수 있습니다.

serving API로 사용 가능한 엔드포인트는 어떤 게 있나요?

현재 서비스에서 제공되는 주요 엔드포인트 목록입니다. 각 엔드포인트는 POST 요청을 기본으로 하며, 자세한 요청/응답 스키마는 요청하는 LLM 모델 문서에서 확인할 수 있습니다.

표 5-1 Serving API 엔드포인트 목록

엔드포인트	용도
/completions	단일 텍스트 답변
/chat/completions	대화형 답변
/generate	단일 텍스트 답변 (TGI 서빙 타입)
/image/generations	이미지 생성

6.

Code Interpreter APIs

6.1 Code Interpreter APIs Overview

Code Interpreter APIs는 서비스에서 제공되는 코드인터프리터의 기능들을 세분화하여 API로 제공합니다.

Code Interpreter APIs 에서 제공하는 API는 다음과 같습니다.

POST /openapi/interpreter/v1/code

사용자의 질의를 기반으로 모델에 코드 생성을 요청하고 실행한 결과를 제공합니다.

POST /openapi/interpreter/v1/execute

사용자가 전달한 코드를 주피터 노트북 환경에서 실행한 결과를 제공합니다.

POST /openapi/interpreter/v1/upload

upload API는 코드인터프리터에 파일을 업로드합니다.

POST /openapi/interpreter/v1/download

코드인터프리터가 생성한 이미지나 파일을 다운로드합니다.

6.2 API 명세

POST /openapi/interpreter/v1/code

주의

코드 생성/실행은 파이썬 언어의 실행 환경만 지원합니다.

Header

Field	Type	Required	Description	Value
Content-Type	string	optional	Type of content	application/json
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-openapi-token	string	required	OpenAPI Token information	ex) Bearer eyJXXX
x-generative-ai-user-email	string	optional	Portal user email	ex) kim.samsung@samsung.com

Request Parameter

Field	Type	Required	Description	Value
stream	bool	optional	Set whether to receive LLM's answers as a stream,. Default value is "False"	ex) False
use_base64	bool	optional	Set whether to apply base64 encoding to image files. If set to True, it responds with base64-encoded image file data. If set to False, it responds with a UUID that allows for file downloads. Default value is "False"	ex) False
kernel_once	boolean	optional	Set whether to create a new Jupyter kernel every time code is executed (set to True to create a new kernel each time). Default value is "False"	ex) False

Request Body

Field	Type	Required	Description	Value
user_prompt	string	required	The user's natural language query.	ex) Find a category data and draw a graph.
chat_history	list[object]	optional	The conversation history between the user and the LLM.	See below.
files	list[object]	optional	Information about files uploaded to a storage server. Up to 5 files are allowed, including 'files' and 'data'. Duplicate file names are not permitted.	See below.
data	list[object]	optional	Text-based CSV format data. Up to 5 data are allowed, including 'files' and 'data'. When specifying a file name, it will be saved with the specified file name. When specifying a file name, only txt and csv extensions are allowed, and the file name must not be duplicated. If no file name is specified, it will be saved as a random ID.csv.	See below.
config	object	optional	If you add this setting, you can additionally apply an LLM prompt and LLM for Code Interpreter. If there is no config item, it follows the tenant default settings.	See below.

| "chat_history" 예시

```
1 "chat_history": [  
2   {  
3     "role": "user",  
4     "content": "Calculate 123 + 456"  
5   },  
6   {  
7     "role": "assistant",  
8     "content": "It is 579."  
9   }  
10  ]
```

| "files" 예시

```
1 "files": [  
2   {  
3     "fileName": "test2.csv",  
4     "fileUuid": "abc0d95f-43ac-4b79-90e6-2qweqweq1231"  
5   }  
6 ]
```

| "data" 예시

```
1 "data": [  
2   {  
3     "fileName": "data1.csv",  
4     "content": "header1,header2\ndata_a,data_b"  
5   }  
6 ]
```

| "config" 예시

```
1 "config": {  
2   "plugin_model_id": 1,  
3   "plugin_model_name": "Llama 3.3",  
4   "plugin_model_parameters": [  
5     "max_tokens": 1024,  
6     "temperature": 0.5,  
7     "top_p": 0.95,  
8     "streaming": true,  
9     "stop_sequences": ["<stop>"]  
10  ]  
11 }
```

```

5      {
6        "key": "max_tokens",
7        "value": 2048
8      }
9    ],
10   "plugin_system_prompt": {
11     "upload_file_info_prompt" : "some prompt",
12     "business_domain_desc_prompt" : "some prompt",
13     "code_generation_and_execution_result" : "some prompt",
14   }
15 }

```

Response Body (stream=False)

Field	Type	Description
code	string	Code generated by LLM
detail	string	Code execution failure information
execution	object	Code execution result details
execution.execute_result_text	string	Text result of print() or standard output. If the cell execution result is an object such as an image, it can be null.
execution.display_data	list[object]	Generated image or chart display data
execution.elapsed_time	float	The time taken to execute the code (unit: seconds)
execution.is_truncated	bool	If the response data is long, cut or exclude the data. The is_truncated value becomes true.
execution.usermachine_expected_shutdown_time	timestamp	Expected end time of user machine
files	list[object]	List of files created by executing code
final_response	string	When the code is executed, the lastly generated final response
intent	string	User query intention classification result
llm_response	string	Full text generated by LLM
llm_url	string	LLM Endpoint
status	string	Code execution state

Response Body (stream=False) 예시

```

1  {
2    "code": "result = 1 + 2\nprint(result)",
3    "detail": null,
4    "execution": {
5      "display_data": [],
6      "elapsed_time": 0.6,
7      "execute_result_text": "3\n",
8      "is_truncated": false,
9      "usermachine_expected_shutdown_time": 1730955478
10   },
11   "files": [],
12   "final_response": "The result of 1+2 is 3.",
13   "intent": "code_generation_execution",
14   "llm_response": "```python\nresult = 1 + 2\nprint(result)\n```",
15   "llm_url": "http://meta-llama-3-70b.xxx/v1",
16   "status": "SUCCESS"
17 }

```

Response Body (stream=True)

Response Body (stream=True) 예시

```

1  event: CODE_GENERATION
2  data: {
3    data:      "code": "```"
4  data: }
5
6  event: CODE_GENERATION
7  data: {
8    data:      "code": "python"
9  data: }
10
11 event: CODE_GENERATION
12 data: {
13 data:      "code": "\n"
14 data: }
15

```

```
16 event: CODE_GENERATION
17 data: {
18   data:      "code": "result"
19 data: }
20
21 event: CODE_GENERATION
22 data: {
23   data:      "code": " ="
24 data: }
25
26 event: CODE_GENERATION
27 data: {
28   data:      "code": " "
29 data: }
30
31 event: CODE_GENERATION
32 data: {
33   data:      "code": "1"
34 data: }
35
36 event: CODE_GENERATION
37 data: {
38   data:      "code": " +"
39 data: }
40
41 event: CODE_GENERATION
42 data: {
43   data:      "code": " "
44 data: }
45
46 event: CODE_GENERATION
47 data: {
48   data:      "code": "2"
49 data: }
50
51 event: CODE_GENERATION
52 data: {
53   data:      "code": "\n"
54 data: }
55
```

```
56 event: CODE_GENERATION
57 data: {
58   data:      "code": "print"
59 data: }
60
61 event: CODE_GENERATION
62 data: {
63   data:      "code": "(result"
64 data: }
65
66 event: CODE_GENERATION
67 data: {
68   data:      "code": ")\n"
69 data: }
70
71 event: CODE_GENERATION
72 data: {
73   data:      "code": "````"
74 data: }
75
76 event: CODE_GENERATION
77 data: {
78   data:      "code": ""
79 data: }
80
81 event: CODE_GENERATION
82 data: {
83   data:      "code": ""
84 data: }
85
86 event: CODE_GENERATION
87 data: {
88   data:      "code": "\n\n````execution_result\n3\n```\n\n"
89 data: }
90
91 event: CODE_GENERATION
92 data: {
93   data:      "code": "The result of 1+2 is indeed 3."
94 data: }
95
```

```

96 event: CODE_GENERATION
97 data: {
98   data:      "code": ""
99 data: }
100
101 event: CODE_GENERATION
102 data: {
103   data:      "code": ""
104 data: }
105
106 event: CODE_RESULT
107 data: {
108   data:      "code": "result = 1 + 2\nprint(result)",
109   data:      "detail": null,
110   data:      "execution": {
111     data:      "display_data": [],
112     data:      "elapsed_time": 0.6,
113     data:      "execute_result_text": "3\n",
114     data:      "is_truncated": false,
115     data:      "usermachine_expected_shutdown_time": 1730955403
116   data:      },
117   data:      "files": [],
118   data:      "final_response": "The result of 1+2 is indeed 3.",
119   data:      "intent": "code_generation_execution",
120   data:      "llm_response": "```python\nresult = 1 + 2\nprint(result)\n```",
121   data:      "llm_url": "http://meta-llama-3-70b.xxx/v1",
122   data:      "status": "SUCCESS"
123 data: }

```

POST /openapi/interpreter/v1/execute

주의

파이썬 코드 실행만 지원 합니다.

Header

Field	Type	Required	Description	Value
Content-Type	string	required	Type of content	application/json
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-openapi-token	string	required	OpenAPI Token information received from catalog connector	ex) Bearer eyJXXX

Request Parameter

Field	Type	Required	Description	Value
use_base64	bool	optional	Set whether to apply base64 encoding to image files. If set to True, it responds with base64-encoded image file data. If set to False, it responds with a UUID that allows for file downloads. Default value is "False"	ex) False
kernel_once	bool	optional	Set whether to create a new Jupyter kernel every time code is executed (set to True to create a new kernel each time). Default value is "False"	ex) False

Request Body

Field	Type	Required	Description	Value
code	string	required	Python code to execute	ex) result = 1 + 2 print(result)
files	list[object]	optional	Information about files uploaded to a storage server. Up to 5 files are allowed,	See below.

Field	Type	Required	Description	Value
			including 'files' and 'data'. Duplicate file names are not permitted.	
data	list[object]	optional	Text-based CSV format data. Up to 5 data are allowed, including 'files' and 'data'. When specifying a file name, it will be saved with the specified file name. When specifying a file name, only txt and csv extensions are allowed, and the file name must not be duplicated. If no file name is specified, it will be saved as a random ID.csv.	See below.

| "files" 예시

```

1 "files": [
2   {
3     "fileName": "test2.csv",
4     "fileUuid": "abc0d95f-43ac-4b79-90e6-2qweqweq1231"
5   }
6 ]

```

| "data" 예시

```

1 "data": [
2   {
3     "fileName": "data1.csv",
4     "content": "header1,header2\ndata_a,data_b"
5   }

```


6]

Response

Field	Type	Description
execute_result_text	string	Text result of print() or standard output. If the cell execution result is an object such as an image, it can be empty.
display_data	list[object]	Generated image or chart display data
elapsed_time	float	The time taken to execute the code (unit: seconds)
is_truncated	bool	If the response data is long, cut or exclude the data. The is_truncated value becomes true.
usermachine_expected_shutdown_time	timestamp	Expected end time of user machine

Response 예시 (When result type is text)

```

1 {
2   "execute_result_text": "3\n",
3   "display_data": [],
4   "elapsed_time": 0.6,
5   "is_truncated": false,
6   "usermachine_expected_shutdown_time": 1730955478
7 }
```

Response 예시 (When result type is image)

```

1 {
2   "execute_result_text": "",
3   "display_data": [
4     {
5       "text": "<Figure size 640x480 with 1 Axes>",
6       "image_format": "image/png",
7       "image_src": "cee9737f-15e3-430b-9abe-cfaf4916f6b8"
8     }
9   ]
10 }
```

```

9    ],
10   "elapsed_time": 1.1,
11   "is_truncated": false,
12   "usermachine_expected_shutdown_time": 1730955478
13  }

```

POST /openapi/interpreter/v1/upload

노트

스토리지 서버에 파일이 업로드 되면 파일 UUID 정보를 응답합니다. 파일 정보를 포함하여 코드 생성과 실행 API를 호출하면 해당 파일을 사용할 수 있습니다.

※ 파일 업로드 제약 사항

- 파일 개수 : 한 번에 최대 5개
- 파일명 중복 : 한 번에 업로드하는 파일 개수가 2개 이상일 때 중복된 파일명 업로드 불가
- 파일 용량 : 파일 하나 당 10MB 미만
- 파일 확장자 : csv, xlsx, xls, txt 확장자만 업로드 가능
- NASCA DRM : 업로드 불가

Header

Field	Type	Required	Description	Value
Content-Type	string	optional	Type of content	application/json
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-openapi-token	string	required	OpenAPI Token information	ex) Bearer eyJXXX
x-generative-ai-user-email	string	optional	Portal user email	ex) kim.samsung@samsung.com

Request

Field	Type	Required	Description	Value
-	File[]	required	Multi part files	

Response

Field	Type	Description
-	list[object]	List of uploaded files information. See below.
[*].file_name	string	File name of uploaded file
[*].file_uuid	string	File UUID of uploaded file
[*].upload_failed_message	string	Reason for upload failure

Response 예시

```

1  [
2    {
3      "file_name": "test1.csv",
4      "file_uuid": "cee9737f-15e3-430b-9abe-cfaf4916f6b8",
5      "upload_failed_message": ""
6    }
7  ]

```

POST /openapi/interpreter/v1/download

이 API로 코드인터프리터가 생성한 이미지나 파일을 다운로드 할 수 있습니다. 코드 생성 및 실행 API의 응답에 포함된 파일 UUID를 확인하세요. 파일 UUID 정보를 포함하여 이 API를 호출하면 스토리지 서버에 업로드 되어있는 파일을 내려받을 수 있습니다.

노트

파일 다운로드 제약 사항

- 해당 사용자의 요청으로 생성된 파일만 다운로드 가능
- 유효기간 이내의 만료되지 않은 파일만 다운로드 가능.

Header

Field	Type	Required	Description	Value
Content-Type	string	optional	Type of content	application/json
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-openapi-token	string	required	OpenAPI Token information	ex) Bearer eyJXXX
x-generative-ai-user-email	string	optional	Portal user email	ex) kim.samsung@samsung.com

Request

Field	Type	Required	Description	Value
file_uuid	string	required	A File UUID to download	ex) cee9737f-15e3-430b-9abe-cfaf4916f6b8

Response

Field	Type	Description
-	File	Binary file (Content-Type: application/octet-stream)

공통 Response Code

Http Status Code	Description
200	Request and response okay. If stream is True, all HTTP statuses except system errors will respond with 200.
400	Requested with incorrect information
401	Authentication information is incorrect
403	Accessing without authorization
404	Request URL information is incorrect.
405	Requested Method not allowed
413	Too long request.
422	Required value missing
500	Unexpected Error
503	Failed to prepare environments. (No user machine

Http Status Code	Description
	available or No kernel available)
504	Code Execution Timeout

6.3 개발 가이드

Code Interpreter APIs 테스트를 위한 curl 샘플 코드입니다. Code Interpreter APIs를 사용 신청한 후, 아래 샘플 코드에서 url 및 인증 정보를 교체하여 테스트 할 수 있습니다.

LLM을 이용한 코드 생성과 실행

Curl 요청 예시

```

1  curl --request POST \
2    --url https://xxx/openapi/interpreter/v1/code \
3    -H 'Content-Type: application/json' \
4    -H 'x-fabrix-client: eyJ...' \
5    -H 'x-openapi-token: Bearer eyJ...' \
6    --data '{
7      "user_prompt": "calculate 1+2",
8      "chat_history": [],
9      "config": {
10        "plugin_system_prompt": {
11          "upload_file_info_prompt": "",
12          "business_domain_desc_prompt": "",
13          "code_generation_and_execution_result": ""
14        }
15      },
16      "data": [],
17      "files": []
18    }'
```

Python 요청 예시

```
1 import requests
2
3 YOUR_CLIENT_KEY = ""
4 YOUR_PASS_KEY = ""
5 ENDPOINT_URL = ""
6
7 headers = {
8     'Content-Type': 'application/json',
9     'x-fabrix-client': YOUR_CLIENT_KEY,
10    'x-openapi-token': YOUR_PASS_KEY}
11
12 body = {
13     "user_prompt": "calculate 1+2",
14     "chat_history": [],
15     "config": {
16         "plugin_system_prompt": {
17             "upload_file_info_prompt": "",
18             "business_domain_desc_prompt": "",
19             "code_generation_and_execution_result": ""
20         }
21     },
22     "data": [],
23     "files": []
24 }
25
26 endpoint_url = f'{ENDPOINT_URL}/openapi/interpreter/v1/code'
27 response = requests.post(endpoint_url, headers=headers, json=body)
28
29 if response.status_code == 200:
30     print("Success!!")
31     print(response.text)
32 else:
33     print(f"Fail: {response.status_code}")
34     print(response.text)
```

주의

※ Endpoint URL은 Open API 신청시 발급되는 URL로 적용하여 사용할 수 있습니다.

응답 예시

```

1  event: CODE_GENERATION
2  data: {
3    data:      "code": "````"
4  data: }
5
6  event: CODE_GENERATION
7  data: {
8    data:      "code": "python"
9  data: }
10
11 event: CODE_GENERATION
12 data: {
13 data:      "code": "\n"
14 data: }
15
16 event: CODE_GENERATION
17 data: {
18 data:      "code": "result"
19 data: }
20
21 event: CODE_GENERATION
22 data: {
23 data:      "code": " ="
24 data: }
25
26 event: CODE_GENERATION
27 data: {
28 data:      "code": " "
29 data: }
30
31 event: CODE_GENERATION
32 data: {
33 data:      "code": "1"

```

```
34 data: }
35
36 event: CODE_GENERATION
37 data: {
38   data:      "code": " +"
39 data: }
40
41 event: CODE_GENERATION
42 data: {
43   data:      "code": " "
44 data: }
45
46 event: CODE_GENERATION
47 data: {
48   data:      "code": "2"
49 data: }
50
51 event: CODE_GENERATION
52 data: {
53   data:      "code": "\n"
54 data: }
55
56 event: CODE_GENERATION
57 data: {
58   data:      "code": "print"
59 data: }
60
61 event: CODE_GENERATION
62 data: {
63   data:      "code": "(result"
64 data: }
65
66 event: CODE_GENERATION
67 data: {
68   data:      "code": ")\n"
69 data: }
70
71 event: CODE_GENERATION
72 data: {
73   data:      "code": "````"
```



```

74 data: }
75
76 event: CODE_GENERATION
77 data: {
78 data:      "code": ""
79 data: }
80
81 event: CODE_GENERATION
82 data: {
83 data:      "code": ""
84 data: }
85
86 event: CODE_GENERATION
87 data: {
88 data:      "code": "\n\n```\nexecution_result\n3\n```\n\n"
89 data: }
90
91 event: CODE_GENERATION
92 data: {
93 data:      "code": "The answer to 1+2 is 3."
94 data: }
95
96 event: CODE_GENERATION
97 data: {
98 data:      "code": ""
99 data: }
100
101 event: CODE_GENERATION
102 data: {
103 data:      "code": ""
104 data: }
105
106 event: CODE_RESULT
107 data: {
108 data:      "code": "result = 1 + 2\nprint(result)",
109 data:      "detail": null,
110 data:      "execution": {
111 data:          "display_data": [],
112 data:          "display_text": null,
113 data:          "elapsed_time": 0.4,

```

```

114 data:      "execute_result_text": "3\n",
115 data:      "is_truncated": false,
116 data:      "usermachine_expected_shutdown_time": 1730970396
117 data:    },
118 data:    "files": [],
119 data:    "final_response": "The answer to 1+2 is 3.",
120 data:    "intent": "code_generation_execution",
121 data:    "llm_response": "```\npython\nresult = 1 + 2\nprint(result)\n```",
122 data:    "llm_url": "http://meta-llama-3-70b.xxx/v1",
123 data:    "status": "SUCCESS"
124 data:  }

```

Python 코드 실행

Curl 요청 예시

```

1 curl --request POST \
2   --url https://xxx/openapi/interpreter/v1/execute \
3   -H 'Content-Type: application/json' \
4   -H 'x-fabrix-client: eyJ...' \
5   -H 'x-openapi-token: Bearer eyJ...' \
6   --data '{
7     "code": "print(1+2)"
8   }'

```

Python 요청 예시

```

1 import requests
2
3 YOUR_CLIENT_KEY = ""
4 YOUR_PASS_KEY = ""
5 ENDPOINT_URL = ""
6
7 headers = {
8     'Content-Type': 'application/json',
9     'x-fabrix-client': YOUR_CLIENT_KEY,
10    'x-openapi-token': YOUR_PASS_KEY}
11

```

```

12 body = {
13     "code": "print(1+2)"
14 }
15
16 endpoint_url = f'{ENDPOINT_URL}/openapi/interpreter/v1/execute'
17 response = requests.post(endpoint_url, headers=headers, json=body)
18
19 if response.status_code == 200:
20     print("Success!!")
21     print(response.text)
22 else:
23     print(f"Fail: {response.status_code}")
24     print(response.text)

```

주의

※ Endpoint URL은 Open API 신청시 발급되는 URL로 적용하여 사용할 수 있습니다.

응답 예시

```

1 {
2     "execute_result_text": "3\n",
3     "display_data": [],
4     "display_text": null,
5     "elapsed_time": 0.4,
6     "is_truncated": false,
7     "usermachine_expected_shutdown_time": 1730970514
8 }

```

파일 업로드

Curl 요청 예시

```

1 curl -X POST "https://xxx/openapi/interpreter/v1/upload" \
2     -H "Content-Type: multipart/form-data" \
3     -H "x-fabrix-client: eyJ..." \

```

```

4 -H "x-openapi-token: Bearer eyJ..." \
5 -F "files=@\"K:/My files/Test/file1.txt\""
```

Python 요청 예시

```

1 import requests
2
3 file_path = 'C:/xxx'
4 YOUR_CLIENT_KEY = ""
5 YOUR_PASS_KEY = ""
6 ENDPOINT_URL = ""
7
8 with open(file_path, 'rb') as file:
9     files = {'files': ('xxx', file)}
10    headers = {
11        'x-fabrix-client': YOUR_CLIENT_KEY,
12        'x-openapi-token': YOUR_PASS_KEY
13    }
14
15    response = requests.post(
16        f'{ENDPOINT_URL}/openapi/interpreter/v1/upload',
17        files=files,
18        headers=headers)
19
20 if response.status_code == 200:
21     print("Success!!")
22     print(response.text)
23 else:
24     print(f"Fail: {response.status_code}")
25     print(response.text)
```

응답 예시

```

1 [
2     {
3         "file_name": "file1.txt",
4         "file_uuid": "8a0336bf-ce21-4256-bf82-9447dda00da2",
5         "upload_failed_message": null
6     }
7 ]
```

생성된 파일 다운로드

Curl 요청 예시

```
1 curl -X GET
  "https://xxx/openapi/interpreter/v1/download?file_uuid=8a0336bf-ce21-4256-bf82-9447dda00da2" \
2   -H "Content-Type: application/json" \
3   -H "x-fabrix-client: eyJ..." \
4   -H "x-openapi-token: Bearer eyJ..." > newfile.txt
```

Python 요청 예시

```
1 import requests
2
3 YOUR_CLIENT_KEY = ""
4 YOUR_PASS_KEY = ""
5 ENDPOINT_URL = ""
6
7 headers = {
8     'Content-Type': 'application/json',
9     'x-fabrix-client': YOUR_CLIENT_KEY,
10    'x-openapi-token': YOUR_PASS_KEY
11 }
12
13 file_uuid = "xxx"
14 endpoint_url = f'{ENDPOINT_URL}/openapi/interpreter/v1/download?file_uuid={file_uuid}'
15
16 response = requests.get(endpoint_url, headers=headers)
17 file_Path = "C:/xxx"
18
19 if response.status_code == 200:
20     with open(file_Path, 'wb') as file:
21         file.write(response.content)
22     print('Success!!')
23 else:
24     print('Fail!!')
```

주의

※ Endpoint URL은 Open API 신청시 발급되는 URL로 적용하여 사용할 수 있습니다.

응답 예시

```

1 % Total      % Received % Xferd  Average Speed   Time    Time     Time  Current
2                               Dload  Upload   Total   Spent    Left   Speed
3 100    750      0    750      0      0   3490      0 --:--:-- --:--:-- --:--:--  3731
4 ... (생략)

```

6.4 FAQ

비동기 방식을 지원하나요?

동기 방식만 지원합니다. 시간이 오래 걸리는 파일 업로드/다운로드는 별도의 API로 제공합니다.

응답으로 실행 결과만 받을 수 있나요?

응답으로 실행 결과만 받을 수는 없습니다. 응답을 파싱해서 사용하면 됩니다.

업로드된 파일을 다운로드할 수 있는 유효 시간이 어떻게 되나요?

이미지 파일은 365일이고 그 외 파일은 30분입니다.

config의 프롬프트 예시를 알려주세요

config	Type	Required	Description	Example
upload_file_info_prompt	string	optional	업로드 파일에 관한 정보를 작성합니다. - 파일에 대한 전반적인 설명을 입력하세요. - 컬럼에 대한 의미를 설명하세요.	업로드 파일은 특정 연도, 도시의 인구 수를 나타내는 파일입니다. 파일에는 다음의 컬럼데이터가 포함될 수 있습니다. year: 1995부터 2024 까 지 의

config	Type	Required	Description	Example
			- 컬럼의 데이터에 대해서 설명하세요. - 데이터 타입 - 날짜 형식(yy-mm-dd, yy-mm-dd, hh:ss:ss 등) - 숫자의 범위 - 문자열 형식 - 샘플 데이터	숫자입니다. - cont : 2자리의 대륙 코드를 의미합니다. AS, OC, EU 값 중에 하나입니다. - city : 3자리의 도시 코드를 의미합니다. - pop : 단위 1000명의 인구를 의미합니다. 즉 100은 100000명을 의미합니다. - updated : yyyy-mm-dd
business_domain_desc_prompt	string	optional	업무 도메인 설명을 작성합니다. 업무에 특화된 용어나 약어를 설명하세요.	사용자는 다음과 같은 업무 용어와 약어를 사용하여 질의할 수 있습니다. - YMS : Yield Management System, 수율 관리 시스템 - BOM : Bill of Materials
code_generation_and_execution_result	string	optional	코드 생성 및 실행 결과 형태를 작성합니다. - 특정 라이브러리 지정을 지정하세요. - 데이터 활용 방식을 지정하세요. - 차트 종류나 스타일을 설명하세요.	사용자의 별도 언급이 없으면 2024년 데이터만 사용합니다. 2024년 데이터만 사용하는 경우 year 컬럼은 제외하고 보일 수 있도록 코드를 작성해주세요. 차트를 그릴 때 pyplot 라이브러리를 사용해주세요. 별도 언급이 없으면 라인 차트로 그려주세요. 차트의 line width는 1로 하고, line은 빨간색으로 그려주세요.

7.

Knowledge APIs

7.1 Knowledge APIs

Knowledge APIs는 Knowledge Asset의 chunk를 조회할 수 있는 API를 제공합니다.

Knowledge APIs 에서 제공하는 API는 다음과 같습니다.

POST /openapi/knowledge/v1/knowledge-retrieval/assets/{asset_id}/search

Knowledge Asset 내 검색된 Chunk 를 조회합니다.

7.2 API 명세

POST /openapi/knowledge/v1/knowledge-retrieval/assets/{asset_id}/search

Header

Field	Type	Required	Description	Value
Content-Type	string	optional	Type of content	application/json
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-openapi-token	string	required	OpenAPI Token information	ex) Bearer eyJXXX
x-generative-ai-user-email	string	optional	Portal user email	ex) kim.samsung@samsung.com

Header 예시

```

1 {
2   'x-fabrix-client': 'ey...',
3   'x-openapi-token': 'Bearer ey...'
4 }

```

Request Body

Field	Type	Description	Value
query	String	query	"origin of mammals"
filters	list	Key-value paris added as filter conditions	[{"system_id":"test_system_id"}]
additionalInformation	object	additional information for query reassembly	{ " c a t e g o r y " : "test_category" }

Request Body 예시

```

1 {
2   "query": "origin of mammals",
3   "filters": [],
4   "additionalInformation": {}
5 }

```

Response

Field	Type	Description
augmentedQuery	string	적용된 질의
results	list[Result]	Type <Result> 참고

Type <Result>

Field	Type	Description
id	Unique ID	Unique ID
systemId	string	System ID
systemCategory	string	System Category
hasPermission	bool	Whether the user has permission
content	string	Content of result
filename	string	Filename of the content

Field	Type	Description
title	string	Title of the content
docId	string	Document ID of the content
url	string	URL of the content
contentType	string	Content Type
rankScore	float	Search score of the content
sourceInfo	object	Source information of the content (Varies depending on data)
contentInfo	object	Information of the content (Varies depending on data)
subSystem	string	Subsystem of the content
catalogId	string	Category ID
useAllRespContent	bool	Whether the entire content is used
rank	DateTime	Rank of the content
executedQuery	string	Excuted query

| Response 예시

```

1  {
2    "augmentedQuery": "Tell me about the origin of mammals",
3    "results": [
4      {
5        "id": "Wv15g5gB6BEVr3lEsoc4",
6        "systemId": "default",
7        "systemCategory": "",
8        "hasPermission": true,
9        "content": "The synapsids, which include mammals and their extinct relatives, split
from the reptile and avian lineages during the Pennsylvanian Period (approximately 330 to 300
million years ago). The crown group mammals evolved from primitive mammals in the Early
Jurassic. The mammalian family tree is represented by the crown group clade.",
10       "filename": "mammals.txt",
11       "title": "mammals",
12       "docId": "3eeb447c-da29-4119-86d9-2e0ffe5751de",
13       "url": "",
14       "contentType": "CT01",
15       "rankScore": 0.9999759197235107,
16       "sourceInfo": {
17         "tags": "",
18         "relatedSystem": [],

```

```

19     "sourceNavi": "",
20     "additionalInfo": ""
21 },
22 "contentInfo": {
23     "anchor": "",
24     "page": "",
25     "subtitle": "",
26     "urlData": [],
27     "chunkSeq": 1,
28     "chunkPageSize": 1
29 },
30 "subSystem": "",
31 "catalogId": "01988379-af10-7eef-b77f-e4191f1a2c56",
32 "useAllRespContent": false,
33 "rank": 1,
34 "executedQuery": "origin of mammals"
35 }
36 ]
37 }

```

| 호출 예시 (python)

```

1  import requests
2
3  YOUR_CLIENT_KEY = ""
4  YOUR_PASS_KEY = ""
5  ENDPOINT_URL = ""
6
7  headers = {
8      "x-fabrix-client": YOUR_CLIENT_KEY,
9      "x-openapi-token": YOUR_PASS_KEY
10 }
11
12 asset_id = '01234567-xxxx-yyyy-...'
13 body = {
14     "query": "origin of mammals",
15     "filters": [],
16     "additionalInformation": {}
17 }

```

```

18
19 api_endpoint_url =
f"{ENDPOINT_URL}/openapi/knowledge/v1/knowledge-retrieval/assets/{asset_id}/search"
20
21 response = requests.post(api_endpoint_url, headers=headers, json=body)
22
23 # 응답 반환
24 print(response.json())

```

공통 Response Code

Http status code	Response code	Reason (response message)
200	R20000	success
200	R20001	need more parameters for api call
200	R20002	only support single-step task
200	R20003	error occurred during api call
200	R20004	api request timeout
200	R20005	exceeded token limit for generating response
200	R20006	only resource uri is provided by cognitive search
200	R20007	skipped answer generation
200	R20009	invalid RAG answer
400	R40000	client error # generic type for client error
400	R40001	request parameter validation error
400	R40002	unsupported model error
400	R40003	invalid plugin or api specification
400	R40004	exceeded token limit for request completion
400	R40005	health check api error
400	R40006	plug-in id not found
400	R40007	search engine not used
400	R40008	blocked by security filter
400	R40009	request timeout
400	R40010	too many LLM request
400	R40011	LLM request timeout
400	R40051	contents not found
400	R40052	contents validation error
400	R40053	llm_model is not exist or no authorization
400	R40054	contents or system_prompt is filtered by FilterServer
400	R40055	no authorization of catalog

Http status code	Response code	Reason (response message)
500	R50000	internal server error # generic type for server error
500	R50001	api specification parsing error
500	R50002	security filter service error
500	R50003	cognitive search service error
500	R50004	LLM request error
500	R50005	LLM response parsing error
500	R50006	Invalid result of cognitive search

7.3 FAQ

Request Body에 넣은 query가 실제 적용된 query와 다릅니다.

Knowledge의 리트리벌 설정에 따라 달라질 수 있습니다. 동의어 적용 여부 및 Query Format 설정에 따라 달라지며 실제 적용된 Query는 Response의 "augmentedQuery" 변수로 확인할 수 있습니다. 적용된 설정은 Store의 Knowledge 상세 정보 > "세부 설정 항목"탭에서 확인하세요.

API로 리트리벌 설정을 변경할 수 있나요?

배포한 사용자가 지정한 리트리벌 설정만 적용할 수 있습니다. 변경이 필요하다면 권한을 가진 사용자가 직접 수정하여 적용하여야 합니다.

8.

Rag Chat APIs

8.1 Rag Chat APIs Overview

FabriX RAG APIs는 Knowledge 및 문서 임베딩을 활용한 Chat서비스를 제공합니다. 개발자는 API에서 제공하는 메소드를 사용하여 재사용 코드를 최소화하고 FabriX RAG Chat 서비스를 이용하는 작업을 할 때 시간을 절약할 수 있습니다.

Rag Chat APIs 에서 제공하는 API는 다음과 같습니다.

GET /openapi/rag-chat/v1/models

API로 사용할 수 있는 대화 모델의 리스트를 조회합니다.

GET /openapi/rag-chat/v1/knowledges

사용 가능한 Knowledge 목록을 조회합니다.

POST /openapi/rag-chat/v1/file

임베딩 대상 문서 파일을 업로드 합니다.

POST /openapi/rag-chat/v1/embedding

임베딩 대상 문서 파일에 대해 임베딩을 요청합니다.

GET /openapi/rag-chat/v1/embedding

임베딩 처리 상태를 확인합니다.

POST /openapi/rag-chat/v1/messages

Knowledge나 사용자가 등록/처리한 Embedding 파일을 대화에 활용하여 모델 응답을 생성합니다.

8.2 API 명세

GET /openapi/rag-chat/v1/models

노트

사용자 포털에서 권한을 가진 모델(Text)을 API로 조회할 수 있습니다.

Text : 대화모델

Header

Field	Type	Required	Description	Value
Content-Type	string	optional	Type of content	application/json
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-openapi-token	string	required	OpenAPI Token information	ex) Bearer eyJXXX
x-generative-ai-user-email	string	optional	Portal user email	ex) kim.samsung@samsung.com

Response

Field	Type	Description	Value
modelId	String	Model Id	
name	list	Model Name	[{ "languageCode": "ko", "content": "모델명" }]
description	list	Model Description	[{ "languageCode": "ko", "content": "설명" }]

호출 예시 (python)

```

1  import requests
2
3  YOUR_CLIENT_KEY = ""
4  YOUR_PASS_KEY = ""
5  ENDPOINT_URL = ""
6  YOUR_EMAIL = ""
7
8  headers = {
9      "x-fabrix-client": YOUR_CLIENT_KEY,
10     "x-openapi-token": YOUR_PASS_KEY,
11     "x-generative-ai-user-email": YOUR_EMAIL
12 }
13
14 api_endpoint_url = f"{ENDPOINT_URL}/openapi/rag-chat/v1/models"
15
16 response=requests.get(
17     api_endpoint_url,
18     headers=headers
19 )
20
21 # 응답 반환
22 print(response.json())

```

주의

※ Endpoint URL은 Open API 신청시 발급되는 URL로 적용하여 사용할 수 있습니다.

GET /openapi/rag-chat/v1/knowledges

노트

사용 가능한 Knowledge 목록을 조회합니다.

Header

Field	Type	Required	Description	Value
Content-Type	string	required	Type of content	application/json
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-openapi-token	string	required	OpenAPI Token information received from catalog connector	ex) Bearer eyJXXX
x-generative-ai-user-email	string	optional	Portal user email	ex) kim.samsung@samsung.com

Query Parameter

Field	Type	Required	Description	Value
keyword	string	optional	knowledge's keyword	ex) Harryporter
page	int	optional	page no	ex) 1
limit	int	optional	limit no	ex) 10

Response

Field	Type	Description
meta.totalItems	integer	total items
meta.itemCount	integer	item count
meta.itemPerPage	integer	item per page
meta.totalPages	integer	total pages
meta.currentPage	integer	current page
items.assetId	string	asset Id
items.category	string	asset category
items.type	string	asset type
items.subtype	string	asset sub type
items.attributes	object	asset attributes
items.providedType	string	asset provided type
items.privacy	string	asset privacy
items.name.languageCode	string	asset name language code
items.name.content	string	asset name content
items.description.languageCode	string	asset description language code
items.description.content	string	asset description content
items.tenantCode	string	asset tenant code
items.labId	string	asset lab id
items.labName	string	asset lab name
items.modifier	object	asset modifier info
items.modifiedAt	DateTime	asset modified datetime

Field	Type	Description
items.deployedAt	DateTime	asset deployed datetime
items.accessControl	string	asset access controll type
items.defaultAsset	boolean	default asset Y/N
items.chatUse	boolean	chat usable
items.labUse	boolean	lab usable
items.chatUsePermission	string	chat use permission
items.isBookmarked	boolean	asset bookmarked Y/N
items.certified	boolean	asset certified Y/N

| Response 예시

```

1  {
2    "meta": {
3      "totalItems": 5,
4      "itemCount": 1,
5      "itemsPerPage": 1,
6      "totalPages": 5,
7      "currentPage": 1
8    },
9    "items": [
10     {
11       "assetId": "0198c1aa-e18f-73e7-966b-7dc5a9668f97",
12       "category": "KNOWLEDGE",
13       "type": "KNOWLEDGE",
14       "subtype": "DEFAULT",
15       "attributes": {},
16       "providedType": "PRE_BUILT",
17       "privacy": "PUBLIC",
18       "name": [
19         {
20           "languageCode": "en",
21           "content": "헨젤과 그레텔"
22         },
23         {
24           "languageCode": "ko",
25           "content": "헨젤과 그레텔"
26         }
27       ],

```

```

28     "description": [
29         {
30             "languageCode": "en",
31             "content": "헨젤과 그레텔 소설"
32         },
33         {
34             "languageCode": "ko",
35             "content": "헨젤과 그레텔 소설"
36         }
37     ],
38     "tenantCode": "sds",
39     "labId": "01987ca2-7bd9-782a-95f6-248bd1e50385",
40     "labName": "My Lab",
41     "modifier": {
42         "id": "5b4c13b6-a36b-4027-ac49-52e7279af131",
43         "userName": "김삼성",
44         "userEmail": "kim.samsung@samsung.com"
45     },
46     "modifiedAt": "2025-08-19T18:30:58.463322+09:00",
47     "deployedAt": "2025-08-19T18:31:42.702721+09:00",
48     "accessControl": "OPEN",
49     "defaultAsset": false,
50     "chatUse": true,
51     "labUse": true,
52     "chatUsePermission": "GRANTED",
53     "isBookmarked": false,
54     "certified": false
55 }
56 ]
57 }

```

호출 예시 (python)

```

1  import requests
2
3  YOUR_CLIENT_KEY = ""
4  YOUR_PASS_KEY = ""
5  ENDPOINT_URL = ""
6  YOUR_EMAIL = ""
7

```

```

8  SEARCH_KEYWORD = ""
9
10 headers = {
11     "x-fabrix-client": YOUR_CLIENT_KEY,
12     "x-openapi-token": YOUR_PASS_KEY,
13     "x-generative-ai-user-email": YOUR_EMAIL
14 }
15
16 api_endpoint_url =
f"{ENDPOINT_URL}/openapi/rag-chat/v1/knowledges?keyword={SEARCH_KEYWORD}"
17
18 response=requests.get(
19     api_endpoint_url,
20     headers=headers
21 )
22
23 # 응답 반환
24 print(response.json())

```

POST /openapi/rag-chat/v1/file

노트

임베딩 대상 문서 파일을 업로드 합니다.

Header

Field	Type	Required	Description	Value
Content-Type	string	required	Type of content	application/json
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-openapi-token	string	required	OpenAPI Token information received from catalog connector	ex) Bearer eyJXXX
x-generative-ai-user-email	string	optional	Portal user email	ex) kim.samsung@samsung.com

Request Body

Field	Type	Required	Description	Value
file	file	required	Document File	

지원대상 : html, docx, xlsx, txt, pdf, json, csv, hwp

※ 부록 A. 지식 기반 대화 문서 형식을 참고하여, 지원가능한 파일 확인 필요

Response

Field	Type	Description	Value
fileUuid		File Uuid	
fileName	String	File Name	
fileSize	int	File Size	
fileStatusCode	String	File Status Code	
fileStatusMessage	String	File Status Message	
createdAt	Datetime	File Uploaded Datetime	
creator	String	File Uploader ID	

Response 예시

```

1 {
2   "createdAt": "2025-08-21T16:28:56.694738+09:00",
3   "creator": "b5bcab1f-051f-4519-b871-9cc827da06c6",
4   "fileUuid": "b55ca7ce-6216-491b-8109-a259c18e4d2b.txt",
5   "fileSize": 10742,
6   "fileName": "라퐁젤.txt",
7   "fileStatusCode": "SUCCESS",
8   "fileStatusMessage": null
9 }
```

호출 예시 (python)

```

1 import requests
2
3 YOUR_CLIENT_KEY = ""
4 YOUR_PASS_KEY = ""
5 ENDPOINT_URL = ""
6 YOUR_EMAIL = ""
```

```

7
8 headers = {
9     "x-fabrix-client": YOUR_CLIENT_KEY,
10    "x-openapi-token": YOUR_PASS_KEY,
11    "x-generative-ai-user-email": YOUR_EMAIL
12 }
13
14 file: UploadFile = open("./test.txt", "rb")
15
16 api_endpoint_url = f"{ENDPOINT_URL}/openapi/rag-chat/v1/file"
17
18 response=requests.post(
19     api_endpoint_url,
20     files=[("file", file)],
21     headers=headers
22 )
23
24 # Response Return
25 print(response.json())
26 EMBEDDING_FILE_UUID = response.json()['fileUuid']

```

주의

※ Endpoint URL은 Open API 신청시 발급되는 URL로 적용하여 사용할 수 있습니다.

POST /openapi/rag-chat/v1/embedding

노트

임베딩 대상 문서 파일에 대해 임베딩을 요청합니다.

Header

Field	Type	Required	Description	Value
Content-Type	string	optional	Type of content	application/json

Field	Type	Required	Description	Value
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-openapi-token	string	required	OpenAPI Token information	ex) Bearer eyJXXX
x-generative-ai-user-email	string	optional	Portal user email	ex) kim.samsung@samsung.com

Request Body

Field	Type	Required	Description	Value
fileUuid	String	required	Embedding File Uuid	

Request Body 예시

```

1 {
2   "fileUuid": "b55ca7ce-6216-491b-8109-a259c18e4d2b.txt"
3 }
```

Response

Field	Type	Description	Value
fileUuid	String	File Uuid	
fileName	String	File Name	
fileSize	Int	File Size	
delYn	Boolean	Is Deleted	
status	String	Embedding Status	
message	String	Embedding Message	
errorType	String	Embedding Error Type	
errorMessage	String	Embedding Error Message	

Response 예시

```

1 {
2   "fileUuid": "b55ca7ce-6216-491b-8109-a259c18e4d2b.txt",
3   "status": "QUEUED",
4   "delYn": false,
5   "fileName": "라퐁젤.txt",
6   "fileSize": 10742,
```

```

7     "message": null,
8     "errorType": null,
9     "errorMessage": null
10  }

```

호출 예시 (python)

```

1  import requests
2
3  YOUR_CLIENT_KEY = ""
4  YOUR_PASS_KEY = ""
5  ENDPOINT_URL = ""
6  YOUR_EMAIL = ""
7
8  EMBEDDING_FILE_UUID = "Get fileUuid from File Upload API"
9
10 headers = {
11     "x-fabrix-client": YOUR_CLIENT_KEY,
12     "x-openapi-token": YOUR_PASS_KEY,
13     "x-generative-ai-user-email": YOUR_EMAIL
14 }
15
16 body = {
17     "fileUuid": f"{EMBEDDING_FILE_UUID}",
18 }
19
20 api_endpoint_url = f"{ENDPOINT_URL}/openapi/rag-chat/v1/embedding"
21
22 response=requests.post(
23     api_endpoint_url,
24     headers=headers,
25     json=body
26 )
27
28 # Response Return
29 print(response.json())

```


주의

※ Endpoint URL은 Open API 신청시 발급되는 URL로 적용하여 사용할 수 있습니다.

GET /openapi/rag-chat/v1/embedding

노트

임베딩 처리 상태를 확인합니다.

Header

Field	Type	Required	Description	Value
Content-Type	string	optional	Type of content	application/json
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-openapi-token	string	required	OpenAPI Token information	ex) Bearer eyJXXX
x-generative-ai-user-email	string	optional	Portal user email	ex)kim.samsung@samsung.com

Param

Field	Type	Required	Description	Value
fileUuid	String	required	Embedding File Uuid	

Response

Field	Type	Description	Value
fileUuid	String	File Uuid	
fileName	String	File Name	
fileSize	Int	File Size	
delYn	Boolean	Is Deleted	
status	String	Embedding Status	
message	String	Embedding Message	
errorType	String	Embedding Error Type	
errorMessage	String	Embedding Error Message	

Response 예시

```

1  {
2      "fileUuid": "b55ca7ce-6216-491b-8109-a259c18e4d2b.txt",
3      "status": "COMPLETED",
4      "delYn": false,
5      "fileName": "라퐁젤.txt",
6      "fileSize": 10742,
7      "message": null,
8      "errorType": null,
9      "errorMessage": null
10 }

```

호출 예시 (python)

```

1  import requests
2  import time
3  from time import sleep
4
5  YOUR_CLIENT_KEY = ""
6  YOUR_PASS_KEY = ""
7  ENDPOINT_URL = ""
8  YOUR_EMAIL = ""
9
10 EMBEDDING_FILE_UUID = "Get fileUuid from File Upload API"
11
12 headers = {
13     "x-fabrix-client": YOUR_CLIENT_KEY,
14     "x-openapi-token": YOUR_PASS_KEY,
15     "x-generative-ai-user-email": YOUR_EMAIL
16 }
17
18 api_endpoint_url =
19 f"{ENDPOINT_URL}/openapi/rag-chat/v1/embedding?fileUuid={EMBEDDING_FILE_UUID}"
20
21 start_time = time.time()
22 embedding_status = ""
23 while(embedding_status != "COMPLETED"):
24     if time.time() - start_time >= 60:
25         print("Embedding Time Out!!!!!!!!!!!!")

```

```

25         break
26
27     response=requests.get(
28         api_endpoint_url,
29         headers=headers
30     )
31
32     response.raise_for_status()
33     # Response Return
34     print(response.json())
35     embedding_status = response.json()['status']
36     if(embedding_status == 'FAILED'):
37         break
38     sleep(1)
39
40 print("Embedding Complete")

```

주의

※ Endpoint URL은 Open API 신청시 발급되는 URL로 적용하여 사용할 수 있습니다.

POST /openapi/rag-chat/v1/messages

노트

Knowledge나 사용자가 등록/처리한 Embedding 파일을 대화에 활용하여 모델 응답을 생성합니다.

Header

Field	Type	Required	Description	Value
Content-Type	string	optional	Type of content	application/json
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-openapi-token	string	required	OpenAPI Token information	ex) Bearer eyJXXX
x-generative-ai-user-	string	optional	Portal user email	ex)kim.samsung@sa

Field	Type	Required	Description	Value
email				msung.com

Request

Field	Type	Required	Description	Value
modelIds	list[string]	required	model id list	ex) ["0196f1fc-2858-70a9-a232-74dbd db971d0"]
contents	list[string]	required	contents	ex) ["Hello"]
isStream	boolean	optional	Stream or not (default: True)	ex) False / True
llmConfig	object	optional	LLM Config info	llmConfig Properties 참고
systemPrompt	string	optional	System prompt for LLM to use	ex) 사용자의 질문에 친절하게 대답해
knowledgeAssetId	string	optional	Knowledge ID	
docId	string	optional	File ID of Knowledge	
embedFileUuid	string	optional	Embedded File ID	
isExecuteRecommendQueries	boolean	optional	Respond to a recommended queries	

llmConfig Properties

Name	Description
temperature	모델의 확률 분포를 조절합니다. 값이 1보다 크면 확률 분포가 더욱 고르게 퍼지게 되고, 값이 1보다 작으면 확률 분포가 더욱 뾰족하게 집중됩니다. (0<temperature<1)
repetition_penalty	반복된 단어의 출현 확률을 조절합니다. 값이 1보다 크면 반복된 단어가 나타날 확률이 줄어들게 되고, 값이 1보다 작으면 반복된 단어가 나타날 확률이 증가하게 됩니다.
decoder_input_details	디코더의 입력 형태를 결정합니다. use_cache를 True로 설정하고 past_key_values를 None으로 설정하면, 디코더는 이전 타임스텝의 캐시값을 이용하여 다음 타임스텝의 입력을 생성합니다. use_cache를 False로 설정하고 past_key_values를 None으로 설정하면, 디코더는 이전 타임스텝의 마지막 히든 상태를 이용하여 다음 타임스텝의 입력을 생성합니다. past_key_values를 임의의 값으로 설정하면, 디코더는 해당 값을 이용하여 다음 타임스텝의 입력을 생성합니다.
seed	모델의 난수 발생기를 시드하는 데 사용됩니다. 동일한 시드를 사용하면 모델은 항상 동일한 순서로 출력을 생성합니다.
top_k	모델이 고려할 후보 단어의 수를 제한합니다. 큰 값을 사용하면

Name	Description
	더 다양한 단어를 선택할 수 있지만, 계산 시간이 오래 걸릴 수 있습니다. ($1 \leq \text{top_k}$)
top_p	확률 분포의 상위 p%를 고려하여 단어를 선택합니다. 이렇게 하면 선택된 단어들이 확률 분포를 고르게 대표하도록 하여 다양성을 높일 수 있습니다. (top_p must be > 0.0 and < 1.0)
max_new_tokens	생성된 텍스트의 최대 길이를 결정합니다. 예를 들어, max_new_tokens=10으로 설정하면 모델은 최대 10개의 토큰을 생성할 수 있습니다.

| Request Body 예시(Knowledge기반 대화)

```

1 {
2   "modelIds":["01983058-7a23-7a9b-955a-5db64a5dcafa"],
3   "contents":["이 이야기의 주인공은?"],
4   "isStream":false,
5   "llmConfig": {},
6   "systemPrompt":"",
7   "knowledgeAssetId": "0198c1aa-e18f-73e7-966b-7dc5a9668f97"
8 }
```

| Request Body 예시(Embedding 기반 대화)

```

1 {
2   "modelIds":["01989bcf-600f-770d-8579-ee46314c80b9"],
3   "contents":["이 파일 내용이 뭐야?"],
4   "isStream":false,
5   "llmConfig": {},
6   "systemPrompt":"",
7   "embedFileUuid": "b55ca7ce-6216-491b-8109-a259c18e4d2b.txt"
8 }
```

| Response

주의

※ Stream 으로 응답을 받을 경우 Field 명이 카멜케이스가 아닌 스네이크케이스로 표시됩니다.

예) modelType -> model_type

Field	Type	Description	
id	integer	openapi message Id	DEPRECATED
parentMessageId	string	openapi message parent id	DEPRECATED
parentMessageCreatedAt	DateTime	parentMessage creation time	DEPRECATED
chatId	integer	chat Id	DEPRECATED
userId	integer	user Id	
modelId	string	LLM ModelId	DEPRECATED
modelType	string	LLM ModelType	
content	string	Answer	
createdAt	DateTime	Message creation time	DEPRECATED
completionToken	integer	Answer Token Usage	DEPRECATED
promptToken	integer	Question Token Usage	DEPRECATED
truncated	string	Whether the question is truncated or not	
finishReason	string	FinishReason (stop : normal, length : Answer length exceeded)	
filterBlockReason.ko	string	Filter block reason(ko)	
filterBlockReason.en	string	Filter block reason(en)	
filterBlockReason.policy_id	string	Filter block reason(policy_id)	
filterBlockReason.message	string	Filter block reason(message)	
filterBlockReason.result_code	string	Filter block reason(result_code)	
filterBlockReason.filter_log_id	string	Filter block reason(filter_log_id)	
status	string	Status	
responseCode	string	ResponseCode	
plugins	list[str]	Requested plugin	
references	list[str]	References used while generating the answer	DEPRECATED
catalogs	list[int]	Requested Catalog	DEPRECATED
eventStatus	string	Event status	
eventData	string	EventData	
filterValidation	boolean	Filter success or not	DEPRECATED
successYn	boolean	Success or failure	DEPRECATED
reasoning_content	string	추론 과정	
processing_content	list	스트리밍 일 경우 중간 처리 과정	

Field	Type	Description	
content_references	list	References used while generating the answer	
recommendQueries	list[str]	Recommended queries	

| Response 예시(Knowledge 기반 대화)

(isStream=false)

```

1  {
2    "userId": "b5bcab1f-051f-4519-b871-9cc827da06c6",
3    "modelType": "Gauss2",
4    "content": "이 이야기의 주인공은 헨젤과 그레텔입니다.",
5    "reasoningContent": null,
6    "processingContent": [],
7    "contentReferences": [
8      {
9        "plugin": "RAG",
10       "answer": "",
11       "references": [
12         {
13           "title": "헨젤과 그레텔",
14           "content": "헨젤과 그레텔 : \n위한 무언가가 있어.\n" 그레텔은 헨젤이
돌을 주머니에 넣었기 때문에 빵을 앞치마 밑으로 가져갔다. 그런 다음 그들은 모두 숲으로 가는 길로
함께 출발했다. 그들이 조금 걸었을때, 헨젤은 여전히 서서 집을 되돌아보았고 책략은 그가 반복했다.
그의 아버지는 그를 보고 말했다. \n헨젤, 너는 거기에서 뭘 봐? 왜, 너는 항상 뒤에 남아있어?
주의를 기울이고 발판을 잃지마.\n \n오! 아버지.\n" 헨젤이 말했다. \n나는 지붕에 앉아 흰 새끼
고양이를 보고, 나에게 작별 인사를 하고있어.\n" 여자는 다음과 같이 외쳤다. \n네가 무슨 당나귀야!
그것은 너의 새끼 고양이가 아니야. 굴뚝에서 아침해가 빛나고있어.\n" 헨젤은 그의 새끼 고양이를
되돌아보지 않았지만, 흰 자갈 중 1개를 항상 그의 주머니에서 길가로 떨어뜨렸다.\n그들이 숲의
중턱에 도착했을때 아버지는 이렇게 말했다. \n자, 아이들은 가서 나무를 많이 가져오고 나는
식지않을 수도 있는 불을 밝힐꺼야.\n" 헨젤과 그레텔은 작은 언덕만한 크기의 더미를 만들때까지
떨감을 넣었다. 떨감은 불이 붙었고 화염이 뛰었을때 여자는 이렇게 말했다. \n이제 불에 누워서,
아이들은 쉬고있어. 우리는 나무를 자르기 위해 숲으로 들어갈꺼야. 우리가 마무리하면 다시 와서
너를 데리고올꺼야.\n" 헨젤과 그레텔은 불 옆에 앉았고 정오에 작은 빵을 먹었다. 그들은 도끼를 치는
소리를 들었기 때문에 아버지가 아주 가까이에 있다고 생각했다. 그들이 도끼 소리를 들은 것은
아니었지만, 그가 죽은 나무에 묶여있었고 그것은 바람때문에 날아갔다. 그리고 그들이 오랫동안
앉아있을때 그들의 눈은 피로로 감겼고 그들은 잠들어있었다. 그들이 마침내 깨어났을때 어둠은
절정이었다. 그레텔은 울기 시작했고 이렇게 말했다. \n우리는 나무에서 어떻게 나와?\n" 그러나
헨젤은 그녀를 위로했다. \n잠깐 기다려.\n" 그가 말했다. \n달이 뜰때까지 우리는 방법을 충분히

```

찾을꺼야\" 그리고 보름달이 뜨자 그는 손으로 여동생을 데리고 자갈을 따라갔다. 자갈은 조약돌처럼
 빛났고 그 길을 보여주었다. 그들은 밤새 걸었고, 새벽에 다시 아버지의 집에 도착했다. 그들은 문을
 두드렸고, 여자가 문을 열때 그녀는 외쳤다. \"당신은 장난꾸러기 아이야. 당신은 한번 나무에서
 잔적이 있어! 우리는 당신이 결코 돌아오지 않을 것이라고 생각했어.\" 그러나 아버지는 자신의
 양심이 자녀를 내버려두고 그를 비난했기 때문에 기뻐했다.\n그 후 얼마 지나지 않아 그 땅에 큰
 기근이 있었고, 아이들은 어느날 밤 그들의 어머니가 그들의 아버지에게 이렇게 말한것을 들었다.
 \"모든것은 한번 더 먹어. 우리는 집에서 한 덩어리만 먹었어. 아이들은 반드시 없애야해. 이번에는
 그들이 나무로 더 깊이 들어가서 다시 길을 찾지 못하게 될꺼야. 우리 자신을 구할 다른 방법은
 없어.\" 남자의 마음은 그를 강타했고 그는 이렇게 생각했다. \"마지막 물기를 아이와 나누는 것이
 나야!\" 그러나 그의 아내는 그의 주장을 듣지 않았고 아무것도 하지 않았다. 그녀는 그를 꾸짖고
 비난했다.\n\n그러나 아이들은 깨어있었고, 대화를 들었다. 노인들이 잠들어있을때 헨젤은 일어나서
 처음으로 했던 것처럼 자갈을 다시 들어야했다. 그러나 여자는 문을 막았고, 헨젤은 나가지 못했다.\"

15 "link": ""

16 },

17 {

18 "title": "헨젤과 그레텔",

19 "content": "헨젤과 그레텔 : \n옛날, 큰 숲의 외곽에 아내와 2명의 자녀와
 함께 가난한 나무꾼이 살았다. 그 소년은 헨젤이었고 소녀는 그레텔이었다. 그는 살기 힘들었고, 한
 때 그 땅에 엄청난 가뭄이 있었을 때 심지어 그들에게 일용할 양식을 주지 못했다. 어느 날 밤,
 침대에 누워서 걱정으로 가득한 한숨을 쉬면서 그는 자신의 아내에게 말했다. \"우리는 어떻게 될까?
 우리는 가난한 아이를 어떻게 도울 것일까? 이제 우리는 더 이상 아무것도 하지 못해.\" \"나는
 남편이 무엇을 말할까? 그 여자가 대답했다. \"내일 아침에 우리는 아이들을 나무의 가장 두꺼운
 부분으로 데리고 나갈거야. 거기에서 우리는 그들에게 불을 붙이고 각각 빵을 주어야 해. 그러면
 우리는 일을 계속하고 그들을 내버려 둘 거야. 그들은 집으로 가는 길을 찾지 못할 것이며, 따라서
 우리는 그렇게 할 거야.\" \"아니, 아내.\" 그녀의 남편이 말했다. \"나는 그렇게 하지 않을 건데, 내
 아이들을 나무에 홀로 두는 것이 내 마음 속에서 어떻게 찾을 수 있어? 야생 동물이 곧 와서 아이들을
 조각으로 찢을 거야.\" \"오! 너는 바보야.\" 그녀가 말했다. \"우리 모두 4명이 굶어 죽어야 해.
 그러면 우리는 관에 가서 널빤지를 타기만 하면 돼.\" 그녀는 동의할 때까지 그에게 평화를 남겨두지
 않았다. \"그러나 나는 가난한 아이들을 유감스럽게 생각해.\" 남편이 말을 덧붙였다.\n아이들도
 굶주림 때문에 잠을 자지 못했고 계모가 그들의 아버지에게 한 말을 들었다. 그레텔은 심하게 울고
 헨젤에게 말했다. \"이제는 우리와 함께 할 거야.\" \"아니, 아니, 그레텔.\" 헨젤이 말했다. \"너
 자신을 두려워 하지 마. 나는 두려움 없이 탈출할 길을 찾을 수 있을 거야.\" 그리고 노인들이 잠든
 채로 일어나서, 작은 외투에 미끄러져 들어갔다가 뒷문을 열고 밖으로 훔쳤다. 달은 분명히 비치고
 있었고, 집 앞에 누워있는 흰 자갈은 은처럼 반짝였다. 헨젤은 아래고 구부리고 그의 주머니에 자신이
 쑤셔 넣을 수 있을 만큼 많은 것을 채웠다. 그 다음 그는 돌아가서 그레텔에게 말했다. \"사랑하는
 그레텔이여, 위로하고 잠자리에 들어. 하나님은 우리를 버리지 않을 거야.\" 그리고 그는 침대에서
 다시 누웠다.\"

20 "link": ""

21 }

22],


```

23     "augmented_standalone_queries": "이 이야기의 주인공은 누구인가요?"
24   }
25 ],
26 "truncated": false,
27 "finishReason": null,
28 "filterBlockReason": {
29   "ko": null,
30   "en": null,
31   "policy_id": null,
32   "message": null,
33   "result_code": "FR-201",
34   "filter_log_id": null
35 },
36 "status": "SUCCESS",
37 "responseCode": "R20000",
38 "plugins": [
39   "RAG"
40 ],
41 "orchestratorType": null,
42 "parentMessageCreatedAt": "2025-08-21T16:22:40.056822+09:00",
43 "references": [
44   {
45     "type": "search",
46     "title": "헨젤과 그레텔",
47     "url": "",
48     "contentType": "CT01",
49     "sourceNavi": "",
50     "sourceInfo": {
51       "sourceNavi": "",
52       "tags": "",
53       "relatedSystem": []
54     },
55     "rank": 1,
56     "rankScore": 0.006541867274791002,
57     "content": "헨젤과 그레텔 : \n위한 무언가가 있어.\n 그레텔은 헨젤이 돌을
주머니에 넣었기 때문에 빵을 앞치마 밑으로 가져갔다. 그런 다음 그들은 모두 숲으로 가는 길로 함께
출발했다. 그들이 조금 걸었을때, 헨젤은 여전히 서서 집을 되돌아보았고 책략은 그가 반복했다. 그의
아버지는 그를 보고 말했다. \n헨젤, 너는 거기에서 뭘 봐? 왜, 너는 항상 뒤에 남아있어? 주의를
기울이고 발판을 잃지마.\n \n오! 아버지.\n 헨젤이 말했다. \n나는 지붕에 앉아 흰 새끼 고양이를
보고, 나에게 작별 인사를 하고있어.\n 여자는 다음과 같이 외쳤다. \n네가 무슨 당나귀야! 그것은

```

너의 새끼 고양이가 아니야. 굴뚝에서 아침해가 빛나고있어.\" 헨젤은 그의 새끼 고양이를 되돌아보지 않았지만, 흰 자갈 중 1개를 항상 그의 주머니에서 길가로 떨어뜨렸다.\n그들이 숲의 중턱에 도착했을때 아버지는 이렇게 말했다. \"자, 아이들은 가서 나무를 많이 가져오고 나는 식지않을 수도 있는 불을 밝힐꺼야.\" 헨젤과 그레텔은 작은 언덕만한 크기의 더미를 만들때까지 땀감을 넣었다. 땀감은 불이 붙었고 화염이 뛰었을때 여자는 이렇게 말했다. \"이제 불에 누워서, 아이들은 쉬고있어. 우리는 나무를 자르기 위해 숲으로 들어갈꺼야. 우리가 마무리하면 다시 와서 너를 데리고올꺼야.\" 헨젤과 그레텔은 불 옆에 앉았고 정오에 작은 빵을 먹었다. 그들은 도끼를 치는 소리를 들었기 때문에 아버지가 아주 가까이에 있다고 생각했다. 그들이 도끼 소리를 들은 것은 아니었지만, 그가 죽은 나무에 묶여있었고 그것은 바람때문에 날아갔다. 그리고 그들이 오랫동안 앉아있을때 그들의 눈은 피로로 감겼고 그들은 잠들어있었다. 그들이 마침내 깨어났을때 어둠은 절정이었다. 그레텔은 울기 시작했고 이렇게 말했다. \"우리는 나무에서 어떻게 나와?\" 그러나 헨젤은 그녀를 위로했다. \"잠깐 기다려.\" 그가 말했다. \"달이 뜰때까지 우리는 방법을 충분히 찾을꺼야\" 그리고 보름달이 뜨자 그는 손으로 여동생을 데리고 자갈을 따라갔다. 자갈은 조약돌처럼 빛났고 그 길을 보여주었다. 그들은 밤새 걸었고, 새벽에 다시 아버지의 집에 도착했다. 그들은 문을 두드렸고, 여자가 문을 열때 그녀는 외쳤다. \"당신은 장난꾸러기 아이야. 당신은 한번 나무에서 잔적이 있어! 우리는 당신이 결코 돌아오지 않을 것이라고 생각했어.\" 그러나 아버지는 자신의 양심이 자녀를 내버려두고 그를 비난했기 때문에 기뻐했다.\n그 후 얼마 지나지 않아 그 땅에 큰 기근이 있었고, 아이들은 어느날 밤 그들의 어머니가 그들의 아버지에게 이렇게 말한것을 들었다. \"모든것은 한번 더 먹어. 우리는 집에서 한 덩어리만 먹었어. 아이들은 반드시 없애야해. 이번에는 그들이 나무로 더 깊이 들어가서 다시 길을 찾지 못하게 될꺼야. 우리 자신을 구할 다른 방법은 없어.\" 남자의 마음은 그를 강타했고 그는 이렇게 생각했다. \"마지막 물기를 아이와 나누는 것이 나야!\" 그러나 그의 아내는 그의 주장을 듣지 않았고 아무것도 하지 않았다. 그녀는 그를 꾸짖고 비난했다.\n\n그러나 아이들은 깨어있었고, 대화를 들었다. 노인들이 잠들어있을때 헨젤은 일어나서 처음으로 했던 것처럼 자갈을 다시 들어야했다. 그러나 여자는 문을 막았고, 헨젤은 나가지 못했다.\",

```
58         "catalogId": "0198a2df-8a28-745e-bfcf-43fd8da052f2"
```

```
59     }
```

```
60 ],
```

```
61     "actions": [
```

```
62         {
```

```
63             "type": "RAG",
```

```
64             "plugin": "RAG",
```

```
65             "api": "RAG",
```

```
66             "params": null,
```

```
67             "responseType": null,
```

```
68             "method": null,
```

```
69             "httpStatus": 200,
```

```
70             "response": "{\"augmentedQuery\": \"이 이야기의 주인공은 누구인가요?\", \"hydeQue
```

```
ry\": \"\", \"results\": [{\"id\": \"B4rfopgB6SBv8YzkjQfS\", \"systemId\": \"default\", \"systemCatego
```

```
ry\": \"\", \"hasPermission\": true, \"content\": \"헨젤과 그레텔 : \n\n위한 무언가가 있어.\n\n\"
```

```
그레텔은 헨젤이 돌을 주머니에 넣었기 때문에 빵을 앞치마 밑으로 가져갔다. 그런 다음 그들은 모두
```

숲으로 가는 길로 함께 출발했다. 그들이 조금 걸었을때, 헨젤은 여전히 서서 집을 되돌아보았고
 책략은 그가 반복했다. 그의 아버지는 그를 보고 말했다. \\\\"헨젤, 너는 거기에서 뭘 봐? 왜, 너는
 항상 뒤에 남아있어? 주의를 기울이고 발판을 잃지마.\\\" \\\\"오! 아버지.\\\" 헨젤이 말했다.
 \\\\"나는 지붕에 앉아 흰 새끼 고양이를 보고, 나에게 작별 인사를 하고있어.\\\" 여자는 다음과 같이
 외쳤다. \\\\"네가 무슨 당나귀야! 그것은 너의 새끼 고양이가 아니야. 굴뚝에서 아침해가
 빛나고있어.\\\" 헨젤은 그의 새끼 고양이를 되돌아보지 않았지만, 흰 자갈 중 1개를 항상 그의
 주머니에서 길가로 떨어뜨렸다.\\n그들이 숲의 중턱에 도착했을때 아버지는 이렇게 말했다. \\\\"자,
 아이들은 가서 나무를 많이 가져오고 나는 식지않을 수도 있는 불을 밝힐꺼야.\\\" 헨젤과 그레텔은
 작은 언덕만한 크기의 더미를 만들때까지 땀감을 넣었다. 땀감은 불이 붙었고 화염이 뛰었을때 여자는
 이렇게 말했다. \\\\"이제 불에 누워서, 아이들은 쉬고있어. 우리는 나무를 자르기 위해 숲으로
 들어갈꺼야. 우리가 마무리하면 다시 와서 너를 데리고올꺼야.\\\" 헨젤과 그레텔은 불 옆에 앉았고
 정오에 작은 빵을 먹었다. 그들은 도끼를 치는 소리를 들었기 때문에 아버지가 아주 가까이 있다고
 생각했다. 그들이 도끼 소리를 들은 것은 아니었지만, 그가 죽은 나무에 묶여있었고 그것은
 바람때문에 날아갔다. 그리고 그들이 오랫동안 앉아있을때 그들의 눈은 피로로 감겼고 그들은
 잠들어있었다. 그들이 마침내 깨어났을때 어둠은 절정이었다. 그레텔은 울기 시작했고 이렇게 말했다.
 \\\\"우리는 나무에서 어떻게 나와?\\\" 그러나 헨젤은 그녀를 위로했다. \\\\"잠깐 기다려.\\\" 그가
 말했다. \\\\"달이 뜰때까지 우리는 방법을 충분히 찾을꺼야\\\" 그리고 보름달이 뜨자 그는 손으로
 여동생을 데리고 자갈을 따라갔다. 자갈은 조약돌처럼 빛났고 그 길을 보여주었다. 그들은 밤새
 걸었고, 새벽에 다시 아버지의 집에 도착했다. 그들은 문을 두드렸고, 여자가 문을 열때 그녀는
 외쳤다. \\\\"당신은 장난꾸러기 아이야. 당신은 한번 나무에서 잔적이 있어! 우리는 당신이 결코
 돌아오지 않을 것이라고 생각했어.\\\" 그러나 아버지는 자신의 양심이 자녀를 내버려두고 그를
 비난했기 때문에 기뻐했다.\\n그 후 얼마 지나지 않아 그 땅에 큰 기근이 있었고, 아이들은 어느날 밤
 그들의 어머니가 그들의 아버지에게 이렇게 말한것을 들었다. \\\\"모든것은 한번 더 먹어. 우리는
 집에서 한 덩어리만 먹었어. 아이들은 반드시 없애야해. 이번에는 그들이 나무로 더 깊이 들어가서
 다시 길을 찾지 못하게 될꺼야. 우리 자신을 구할 다른 방법은 없어.\\\" 남자의 마음은 그를
 강타했고 그는 이렇게 생각했다. \\\\"마지막 물기를 아이와 나누는 것이 나아!\\\" 그러나 그의
 아내는 그의 주장을 듣지 않았고 아무것도 하지 않았다. 그녀는 그를 꾸짖고 비난했다.\\n\\n그러나
 아이들은 깨어있었고, 대화를 들었다. 노인들이 잠들어있을때 헨젤은 일어나서 처음으로 했던 것처럼
 자갈을 다시 들어야했다. 그러나 여자는 문을 막았고, 헨젤은 나가지
 못했다.\\\",\\\"filename\\\":\\\"헨젤과 그레텔.txt\\\",\\\"title\\\":\\\"헨젤과 그레텔\\\",\\\"docId\\\":\\\"f27d7246-e
 864-4d84-9798-a22f9b023333\\\",\\\"url\\\":\\\"\\\",\\\"contentType\\\":\\\"CT01\\\",\\\"rankScore\\\":0.006541867274
 791002,\\\"rerankScore\\\":\\\"\\\",\\\"sourceInfo\\\":{\\\"tags\\\":\\\"\\\",\\\"relatedSystem\\\":[],\\\"sourceNavi\\\":\\\"
 \\\",\\\"contentInfo\\\":{\\\"anchor\\\":\\\"\\\",\\\"page\\\":\\\"\\\",\\\"subtitle\\\":\\\"\\\",\\\"urlData\\\":[],\\\"chunkSeq\\
 :4,\\\"chunkPageSize\\\":13,\\\"subSystem\\\":\\\"\\\",\\\"catalogId\\\":\\\"0198a2df-8a28-745e-bfcf-43fd8da052f
 2\\\",\\\"useAllRespContent\\\":false,\\\"rank\\\":1,\\\"executedQuery\\\":\\\"이 이야기의 주인공은 누구인가요?
 \\\",{\\\"id\\\":\\\"CYrfopgB6SBv8YzkjQfS\\\",\\\"systemId\\\":\\\"default\\\",\\\"systemCategory\\\":\\\"\\\",\\\"hasPerm
 ission\\\":true,\\\"content\\\":\\\"헨젤과 그레텔 : \\n말한것을 들었다. \\\\"모든것은 한번 더 먹어.
 우리는 집에서 한 덩어리만 먹었어. 아이들은 반드시 없애야해. 이번에는 그들이 나무로 더 깊이
 들어가서 다시 길을 찾지 못하게 될꺼야. 우리 자신을 구할 다른 방법은 없어.\\\" 남자의 마음은
 그를 강타했고 그는 이렇게 생각했다. \\\\"마지막 물기를 아이와 나누는 것이 나아!\\\" 그러나 그의

아내는 그의 주장을 듣지 않았고 아무것도 하지 않았다. 그녀는 그를 꾸짖고 비난했다.\\n\\n그러나 아이들은 깨어있었고, 대화를 들었다. 노인들이 잠들어있을때 헨젤은 일어나서 처음으로 했던 것처럼 자갈을 다시 들어야했다. 그러나 여자는 문을 막았고, 헨젤은 나가지 못했다. 그러나 그는 여동생을 위로하고 이렇게 말했다. \\\"울지마, 그레텔. 평화롭게 잠자. 하나님이 우리를 도울꺼야.\\\"\\n이른 새벽에 여자가 와서 아이들을 깨웠다. 그들을 빵을 조금 받았지만 이전보다 훨씬 작았다. 헨젤은 나무로 가는 길에 그것을 주머니에 넣고, 몇 분마다 서서 부스러기를 땅에 떨어뜨렸다. \\\"헨젤, 너는 왜 멈추고 너를 찾고있어?\\\" 아버지가 말했다. \\\"나는 지붕에서 작별인사를 하고있는 비둘기를 보고있어.\\\" 헨젤이 대답했다. \\\"바보!\\\" 아내가 말했다. \\\"그것은 비둘기가 아니야. 그것은 굴뚝에서 반짝이는 아침 해야.\\\" 그러나 헨젤은 점차 모든 부스러기를 길에 던졌다. 그 여자는 아이들이 전에 살았던 것보다 더 멀리 숲속으로 가게 했다. 그런 다음 큰 불이 다시 켜졌고 어머니는 이렇게 말했다. \\\"그냥 저기 앉아, 아이들. 피곤하면 조금 자면 돼. 우리는 나무를 자르기 위해 숲에 들어가고 저녁에 다하면 다시와서 당신을 데려올꺼야.\\\" 정오에 그레텔은 헨젤과 빵을 나누어 썼다. 그리고 그들을 잠이 들었고 저녁은 지나갔다. 그들은 어둠이 최고조가 될때까지 깨어있지 않았고, 헨젤은 여동생을 위로하고 이렇게 말했다. \\\"그레텔, 달이 뜰때까지 기다려. 그러면 내가 길을 따라 흩어져있는 빵 부스러기를 보게 될꺼야. 그들은 우리에게 집으로 되돌아가는 길을 보여줄꺼야.\\\" 달이 나타났을때 그들은 일어났지만, 아무 부스러기도 찾지 못했다. 숲과 들판을 날아다니는 수천 마리의 새들이 그들을 모두 데려왔기 때문이다. \\\"상관하지 마.\\\" 헨젤이 그레텔에게 말했다. \\\"우리는 탈출구를 찾을꺼야.\\\" 그러나 그들은 해내지 못했다. 그들은 밤새도록, 그리고 아침부터 저녁까지 방랑했지만 나무에서 길을 찾지 못했다. 그들은 배가 고팠다. 그들은 먹을 것이\\n 없었지만 땅에서 자라는 열매 몇 개가 있었다. 그리고 마침내 그들은 너무 피곤해서 나무 아래에서 누워서 잠들었다.탈출구를 찾을꺼야.\\\" 그러나 그들은 해내지 못했다. 그들은 밤새도록, 그리고 아침부터 저녁까지 방랑했지만 나무에서 길을 찾지 못했다. 그들은 배가 고팠다. 그들은 먹을 것이 없었지만 땅에서 자라는 열매 몇 개가 있었다. 그리고 마침내 그들은 너무 피곤해서 나무 아래에서 누워서 잠들었다.셋째날 아침에 그들은 자신의 아버지 집을 떠난 후에 다시 돌아다니는 것에 대해 이야기했지만, 나무 속은 깊어졌을 뿐이다. 정오에

그들은\\\",\\\"filename\\\":\\\"헨젤과 그레텔.txt\\\",\\\"title\\\":\\\"헨젤과 그레텔\\\",\\\"docId\\\":\\\"f27d7246-e864-4d84-9798-a22f9b023333\\\",\\\"url\\\":\\\"\\\",\\\"contentType\\\":\\\"CT01\\\",\\\"rankScore\\\":0.00054055021610111,\\\"rerankScore\\\":\\\"\\\",\\\"sourceInfo\\\":{\\\"tags\\\":\\\"\\\",\\\"relatedSystem\\\":[],\\\"sourceNavi\\\":\\\"\\\",\\\"contentInfo\\\":{\\\"anchor\\\":\\\"\\\",\\\"page\\\":\\\"\\\",\\\"subtitle\\\":\\\"\\\",\\\"urlData\\\":[],\\\"chunkSeq\\\":6,\\\"chunkPageSize\\\":13},\\\"subSystem\\\":\\\"\\\",\\\"catalogId\\\":\\\"0198a2df-8a28-745e-bfcf-43fd8da052f2\\\",\\\"useAllRespContent\\\":false,\\\"rank\\\":2,\\\"executedQuery\\\":\\\"이 이야기의 주인공은 누구인가요?\\\"},{\\\"id\\\":\\\"BIrfopgB6SBv8YzkjQfS\\\",\\\"systemId\\\":\\\"default\\\",\\\"systemCategory\\\":\\\"\\\",\\\"hasPermission\\\":true,\\\"content\\\":\\\"헨젤과 그레텔 : \\n옛날, 큰 숲의 외곽에 아내와 2명의 자녀와 함께 가난한 나무꾼이 살았다. 그 소년은 헨젤이었고 소녀는 그레텔이었다. 그는 살기 힘겨웠고, 한 때 그 땅에 엄청난 가뭄이 있었을 때 심지어 그들에게 일용할 양식을 주지 못했다. 어느 날 밤, 침대에 누워서 걱정으로 가득한 한숨을 쉬면서 그는 자신의 아내에게 말했다. \\\"우리는 어떻게 될까? 우리는 가난한 아이를 어떻게 도울 것일까? 이제 우리는 더 이상 아무것도 하지 못해.\\\" \\\"나는 남편이 무엇을 말할까? 그 여자가 대답했다. \\\"내일 아침에 우리는 아이들을 나무의 가장 두꺼운 부분으로 데리고 나갈거야. 거기에서 우리는 그들에게 불을 붙이고 각각 빵을 주어야 해. 그러면 우리는 일을 계속하고 그들을 내버려 둘 거야. 그들은 집으로 가는 길을 찾지 못할 것이며, 따라서

우리는 그렇게 할 거야.\\" \\"아니, 아내.\\" 그녀의 남편이 말했다. \\"나는 그렇게 하지 않을 건데, 내 아이들을 나무에 홀로 두는 것이 내 마음 속에서 어떻게 찾을 수 있어? 야생 동물이 곧 와서 아이들을 조각으로 찢을 거야.\\" \\"오! 너는 바보야.\\" 그녀가 말했다. \\"우리 모두 4명이 굶어 죽어야 해. 그러면 우리는 관에 가서 널빤지를 타기만 하면 돼.\\" 그녀는 동의할 때까지 그에게 평화를 남겨두지 않았다. \\"그러나 나는 가난한 아이들을 유감스럽게 생각해.\\" 남편이 말을 덧붙였다.\n아이들도 굶주림 때문에 잠을 자지 못했고 계모가 그들의 아버지에게 한 말을 들었다. 그레텔은 심하게 울고 헨젤에게 말했다. \\"이제는 우리와 함께 할 거야.\\" \\"아니, 아니, 그레텔.\\" 헨젤이 말했다. \\"너 자신을 두려워 하지 마. 나는 두려움 없이 탈출할 길을 찾을 수 있을 거야.\\" 그리고 노인들이 잠든 채로 일어나서, 작은 외투에 미끄러져 들어갔다가 뒷문을 열고 밖으로 훔쳤다. 달은 분명히 비치고 있었고, 집 앞에 누워있는 흰 자갈은 은처럼 반짝였다. 헨젤은 아래고 구부리고 그의 주머니에 자신이 쑤셔 넣을 수 있을 만큼 많은 것을 채웠다. 그 다음 그는 돌아가서 그레텔에게 말했다. \\"사랑하는 그레텔이여, 위로하고 잠자리에 들어. 하나님은 우리를 버리지 않을 거야.\\" 그리고 그는 침대에서 다시 누웠다.\",\"filename\": \"헨젤과 그레텔.txt\", \"title\": \"헨젤과 그레텔\", \"docId\": \"f27d7246-e864-4d84-9798-a22f9b023333\", \"url\": \"\", \"contentType\": \"CT01\", \"rankScore\": 0.000332260417053476, \"rerankScore\": \"\", \"sourceInfo\": {\"tags\": \"\", \"relatedSystem\": [], \"sourceNavi\": \"\"}, \"contentInfo\": {\"anchor\": \"\", \"page\": \"\", \"subtitle\": \"\", \"urlData\": [], \"chunkSeq\": 1, \"chunkPageSize\": 13}, \"subSystem\": \"\", \"catalogId\": \"0198a2df-8a28-745e-bfcf-43fd8da052f2\", \"useAllRespContent\": false, \"rank\": 3, \"executedQuery\": \"이 이야기의 주인공은 누구인가요?\"}}\",

```

71         "properties": {
72             "is_multi_query": false,
73             "augmented_standalone_queries": [
74                 "이 이야기의 주인공은 누구인가요?"
75             ]
76         },
77         "answer": "Data information retrieved through 이 이야기의 주인공은
누구인가요?.",
78         "question": null
79     }
80 ],
81     "eventStatus": "CHUNK",
82     "eventData": ""
83 }
```

| Response 예시(Embedding 기반 대화)

(isStream=true)

```

1  data: {"user_id":"b5bcab1f-051f-4519-b871-9cc827da06c6","model_type":"Llama 3.3","content":",
", "reasoning_content":null, "processing_content":[], "content_references":[], "truncated":null, "fi
nish_reason":null, "filter_block_reason":null, "status":"SUCCESS", "response_code":null, "plugins":
[], "orchestrator_type":null, "parent_message_created_at":"2025-08-21T16:31:34.104164+09:00", "ref
erences":null, "actions":null, "event_status":"CHUNK", "event_data":""}

2  data:
3
4  data: {"user_id":"b5bcab1f-051f-4519-b871-9cc827da06c6","model_type":"Llama
3.3","content":", 왕", "reasoning_content":null, "processing_content":[], "content_references":[], "
truncated":null, "finish_reason":null, "filter_block_reason":{"ko":"Default", "en":"Default", "poli
cy_id":"34", "message":"The content was allowed by the filter", "result_code":"FR-201", "filter_lo
g_id":"1861"}, "status":"SUCCESS", "response_code":null, "plugins":[], "orchestrator_type":null, "pa
rent_message_created_at":"2025-08-21T16:31:34.104164+09:00", "references":null, "actions":null, "e
vent_status":"CHUNK", "event_data":""}

5  data:
6
7  data: {"user_id":"b5bcab1f-051f-4519-b871-9cc827da06c6","model_type":"Llama 3.3","content":",
자", "reasoning_content":null, "processing_content":[], "content_references":[], "truncated":null, "
finish_reason":null, "filter_block_reason":{"ko":"Default", "en":"Default", "policy_id":"34", "mess
age":"The content was allowed by the filter", "result_code":"FR-201", "filter_log_id":"1861"}, "st
atus":"SUCCESS", "response_code":null, "plugins":[], "orchestrator_type":null, "parent_message_crea
ted_at":"2025-08-21T16:31:34.104164+09:00", "references":null, "actions":null, "event_status":"CHU
NK", "event_data":""}

8  data:
9
10 data: {"user_id":"b5bcab1f-051f-4519-b871-9cc827da06c6","model_type":"Llama 3.3","content":",
님", "reasoning_content":null, "processing_content":[], "content_references":[], "truncated":null, "
finish_reason":null, "filter_block_reason":{"ko":"Default", "en":"Default", "policy_id":"34", "mess
age":"The content was allowed by the filter", "result_code":"FR-201", "filter_log_id":"1861"}, "st
atus":"SUCCESS", "response_code":null, "plugins":[], "orchestrator_type":null, "parent_message_crea
ted_at":"2025-08-21T16:31:34.104164+09:00", "references":null, "actions":null, "event_status":"CHU
NK", "event_data":""}

11 data:
12
13 data: {"user_id":"b5bcab1f-051f-4519-b871-9cc827da06c6","model_type":"Llama 3.3","content":",
과", "reasoning_content":null, "processing_content":[], "content_references":[], "truncated":null, "
finish_reason":null, "filter_block_reason":{"ko":"Default", "en":"Default", "policy_id":"34", "mess
age":"The content was allowed by the filter", "result_code":"FR-201", "filter_log_id":"1861"}, "st
atus":"SUCCESS", "response_code":null, "plugins":[], "orchestrator_type":null, "parent_message_crea
ted_at":"2025-08-21T16:31:34.104164+09:00", "references":null, "actions":null, "event_status":"CHU

```

```

NK", "event_data": ""}
14 data:
15
16 data: {"user_id": "b5bcab1f-051f-4519-b871-9cc827da06c6", "model_type": "Llama
3.3", "content": " 라", "reasoning_content": null, "processing_content": [], "content_references": [], "
truncated": null, "finish_reason": null, "filter_block_reason": {"ko": "Default", "en": "Default", "poli
cy_id": "34", "message": "The content was allowed by the filter", "result_code": "FR-201", "filter_lo
g_id": "1861"}, "status": "SUCCESS", "response_code": null, "plugins": [], "orchestrator_type": null, "pa
rent_message_created_at": "2025-08-21T16:31:34.104164+09:00", "references": null, "actions": null, "e
vent_status": "CHUNK", "event_data": ""}
17 data:
18
19 data: {"user_id": "b5bcab1f-051f-4519-b871-9cc827da06c6", "model_type": "Llama 3.3", "content": "
핀", "reasoning_content": null, "processing_content": [], "content_references": [], "truncated": null, "
finish_reason": null, "filter_block_reason": {"ko": "Default", "en": "Default", "policy_id": "34", "mess
age": "The content was allowed by the filter", "result_code": "FR-201", "filter_log_id": "1861"}, "st
atus": "SUCCESS", "response_code": null, "plugins": [], "orchestrator_type": null, "parent_message_crea
ted_at": "2025-08-21T16:31:34.104164+09:00", "references": null, "actions": null, "event_status": "CHU
NK", "event_data": ""}
20 data:
21
22 data: {"user_id": "b5bcab1f-051f-4519-b871-9cc827da06c6", "model_type": "Llama 3.3", "content": "
첼", "reasoning_content": null, "processing_content": [], "content_references": [], "truncated": null, "
finish_reason": null, "filter_block_reason": {"ko": "Default", "en": "Default", "policy_id": "34", "mess
age": "The content was allowed by the filter", "result_code": "FR-201", "filter_log_id": "1861"}, "st
atus": "SUCCESS", "response_code": null, "plugins": [], "orchestrator_type": null, "parent_message_crea
ted_at": "2025-08-21T16:31:34.104164+09:00", "references": null, "actions": null, "event_status": "CHU
NK", "event_data": ""}
23 data:
24
25 data: {"user_id": "b5bcab1f-051f-4519-b871-9cc827da06c6", "model_type": "Llama 3.3", "content": "
은", "reasoning_content": null, "processing_content": [], "content_references": [], "truncated": null, "
finish_reason": null, "filter_block_reason": {"ko": "Default", "en": "Default", "policy_id": "34", "mess
age": "The content was allowed by the filter", "result_code": "FR-201", "filter_log_id": "1861"}, "st
atus": "SUCCESS", "response_code": null, "plugins": [], "orchestrator_type": null, "parent_message_crea
ted_at": "2025-08-21T16:31:34.104164+09:00", "references": null, "actions": null, "event_status": "CHU
NK", "event_data": ""}
26 data:
27
28 data: {"user_id": "b5bcab1f-051f-4519-b871-9cc827da06c6", "model_type": "Llama

```

```
3.3", "content": " 행복", "reasoning_content": null, "processing_content": [], "content_references": [],
, "truncated": null, "finish_reason": null, "filter_block_reason": {"ko": "Default", "en": "Default", "policy_id": "34", "message": "The content was allowed by the filter", "result_code": "FR-201", "filter_log_id": "1861"}, "status": "SUCCESS", "response_code": null, "plugins": [], "orchestrator_type": null, "parent_message_created_at": "2025-08-21T16:31:34.104164+09:00", "references": null, "actions": null, "event_status": "CHUNK", "event_data": ""}
```

```
29 data:
```

```
30
```

```
31 data: {"user_id": "b5bcab1f-051f-4519-b871-9cc827da06c6", "model_type": "Llama 3.3", "content": "
하게", "reasoning_content": null, "processing_content": [], "content_references": [], "truncated": null
, "finish_reason": null, "filter_block_reason": {"ko": "Default", "en": "Default", "policy_id": "34", "message": "The content was allowed by the filter", "result_code": "FR-201", "filter_log_id": "1861"}, "status": "SUCCESS", "response_code": null, "plugins": [], "orchestrator_type": null, "parent_message_created_at": "2025-08-21T16:31:34.104164+09:00", "references": null, "actions": null, "event_status": "CHUNK", "event_data": ""}
```

```
32 data:
```

```
33
```

```
34 data: {"user_id": "b5bcab1f-051f-4519-b871-9cc827da06c6", "model_type": "Llama
3.3", "content": " 함께", "reasoning_content": null, "processing_content": [], "content_references": [],
, "truncated": null, "finish_reason": null, "filter_block_reason": {"ko": "Default", "en": "Default", "policy_id": "34", "message": "The content was allowed by the filter", "result_code": "FR-201", "filter_log_id": "1861"}, "status": "SUCCESS", "response_code": null, "plugins": [], "orchestrator_type": null, "parent_message_created_at": "2025-08-21T16:31:34.104164+09:00", "references": null, "actions": null, "event_status": "CHUNK", "event_data": ""}
```

```
35 data:
```

```
36
```

```
37 data: {"user_id": "b5bcab1f-051f-4519-b871-9cc827da06c6", "model_type": "Llama
3.3", "content": " 살", "reasoning_content": null, "processing_content": [], "content_references": [],
, "truncated": null, "finish_reason": null, "filter_block_reason": {"ko": "Default", "en": "Default", "policy_id": "34", "message": "The content was allowed by the filter", "result_code": "FR-201", "filter_log_id": "1861"}, "status": "SUCCESS", "response_code": null, "plugins": [], "orchestrator_type": null, "parent_message_created_at": "2025-08-21T16:31:34.104164+09:00", "references": null, "actions": null, "event_status": "CHUNK", "event_data": ""}
```

```
38 data:
```

```
39
```

```
40 data: {"user_id": "b5bcab1f-051f-4519-b871-9cc827da06c6", "model_type": "Llama 3.3", "content": "
계", "reasoning_content": null, "processing_content": [], "content_references": [], "truncated": null,
, "finish_reason": null, "filter_block_reason": {"ko": "Default", "en": "Default", "policy_id": "34", "message": "The content was allowed by the filter", "result_code": "FR-201", "filter_log_id": "1861"}, "status": "SUCCESS", "response_code": null, "plugins": [], "orchestrator_type": null, "parent_message_created_at": "2025-08-21T16:31:34.104164+09:00", "references": null, "actions": null, "event_status": "CHUNK", "event_data": ""}
```



```

ted_at":"2025-08-21T16:31:34.104164+09:00","references":null,"actions":null,"event_status":"CHUNK",
"event_data":""}
41 data:
42
43 data: {"user_id":"b5bcab1f-051f-4519-b871-9cc827da06c6","model_type":"Llama
3.3","content":" 됩니다","reasoning_content":null,"processing_content":[],"content_references":
[],"truncated":null,"finish_reason":null,"filter_block_reason":{"ko":"Default","en":"Default","
policy_id":"34","message":"The content was allowed by the filter","result_code":"FR-201","filter_log_id":"1861"},"status":"SUCCESS","response_code":null,"plugins":[],"orchestrator_type":null
,"parent_message_created_at":"2025-08-21T16:31:34.104164+09:00","references":null,"actions":null
,"event_status":"CHUNK","event_data":""}
44 data:
45
46 data: {"user_id":"b5bcab1f-051f-4519-b871-9cc827da06c6","model_type":"Llama 3.3","content":"
.", "reasoning_content":null,"processing_content":[],"content_references":[],"truncated":null,"f
inish_reason":null,"filter_block_reason":{"ko":"Default","en":"Default","policy_id":"34","messa
ge":"The content was allowed by the filter","result_code":"FR-201","filter_log_id":"1861"},"sta
tus":"SUCCESS","response_code":null,"plugins":[],"orchestrator_type":null,"parent_message_creat
ed_at":"2025-08-21T16:31:34.104164+09:00","references":null,"actions":null,"event_status":"CHUN
K","event_data":""}
47 data:
48
49 data: {"user_id":"b5bcab1f-051f-4519-b871-9cc827da06c6","model_type":"Llama 3.3","content":"
","reasoning_content":null,"processing_content":[],"content_references":[{"plugin":"RAG","answe
r":"","references":[{"title":"라퐁첼.txt","content":"라퐁첼.txt : \n왕자님은 멈춰 서서 듣고
있었습니다.\n그 목소리는 라퐁첼의 목소리였어요. 그녀는 사랑스러운 목소리를 가졌는데 심심할 때면
고독함을 달래기 위해 노래를 불렀답니다.\n왕자님은 그녀가 있는 곳까지 올라가고 싶어서 탑의 문을
찾아보았지만 그 어디에도 출입구는 없었어요.\n그래서 일단 돌아갔는데 어쨌든 이번 일로 왕자님은
그녀의 노랫소리에 큰 감명을 받았답니다.\n이후 왕자님은 매일 숲으로 와서 그녀의 노랫소리를
들었어요.\n그러다 한 번은 나무 아래에 서 있는데, 여자마법사가 거리로 오지 뭐예요.\n곧 그
여자마법사가 어찌어찌 소리치는 게 들렸어요.\n“ 라퐁첼아, 라퐁첼아, 네 머리카락을 아래로
내려주렴. ” \n라퐁첼이 자신의 땃은 머리를 내려주자 여자마법사가 그걸 잡고 탑으로
올라갔어요.\n“ 라퐁첼아, 라퐁첼아, 네 머리카락을 아래로 내려주렴. ” \n라퐁첼이 자신의 땃은
머리를 내려주자 여자마법사가 그걸 잡고 탑으로 올라갔어요.\n“ 그녀의 머리카락이 사다리 역할을
하는 거라면, 나도 한번 운을 시험해보리라. ” 라고 왕자님은 말했어요.\n다음날 어둠이 깔리기 시작할
무렵에 왕자님은 탑으로 가 소리쳤어요.\n“ 라퐁첼아, 라퐁첼아, 네 머리카락을 아래로
내려주렴. ” \n그 즉시 머리카락이 내려와 왕자님은 부여잡고 올라갔어요.\n한 번도 본 적이 없는 웬
남자가 들어오는 것을 보고 라퐁첼도 처음엔 깜짝 놀랐어요.\n하지만 왕자님이 친구처럼 차분히
말씀을 걸기 시작하시는 거예요.\n“ 당신이 내 마음을 온통 휘저어놓은 이후 내게 안식이란 없었소.
당신을 보지 않곤 못 버티겠지 뭐요. ” \n그 말에 라퐁첼도 두려움을 누그러뜨렸어요.\n“ 저를 당신의

```

남편으로 받아들여줄 수 있겠소?” \n그제야 라퐁첼의 눈에 이분이 젊고 잘생긴 게 보였어요. \n라퐁첼이 생각했어요. \n“ 마녀 엄마보다 이 분이 더 나를 사랑해주실 거야. ” \n그래서 그녀는 그에게 “ 네 ” 라고 말하곤 자신의 손을 왕자님의 손에 안겼어요. \n라퐁첼이 말했어요. \n“ 기꺼이 당신을 따라 갈래요, 하지만 저는 여기서 내려가는 방법을 모르는 걸요. 매일 제게 비단 한 타래(뭉쳐놓은 거)씩 가져다주시면, 제가 그걸로 사다리를 짜 볼게요. 그럼 준비 되는 대로 게가 내려가고, 저를 말 등에 태우고 가 주세요. ” \n그들은 왕자님이 매일 저녁 들르기로 말을 맞추었어요. 왜냐면 늙은 여자마법사가 낮에 오기 때문이죠. \n여자마법사 또한 이 사실을 모르고 있었어요. \n그들은 왕자님이 매일 저녁 들르기로 말을 맞추었어요. 왜냐면 늙은 여자마법사가 낮에 오기 때문이죠. \n여자마법사 또한 이 사실을 모르고 있었어요. \n그러다 한 번은 라퐁첼이 이렇게 말하는 거예요. \n“ 알고 싶어요, 나이 많으신 엄마, 어째서 왕자님을 들어 올리는 것보다 당신을 들어 올리는 것이 훨씬 더 무겁죠…. 왕자님은 얼마 있지도 않는데. ” \n“ 뭐! 이 고약한 것, ” 라며 여자마법사가 소리쳤어요. “ 뭐가 어찌고 어찌! 너를 세상과 완전히 단절시켰는데도 이렇게 나를 속여. ” \n화가 난 여자마법사가 라퐁첼의 아름다운 머리채를 부여잡곤 자신의 왼손에 두 번 감은 다음 오른손에 가위를 들고서 “ 싹둑! ” 또 “ 싹둑! ” 자르고 말았어요. \n그래서 그녀의 사랑스럽던 머리카락들이 바닥에 푹푹 떨어졌어요. \n그럼에도, {"link":"","title":"라퐁첼.txt","content":"라퐁첼.txt : \n말았어요. \n그래서 그녀의 사랑스럽던 머리카락들이 바닥에 푹푹 떨어졌어요. \n그럼에도 불구하고 여전히 화가 안 풀린 이 무자비한 여자마법사는 불쌍한 라퐁첼을 ‘ 황무지 ’ (사람의 손이 가지 않은 거칠고 쓸모없는 땅)에 가둬버렸어요. \n거기서 라퐁첼은 큰 슬픔과 고통 속에 살아야했답니다. \n하지만, 라퐁첼을 내쫓아 가둔 바로 그 날 저녁 여자마법사는 라퐁첼의 머리카락을 창고 고리에 다시 이었어요. \n곧 왕자님이 나타나 소리쳤어요. \n“ 라퐁첼아, 라퐁첼아, 네 머리카락을 아래로 내려주렴. ” \n여자마법사가 창에 매단 그 머리카락을 내려주었어요. \n왕자님이 올라왔어요. 하지만 아리따운 라퐁첼은 온데간데없고 웬 여자마법사가 사악하고 이글이글거리는 눈빛으로 자신을 노려보고 있지 뭐예요. \n왕자님이 올라왔어요. 하지만 아리따운 라퐁첼은 온데간데없고 웬 여자마법사가 사악하고 이글이글거리는 눈빛으로 자신을 노려보고 있지 뭐예요. \n“ 아하! ” 라며 여자마법사가 조롱하듯 큰소리로 말했어요. “ 연인을 찾고 있군, 하지만 아름다운 새는 더는 둥지에서 노래 부르고 있지 않지. 고양이가 낚아채갔거든. 그리고 그 고양이가 이제 또한 네 놈의 눈을 핏줄 참이다. 라퐁첼은 네게서 사라졌다. 너는 이제 다신 그녀를 볼 수 없을 게다. ” \n왕자님은 상심에 그만 제 정신을 잃고 절망감에 그만 탑에서 뛰어내리고 말았어요. \n다행히 죽지는 않았지만 떨어질 때 가시나무에 떨어지는 바람에 두 눈을 가시에 찔리고 말았어요. \n이후 왕자님은 눈이 먼 상태에서 숲을 방황했어요. 그렇게 왕자님은 뿌리와 열매 외엔 아무 것도 먹지 못했답니다. 그리고 사랑하는 아내를 잃은 것이 너무도 슬퍼 하염없는 눈물말 흘리셨어요. \n그렇게 수년간을 비탄에 잠겨 떠돌아다니시다 마침내 라퐁첼이 살고 있는 ‘ 황무지 ’ (사람의 손이 가지 않은 거칠고 쓸모없는 땅)에까지 다다르게 되셨어요. \n그 황무지는 라퐁첼이 쌍둥이 남매(오빠와 여동생)를 낳고 가엾게 살고 있던 곳이었어요. \n왕자님은 너무도 친숙한 라퐁첼의 목소리가 들리자 앞으로 나아갔어요. \n왕자님이 다가오자마자 라퐁첼은 그를 알아보고 목을 감사며 안겨서 울었어요. \n그때 라퐁첼의 눈물 두 방울이 왕자님의 눈들을 적셨어요. \n그러자 왕자님의 시력이 점점 밝아지며 급기야 예전처럼 라퐁첼을 볼 수 있게 되었어요. \n왕자님은 라퐁첼을 데리고 자신의 왕국으로 돌아갔어요. \n왕국에서도 왕자님 일행을 대환영해주었어요. \n왕자님은 라퐁첼을 데리고 자신의 왕국으로 돌아갔어요. \n왕국에서도 왕자님 일행을 대환영해주었어요. \n이후 그들은 행복해하고 만족해하며 오래토록 잘 살았답니다. (해피엔딩

:) ㅎㅎ)", "link": ""}, {"title": "라퐁첼.txt", "content": "라퐁첼.txt : \n아내가 창문으로 이 상추를 보고 죽기 전에 먹어보면 소원이 없겠다고 해서요. " \n그 말에 여자마법사의 노여움이 좀 가셨어요, 그래서 그에게 말했어요.\n“ 그렇다면야, 네가 원하는 만큼 상추를 따서 갈 수 있게 허락해주겠다. 하지만 조건이 하나 있다, 네 아내가 장차 나올 그 아이를 내게 주어야겠다. 난 그 아이를 해치려는 게 아니라 엄마처럼 돌볼 생각이거든. " \n공포에 잡혀 있던 그 남자는 앞뒤 가리지 않고 동의하고 말았어요.\n\n여자 아이가 태어났어요. 그와 동시에 여자마법사가 오더니 아이에게 ‘ 라퐁첼 ’ (상추, 독일양배추. 라퐁첼 사진 링크 ▶ <https://goo.gl/4gW9wK>)이라는 이름을 지어주고는 아이를 데리고 가버렸어요.\n\n라퐁첼은 하늘 아래 가장 예쁜 여자아이를 자랐어요.\n\n라퐁첼은 하늘 아래 가장 예쁜 여자아이를 자랐어요.\n\n그녀가 12살이 되자, 여자마법사가 숲 속 탑에 가두었어요. 나오는 계단도 문도 없는 탑이었어요.\n\n맨 꼭대기에 작은 창이 하나 나 있었어요.\n\n여자마법사는 탑에 들어오고 싶을 때면, 창 아래로 가 외쳤어요.\n\n“ 라퐁첼아, 라퐁첼아, 네 머리카락을 아래로 내려다오. " \n\n라퐁첼은 금실 같이 훌륭하고 엄청나게 기다란 머리카락을 가지고 있었거든요.\n\n그래서 여자마법사의 목소리가 들리면, 땀뻘 머리카락을 풀어 창문 고리들 중 하나에 감고서 20미터 아래까지 머리카락을 늘어뜨리면 여자마법사가 그걸 잡고서 올라왔어요.\n\n일 이 년 후, 왕자님이 말을 타고 숲에 들어오셨다 그 탑을 지나가게 되었어요.\n\n그런데 어디서 매우 아름다운 노랫소리가 들려 왕자님은 멈춰 서서 듣고 있었답니다.\n\n그 목소리는 라퐁첼의 목소리였어요. 그녀는 사랑스러운 목소리를 가졌는데 심심할 때면 고독함을 달래기 위해 노래를 불렀답니다.\n\n왕자님은 그녀가 있는 곳까지 올라가고 싶어서 탑의 문을 찾아보았지만 그 어디에도 출입구는 없었어요.\n\n그래서 일단 돌아갔는데 어쨌든 이번 일로 왕자님은 그녀의 노랫소리에 큰 감명을 받았답니다.\n\n이후 왕자님은 매일 숲으로 와서 그녀의 노랫소리를 들었어요.\n\n그러다 한 번은 나무 아래에 서 있는데, 여자마법사가 거리로 오지 뭐예요.\n\n곧 그 여자마법사가 어찌어찌 소리치는 게 들렸어요.\n\n“ 라퐁첼아, 라퐁첼아, 네 머리카락을 아래로 내려주렴. " \n\n라퐁첼이 자신의 땀은 머리를 내려주자 여자마법사가 그걸 잡고 탑으로 올라갔어요.\n\n“ 라퐁첼아, 라퐁첼아, 네 머리카락을 아래로 내려주렴. " \n\n라퐁첼이 자신의 땀은 머리를 내려주자 여자마법사가 그걸 잡고 탑으로 올라갔어요.\n\n“ 그녀의 머리카락이 사다리 역할을 하는 거라면, 나도 한번 운을 시험해보리라. " 라고 왕자님은 말했어요.\n\n다음날 어둠이 깔리기 시작할 무렵에 왕자님은 탑으로 가 소리쳤어요.\n\n“ 라퐁첼아, 라퐁첼아, 네 머리카락을 아래로 내려주렴. " \n\n그 즉시 머리카락이 내려와 왕자님은 부여잡고 올라갔어요.\n\n한 번도 본 적이 없는 웬 남자가 들어오는 것을 보고 라퐁첼도 처음엔 깜짝 놀랐어요.\n\n하지만 왕자님이 친구처럼 차분히 말씀을 걸기 시작하시는 거예요.\n\n“ 당신이 내 마음을 온통 휘저어놓은 이후 내게 안식이란 없었소. 당신을 보지 않곤 못 버티겠지 뭐요. " \n\n그", "link": ""}], "augmented_standalone_queries": "이 파일의 내용을 알려주세요."}], "truncated": false, "finish_reason": null, "filter_block_reason": {"ko": "Default", "en": "Default", "policy_id": "34", "message": "The content was allowed by the filter", "result_code": "FR-201", "filter_log_id": "1861"}, "status": "SUCCESS", "response_code": "R20000", "plugins": [], "orchestrator_type": null, "parent_message_created_at": "2025-08-21T16:31:34.104164+09:00", "references": [{"type": "search", "title": "라퐁첼.txt", "url": "", "content_type": "CT01", "source_navi": "", "source_info": {"source_navi": "", "tags": "", "related_system": []}, "rank": 1, "rank_score": 0.000014824562640569638, "content": "라퐁첼.txt : \n왕자님은 멈춰 서서 듣고 있었답니다.\n\n그 목소리는 라퐁첼의 목소리였어요. 그녀는 사랑스러운 목소리를 가졌는데 심심할 때면 고독함을 달래기 위해 노래를 불렀답니다.\n\n왕자님은 그녀가 있는 곳까지 올라가고 싶어서 탑의 문을 찾아보았지만 그 어디에도 출입구는 없었어요.\n\n그래서 일단 돌아갔는데 어쨌든 이번 일로 왕자님은 그녀의 노랫소리에 큰

감명을 받았답니다.\n이후 왕자님은 매일 숲으로 와서 그녀의 노랫소리를 들었어요.\n그러다 한 번은 나무 아래에 서 있는데, 여자마법사가 거리로 오지 뭐예요.\n곧 그 여자마법사가 어찌어찌 소리치는 게 들렸어요.\n“ 라퐼첼아, 라퐼첼아, 네 머리카락을 아래로 내려주렴. ”\n라퐼첼이 자신의 딸은 머리를 내려주자 여자마법사가 그걸 잡고 탑으로 올라갔어요.\n“ 라퐼첼아, 라퐼첼아, 네 머리카락을 아래로 내려주렴. ”\n라퐼첼이 자신의 딸은 머리를 내려주자 여자마법사가 그걸 잡고 탑으로 올라갔어요.\n“ 그녀의 머리카락이 사다리 역할을 하는 거라면, 나도 한번 운을 시험해보리라. ” 라고 왕자님은 말했어요.\n다음날 어둠이 깔리기 시작할 무렵에 왕자님은 탑으로 가 소리쳤어요.\n“ 라퐼첼아, 라퐼첼아, 네 머리카락을 아래로 내려주렴. ”\n그 즉시 머리카락이 내려와 왕자님은 부여잡고 올라갔어요.\n한 번도 본 적이 없는 웬 남자가 들어오는 것을 보고 라퐼첼도 처음엔 깜짝 놀랐어요.\n하지만 왕자님이 친구처럼 차분히 말씀을 걸기 시작하시는 거예요.\n“ 당신이 내 마음을 온통 휘저어놓은 이후 내게 안식이란 없었소. 당신을 보지 않곤 못 버티겠지 뭐요. ”\n그 말에 라퐼첼도 두려움을 누그러뜨렸어요.\n“ 저를 당신의 남편으로 받아들여줄 수 있겠소? ”\n그제야 라퐼첼의 눈에 이분이 젊고 잘생긴 게 보였어요.\n라퐼첼이 생각했어요.\n“ 마녀 엄마보다 이 분이 더 나를 사랑해주실 거야. ”\n그래서 그녀는 그에게 “ 네 ” 라고 말하곤 자신의 손을 왕자님의 손에 안겼어요.\n라퐼첼이 말했어요.\n“ 기꺼이 당신을 따라 갈래요, 하지만 저는 여기서 내려가는 방법을 모르는 걸요. 매일 제게 비단 한 타래(뭉쳐놓은 거)씩 가져다주시면, 제가 그걸로 사다리를 짜 볼게요. 그럼 준비 되는 대로 게가 내려가고, 저를 말 등에 태우고 가 주세요. ”\n그들은 왕자님이 매일 저녁 들르기로 말을 맞추었어요. 왜냐면 늙은 여자마법사가 낮에 오기 때문이죠.\n여자마법사 또한 이 사실을 모르고 있었어요.\n그들은 왕자님이 매일 저녁 들르기로 말을 맞추었어요. 왜냐면 늙은 여자마법사가 낮에 오기 때문이죠.\n여자마법사 또한 이 사실을 모르고 있었어요.\n그러다 한 번은 라퐼첼이 이렇게 말하는 거예요.\n“ 알고 싶어요, 나이 많으신 엄마, 어쩌서 왕자님을 들어 올리는 것보다 당신을 들어 올리는 것이 훨씬 더 무겁죠... 왕자님은 얼마 있지도 않는데. ”\n“ 뭐! 이 고약한 것, ” 라며 여자마법사가 소리쳤어요. “ 뭐가 어찌고 어찌! 너를 세상과 완전히 단절시켰는데도 이렇게 나를 속여. ”\n화가 난 여자마법사가 라퐼첼의 아름다운 머리채를 부여잡곤 자신의 왼손에 두 번 감은 다음 오른손에 가위를 들고서 “ 싹둑! ” 또 “ 싹둑! ” 자르고 말았어요.\n그래서 그녀의 사랑스럽던 머리카락들이 바닥에 뚝뚝 떨어졌어요.\n그럼에도", "catalog_id": "b55ca7ce-6216-491b-8109-a259c18e4d2b.txt"}], "actions": [{"type": "RAG", "plugin": "RAG", "api": "RAG", "params": null, "response_type": null, "method": null, "http_status": 200, "response": "{\"augmentedQuery\": \"이 파일의 내용을 알려주세요.\", \"hydeQuery\": \"\", \"results\": [{\"id\": \"UESIy5gBKnwI2hpnqrR2\", \"systemId\": \"personal_file\", \"systemCategory\": \"chat\", \"hasPermission\": true, \"content\": \"라퐼첼.txt : \\n왕자님은 멈춰 서서 듣고 있었답니다.\\n그 목소리는 라퐼첼의 목소리였어요. 그녀는 사랑스러운 목소리를 가졌는데 심심할 때면 고독함을 달래기 위해 노래를 불렀답니다.\\n왕자님은 그녀가 있는 곳까지 올라가고 싶어서 탑의 문을 찾아보았지만 그 어디에도 출입구는 없었어요.\\n그래서 일단 돌아갔는데 어쨌든 이번 일로 왕자님은 그녀의 노랫소리에 큰 감명을 받았답니다.\\n이후 왕자님은 매일 숲으로 와서 그녀의 노랫소리를 들었어요.\\n그러다 한 번은 나무 아래에 서 있는데, 여자마법사가 거리로 오지 뭐예요.\\n곧 그 여자마법사가 어찌어찌 소리치는 게 들렸어요.\\n\" 라퐼첼아, 라퐼첼아, 네 머리카락을 아래로 내려주렴. \"\\n라퐼첼이 자신의 딸은 머리를 내려주자 여자마법사가 그걸 잡고 탑으로 올라갔어요.\\n\" 라퐼첼아, 라퐼첼아, 네 머리카락을 아래로 내려주렴. \"\\n라퐼첼이 자신의 딸은 머리를 내려주자 여자마법사가 그걸 잡고 탑으로 올라갔어요.\\n\" 그녀의 머리카락이 사다리 역할을 하는 거라면, 나도 한번 운을

시험해보리라. ” 라고 왕자님은 말했어요.\n\n다음날 어둠이 깔리기 시작할 무렵에 왕자님은 탑으로 가 소리쳤어요.\n\n“ 라퐁첼아, 라퐁첼아, 네 머리카락을 아래로 내려주렴. ” \n\n그 즉시 머리카락이 내려와 왕자님은 부여잡고 올라갔어요.\n\n한 번도 본 적이 없는 웬 남자가 들어오는 것을 보고 라퐁첼도 처음엔 깜짝 놀랐어요.\n\n하지만 왕자님이 친구처럼 차분히 말씀을 걸기 시작하시는 거예요.\n\n“ 당신이 내 마음을 온통 휘저어놓은 이후 내게 안식이란 없었소. 당신을 보지 않곤 못 버티겠지 뭐요. ” \n\n그 말에 라퐁첼도 두려움을 누그러뜨렸어요.\n\n“ 저를 당신의 남편으로 받아들여줄 수 있겠소? ” \n\n그제야 라퐁첼의 눈에 이분이 젊고 잘생긴 게 보였어요.\n\n라퐁첼이 생각했어요.\n\n“ 마녀 엄마보다 이 분이 더 나를 사랑해주실 거야. ” \n\n그래서 그녀는 그에게 “ 네 ” 라고 말하곤 자신의 손을 왕자님의 손에 안겼어요.\n\n라퐁첼이 말했어요.\n\n“ 기꺼이 당신을 따라 갈래요, 하지만 저는 여기서 내려가는 방법을 모르는 걸요. 매일 제게 비단 한 타래(뭉쳐놓은 거)씩 가져다주시면, 제가 그걸로 사다리를 짜 볼게요. 그럼 준비 되는 대로 게가 내려가고, 저를 말 등에 태우고 가 주세요. ” \n\n그들은 왕자님이 매일 저녁 들르기로 말을 맞추었어요. 왜냐면 늙은 여자마법사가 낮에 오기 때문이죠.\n\n여자마법사 또한 이 사실을 모르고 있었어요.\n\n그들은 왕자님이 매일 저녁 들르기로 말을 맞추었어요. 왜냐면 늙은 여자마법사가 낮에 오기 때문이죠.\n\n여자마법사 또한 이 사실을 모르고 있었어요.\n\n그러다 한 번은 라퐁첼이 이렇게 말하는 거예요.\n\n“ 알고 싶어요, 나이 많으신 엄마, 어째서 왕자님을 들어 올리는 것보다 당신을 들어 올리는 것이 훨씬 더 무겁죠... 왕자님은 얼마 있지도 않는데. ” \n\n“ 뭐! 이 고약한 것, ” 라며 여자마법사가 소리쳤어요. “ 뭐가 어찌고 어찌! 너를 세상과 완전히 단절시켰는데도 이렇게 나를 속여. ” \n\n화가 난 여자마법사가 라퐁첼의 아름다운 머리채를 부여잡곤 자신의 왼손에 두 번 감은 다음 오른손에 가위를 들고서 “ 싹둑! ” 또 “ 싹둑! ” 자르고 말았어요.\n\n그래서 그녀의 사랑스럽던 머리카락들이 바닥에 뚝뚝 떨어졌어요.\n\n그럼에도\n\n{\n \"filename\": \"\", \"title\": \"라퐁첼.txt\", \"docId\": \"b55ca7ce-6216-491b-8109-a259c18e4d2b.txt\", \"url\": \"\", \"contentType\": \"CT01\", \"rankScore\": 1.4824562640569638e-05, \"rerankScore\": \"\", \"sourceInfo\": {\n \"tags\": \"\", \"relatedSystem\": [], \"sourceNav\": \"\", \"contentInfo\": {\n \"anchor\": \"\", \"page\": \"\", \"subtitle\": \"\", \"urlData\": [], \"chunkSeq\": 4, \"chunkPageSize\": 6, \"subSystem\": \"\", \"catalogId\": \"b55ca7ce-6216-491b-8109-a259c18e4d2b.txt\", \"useAllRespContent\": false, \"rank\": 1, \"executedQuery\": \"이 파일의 내용을 알려주세요.\"\n }, {\n \"id\": \"UkSIy5gBKwI2hpnqrR2\", \"systemId\": \"personal_file\", \"systemCategory\": \"chat\", \"hasPermission\": true, \"content\": \"라퐁첼.txt : \n\n말았어요.\n\n그래서 그녀의 사랑스럽던 머리카락들이 바닥에 뚝뚝 떨어졌어요.\n\n그럼에도 불구하고 여전히 화가 안 풀린 이 무자비한 여자마법사는 불쌍한 라퐁첼을 ‘ 황무지 ’ (사람의 손이 가지 않은 거칠고 쓸모없는 땅)에 가둬버렸어요.\n\n거기서 라퐁첼은 큰 슬픔과 고통 속에 살아야했답니다.\n\n하지만, 라퐁첼을 내쫓아가둔 바로 그 날 저녁 여자마법사는 라퐁첼의 머리카락을 창외 고리에 다시 이었어요.\n\n곧 왕자님이 나타나 소리쳤어요.\n\n“ 라퐁첼아, 라퐁첼아, 네 머리카락을 아래로 내려주렴. ” \n\n여자마법사가 창에 매단 그 머리카락을 내려주었어요.\n\n왕자님이 올라왔어요. 하지만 아리따운 라퐁첼은 온데간데없고 웬 여자마법사가 사악하고 이글이글거리는 눈빛으로 자신을 노려보고 있지 뭐예요.\n\n왕자님이 올라왔어요. 하지만 아리따운 라퐁첼은 온데간데없고 웬 여자마법사가 사악하고 이글이글거리는 눈빛으로 자신을 노려보고 있지 뭐예요.\n\n“ 아하! ” 라며 여자마법사가 조롱하듯 큰소리로 말했어요. “ 연인을 찾고 있군, 하지만 아름다운 새는 더는 둥지에서 노래 부르고 있지 않지. 고양이 가 낚아챘거든. 그리고 그 고양이가 이제 또한 네 놈의 눈을 핥릴 참이다. 라퐁첼은 네게서 사라졌다. 너는 이제 대신 그녀를 볼 수 없을 게다. ” \n\n왕자님은 상심에 그만 제 정신을 잃고 절망감에 그만

tap에서 뛰어내리고 말았어요.\n\n다행히 죽지는 않았지만 떨어질 때 가시나무에 떨어지는 바람에 두
 눈을 가시에 찔리고 말았어요.\n\n이후 왕자님은 눈이 먼 상태에서 숲을 방황했어요. 그렇게 왕자님은
 뿌리와 열매 외엔 아무 것도 먹지 못했습니다. 그리고 사랑하는 아내를 잃은 것이 너무도 슬퍼
 하염없는 눈물말 흘리셨어요.\n\n그렇게 수년간을 비탄에 잠겨 떠돌아다니시다 마침내 라푼첼이 살고
 있는 ‘ 황무지 ’ (사람의 손이 가지 않은 거칠고 쓸모없는 땅)에까지 다다르게 되셨어요.\n\n그
 황무지는 라푼첼이 쌍둥이 남매(오빠와 여동생)를 낳고 가엾게 살고 있던 곳이었어요.\n\n왕자님은
 너무도 친숙한 라푼첼의 목소리가 들리자 앞으로 나아갔어요.\n\n왕자님이 다가오자마자 라푼첼은 그를
 알아보고 목을 감사며 안겨서 울었어요.\n\n그때 라푼첼의 눈물 두 방울이 왕자님의 눈들을
 젖혔어요.\n\n그러자 왕자님의 시력이 점점 밝아지며 급기야 예전처럼 라푼첼을 볼 수 있게
 되었어요.\n\n왕자님은 라푼첼을 데리고 자신의 왕국으로 돌아갔어요.\n\n왕국에서도 왕자님 일행을
 대환영해주었어요.\n\n왕자님은 라푼첼을 데리고 자신의 왕국으로 돌아갔어요.\n\n왕국에서도 왕자님
 일행을 대환영해주었어요.\n\n이후 그들은 행복해하고 만족해하며 오래토록 잘 살았습니다. (해피엔딩
 :) ㅎㅎ)\n\n

```

  "filename": "라푼첼.txt", "docId": "b55ca7ce-6216-491b-8109-a259c18e4d2b.txt", "url": "", "contentType": "CT01", "rankScore": 1.0608756383589935e-05, "rerankScore": "", "sourceInfo": {"tags": "", "relatedSystem": [], "sourceNavi": ""}, "contentInfo": {"anchor": "", "page": "", "subtitle": "", "urlData": [], "chunkSeq": 6, "chunkPageSize": 6}, "subSystem": "", "catalogId": "b55ca7ce-6216-491b-8109-a259c18e4d2b.txt", "useAllRespContent": false, "rank": 2, "executedQuery": "이 파일의 내용을 알려주세요."}, {"id": "T0SiY5gBKnwI2hpnqrR2", "systemId": "personal_file", "systemCategory": "chat", "hasPermission": true, "content": "라푼첼.txt : \n\n아내가 창문으로 이 상추를 보고 죽기 전에 먹어보면 소원이 없겠다고 해서요. "
  
```

\n\n그 말에 여자마법사의 노여움이 좀 가셨어요, 그래서 그에게
 말했어요.\n\n“ 그렇다면야, 네가 원하는 만큼 상추를 따서 갈 수 있게 허락해주겠다. 하지만 조건이
 하나 있다, 네 아내가 장차 나올 그 아이를 내게 주어야겠다. 난 그 아이를 해치려는 게 아니라
 엄마처럼 돌볼 생각이거든. ”\n\n공포에 잡혀 있던 그 남자는 앞뒤 가리지 않고 동의하고
 말했어요.\n\n여자 아이가 태어났어요. 그와 동시에 여자마법사가 오더니 아이에게 ‘ 라푼첼 ’ (상추.
 독일양배추. 라푼첼 사진 링크 ▶ <https://goo.gl/4gW9wK>)이라는 이름을 지어주고는 아이를 데리고
 가버렸어요.\n\n라푼첼은 하늘 아래 가장 예쁜 여자아이로 자랐어요.\n\n라푼첼은 하늘 아래 가장 예쁜
 여자아이로 자랐어요.\n\n그녀가 12살이 되자, 여자마법사가 숲 속 탑에 가두었어요. 나오는 계단도
 문도 없는 탑이었어요.\n\n맨 꼭대기에 작은 창이 하나 나 있었어요.\n\n여자마법사는 탑에 들어오고
 싶을 때면, 창 아래로 가 외쳤어요.\n\n“ 라푼첼아, 라푼첼아, 네 머리카락을 아래로
 내려다오. ”\n\n라푼첼은 금실 같이 훌륭하고 엄청나게 기다란 머리카락을 가지고
 있었거든요.\n\n그래서 여자마법사의 목소리가 들리면, 땀났던 머리카락을 풀어 창문 고리들 중 하나에
 감고서 20미터 아래까지 머리카락을 늘어뜨리면 여자마법사가 그걸 잡고서 올라왔어요.\n\n일 이 년
 후, 왕자님이 말을 타고 숲에 들어오셨다 그 탑을 지나가게 되었어요.\n\n그런데 어디서 매우 아름다운
 노랫소리가 들려 왕자님은 멈춰 서서 듣고 있었답니다.\n\n그 목소리는 라푼첼의 목소리였어요. 그녀는
 사랑스러운 목소리를 가졌는데 심심할 때면 고독함을 달래기 위해 노래를 불렀답니다.\n\n왕자님은
 그녀가 있는 곳까지 올라가고 싶어서 탑의 문을 찾아보았지만 그 어디에도 출입구는
 없었어요.\n\n그래서 일단 돌아갔는데 어쨌든 이번 일로 왕자님은 그녀의 노랫소리에 큰 감명을
 받았답니다.\n\n이후 왕자님은 매일 숲으로 와서 그녀의 노랫소리를 들었어요.\n\n그러다 한 번은 나무
 아래에 서 있는데, 여자마법사가 거리로 오지 뭐예요.\n\n곧 그 여자마법사가 어찌어찌 소리치는 게

들렸어요.\n\n“ 라푼첼아, 라푼첼아, 네 머리카락을 아래로 내려주렴.”\n\n라푼첼이 자신의 딸은 머리를 내려주자 여자마법사가 그걸 잡고 탑으로 올라갔어요.\n\n“ 라푼첼아, 라푼첼아, 네 머리카락을 아래로 내려주렴.”\n\n라푼첼이 자신의 딸은 머리를 내려주자 여자마법사가 그걸 잡고 탑으로 올라갔어요.\n\n“ 그녀의 머리카락이 사다리 역할을 하는 거라면, 나도 한번 운을 시험해보리라.” 라고 왕자님은 말했어요.\n\n다음날 어둠이 깔리기 시작할 무렵에 왕자님은 탑으로 가 소리쳤어요.\n\n“ 라푼첼아, 라푼첼아, 네 머리카락을 아래로 내려주렴.”\n\n그 즉시 머리카락이 내려와 왕자님은 부여잡고 올라갔어요.\n\n한 번도 본 적이 없는 웬 남자가 들어오는 것을 보고 라푼첼도 처음엔 깜짝 놀랐어요.\n\n하지만 왕자님이 친구처럼 차분히 말씀을 걸기 시작하시는 거예요.\n\n“ 당신이 내 마음을 온통 휘저어놓은 이후 내게 안식이란 없었소. 당신을 보지 않곤 못 버티겠지 뭐요.”\n\n그",\n\n"filename":"",\n\n"title":"라푼첼.txt",\n\n"docId":"b55ca7ce-6216-491b-8109-a259c18e4d2b.txt",\n\n"url":"",\n\n"contentType":"CT01",\n\n"rankScore":7.356095011346042e-06,\n\n"rerankScore":"",\n\n"sourceInfo":{"tags":"","relatedSystem":[],\n\n"sourceNav":"","contentInfo":{"anchor":"","page":"","subtitle":"","urlData":[],\n\n"chunkSeq":3,\n\n"chunkPageSize":6},\n\n"subSystem":"","catalogId":"b55ca7ce-6216-491b-8109-a259c18e4d2b.txt",\n\n"useAllRespContent":false,\n\n"rank":3,\n\n"executedQuery":"이 파일의 내용을 알려주세요."}}},\n\n"properties":{"is_multi_query":false,\n\n"augmented_standalone_queries":["이 파일의 내용을 알려주세요."]},\n\n"answer":"Data information retrieved through 이 파일의 내용을 알려주세요..",\n\n"question":null}},\n\n"event_status":"CHUNK",\n\n"event_data":""}\n\n50 data:

호출 예시 (python)

Knowledge 기반 대화

(isStream=false)

LLM 목록 조회를 이용하여 사용가능한 MODEL_ID를 확보한다.

Knowledge 목록 조회를 이용하여 대화에 사용할 Knowledge의 ID를 확보한다.

```
1 import requests
2
3 YOUR_CLIENT_KEY = ""
4 YOUR_PASS_KEY = ""
5 ENDPOINT_URL = ""
6 YOUR_EMAIL = ""
7
8 headers = {
9     "x-fabrix-client": YOUR_CLIENT_KEY,
10    "x-openapi-token": YOUR_PASS_KEY,
11    "x-generative-ai-user-email": YOUR_EMAIL
```

```

12 }
13
14 MODEL_ID = "Get model id from Model API"
15 KNOWLEDGE_ASSET_ID = "Get knowledge asset id from Knowledge API"
16
17 body = {
18     "modelIds":[MODEL_ID],
19     "contents":["이 이야기의 주인공은?"],
20     "isStream":False,
21     "llmConfig": {},
22     "systemPrompt": "",
23     "knowledgeAssetId": KNOWLEDGE_ASSET_ID
24 }
25
26 api_endpoint_url = f"{ENDPOINT_URL}/openapi/rag-chat/v1/messages"
27
28 response=requests.post(
29     api_endpoint_url,
30     headers=headers,
31     json=body
32 )
33
34 # 응답 반환
35 print(response.json())

```

주의

※ Endpoint URL은 Open API 신청시 발급되는 URL로 적용하여 사용할 수 있습니다.

| Embedding 기반 대화

LLM 목록 조회를 이용하여 사용가능한 MODEL_ID를 확보한다.

문서파일을 업로드하고 임베딩 하여 대화에 사용할 Embed File ID를 확보한다.

(isStream=true)

팁

Stream = true 인 경우에 최종 응답 확인은 status 값을 이용하여 확인합니다.

- 정상 답변 : status = 'SUCCESS' and response_code = 'R20000'
- 비정상 답변 : status='ERROR' or 'FILTER_INVALID'

```

1  import requests
2  import sseclient
3  import json
4
5  YOUR_CLIENT_KEY = ""
6  YOUR_PASS_KEY = ""
7  ENDPOINT_URL = ""
8  YOUR_EMAIL = ""
9
10 headers = {
11     "x-fabrix-client": YOUR_CLIENT_KEY,
12     "x-openapi-token": YOUR_PASS_KEY,
13     "x-generative-ai-user-email": YOUR_EMAIL
14 }
15
16 MODEL_ID = "Get model id from Model API"
17 EMBEDDING_FILE_UUID= "Get file uuid from file upload API"
18
19 body = {
20     "modelIds":[MODEL_ID],
21     "contents":["이 파일 내용이 뭐야?"],
22     "isStream":True,
23     "llmConfig": {},
24     "systemPrompt": "",
25     "embedFileUuid": EMBEDDING_FILE_UUID
26 }
27
28 api_endpoint_url = f"{ENDPOINT_URL}/openapi/rag-chat/v1/messages"
29
30 response = requests.post(
31     api_endpoint_url,
```

```

32     headers=headers,
33     json=body
34 )
35
36 # return response
37 if response.status_code != 200:
38     print(f"Error: Status_code=[{response.status_code}], Status message=[{response.text}]")
39 client = sseclient.SSEClient(response)
40
41 # make result contents
42 def get_result():
43     for event in client.events():
44         # print(f"Event: {event.event}")
45         print(f"Data: {event.data}")
46         yield event.data
47
48 result_message = ""
49 for ch in get_result():
50     ch_json = json.loads(ch)
51     if ch_json['status'] == 'SUCCESS' and ch_json['response_code'] == 'R200000':
52         print("Stream Finished!!")
53     if ch_json['event_status'] == 'CHUNK' and ch_json['content']:
54         result_message += ch_json['content']
55
56 print(result_message)

```

주의

※ Endpoint URL은 Open API 신청시 발급되는 URL로 적용하여 사용할 수 있습니다.

공통 Response Code

노트

Response 가 Success (Http Status Code 가 200) 인 경우 Response code 를 통해 상세 내용 확인이 가능합니다.

Response 가 False (Http Status 가 200이 아닌) 인 경우 Error Code 를 통해 상세 내용 확인이

가능합니다.

Success

Http status code	Response code	Reason (response message)
200	R20000	success
200	R20001	(DEPRECATED) need more parameters for api call
200	R20002	(DEPRECATED) only support single-step task
200	R20003	(DEPRECATED) error occurred during api call
200	R20004	(DEPRECATED) api request timeout
200	R20005	(DEPRECATED) exceeded token limit for generating response
200	R40000	client error # generic type for client error
200	R40001	request parameter validation error
200	R40002	unsupported model error
200	R40003	invalid plugin or api specification
200	R40004	exceeded token limit for request completion
200	R40005	health check api error
200	R40006	plug-in id not found
200	R40007	search engine not used
200	R40008	blocked by security filter
200	R40009	request timeout
200	R40010	too many LLM request
200	R40011	LLM request timeout
200	R40012	Exceeding the token usage limit on the client.
200	R40013	Exceeding the filter check content limit on the client.
200	R50000	internal server error # generic type for server error
200	R50001	api specification parsing error
200	R50002	security filter service error
200	R50003	cognitive search service error
200	R50004	LLM request error
200	R50005	LLM response parsing error
200	R50030	tool hallucination
200	R50050	Plug-in server error
200	R50051	Plug-in API connect error
200	R50052	Code Interpreter call error.
200	R50053	File summary/translate call error.
200	R50060	MCP server error
200	R50061	MCP tool connect error
200	R50070	Agent error

Http status code	Response code	Reason (response message)
200	R50071	Agent(standard) connect error
200	R50072	Agent(Langflow) connect error
200	R50073	Agent(custom) connect error
200	R50074	Agent(A2A) connect error
200	R50075	Agent Builder connect error
200	R50080	Selection flow error
200	R50081	Workflow Execution error

Fail

Http status code	Error Code	Reason (response message)
413	FILTER_ERROR_2	The content length has been exceeded
500	COMMON_ERROR_1	Exception occurred
404	COMMON_ERROR_2	The table does not exist
500	COMMON_ERROR_3	Updating the table was failed
404	COMMON_ERROR_4	It does not exist
400	COMMON_ERROR_5	Request parameter or body is invalid
422	COMMON_ERROR_6	Unprocessable Entity is requeste
403	COMMON_ERROR_7	Access is denied
423	COMMON_ERROR_8	Resource is locked while being processed. Please try again later

8.3 FAQ

기본 System Prompt를 알려주세요.

기본 System Prompt도 FabriX의 지식재산에 해당하므로 관리자에게 문의 주시면 검토하여 답변 드리겠습니다. 참고로 Rag Chat API를 사용할 때, “systemPrompt” 매개변수를 전달하지 않으면 기본 System Prompt를 사용합니다.

Rag Chat API를 사용할 때, 이전 대화를 고려한 응답을 받으려면 어떻게 해야 하나요?

Rag Chat API의 “contents” 매개변수를 활용하여 대화를 구성할 수 있습니다. 예를 들어, Request Body로 “contents”: [“사용자의 첫번째 질의”, “모델의 첫번째 답변”, “사용자의 두번째 질의”, “모델의 두번째 답변”, “사용자의 마지막 질의”] 를 전달하면 contents가 대화이력 역할을 하여 이전 대화를 활용할 수 있습니다.

llmConfig 매개변수에 대해서 조금 더 상세한 내용을 알고 싶어요. 기본 llmConfig를 알려주실 수 있나요?

llmConfig 매개변수에 해당하는 변수들을 입력하지 않는 경우 Default 설정값을 적용하여 응답합니다. FabriX의 기본 llmConfig 또한 모델 연구자들의 연구 결과에 의한 지식재산에 해당하므로 추가로 필요하신 내용이 있으시면, 관리자에게 문의 바랍니다.

9.

A2A APIs

9.1 A2A APIs Overview

A2A APIs는 A2A Spec을 이용한 Agent 호출 서비스를 제공합니다.

A2A APIs 에서 제공하는 API는 다음과 같습니다.

GET /openapi/agent-chat/v1/agents

사용 권한을 가진 Agent 목록을 조회할 수 있습니다.

GET /openapi/orchestrator/v1/a2a/{agent_id}/.well-known/agent-card.json

Agent Card를 조회합니다.

POST /openapi/orchestrator/v1/a2a/{agent_id}

Agent에 사용자 질의에 대한 Agent 응답을 생성합니다.

9.2 API 명세

GET /openapi/agent-chat/v1/agents

Header

Field	Type	Required	Description	Value
Content-Type	string	optional	Type of content	application/json
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-openapi-token	string	required	OpenAPI Token information	ex) Bearer eyJXXX

Field	Type	Required	Description	Value
x-generative-ai-user-email	string	optional	Portal user email	ex) kim.samsung@samsung.com

Query Parameter

Key	Type	Default	Description	Default Value
keyword	String	Optional	Search keyword	
page	Integer	Required	Pagination page value	1
limit	Integer	Required	Pagination limit value	10
sort	String	Optional	Sort Order	updatedAt
categories	String	Optional	Category filter	
favorite	Boolean	Optional	Whether the item is marked as a favorite.	

Response

Field	Type	Description
meta	Pagination	Pagination meta information
items	AgentInfo[]	Agent information list

Type <Pagination>

Field	Type	Description
totalItems	Integer	Total Item Count
itemCount	Integer	Current Item count
itemsPerPage	Integer	Items per page
totalPages	Integer	Total Page Count
currentPage	Integer	Current Page

Type <AgentInfo>

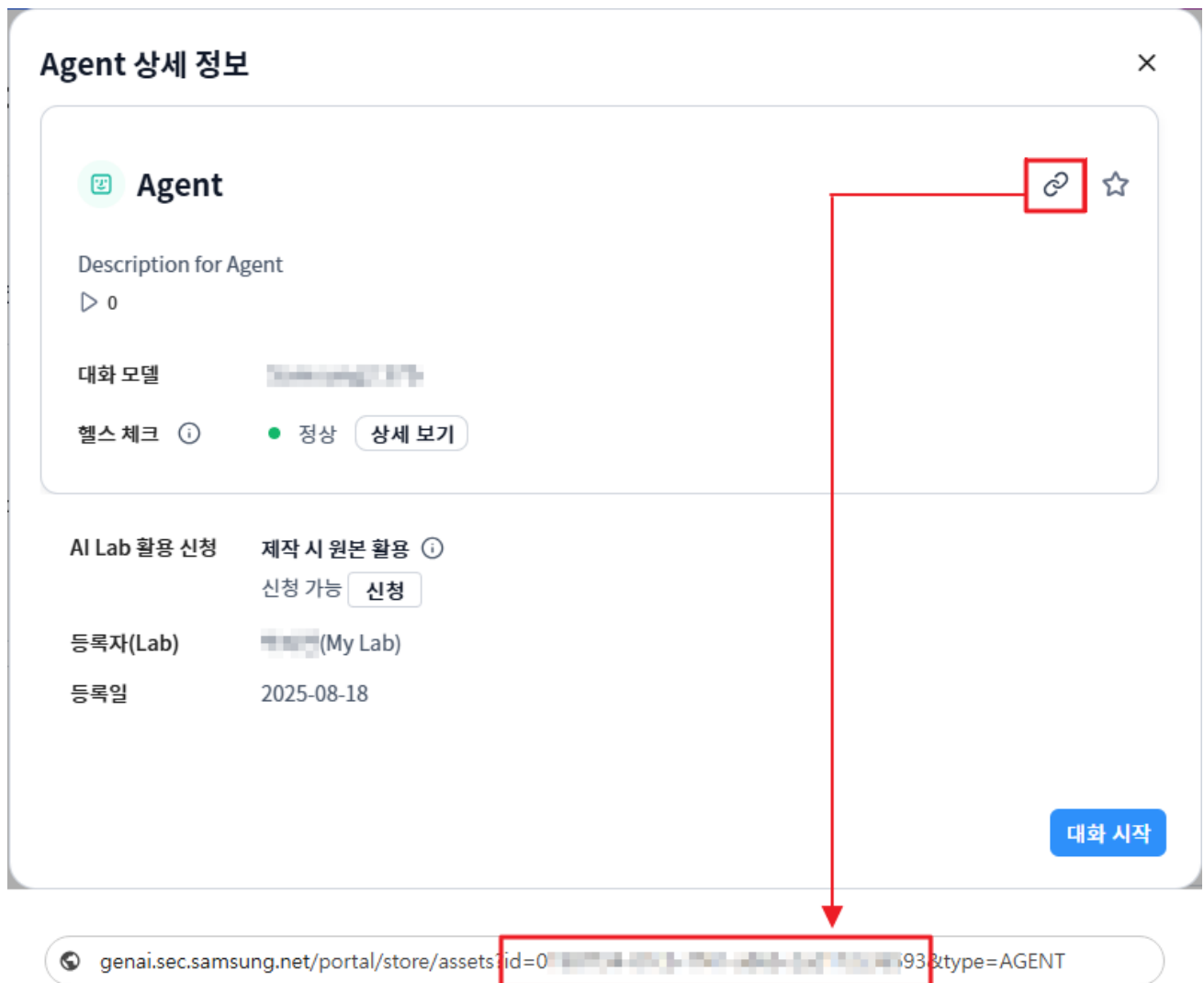
Field	Type	Description
agentId	UUID	Agent Id
label	String	Agent Label
subType	String	Agent Type ("SINGLE" or "MULTI")
description	String	Agent detail description
deployedAt	Datetime	Agent Deployed Date
modifiedAt	Datetime	Agent Modified Date

노트

agentID 확인 방법

1. "Store > Agent > Agent 상세 정보" 화면 진입
2. Agent URL 링크 복사 버튼 클릭
3. URL 내 agentID 정보 확인

그림 9-1 Agent URL 링크 내 agentID 확인하기



Response 예시

```

1 {
2   "meta": {
3     "totalItems": 11,

```



```

4     "itemCount": 10,
5     "itemsPerPage": 10,
6     "totalPages": 2,
7     "currentPage": 1
8 },
9     "items": [
10    {
11        "agentId": "019806a8-12be-7495-ac12-67255b808aa4",
12        "label": "Plugin 요리사 봇",
13        "subType": "SINGLE"
14        "description": "안녕하세요, 저는 Plugin 요리사 봇 니다. 요리 아이디어나 지침이
필요하신 가정 요리사나 음식 애호가를 위해 제가 도와드릴 수 있습니다.\n사용 가능한 재료와
식이제한, 요리시간 등을 고려하여 레시피를 추천해드리고, 단계별 요리 지침과 와인 페어링 팁도
제공합니다. ",
15        "deployedAt": "2025-05-19T08:58:45.679785+09:00",
16        "modifiedAt": "2025-05-05T14:51:56.133896+09:00"
17    },
18    {
19        "agentId": "0197f289-a2fd-7daf-89e0-e27e13dd7ffd",
20        "label": "RAG 요리사 봇",
21        "subType": "MULTI"
22        "description": "안녕하세요, RAG 봇입니다. 요리 아이디어와 지침이 필요하신 분들께
도와드리겠습니다. 재료, 식이 제한, 요리 시간에 따라 맞춤형 레시피를 제공하고, 요리 지침과 와인
페어링 팁을 제공해드리겠습니다. 하지만 실시간 영양 정보나 식이 추적 기능은 없으니 이 점
참고해주시기 바랍니다.",
23        "deployedAt": "2025-05-19T15:35:56.153453+09:00",
24        "modifiedAt": "2025-05-05T14:19:24.034232+09:00"
25    },
26    ...(생략)

```

| 호출 예시 (python)

```

1 import requests
2
3 YOUR_CLIENT_KEY = ""
4 YOUR_PASS_KEY = ""
5 ENDPOINT_URL = ""
6 YOUR_EMAIL = ""
7

```

```

8  headers = {
9      "x-fabrix-client": YOUR_CLIENT_KEY,
10     "x-openapi-token": YOUR_PASS_KEY,
11     "x-generative-ai-user-email": YOUR_EMAIL
12 }
13
14 page = 1
15 limit = 10
16 keyword = None
17
18 query_params = f"page={page}&limit={limit}"
19
20 if keyword is not None:
21     query_params += f"&keyword={keyword}"
22
23 api_endpoint_url = f"{ENDPOINT_URL}/openapi/agent-chat/v1/agents?{query_params}"
24
25 response = requests.get(
26     api_endpoint_url,
27     headers=headers
28 )
29
30 # 응답 반환
31 print(response.json())

```

GET /openapi/orchestrator/v1/a2a/{agent_id}/.well-known/agent-card.json

Header

Field	Type	Required	Description	Value
Content-Type	string	optional	Type of content	application/json
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-openapi-token	string	required	OpenAPI Token information	ex) Bearer eyJXXX
x-generative-ai-user-email	string	optional	Portal user email	ex) kim.samsung@samsung.com

Field	Type	Required	Description	Value
A2A-Version	string	required	Version of A2A	1.0

Response

Field	Type	Description
name	string	agent 이름
description	string	agent 설명
supportedInterfaces	list[AgentInterface]	지원 인터페이스 목록
provider	AgentProvider	agent 제공자 정보
version	string	agent card 버전
documentationUrl	string	추가 문서 URL
capabilities	AgentCapabilities	streaming 지원 여부 등 capability 정보
securitySchemes	SecuritySchemes	JWT Bearer 등 보안 스키마 정의
securityRequirements	SecurityRequirements	적용 보안 요구사항
defaultInputModes	list[string]	기본 입력 MIME 타입 목록
defaultOutputModes	list[string]	기본 출력 MIME 타입 목록
skills	list[AgentSkill]	agent가 제공하는 skill 목록
signatures	list[AgentCardSignature]	card signature 정보
iconUrl	string	agent 아이콘 URL

Type <AgentInterface>

Field	Type	Required	Description
url	string	required	이 인터페이스를 사용할 수 있는 URL. 유효한 절대 HTTPS URL ex: "https://grpc.example.com/a2a"
protocolBinding	string	required	URL에서 지원되는 프로토콜 바인딩. JSONRPC만 지원
tenant	string	optional	agent 연결 시 요청에 사용할 테넌트 ID
protocolVersion	string	required	인터페이스에서 노출하는 A2A 프로토콜 버전. "1.0"만 지원.

Type <AgentProvider>

Field	Type	Required	Description
url	string	required	에이전트 제공업체의 웹사이트 또는 관련 문서의 URL 예: "https://example.com"
organization	string	required	에이전트 제공업체 조직의 이름.

Type <AgentCapabilities>

표 9-1 Type<AgentCapabilities>

Field	Type	Required	Description
streaming	boolean	optional	에이전트가 스트리밍 응답을 지원하는지 여부를 나타냄
pushNotifications	boolean	optional	에이전트가 비동기 작업 업데이트에 대한 푸시 알림 전송을 지원하는지 여부를 나타냄
extensions	list[AgentExtension]	optional	에이전트에서 지원하는 프로토콜 확장 목록
extendedAgentCard	boolean	optional	인증 시 확장 에이전트 카드 제공을 지원하는지 여부를 나타냄

Type <AgentSkill>

Field	Type	Required	Description
id	string	required	에이전트의 스킬을 식별하는 고유 식별자
name	string	required	스킬 이름
description	string	required	스킬에 대한 자세한 설명
tags	list[string]	required	스킬의 기능을 설명하는 키워드 집합
examples	list[string]	optional	스킬이 처리할 수 있는 예시 프롬프트 또는 시나리오
inputModes	list[string]	optional	스킬에서 지원되는 입력 미디어 유형 집합
outputModes	list[string]	optional	스킬에서 지원하는 출력 미디어 유형 집합
securityRequirements	list[SecurityRequirement]	optional	이 기술에 필요한 보안 체계

Type <AgentCardSignature>

Field	Type	Required	Description
protected	string	required	서명을 위한 보호된 JWS 헤더. 이 헤더는 항상 base64url로 인코딩된 JSON 객체
signature	string	required	계산된 서명을 base64로 URL 인코딩한 값
header	object	optional	보호되지 않은 JWS 헤더 값

Response 예시

```

1  {
2    "name": "A2A Agent",
3    "description": "A2A Agent",
4    "supportedInterfaces": [
5      {
6        "url": "http://example.com/a2a",
7        "protocolBinding": "JSONRPC",
8        "protocolVersion": "1.0"
9      }
10   ],
11   "version": "1.7.0",
12   "capabilities": {
13     "streaming": true,
14     "pushNotifications": false,
15     "extendedAgentCard": false
16   },
17   "securitySchemes": {
18     "bearerAuth": {
19       "httpAuthSecurityScheme": {
20         "scheme": "bearer",
21         "bearerFormat": "JWT"
22       }
23     }
24   },
25   "securityRequirements": [
26     {
27       "schemes": {
28         "bearerAuth": { }

```

```

29     }
30   }
31 ],
32 "defaultInputModes": [
33   "application/json"
34 ],
35 "defaultOutputModes": [
36   "application/json"
37 ],
38 "skills": [
39   {
40     "id": "019daf80-1674-75b9-ab11-c5e1c07fafa2",
41     "name": "인사",
42     "description": "인사를 잘한다",
43     "examples": [
44       "안녕하세요"
45     ],
46     "inputModes": [
47       "application/json"
48     ],
49     "outputModes": [
50       "application/json"
51     ]
52   }
53 ]
54 }

```

| 호출 예시 (python)

```

1  import requests
2
3  YOUR_CLIENT_KEY = ""
4  YOUR_PASS_KEY = ""
5  ENDPOINT_URL = ""
6  AGENT_ID = ""
7  YOUR_EMAIL = ""
8
9  headers: dict = {
10     "x-fabrix-client": YOUR_CLIENT_KEY,

```

```

11     "x-openapi-token": YOUR_PASS_KEY,
12     "x-generative-ai-user-email": YOUR_EMAIL,
13     "A2A-Version" : "1.0"
14 }
15
16 api_endpoint_url =
17 f"{ENDPOINT_URL}/openapi/orchestrator/v1/a2a/{AGENT_ID}/.well-known/agent-card.json"
18
19 response: requests.Response = requests.get(
20     api_endpoint_url,
21     headers=headers
22 )
23 # 응답 반환
24 print(response.json())

```

A2A Spec 제약사항

이 문서는 A2A 1.0.0 기준으로 작성되었습니다.

기본적으로 A2A Spec을 준수하며, A2A Spec 관련 상세 내용은 아래 링크를 참고하세요.

<https://a2a-protocol.org/latest/specification/#1-introduction>

본 문서는 제약사항 및 Custom Logic에 초점을 맞추어 작성되어있습니다.

transport는 JSONRPC만 지원합니다.

현재 2개 method만 지원합니다.

- SendMessage

- SendStreamingMessage

주의 : 구버전 slash 표기인 message/send, message/stream method는 지원되지 않습니다.

Agent 호출 시, 내부적으로 필터가 동작하니 참고하세요. 필터 정책은 Admin이 관리합니다.

POST /openapi/orchestrator/v1/a2a/{agent_id}

Header

Field	Type	Required	Description	Value
Content-Type	string	optional	Type of content	application/json
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-openapi-token	string	required	OpenAPI Token information	ex) Bearer eyJXXX
x-generative-ai-user-email	string	optional	Portal user email	ex) kim.samsung@samsung.com
A2A-Version	string	required	Version of A2A	1.0

Request Body

표 9-2 JSON-RPC Envelope

Field	Type	Required	Description	Value
jsonrpc	string	required	JSON-RPC 버전	"2.0" (고정값)
id	string integer	required	클라이언트가 생성한 고유 식별자	
method	string	required	메서드 명	SendMessage, SendStreamingMessage, GetTask, CancelTask만 지원
params	SendMessageRequest	required	메세지 전송 파라미터	

Type<SendMessageRequest>

Field	Type	Required	Description
tenant	string	optional	테넌트 ID는 경로 매개변수로 제공
message	Message	required	Agent에게 보낼 메시지
configuration	SendMessageConfiguration	optional	전송 요청에 대한 구성
metadata	object	optional	추가적인 컨텍스트나 매개변수를 전달하기 위한 유연한 키-값 맵

Type<Message>

Field	Type	Required	Description
messageId	string	required	메시지의 고유 식별자(UUID). 메시지 생성자가 생성
contextId	string	required	context id(UUID)
referenceTaskIds	list	optional	agent 내에서 하위 agent를 호출하는 경우 본인의 task id 전달 필요
role	Role	required	메시지 발신자를 식별
parts	list[Part]	required	Parts는 메시지 내용을 담는 컨테이너
metadata	object	optional	메세지와 함께 제공할 메타데이터가 있으면 입력

Type <Role>

Value	Description
ROLE_UNSPECIFIED	역할은 구체적으로 명시되지 않음.
ROLE_USER	클라이언트에서 서버로 전송되는 메시지
ROLE_AGENT	서버에서 클라이언트로 전송

Type <Parts>

Field	Type	Required	Description
metadata	Object	optional	파트와 관련된 메타데이터
text	string	optional(Oneof)	테스트 본문
raw	bytes	optional(Oneof)	raw binary payload
data	any	optional(Oneof)	구조화 데이터

Request 예시

```

1 {
2   "jsonrpc": "2.0",
3   "method": "SendMessage",
4   "params": {
5     "message": {
6       "contextId": "01491f0b-7623-441f-bcde-f12a76badac7",
7       "messageId": "b039c301-f46f-4d1e-9e1f-7ceac721dd1b",
8       "role": "ROLE_USER",

```

```

9      "parts": [
10      {
11          "text": "안녕하세요?"
12      }
13  ]
14  }
15  }
16  }

```

Response

표 9-3 JSON-RPC Envelope

Field	Type	Required	Description	Value
jsonrpc	string	Y	JSON-RPC 버전	"2.0"
id	string integer	Y	요청 시 전송한 ID	
result	Task	Y	생성 된 작업	

Type <Task>

Field	Type	Required	Description
id	string	required	task ID(UUID)
contextId	string	optional	task가 속한 컨텍스트 ID(UUID)
status	TaskStatus	required	현재 task 상태
artifacts	list[Artifact]	optional	최종/중간 산출물 목록
metadata	object	optional	task metadata

Type <TaskStatus>

Field	Type	Required	Description
state	TaskState	required	현재 상태
message	Message	optional	상태와 관련된 메시지
timestamp	timestamp	optional	상태가 기록된 시간 ex.2023-10-27T10:00:00Z

Type <TaskState>

value	Description
TASK_STATE_UNSPECIFIED	해당 작업은 알 수 없거나 불확정적인 상
TASK_STATE_SUBMITTED	작업이 성공적으로 제출되고 확인되었음(대기 중)

value	Description
TASK_STATE_WORKING	에이전트가 해당 작업을 활발히 처리 중
TASK_STATE_COMPLETED	작업이 성공적으로 완료되었음. 최종 상태
TASK_STATE_FAILED	작업이 오류와 함께 종료. 최종 상태
TASK_STATE_CANCELED	작업이 완료되기 전에 취소되었음. 최종 상태
TASK_STATE_INPUT_REQUIRED	에이전트를 진행하기 위해 추가 사용자 입력이 필요함. 중단된 상태
TASK_STATE_REJECTED	에이전트가 작업을 수행하지 않기로 결정. 초기 작업 생성 시 또는 에이전트가 작업을 진행할 수 없거나 진행하지 않기로 결정한 후에 발생. 중단된 상태
TASK_STATE_AUTH_REQUIRED	인증이 필요함. 중단된 상태

Type <Artifact>

Field	Type	Required	Description
artifactId	string	required	아티팩트에 대한 고유 식별자(예: UUID). 태스크 내에서 고유한 값.
name	string	optional	아티팩트에 대한 이름
description	string	optional	아티팩트에 대한 설명
parts	list[Part]	required	아티팩트의 내용. 최소 하나의 파트를 포함해야함
metadata	object	optional	아티팩트에 대한 메타데이터
extensions	list[string]	optional	아티팩트에 존재하거나 추가된 확장 프로그램의 URI

노트

Response 예시는 method 별로 하단에 정리되어있습니다.

Send Streaming Message

Request 예시

```

1 {
2   "id": "msg-1",
3   "jsonrpc": "2.0",
4   "method": "SendStreamingMessage",

```

```

5  "params": {
6    "message": {
7      "contextId": "01491f0b-7623-441f-bcde-f12a76badac7",
8      "messageId": "b039c301-f46f-4d1e-9e1f-7ceac721dd1b",
9      "role": "ROLE_USER",
10     "parts": [
11       {
12         "text": "안녕하세요?,"
13       }
14     ]
15   }
16 }
17 }

```

| 호출 예시 (python)

```

1  import requests
2  import uuid
3
4  YOUR_CLIENT_KEY = ""
5  YOUR_PASS_KEY = ""
6  ENDPOINT_URL = ""
7  AGENT_ID = ""
8  YOUR_EMAIL = ""
9
10 headers: dict = {
11     "x-fabrix-client": YOUR_CLIENT_KEY,
12     "x-openapi-token": YOUR_PASS_KEY,
13     "x-generative-ai-user-email": YOUR_EMAIL,
14     "A2A-Version" : "1.0"
15 }
16
17 api_endpoint_url = f"{ENDPOINT_URL}/openapi/orchestrator/v1/a2a/{AGENT_ID}"
18
19 contextId = str(uuid.uuid4())
20 messageId = str(uuid.uuid4())
21
22 payload = {
23     "id": "msg-1",

```

```

24  "jsonrpc": "2.0",
25  "method": "SendStreamingMessage",
26  "params": {
27    "message": {
28      "contextId": contextId,
29      "messageId": messageId,
30      "role": "ROLE_USER",
31      "parts": [
32        {
33          "text": "안녕하세요?",
34        }
35      ]
36    }
37  }
38 }
39
40 response = requests.post(url, headers = headers, json=payload)
41 response: requests.Response = requests.post(
42     api_endpoint_url,
43     headers=headers
44 )
45
46 # 응답 반환
47 print(response.json())

```

| Response 예시

```

1  data: {
2    "result": {
3      "task": {
4        "id": "6ca4e05e-c62d-405a-a205-b7185775aba1",
5        "contextId": "108347f7-2cd7-4510-8e0e-e42b337bc2ee",
6        "status": {
7          "state": "TASK_STATE_SUBMITTED"
8        }
9      }
10   },
11   "id": "msg-1",
12   "jsonrpc": "2.0"

```

```

13 }
14
15 data: {
16   "result": {
17     "statusUpdate": {
18       "taskId": "6ca4e05e-c62d-405a-a205-b7185775aba1",
19       "contextId": "108347f7-2cd7-4510-8e0e-e42b337bc2ee",
20       "status": {
21         "state": "TASK_STATE_WORKING",
22         "timestamp": "2026-04-24T06:19:03.265990Z"
23       }
24     }
25   },
26   "id": "msg-1",
27   "jsonrpc": "2.0"
28 }
29 data: {
30   "result": {
31     "artifactUpdate": {
32       "taskId": "6ca4e05e-c62d-405a-a205-b7185775aba1",
33       "contextId": "108347f7-2cd7-4510-8e0e-e42b337bc2ee",
34       "artifact": {
35         "artifactId": "final_answer",
36         "name": "final_answer",
37         "description": "Final answer",
38         "parts": [
39           {
40             "text": "안녕하세요?"
41           }
42         ]
43       }
44     }
45   },
46   "id": "msg-1",
47   "jsonrpc": "2.0"
48 }
49
50 data: {
51   "result": {
52     "artifactUpdate": {

```

```

53     "taskId": "6ca4e05e-c62d-405a-a205-b7185775aba1",
54     "contextId": "108347f7-2cd7-4510-8e0e-e42b337bc2ee",
55     "artifact": {
56         "artifactId": "final_answer",
57         "name": "final_answer",
58         "description": "Final answer",
59         "parts": [
60             {
61                 "text": "반가워요."
62             }
63         ]
64     },
65 },
66 {
67     "id": "msg-1",
68     "jsonrpc": "2.0"
69 }
70
71 data: {
72     "result": {
73         "statusUpdate": {
74             "taskId": "6ca4e05e-c62d-405a-a205-b7185775aba1",
75             "contextId": "108347f7-2cd7-4510-8e0e-e42b337bc2ee",
76             "status": {
77                 "state": "TASK_STATE_COMPLETED",
78                 "message": {
79                     "messageId": "7e750d65-18a1-4213-aaaf-af9762244d1a",
80                     "contextId": "108347f7-2cd7-4510-8e0e-e42b337bc2ee",
81                     "taskId": "6ca4e05e-c62d-405a-a205-b7185775aba1",
82                     "role": "ROLE_AGENT",
83                     "parts": [
84                         {
85                             "text": "COMPLETED"
86                         }
87                     ],
88                     "metadata": {
89                         "status": "RESPONSE"
90                     }
91                 },
92                 "timestamp": "2026-04-24T06:19:03.476802Z"

```

```

93     }
94     }
95 },
96 "id": "msg-1",
97 "jsonrpc": "2.0"
98 }

```

Send Message

| Request 예시

```

1  {
2    "id": "msg-1",
3    "jsonrpc": "2.0",
4    "method": "SendMessage",
5    "params": {
6      "message": {
7        "contextId": "01491f0b-7623-441f-bcde-f12a76badac7",
8        "messageId": "b039c301-f46f-4d1e-9e1f-7ceac721dd1b",
9        "role": "ROLE_USER",
10       "parts": [
11         {
12           "text": "안녕하세요?",
13         }
14       ]
15     }
16   }
17 }

```

| 호출 예시 (python)

```

1  import requests
2  import uuid
3
4  YOUR_CLIENT_KEY = ""
5  YOUR_PASS_KEY = ""
6  ENDPOINT_URL = ""
7  AGENT_ID = ""

```



```

8 YOUR_EMAIL = ""
9
10 headers: dict = {
11     "x-fabrix-client": YOUR_CLIENT_KEY,
12     "x-openapi-token": YOUR_PASS_KEY,
13     "x-generative-ai-user-email": YOUR_EMAIL,
14     "A2A-Version" : "1.0"
15 }
16
17 api_endpoint_url = f"{ENDPOINT_URL}/openapi/orchestrator/v1/a2a/{AGENT_ID}"
18
19 contextId = str(uuid.uuid4())
20 messageId = str(uuid.uuid4())
21
22 payload = {
23     "id": "msg-1",
24     "jsonrpc": "2.0",
25     "method": "SendMessage",
26     "params": {
27         "message": {
28             "contextId": contextId,
29             "messageId": messageId,
30             "role": "ROLE_USER",
31             "parts": [
32                 {
33                     "text": "안녕하세요?,"
34                 }
35             ]
36         }
37     }
38 }
39
40 response = requests.post(url, headers = headers, json=payload)
41 response: requests.Response = requests.post(
42     api_endpoint_url,
43     headers=headers
44 )
45
46 # 응답 반환
47 print(response.json())

```

| Response 예시

```

1  {
2    "result": {
3      "task": {
4        "id": "fb58f7a6-7516-4fb1-a023-2d785015fdaa",
5        "contextId": "cd3c5868-dac7-4463-ad25-40c6f53d6c1a",
6        "status": {
7          "state": "TASK_STATE_COMPLETED",
8          "message": {
9            "messageId": "5b33746d-b3cf-4b3d-9584-99fabf7b5688",
10           "contextId": "cd3c5868-dac7-4463-ad25-40c6f53d6c1a",
11           "taskId": "fb58f7a6-7516-4fb1-a023-2d785015fdaa",
12           "role": "ROLE_AGENT",
13           "parts": [
14             {
15               "text": "COMPLETED"
16             }
17           ],
18           "metadata": {
19             "status": "RESPONSE"
20           }
21         },
22         "timestamp": "2026-04-24T08:14:17.010622Z"
23       },
24       "artifacts": [
25         {
26           "artifactId": "final_answer",
27           "name": "final_answer",
28           "description": "Final answer",
29           "parts": [
30             {
31               "text": "안녕하세요? 반가워요."
32             }
33           ]
34         }
35       ]
36     },
37   },
38   "id": "msg-1",

```

```

39     "jsonrpc": "2.0"
40 }

```

Agent 요청시 첨부파일 추가하기

| 호출 예시 (python)

```

1  import requests
2  import uuid
3
4  YOUR_CLIENT_KEY = ""
5  YOUR_PASS_KEY = ""
6  ENDPOINT_URL = ""
7  AGENT_ID = ""
8  YOUR_EMAIL = ""
9
10
11 headers: dict = {
12     "x-fabrix-client": YOUR_CLIENT_KEY,
13     "x-openapi-token": YOUR_PASS_KEY,
14     "x-generative-ai-user-email": YOUR_EMAIL,
15     "A2A-Version" : "1.0"
16 }
17
18 api_endpoint_url = f"{ENDPOINT_URL}/openapi/orchestrator/v1/a2a/{AGENT_ID}"
19 FILE_PATH1 = "../file/파일1.csv"
20 file_path1 = Path(FILE_PATH1)
21 mime_type1 = "text/csv"
22
23 FILE_PATH2 = "../file/image.png"
24 file_path2 = Path(FILE_PATH2)
25 mime_type2 = "image/png"
26
27 with open(file_path1, "rb") as f:
28     encoded_file1 = base64.b64encode(f.read()).decode("utf-8")
29 with open(file_path2, "rb") as f:
30     encoded_file2 = base64.b64encode(f.read()).decode("utf-8")
31
32 payload={

```

```

33  "jsonrpc": "2.0",
34  "method": "SendStreamingMessage",
35  "params": {
36    "message": {
37      "role": "ROLE_USER",
38      "parts": [
39        {
40          "text": "파일1을 분석하고, image.png는 무슨 그림인지 분석해줘."
41        },
42        {
43          "raw": encoded_file1,
44          "filename": file_path1.name,
45          "mediaType": mime_type1,
46        },
47        {
48          "raw": encoded_file2,
49          "filename": file_path2.name,
50          "mediaType": mime_type2,
51        }
52      ],
53      "messageId": str(uuid.uuid4()),
54      "metadata": {
55      }
56    }
57  }
58 }
59
60
61 response = requests.post(
62     API_URL,
63     headers=headers,
64     json=payload,
65     timeout=600,
66 )
67
68 # 응답 반환
69 print(response.json())

```

표 9-4 업로드 가능한 파일 확장자 및 MIME Type

확장자	MIME Type
txt	text/plain
docx	application/vnd.openxmlformats-officedocument.wordprocessingml.document
xlsx	application/vnd.openxmlformats-officedocument.spreadsheetml.sheet
pdf	application/pdf
html	text/html
csv	text/csv
json	application/json
hwp	application/x-hwp
hwpx	application/haansofthwp
jpg/jpeg	image/jpeg
png	image/png
pptx	application/vnd.openxmlformats-officedocument.presentationml.presentation
md	text/markdown

주의

파일 관련 제약사항

- 동시 업로드는 최대 5개
- 용량은 최대 3MB

Input Required 발생 및 Session 재개

Input Required 발생 및 Session 재개 시나리오는 다음과 같습니다.

(첫번째 Turn)

Agent에 질문을 합니다. -> 응답으로 TASK_STATE_INPUT_REQUIRED state를 수신합니다.

(두번째 Turn)

첫번째 턴에서 받은 응답의 taskId와 contextId를 요청에 포함시켜, Agent에 재요청합니다.

| 첫번째 Turn의 Request

```

1 {
2   "id": "msg-1",
3   "jsonrpc": "2.0",
4   "method": "SendMessage",
5   "params": {
6     "message": {
7       "messageId": "be88612c-7556-4483-8986-fccb27a9af86",
8       "role": "ROLE_USER",
9       "parts": [
10        {
11          "text": "호텔을 예약하고 싶어."
12        }
13      ]
14    }
15  }
16 }

```

| 첫번째 Turn의 Response

```

1 {
2   "result": {
3     "task": {
4       "id": "97968eff-abf6-405a-8ee1-e11aaa7c7041",
5       "contextId": "04c84289-a936-4ff9-9982-07fe42ca5750",
6       "status": {
7         "state": "TASK_STATE_INPUT_REQUIRED",
8         "message": {
9           "messageId": "9ef2ec92-7610-41c6-a125-e05091c0bb99",
10          "contextId": "04c84289-a936-4ff9-9982-07fe42ca5750",
11          "taskId": "97968eff-abf6-405a-8ee1-e11aaa7c7041",
12          "role": "ROLE_AGENT",
13          "parts": [
14            {
15              "text": "호텔 예약을 위해서는 예약자 이름, 체크인 날짜, 체크아웃 날짜, 숙박
지역(도시) 및 인원 수가 필요합니다. 해당 정보를 알려 주세요."
16            }
17          ],
18          "metadata": {
19            "status": "INPUT_REQUIRED"

```

```

20     }
21     },
22     "timestamp": "2026-06-10T13:28:40.482886Z"
23   }
24 }
25 },
26 "id": "id123",
27 "jsonrpc": "2.0"
28 }

```

| 두번째 Turn의 Request

```

1  {
2    "id" : "id123",
3    "jsonrpc": "2.0",
4    "method": "SendMessage",
5    "params": {
6      "message": {
7        "role": "ROLE_USER",
8        "parts": [
9          {
10             "text": "2026년 6월 25~30까지 뉴욕 인터콘티넨탈 호텔 예약해줘 방은
스텐다드로 1개, 예약자 명은 홍길동, 투숙인원은 2명"
11         }
12       ],
13       "taskId": "97968eff-abf6-405a-8ee1-e11aaa7c7041",
14       "contextId": "04c84289-a936-4ff9-9982-07fe42ca5750",
15       "messageId": "944fa9ad-d0b2-4b0d-8e84-ab424d8546a8",
16       "metadata": {
17         }
18     }
19   }
20 }

```

| 두번째 Turn의 Response

```

1  {
2    "result": {

```

```

3    "task": {
4      "id": "97968eff-abf6-405a-8ee1-e11aaa7c7041",
5      "contextId": "04c84289-a936-4ff9-9982-07fe42ca5750",
6      "status": {
7        "state": "TASK_STATE_COMPLETED",
8        "message": {
9          "messageId": "9ef2ec92-7610-41c6-a125-e05091c0bb99",
10         "contextId": "04c84289-a936-4ff9-9982-07fe42ca5750",
11         "taskId": "97968eff-abf6-405a-8ee1-e11aaa7c7041",
12         "role": "ROLE_AGENT",
13         "parts": [
14           {
15             "text": "COMPLETED"
16           }
17         ],
18         "metadata": {
19           "status": "RESPONSE"
20         }
21       },
22       "timestamp": "2026-06-10T13:28:40.482886Z"
23     },
24     "artifacts": [
25       {
26         "artifactId": "final_answer",
27         "name": "final_answer",
28         "description": "Final answer",
29         "parts": [
30           {
31             "text": "홍길동님, 뉴욕 인터콘티넨탈 호텔, 스텐다드 룸 1개 예약했습니다.
예약자 : 홍길동, 투숙 인원 : 2명, 체크인 : 2026년 6월 25일, 체크아웃 : 2026년 6월 30일 입니다.
"
32           }
33         ]
34       }
35     ]
36   }
37 },
38 "id": "id123",
39 "jsonrpc": "2.0"
40 }

```


Auth Required 발생

Auth Required 발생 및 Session 재개 시나리오는 다음과 같습니다.

(첫번째 Turn)

Agent에 질문을 합니다. -> 응답으로 TASK_STATE_AUTH_REQUIRED state를 수신합니다.

(두번째 Turn)

첫번째 턴에서 받은 응답의 taskId와 contextId를 요청에 포함시켜, Agent에 재요청합니다.

공통 Response Code

JSON-RPC 표준 에러 코드

코드	에러	설명
-32700	JSONParseError	잘못된 JSON 페이로드
-32600	InvalidRequestError	잘못된 요청 객체
-32601	MethodNotFoundError	메서드를 찾을 수 없음
-32602	InvalidParamsError	잘못된 파라미터
-32603	InternalError	내부 서버 에러

A2A 프로토콜 에러 코드

코드	에러	설명
-32001	TaskNotFoundError	작업을 찾을 수 없음
-32002	TaskNotCancelableError	작업을 취소할 수 없음
-32003	PushNotificationNotSupportedError	푸시 알림이 지원되지 않음
-32004	UnsupportedOperationError	지원하지 않는 작업
-32005	ContentTypeNotSupportedError	호환되지 않는 콘텐츠 타입
-32006	InvalidAgentResponseError	잘못된 에이전트 응답
-32007	AuthenticatedExtendedCardNotConfiguredError	인증된 확장 카드 미설정

9.3 FAQ

■ 추가 method 지원 시점

GetTask, CancelTask method는 8월 경 지원 예정입니다.

■ 비동기 실행 지원 시점

8월 경 지원 예정입니다.

10.

Data Ingestion APIs

10.1 Data Ingestion APIs Overview

Data Ingestion APIs는 Data Ingestion Spec을 이용한 파일 업로드 및 전처리 호출 서비스를 제공합니다.

Data Ingestion APIs 에서 제공하는 API는 다음과 같습니다.

GET /openapi/data/v1/data-ingestions/labs/{labId}/folders/tree

생성된 폴더 트리를 조회할 수 있습니다.

POST /openapi/data/v1/data-ingestions/folders/{folderId}

부모 폴더에 자식 폴더를 생성 할 수 있습니다,

POST /openapi/data/v1/data-ingestions/folders/{folderId}/files

생성된 폴더내 파일을 업로드(다중 파일 제공) 할 수 있습니다.

GET /openapi/data/v1/data-ingestions/folders/{folderId}/files

생성된 폴더내 관리되는 파일 리스트를 조회 할 수 있습니다.

POST /openapi/data/v1/data-ingestions/folders/{folderId}/files/status

생성된 폴더내 업로드 중인 파일 처리(다중 파일 제공) 상태를 조회 할 수 있습니다.

GET /openapi/data/v1/data-ingestions/files/{fileId}

파일 정보를 조회 할 수 있습니다.

PUT /openapi/data/v1/data-ingestions/files/{fileId}

파일 정보를 수정 할 수 있습니다.

DELETE /openapi/data/v1/data-ingestions/files/{fileId}

파일을 삭제 할 수 있습니다.

DELETE /openapi/data/v1/data-ingestions/files

파일을 다중으로 삭제 할 수 있습니다.

10.2 API 명세

G E T
/openapi/data/v1/data-ingestions/labs/{labId}/folders/tree

Header

Field	Type	Required	Description	Value
Content-Type	string	optional	Type of content	application/json
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-openapi-token	string	required	OpenAPI Token information	ex) Bearer eyJXXX

Query Parameter

Key	Type	Default	Description	Default Value
labId	String	Required	개인이 속한 Lab ID	

Response

Field	Type	Description
meta	ApiResponse	

Type <ApiResponse>

Field	Type	Description
success	boolean	
data	object[UploadResult]	
error		

Type <UploadResult>

Field	Type	Description
total	Integer	
result	list[TreeData]	uuid
error	string	

Type <TreeData>

Field	Type	Description
title	string	
key	string	uuid
uuid	string	
file_cnt	Integer	
file_data	string	
crt_user	string	
upd_user	string	
parent	string	
depth	string	
desc	string	
crt_ts	string	"yyyy-mm-dd"
children	list[TreeData]	

Response 예시

```

1  {
2    'success': True,
3    'data': {
4      'total': 1,
5      'result': [
6        {
7          'title': 'root',
8          'key': 'root/',
9          'uuid': 'b52867f4-396b-4385-92bc-766fd8c13b19',
10         'file_cnt': 0,
11         'file_data': 'N',
12         'crt_user': ' 아무개',
13         'upd_user': '아무개',
14         'parent': '',
15         'depth': '1',
16         'desc': 'root',
17         'crt_ts': '2026-03-03',
18         'children': [
19           {
20             'title': 'tc',
21             'key': 'root/tc/',
22             'uuid': '6a0c9a8f-3735-49de-9bdf-afac8a9bfd39',

```

```

23         'file_cnt': 15,
24         'file_data': 'Y',
25         'crt_user': '아무개',
26         'upd_user': '아무개',
27         'parent': 'root/',
28         'depth': '2',
29         'desc': '',
30         'crt_ts': '2026-06-15',
31         'children': []
32     },
33 ]
34 }
35 ]
36 },
37 'error': None
38 }

```

| 호출 예시 (python)

```

1  import requests
2
3  YOUR_CLIENT_KEY = ""
4  YOUR_PASS_KEY = ""
5  ENDPOINT_URL = ""
6
7  headers = {
8      "x-fabrix-client": YOUR_CLIENT_KEY,
9      "x-openapi-token": YOUR_PASS_KEY
10 }
11
12 LAB_ID = ""
13
14 api_endpoint_url =
15 f"{ENDPOINT_URL}/openapi/data/v1/data-ingestions/labs/{LAB_ID}/folders/tree"
16
17 response = requests.get(
18     api_endpoint_url,
19     headers=headers
20 )

```

```

20
21 # 응답 반환
22 print(response.json())

```

POST /openapi/data/v1/data-ingestions/folders/{folderId}

Header

Field	Type	Required	Description	Value
Content-Type	string	optional	Type of content	application/json
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-openapi-token	string	required	OpenAPI Token information	ex) Bearer eyJXXX

Query Parameter

Key	Type	Default	Description	Default Value
folderId	String	Required	root 대상 folderId	

Request Body

Field	Type	Description
targetPath	string	folder3을 생성한다고 치면 부모 노드까지 포함하여 "root/folder3/" 로 구성
folderDesc	string	

Response

Field	Type	Description
meta	ApiResponse	

Type <ApiResponse>

Field	Type	Description
success	boolean	
data	object[FolderResult]	
error		

Type <FolderResult>

Field	Type	Description
folderId	string	
fullPath	string	
folderDesc	string	

Response 예시

```

1 {
2     'success': True,
3     'data': {
4         'folderId': 'f63bbc88-d8e3-4f87-bd7b-aebea0b598c3',
5         'fullPath': 'root/folder3/',
6         'folderDesc': ''},
7     'error': None
8 }
```

호출 예시 (python)

```

1 import requests
2
3 YOUR_CLIENT_KEY = ""
4 YOUR_PASS_KEY = ""
5 ENDPOINT_URL = ""
6 ROOT_FOLDER_ID = ""
7
8 headers: dict = {
9     "x-fabrix-client": YOUR_CLIENT_KEY,
10    "x-openapi-token": YOUR_PASS_KEY
11 }
12
13 api_endpoint_url =
14 f"{ENDPOINT_URL}/openapi/data/v1/data-ingestions/folders/{ROOT_FOLDER_ID}"
15
16 payload = {
17     "targetPath": "root/folder3/",
18     "folderDesc": "",
19 }
```



```

19
20 response: requests.Response = requests.get(
21     api_endpoint_url,
22     headers=headers,
23     json=payload
24 )
25
26 # 응답 반환
27 print(response.json())

```

POST /openapi/data/v1/data-ingestions/folders/{folderId}/files

Header

Field	Type	Required	Description	Value
Content-Type	string	optional	Type of content	application/json
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-openapi-token	string	required	OpenAPI Token information	ex) Bearer eyJXXX

Query Parameter

Key	Type	Default	Description	Default Value
folderId	String	Required	root 대상 folderId	

Request Body

Field	Type	Description
parserId	string	Default: FABRIX
sourceNavi	string	
url	string	
tags	string	
documentPurpose	string	
additionalInfo	object	JSON format
filtersInfo	object	JSON format
referUrls	object	JSON format
useOcr	bool	
useLmm	bool	

Field	Type	Description
useTsr	bool	
ragMode	string	
parsingTimeout	Integer	

Multipart form

Field	Type	Description
files	list[filename, binary]	

Response 예시

```

1  {
2      'success': True,
3      'data': {
4          'total': 1,
5          'result': [
6              {
7                  'folderId': 'f63bbc88-d8e3-4f87-bd7b-aebea0b598c3',
8                  'fileId': '0ef7c074-bb79-4973-8c85-5fa498e85025',
9                  'fileName': 'openapi_upload_20260626_091729.txt',
10                 'parserId': '8d8f1e0d-5690-482f-a4f3-0584310c358c',
11                 'status': 'EXECUTING',
12                 'detailStatus': 'UPLOADING',
13                 'errorMessage': ''}
14             ]
15         }
16     }

```

호출 예시 (python)

```

1  import requests
2
3  YOUR_CLIENT_KEY = ""
4  YOUR_PASS_KEY = ""
5  ENDPOINT_URL = ""
6  ROOT_FOLDER_ID = ""
7

```

```

8  headers: dict = {
9      "x-fabrix-client": YOUR_CLIENT_KEY,
10     "x-openapi-token": YOUR_PASS_KEY
11 }
12
13 api_endpoint_url =
14 f"{ENDPOINT_URL/openapi/data/v1/data-ingestions/folders/{ROOT_FOLDER_ID}"
15
16 payload = {
17     "targetPath": "root/folder3/",
18     "folderDesc": "",
19 }
20
21 response: requests.Response = requests.get(
22     api_endpoint_url,
23     headers=headers,
24     json=payload
25 )
26 # 응답 반환
27 print(response.json())

```

POST /openapi/orchestrator/v1/a2a/{agent_id}

Header

Field	Type	Required	Description	Value
Content-Type	string	optional	Type of content	application/json
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-openapi-token	string	required	OpenAPI Token information	ex) Bearer eyJXXX
x-generative-ai-user-email	string	optional	Portal user email	ex) kim.samsung@samsung.com
A2A-Version	string	required	Version of A2A	1.0

Request Body

표 10-1 JSON-RPC Envelope

Field	Type	Required	Description	Value
jsonrpc	string	required	JSON-RPC 버전	"2.0" (고정값)
id	string integer	required	클라이언트가 생성한 고유 식별자	
method	string	required	메서드 명	SendMessage, SendStreamingMessage, GetTask, CancelTask만 지원
params	SendMessageRequest	required	메세지 전송 파라미터	

Type<SendMessageRequest>

Field	Type	Required	Description
tenant	string	optional	테넌트 ID는 경로 매개변수로 제공
message	Message	required	Agent에게 보낼 메시지
configuration	SendMessageConfiguration	optional	전송 요청에 대한 구성
metadata	object	optional	추가적인 컨텍스트나 매개변수를 전달하기 위한 유연한 키-값 맵

Type<Message>

Field	Type	Required	Description
messageId	string	required	메시지의 고유 식별자(UUID). 메시지 생성자가 생성
contextId	string	required	context id(UUID)
referenceTaskIds	list	optional	agent 내에서 하위 agent를 호출하는 경우 본인의 task id 전달 필요
role	Role	required	메시지 발신자를 식별
parts	list[Part]	required	Parts는 메시지 내용을 담은 컨테이너
metadata	object	optional	메세지와 함께 제공할 메타데이터가 있으면 입력

Type <Role>

Value	Description
ROLE_UNSPECIFIED	역할은 구체적으로 명시되지 않음.
ROLE_USER	클라이언트에서 서버로 전송되는 메시지
ROLE_AGENT	서버에서 클라이언트로 전송

Type <Parts>

Field	Type	Required	Description
metadata	Object	optional	파트와 관련된 메타데이터
text	string	optional(Oneof)	테스트 본문
raw	bytes	optional(Oneof)	raw binary payload
data	any	optional(Oneof)	구조화 데이터

Type <metadata>

표 10-2 Type<metadata>

Field	Type	Required	Description
respondAsync	boolean	optional	비동기 실행 여부 (지원 예정)

Request 예시

```

1  {
2    "jsonrpc": "2.0",
3    "method": "SendMessage",
4    "params": {
5      "message": {
6        "contextId": "01491f0b-7623-441f-bcde-f12a76badac7",
7        "messageId": "b039c301-f46f-4d1e-9e1f-7ceac721dd1b",
8        "role": "ROLE_USER",
9        "parts": [
10         {
11           "text": "안녕하세요?"
12         }
13       ]
14     }
15   }
16 }
```

Response

표 10-3 JSON-RPC Envelope

Field	Type	Required	Description	Value
jsonrpc	string	Y	JSON-RPC 버전	"2.0"
id	string integer	Y	요청 시 전송한 ID	
result	Task	Y	생성 된 작업	

Type <Task>

Field	Type	Required	Description
id	string	required	task ID(UUID)
contextId	string	optional	task가 속한 컨텍스트 ID(UUID)
status	TaskStatus	required	현재 task 상태
artifacts	list[Artifact]	optional	최종/중간 산출물 목록
metadata	object	optional	task metadata

Type <TaskStatus>

Field	Type	Required	Description
state	TaskState	required	현재 상태
message	Message	optional	상태와 관련된 메시지
timestamp	timestamp	optional	상태가 기록된 시간 ex.2023-10-27T10:00:00Z

Type <TaskState>

value	Description
TASK_STATE_UNSPECIFIED	해당 작업은 알 수 없거나 불확정적인 상
TASK_STATE_SUBMITTED	작업이 성공적으로 제출되고 확인되었음(대기 중)
TASK_STATE_WORKING	에이전트가 해당 작업을 활발히 처리 중
TASK_STATE_COMPLETED	작업이 성공적으로 완료되었음. 최종 상태
TASK_STATE_FAILED	작업이 오류와 함께 종료. 최종 상태
TASK_STATE_CANCELED	작업이 완료되기 전에 취소되었음. 최종 상태
TASK_STATE_INPUT_REQUIRED	에이전트를 진행하기 위해 추가 사용자 입력이 필요함. 중단된 상태
TASK_STATE_REJECTED	에이전트가 작업을 수행하지 않기로 결정. 초기 작업 생성 시 또는 에이전트가 작업을 진행할 수 없거나 진행하지 않기로 결정한 후에 발생. 중단된 상태

value	Description
TASK_STATE_AUTH_REQUIRED	인증이 필요함. 중단된 상태

Type <Artifact>

Field	Type	Required	Description
artifactId	string	required	아티팩트에 대한 고유 식별자(예: UUID). 태스크 내에서 고유한 값.
name	string	optional	아티팩트에 대한 이름
description	string	optional	아티팩트에 대한 설명
parts	list[Part]	required	아티팩트의 내용. 최소 하나의 파트를 포함해야함
metadata	object	optional	아티팩트에 대한 메타데이터
extentions	list[string]	optional	아티팩트에 존재하거나 추가된 확장 프로그램의 URI

노트

Response 예시는 method 별로 하단에 정리되어있습니다.

Send Streaming Message

Request 예시

```

1  {
2    "id": "msg-1",
3    "jsonrpc": "2.0",
4    "method": "SendStreamingMessage",
5    "params": {
6      "message": {
7        "contextId": "01491f0b-7623-441f-bcde-f12a76badac7",
8        "messageId": "b039c301-f46f-4d1e-9e1f-7ceac721dd1b",
9        "role": "ROLE_USER",
10       "parts": [
11         {
12           "text": "안녕하세요?,"

```

```

13     }
14   ]
15 }
16 }
17 }

```

| 호출 예시 (python)

```

1  import requests
2  import uuid
3
4  YOUR_CLIENT_KEY = ""
5  YOUR_PASS_KEY = ""
6  ENDPOINT_URL = ""
7  AGENT_ID = ""
8  YOUR_EMAIL = ""
9
10 headers: dict = {
11     "x-fabrix-client": YOUR_CLIENT_KEY,
12     "x-openapi-token": YOUR_PASS_KEY,
13     "x-generative-ai-user-email": YOUR_EMAIL,
14     "A2A-Version" : "1.0"
15 }
16
17 api_endpoint_url = f"{ENDPOINT_URL}/openapi/orchestrator/v1/a2a/{AGENT_ID}"
18
19 contextId = str(uuid.uuid4())
20 messageId = str(uuid.uuid4())
21
22 payload = {
23     "id": "msg-1",
24     "jsonrpc": "2.0",
25     "method": "SendStreamingMessage",
26     "params": {
27         "message": {
28             "contextId": contextId,
29             "messageId": messageId,
30             "role": "ROLE_USER",
31             "parts": [

```



```

32     {
33         "text": "안녕하세요?,"
34     }
35 ]
36 }
37 }
38 }
39
40 response = requests.post(url, headers = headers, json=payload)
41 response: requests.Response = requests.post(
42     api_endpoint_url,
43     headers=headers
44 )
45
46 # 응답 반환
47 print(response.json())

```

| Response 예시

```

1  data: {
2    "result": {
3      "task": {
4        "id": "6ca4e05e-c62d-405a-a205-b7185775aba1",
5        "contextId": "108347f7-2cd7-4510-8e0e-e42b337bc2ee",
6        "status": {
7          "state": "TASK_STATE_SUBMITTED"
8        }
9      }
10   },
11   "id": "msg-1",
12   "jsonrpc": "2.0"
13 }
14
15 data: {
16   "result": {
17     "statusUpdate": {
18       "taskId": "6ca4e05e-c62d-405a-a205-b7185775aba1",
19       "contextId": "108347f7-2cd7-4510-8e0e-e42b337bc2ee",
20       "status": {

```

```

21     "state": "TASK_STATE_WORKING",
22     "timestamp": "2026-04-24T06:19:03.265990Z"
23   }
24 }
25 },
26 "id": "msg-1",
27 "jsonrpc": "2.0"
28 }
29 data: {
30   "result": {
31     "artifactUpdate": {
32       "taskId": "6ca4e05e-c62d-405a-a205-b7185775aba1",
33       "contextId": "108347f7-2cd7-4510-8e0e-e42b337bc2ee",
34       "artifact": {
35         "artifactId": "final_answer",
36         "name": "final_answer",
37         "description": "Final answer",
38         "parts": [
39           {
40             "text": "안녕하세요?"
41           }
42         ]
43       }
44     }
45   },
46   "id": "msg-1",
47   "jsonrpc": "2.0"
48 }
49
50 data: {
51   "result": {
52     "artifactUpdate": {
53       "taskId": "6ca4e05e-c62d-405a-a205-b7185775aba1",
54       "contextId": "108347f7-2cd7-4510-8e0e-e42b337bc2ee",
55       "artifact": {
56         "artifactId": "final_answer",
57         "name": "final_answer",
58         "description": "Final answer",
59         "parts": [
60           {

```

```

61         "text": "반가워요."
62     }
63 ]
64 }
65 }
66 },
67 "id": "msg-1",
68 "jsonrpc": "2.0"
69 }
70
71 data: {
72     "result": {
73         "statusUpdate": {
74             "taskId": "6ca4e05e-c62d-405a-a205-b7185775aba1",
75             "contextId": "108347f7-2cd7-4510-8e0e-e42b337bc2ee",
76             "status": {
77                 "state": "TASK_STATE_COMPLETED",
78                 "message": {
79                     "messageId": "7e750d65-18a1-4213-aaaf-af9762244d1a",
80                     "contextId": "108347f7-2cd7-4510-8e0e-e42b337bc2ee",
81                     "taskId": "6ca4e05e-c62d-405a-a205-b7185775aba1",
82                     "role": "ROLE_AGENT",
83                     "parts": [
84                         {
85                             "text": "COMPLETED"
86                         }
87                     ],
88                     "metadata": {
89                         "status": "RESPONSE"
90                     }
91                 },
92                 "timestamp": "2026-04-24T06:19:03.476802Z"
93             }
94         }
95     },
96     "id": "msg-1",
97     "jsonrpc": "2.0"
98 }

```

Send Message

| Request 예시

```

1  {
2    "id": "msg-1",
3    "jsonrpc": "2.0",
4    "method": "SendMessage",
5    "params": {
6      "message": {
7        "contextId": "01491f0b-7623-441f-bcde-f12a76badac7",
8        "messageId": "b039c301-f46f-4d1e-9e1f-7ceac721dd1b",
9        "role": "ROLE_USER",
10       "parts": [
11         {
12           "text": "안녕하세요?,"
13         }
14       ]
15     }
16   }
17 }
```

| 호출 예시 (python)

```

1  import requests
2  import uuid
3
4  YOUR_CLIENT_KEY = ""
5  YOUR_PASS_KEY = ""
6  ENDPOINT_URL = ""
7  AGENT_ID = ""
8  YOUR_EMAIL = ""
9
10 headers: dict = {
11     "x-fabrix-client": YOUR_CLIENT_KEY,
12     "x-openapi-token": YOUR_PASS_KEY,
13     "x-generative-ai-user-email": YOUR_EMAIL,
14     "A2A-Version" : "1.0"
15 }
```

```

16
17 api_endpoint_url = f"{ENDPOINT_URL}/openapi/orchestrator/v1/a2a/{AGENT_ID}"
18
19 contextId = str(uuid.uuid4())
20 messageId = str(uuid.uuid4())
21
22 payload = {
23     "id": "msg-1",
24     "jsonrpc": "2.0",
25     "method": "SendMessage",
26     "params": {
27         "message": {
28             "contextId": contextId,
29             "messageId": messageId,
30             "role": "ROLE_USER",
31             "parts": [
32                 {
33                     "text": "안녕하세요?,"
34                 }
35             ]
36         }
37     }
38 }
39
40 response = requests.post(url, headers = headers, json=payload)
41 response: requests.Response = requests.post(
42     api_endpoint_url,
43     headers=headers
44 )
45
46 # 응답 반환
47 print(response.json())

```

| Response 예시

```

1 {
2     "result": {
3         "task": {
4             "id": "fb58f7a6-7516-4fb1-a023-2d785015fdaa",

```

```

5     "contextId": "cd3c5868-dac7-4463-ad25-40c6f53d6c1a",
6     "status": {
7         "state": "TASK_STATE_COMPLETED",
8         "message": {
9             "messageId": "5b33746d-b3cf-4b3d-9584-99fabf7b5688",
10            "contextId": "cd3c5868-dac7-4463-ad25-40c6f53d6c1a",
11            "taskId": "fb58f7a6-7516-4fb1-a023-2d785015fdaa",
12            "role": "ROLE_AGENT",
13            "parts": [
14                {
15                    "text": "COMPLETED"
16                }
17            ],
18            "metadata": {
19                "status": "RESPONSE"
20            }
21        },
22        "timestamp": "2026-04-24T08:14:17.010622Z"
23    },
24    "artifacts": [
25        {
26            "artifactId": "final_answer",
27            "name": "final_answer",
28            "description": "Final answer",
29            "parts": [
30                {
31                    "text": "안녕하세요? 반가워요."
32                }
33            ]
34        }
35    ]
36 },
37 {
38     "id": "msg-1",
39     "jsonrpc": "2.0"
40 }

```

Get Task (지원 예정)

| Request 예시

```
1 {
2   "id" : "id123",
3   "jsonrpc": "2.0",
4   "method": "GetTask",
5   "params": {
6     "id": "b039c301-f46f-4d1e-9e1f-7ceac721dd1b"
7   }
8 }
```

| Response 예시

```
1 {
2   "result": {
3     "id": "9e91bc96-c0e5-4939-b6eb-b4d59ae511e0",
4     "contextId": "ccb8547a-f8fa-447f-a753-2518352625a5",
5     "status": {
6       "state": "TASK_STATE_COMPLETED",
7       "message": {
8         "messageId": "17bfc846-d80f-4b56-a8e8-a21a97a46eb7",
9         "contextId": "ccb8547a-f8fa-447f-a753-2518352625a5",
10        "taskId": "9e91bc96-c0e5-4939-b6eb-b4d59ae511e0",
11        "role": "ROLE_AGENT",
12        "parts": [
13          {
14            "text": "TASK_STATE_COMPLETED"
15          }
16        ],
17        "metadata": {
18          "status": "RESPONSE"
19        }
20      }
21    },
22    "artifacts": [
23      {
24        "artifactId": "final_answer",
```

```

25     "name": "final_answer",
26     "description": "Final answer",
27     "parts": [
28         {
29             "text": "안녕!\n무엇을 도와줄까? 언제든 편하게 말해줘."
30         }
31     ],
32     "metadata": { }
33 }
34 ]
35 },
36 "id": "id123",
37 "jsonrpc": "2.0"
38 }

```

| 호출 예시 (python)

```

1  import requests
2  import uuid
3
4  YOUR_CLIENT_KEY = ""
5  YOUR_PASS_KEY = ""
6  ENDPOINT_URL = ""
7  AGENT_ID = ""
8  YOUR_EMAIL = ""
9  PREVIOUS_TASK_ID = "" #agent 요청 후 받은 taskId
10
11  headers: dict = {
12      "x-fabrix-client": YOUR_CLIENT_KEY,
13      "x-openapi-token": YOUR_PASS_KEY,
14      "x-generative-ai-user-email": YOUR_EMAIL,
15      "A2A-Version" : "1.0"
16  }
17
18  api_endpoint_url = f"{ENDPOINT_URL}/openapi/orchestrator/v1/a2a/{AGENT_ID}"
19
20  payload = {
21      "id" : "id123",
22      "jsonrpc": "2.0",

```



```

23     "method": "GetTask",
24     "params": {
25         "id": PREVIOUS_TASK_ID
26     }
27 }
28
29 response = requests.post(url, headers = headers, json=payload)
30 response: requests.Response = requests.post(
31     api_endpoint_url,
32     headers=headers
33 )
34
35 # 응답 반환
36 print(response.json())

```

Agent 요청시 첨부파일 추가하기

| 호출 예시 (python)

```

1  import requests
2  import uuid
3
4  YOUR_CLIENT_KEY = ""
5  YOUR_PASS_KEY = ""
6  ENDPOINT_URL = ""
7  AGENT_ID = ""
8  YOUR_EMAIL = ""
9
10
11 headers: dict = {
12     "x-fabrix-client": YOUR_CLIENT_KEY,
13     "x-openapi-token": YOUR_PASS_KEY,
14     "x-generative-ai-user-email": YOUR_EMAIL,
15     "A2A-Version" : "1.0"
16 }
17
18 api_endpoint_url = f"{ENDPOINT_URL}/openapi/orchestrator/v1/a2a/{AGENT_ID}"
19 FILE_PATH1 = "../file/파일1.csv"
20 file_path1 = Path(FILE_PATH1)

```

```

21 mime_type1 = "text/csv"
22
23 FILE_PATH2 = "../file/image.png"
24 file_path2 = Path(FILE_PATH2)
25 mime_type2 = "image/png"
26
27 with open(file_path1, "rb") as f:
28     encoded_file1 = base64.b64encode(f.read()).decode("utf-8")
29 with open(file_path2, "rb") as f:
30     encoded_file2 = base64.b64encode(f.read()).decode("utf-8")
31
32 payload={
33     "jsonrpc": "2.0",
34     "method": "SendStreamingMessage",
35     "params": {
36         "message": {
37             "role": "ROLE_USER",
38             "parts": [
39                 {
40                     "text": "파일1을 분석하고, image.png는 무슨 그림인지 분석해줘."
41                 },
42                 {
43                     "raw": encoded_file1,
44                     "filename": file_path1.name,
45                     "mediaType": mime_type1,
46                 },
47                 {
48                     "raw": encoded_file2,
49                     "filename": file_path2.name,
50                     "mediaType": mime_type2,
51                 }
52             ],
53             "messageId": str(uuid.uuid4()),
54             "metadata": {
55             }
56         }
57     }
58 }
59
60

```

```

61 response = requests.post(
62     API_URL,
63     headers=headers,
64     json=payload,
65     timeout=600,
66 )
67
68 # 응답 반환
69 print(response.json())

```

표 10-4 업로드 가능한 파일 확장자 및 MIME Type

확장자	MIME Type
txt	text/plain
docx	application/vnd.openxmlformats-officedocument.wordprocessingml.document
xlsx	application/vnd.openxmlformats-officedocument.spreadsheetml.sheet
pdf	application/pdf
html	text/html
csv	text/csv
json	application/json
hwp	application/x-hwp
hwpx	application/haansofthwp
jpg/jpeg	image/jpeg
png	image/png
pptx	application/vnd.openxmlformats-officedocument.presentationml.presentation
md	text/markdown

주의

파일 관련 제약사항

- 동시 업로드는 최대 5개
- 용량은 최대 3MB

Input Required 발생 및 Session 재개

Input Required 발생 및 Session 재개 시나리오는 다음과 같습니다.

(첫번째 Turn)

Agent에 질문을 합니다. -> 응답으로 TASK_STATE_INPUT_REQUIRED state를 수신합니다.

(두번째 Turn)

첫번째 턴에서 받은 응답의 taskId와 contextId를 요청에 포함시켜, Agent에 재요청합니다.

| 첫번째 Turn의 Request

```

1  {
2    "id": "msg-1",
3    "jsonrpc": "2.0",
4    "method": "SendMessage",
5    "params": {
6      "message": {
7        "messageId": "be88612c-7556-4483-8986-fccb27a9af86",
8        "role": "ROLE_USER",
9        "parts": [
10         {
11           "text": "호텔을 예약하고 싶어."
12         }
13       ]
14     }
15   }
16 }
```

| 첫번째 Turn의 Response

```

1  {
2    "result": {
3      "task": {
4        "id": "97968eff-abf6-405a-8ee1-e11aaa7c7041",
5        "contextId": "04c84289-a936-4ff9-9982-07fe42ca5750",
6        "status": {
7          "state": "TASK_STATE_INPUT_REQUIRED",
8          "message": {
9            "messageId": "9ef2ec92-7610-41c6-a125-e05091c0bb99",
10           "contextId": "04c84289-a936-4ff9-9982-07fe42ca5750",
```

```

11      "taskId": "97968eff-abf6-405a-8ee1-e11aaa7c7041",
12      "role": "ROLE_AGENT",
13      "parts": [
14          {
15              "text": "호텔 예약을 위해서는 예약자 이름, 체크인 날짜, 체크아웃 날짜, 숙박
지역(도시) 및 인원 수가 필요합니다. 해당 정보를 알려 주세요."
16          }
17      ],
18      "metadata": {
19          "status": "INPUT_REQUIRED"
20      }
21  },
22  "timestamp": "2026-06-10T13:28:40.482886Z"
23  }
24  }
25  },
26  "id": "id123",
27  "jsonrpc": "2.0"
28  }

```

| 두번째 Turn의 Request

```

1  {
2      "id" : "id123",
3      "jsonrpc": "2.0",
4      "method": "SendMessage",
5      "params": {
6          "message": {
7              "role": "ROLE_USER",
8              "parts": [
9                  {
10                     "text": "2026년 6월 25~30까지 뉴욕 인터콘티넨탈 호텔 예약해줘 방은
스텐다드로 1개, 예약자 명은 홍길동, 투숙인원은 2명"
11                 }
12             ],
13             "taskId": "97968eff-abf6-405a-8ee1-e11aaa7c7041",
14             "contextId": "04c84289-a936-4ff9-9982-07fe42ca5750",
15             "messageId": "944fa9ad-d0b2-4b0d-8e84-ab424d8546a8",
16             "metadata": {

```

```

17     }
18   }
19 }
20 }

```

| 두번째 Turn의 Response

```

1  {
2    "result": {
3      "task": {
4        "id": "97968eff-abf6-405a-8ee1-e11aaa7c7041",
5        "contextId": "04c84289-a936-4ff9-9982-07fe42ca5750",
6        "status": {
7          "state": "TASK_STATE_COMPLETED",
8          "message": {
9            "messageId": "9ef2ec92-7610-41c6-a125-e05091c0bb99",
10           "contextId": "04c84289-a936-4ff9-9982-07fe42ca5750",
11           "taskId": "97968eff-abf6-405a-8ee1-e11aaa7c7041",
12           "role": "ROLE_AGENT",
13           "parts": [
14             {
15               "text": "COMPLETED"
16             }
17           ],
18           "metadata": {
19             "status": "RESPONSE"
20           }
21         },
22         "timestamp": "2026-06-10T13:28:40.482886Z"
23       },
24       "artifacts": [
25         {
26           "artifactId": "final_answer",
27           "name": "final_answer",
28           "description": "Final answer",
29           "parts": [
30             {
31               "text": "홍길동님, 뉴욕 인터콘티넨탈 호텔, 스텐다드 룸 1개 예약했습니다.
예약자 : 홍길동, 투숙 인원 : 2명, 체크인 : 2026년 6월 25일, 체크아웃 : 2026년 6월 30일 입니다.

```

```

"
32         }
33     ]
34 }
35 ]
36 }
37 },
38 "id": "id123",
39 "jsonrpc": "2.0"
40 }

```

Auth Required 발생

Auth Required 발생 및 Session 재개 시나리오는 다음과 같습니다.

(첫번째 Turn)

Agent에 질문을 합니다. -> 응답으로 TASK_STATE_AUTH_REQUIRED state를 수신합니다.

(두번째 Turn)

첫번째 턴에서 받은 응답의 taskId와 contextId를 요청에 포함시켜, Agent에 재요청합니다.

비동기 요청 (지원예정)

비동기 요청은 SendMessage method만 지원합니다.

respondAsync의 기본값은 False입니다.

비동기 요청시 Agent 답변은 GetTask를 통해 받을 수 있습니다.

| SendMessage Request 예시

```

1  {
2      "jsonrpc": "2.0",
3      "id": "stream-1",
4      "method": "SendMessage",
5      "params": {
6          "message": {
7              "messageId": "b039c301-f46f-4d1e-9e1f-7ceac721dd1b",
8              "role": "ROLE_USER",
9              "parts": [
10                 { "text": "6/22~6/25 뉴욕 인터콘티넨탈 예약 도와줘" }
11             ]
12         },

```

```

13     "metadata": {
14         "respondAsync": true
15     }
16 }
17 }

```

| SendMessage Response 예시

```

1  {
2      "jsonrpc": "2.0",
3      "id": "stream-1",
4      "result": {
5          "id": "8b8c3d2a-6f0b-4f1d-9d65-2d8b1b2c4e11",
6          "contextId": "3a9f2d77-2c4d-4bb0-a4c2-19b2a0d5c901",
7          "status": {
8              "state": "TASK_STATE_SUBMITTED",
9              "message": {
10                  "messageId": "b039c301-f46f-4d1e-9e1f-7ceac721dd1b",
11                  "contextId": "3a9f2d77-2c4d-4bb0-a4c2-19b2a0d5c901",
12                  "taskId": "8b8c3d2a-6f0b-4f1d-9d65-2d8b1b2c4e11",
13                  "role": "ROLE_USER",
14                  "parts": [
15                      {
16                          "text": "6/22~6/25 뉴욕 인터콘티넨탈 예약 도와줘"
17                      }
18                  ]
19              }
20          }
21      }
22  }

```

| GetTask Request 예시

```

1  {
2      "jsonrpc": "2.0",
3      "id": "gettask-1",
4      "method": "GetTask",
5      "params": {

```



```

6      "id": "8b8c3d2a-6f0b-4f1d-9d65-2d8b1b2c4e11"
7    }
8  }

```

| GetTask Response 예시

```

1  {
2    "jsonrpc": "2.0",
3    "id": "gettask-1",
4    "result": {
5      "id": "8b8c3d2a-6f0b-4f1d-9d65-2d8b1b2c4e11",
6      "contextId": "3a9f2d77-2c4d-4bb0-a4c2-19b2a0d5c901",
7      "status": {
8        "state": "TASK_STATE_COMPLETED",
9        "message": {
10         "messageId": "f4c1f9a9-9989-4c13-8c9a-5ab1ed4b8c77",
11         "contextId": "3a9f2d77-2c4d-4bb0-a4c2-19b2a0d5c901",
12         "taskId": "8b8c3d2a-6f0b-4f1d-9d65-2d8b1b2c4e11",
13         "role": "ROLE_AGENT",
14         "parts": [
15           {
16             "text": "TASK_STATE_COMPLETED"
17           }
18         ],
19         "metadata": {
20           "status": "RESPONSE"
21         }
22       }
23     },
24     "artifacts": [
25       {
26         "artifactId": "a7d0d6b4-4d18-4d6e-a7d3-8b4b6c2f9e10",
27         "name": "final_answer",
28         "description": "Final answer",
29         "parts": [
30           {
31             "text": "6/22부터 6/25까지 뉴욕 인터콘티넨탈 예약을 도와드릴게요. 투숙 인원,
32             객실 수, 투숙자 영문명, 결제 가능 카드 정보를 알려주시면 다음 단계로 진행하겠습니다."

```

```

33     ]
34   }
35 ]
36 }
37 }

```

공통 Response Code

JSON-RPC 표준 에러 코드

코드	에러	설명
-32700	JSONParseError	잘못된 JSON 페이로드
-32600	InvalidRequestError	잘못된 요청 객체
-32601	MethodNotFoundError	메서드를 찾을 수 없음
-32602	InvalidParamsError	잘못된 파라미터
-32603	InternalError	내부 서버 에러

A2A 프로토콜 에러 코드

코드	에러	설명
-32001	TaskNotFoundError	작업을 찾을 수 없음
-32002	TaskNotCancelableError	작업을 취소할 수 없음
-32003	PushNotificationNotSupportedError	푸시 알림이 지원되지 않음
-32004	UnsupportedOperationError	지원하지 않는 작업
-32005	ContentTypeNotSupportedError	호환되지 않는 콘텐츠 타입
-32006	InvalidAgentResponseError	잘못된 에이전트 응답
-32007	AuthenticatedExtendedCardNotConfiguredError	인증된 확장 카드 미설정

10.3 FAQ

비동기 실행 지원 시점

8월 경 지원 예정입니다.

11.

Data Parsing APIs

11.1 Data Parsing APIs Overview

Data Parsing APIs는 Parsing Spec을 이용한 1회성 데이터 파싱 호출 서비스를 제공합니다.

(주의) 1회성 데이터 파싱 대상은 1일 후 삭제 됩니다.

Data Parsing APIs 에서 제공하는 API는 다음과 같습니다.

GET /openapi/parsing/v1/parsers

지원되는 파서의 목록을 조회합니다.

POST /openapi/parsing/v1/documents/parsing-jobs (사전협의필요)

스토리지에 업로드된 문서의 파싱을 요청합니다.(Object Storage 연동 파라미터는 사전 협의 시 사용 가능합니다).

POST /openapi/parsing/v1/documents/parsing-jobs/files

문서 스트림에 대한 파싱을 요청합니다..

GET /openapi/parsing/v1/documents/parsing-jobs/{parsingJobId}

문서의 파싱 상태를 조회합니다.

GET /openapi/parsing/v1/documents/parsing-jobs/{parsingJobId}/result

문서 파싱 결과를 조회 할 수 있습니다.

11.2 API 명세

GET /openapi/parsing/v1/parsers

Header

Field	Type	Required	Description	Value
Content-Type	string	optional	Type of content	application/json
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-openapi-token	string	required	OpenAPI Token information	ex) Bearer eyJXXX
x-generative-ai-user-email	string	optional	Portal user email	ex) kim.samsung@samsung.com

Header 예시

```

1 {
2   'x-fabrix-client': 'eyJ...',
3   'x-openapi-token': 'Bearer ey...'
4 }
```

Query Parameters

Field	Type	Required	Description	Value
paging_num	int	optional	page number	1
paging_count	int	optional	page size	20
search	string	optional	search keyword	""

Response

Field	Type	Description
	list[Result]	Refer to Result type

Type <Result>

Field	Type	Description
parserId	String	Id of parser
parserName	String	Name of parser
supportFormat	list[String]	support formats
defaultFormat	list[String]	List of file extensions for which this

Field	Type	Description
		parser is the default choice

| Response 예시

```

1  [
2    {
3      "parserId": "8d8f1e0..",
4      "parserName": "DOC.AI",
5      "supportFormat": [".pdf", ".hwp", ".hwp", ".xlsx", ".csv"],
6      "defaultFormat": [".pdf", ".hwp", ".hwp", ".xlsx", ".csv"]
7    }
8  ]

```

| 호출 예시 (python)

```

1  import requests
2
3  YOUR_CLIENT_KEY = ""
4  YOUR_PASS_KEY = ""
5  ENDPOINT_URL = ""
6  YOUR_EMAIL = ""
7
8  headers = {
9      "x-fabrix-client": YOUR_CLIENT_KEY,
10     "x-openapi-token": YOUR_PASS_KEY,
11     "x-generative-ai-user-email": YOUR_EMAIL
12 }
13
14 api_endpoint_url = f"{ENDPOINT_URL}/openapi/parsing/v1/parsers"
15
16 response = requests.get(
17     api_endpoint_url,
18     headers=headers,
19 )
20
21 # Response
22 print(response.json())

```

Header

Field	Type	Required	Description	Value
Content-Type	string	optional	Type of content	application/json
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-openapi-token	string	required	OpenAPI Token information	ex) Bearer eyJXXX

Query Parameter

Key	Type	Default	Description	Default Value
paging_num	Integer	Optional	Pagination page value	1
paging_count	Integer	Optional	Pagination limit value	10
search	string	Optional	Search keyword	

Response

Field	Type	Description
meta	ApiResponse	

Type <ApiResponse>

Field	Type	Description
success	boolean	
data	object[ParserResult]	
error		

Type <ParserResult>

Field	Type	Description
totalCount	Integer	
items	list[string]	uuid
parserName	string	
supportFormat	list[string]	
defaultFormat	list[string]	

Response 예시

```

1  {
2      'success': True,
3      'data': {
4          'totalCount': 2,
5          'items': [
6              {
7                  'parserId': '9d8f1e0d-5690-482f-a4f3-0584310c358c',
8                  'parserName': 'Lomin',
9                  'supportFormat': ['.html'],
10                 'defaultFormat': ['.html']
11             }
12         ]
13     },
14     'error': None
15 }

```

| 호출 예시 (python)

```

1  import requests
2
3  YOUR_CLIENT_KEY = ""
4  YOUR_PASS_KEY = ""
5  ENDPOINT_URL = ""
6
7  headers = {
8      "x-fabrix-client": YOUR_CLIENT_KEY,
9      "x-openapi-token": YOUR_PASS_KEY
10 }
11
12 api_endpoint_url = f"{ENDPOINT_URL}/openapi/parsing/v1/parsers"
13
14 response = requests.get(
15     api_endpoint_url,
16     headers=headers
17 )
18
19 # 응답 반환
20 print(response.json())

```

POST /openapi/parsing/v1/documents/parsing-jobs (활용 시 사전협의)

Header

Field	Type	Required	Description	Value
Content-Type	string	optional	Type of content	application/json
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-openapi-token	string	required	OpenAPI Token information	ex) Bearer eyJXXX
x-generative-ai-user-email	string	optional	Portal user email	ex) kim.samsung@samsung.com

Header 예시

```

1 {
2   'x-fabrix-client': 'eyJ...',
3   'x-openapi-token': 'Bearer ey...'
4 }
```

Request Body

Field	Type	Required	Description	Value
parserName	String	optional	Name of the registered parser	"FABRIX"
sourceObjUrl	String	required	Storage URL where the document is stored	"s3://my-preprocessing/raw/HTML_10mb.html"
targetObjUrl	String	optional	Storage URL where the parsed document will be stored	"s3://my-preprocessing/parsed"
sourceVerifyKey	String	optional	Source Storage Key (pre-registration required)	"ardcwe.."
targetVerifyKey	String	optional	Target Storage Key (pre-registration required)	"kkrpes2.."
fileName	String	required	document name	"HTML_10mb.html"

Field	Type	Required	Description	Value
callbackUrl	String	optional	URL to receive results	"https://dev150.genaiqa.samsungsds.com/abcefg"
callbackSecret	String	optional	Secret of callbackUrl	""
outputType	String	optional	Format of parsing result file · JSON_FOR_RAG · JSON_FOR_RAG_IMAGE · HTML · MARKDOWN	"JSON_FOR_RAG"
useOcr	String	optional	Using ocr model or not	"True" or "False"
useTsr	String	optional	Using tsr model or not	"True" or "False"
useLmm	String	optional	Using lmm model or not	"True" or "False"

Request Body 예시

```

1 {
2   "parserName": "FABRIX",
3   "sourceObjUrl": "s3://my-preprocessing/raw/HTML_10mb.html",
4   "targetObjUrl": "s3://my-preprocessing/parsed",
5   "sourceVerifyKey": "ardcwe...",
6   "targetVerifyKey": "kkrpes2...",
7   "fileName": "HTML_10mb.html",
8   "callbackUrl": "https://dev150.genaiqa.samsungsds.com/abcefg",
9   "callbackSecret": "",
10  "outputType": "JSON_FOR_RAG",
11  "useOcr": "True",
12  "useTsr": "True",
13  "useLmm": "True"
14 }
```

Response

Field	Type	Description
parsingJobId	String	Id of parsing job

Field	Type	Description
status	String	status of job
fileName	String	document name
targetObjUrl	String	Storage URL where the parsed document will be stored
errorType	String	error type
message	Object	Error message description values

| Response 예시

```

1  {
2    "parsingJobId": "202511031...",
3    "status": "QUEUED",
4    "fileName": "string",
5    "targetObjUrl": "",
6    "errorType": "string",
7    "message": {
8      "ko": "",
9      "en": ""
10   }
11 }
```

| 호출 예시 (python)

```

1  import requests
2
3  YOUR_CLIENT_KEY = ""
4  YOUR_PASS_KEY = ""
5  ENDPOINT_URL = ""
6  YOUR_EMAIL = ""
7
8  headers = {
9    "x-fabrix-client": YOUR_CLIENT_KEY,
10   "x-openapi-token": YOUR_PASS_KEY,
11   "x-generative-ai-user-email": YOUR_EMAIL
12 }
13
14 body = {
```

```

15     "parserName": "FABRIX",
16     "sourceObjUrl": "s3://my-preprocessing/raw/HTML_10mb.html",
17     "targetObjUrl": "s3://my-preprocessing/parsed",
18     "sourceVerifyKey": "ardcwe...",
19     "targetVerifyKey": "kkrpes2..",
20     "fileName": "HTML_10mb.html",
21     "callbackUrl": "https://dev150.genaiqa.samsungsds.com/abcefg",
22     "callbackSecret": "",
23     "outputType": "JSON_FOR_RAG",
24     "useOcr": "True",
25     "useTsr": "True",
26     "useLmm": "True"
27 }
28
29 api_endpoint_url = f"{ENDPOINT_URL}/openapi/parsing/v1/documents/parsing-jobs"
30
31 response = requests.post(
32     api_endpoint_url,
33     headers=headers,
34     json=body
35 )
36
37 # 응답 반환
38 print(response.json())

```

POST /openapi/parsing/v1/documents/parsing-jobs/files

Header

Field	Type	Required	Description	Value
Content-Type	string	optional	Type of content	multipart/form-data
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-openapi-token	string	required	OpenAPI Token information	ex) Bearer eyJXXX

Header 예시

```

1 {
2   'x-fabrix-client': 'ey...',
3   'x-openapi-token': 'Bearer ey...'
4 }

```

Query Parameters

Field	Type	Required	Description	Value
parserName	String	optional	Name of the registered parser	"FABRIX"
targetObjUrl	String	optional	Storage URL where the parsed document will be stored	"s3://my-preprocessing/parsed"
targetVerifyKey	String	optional	Target Storage Key (pre-registration required)	"kkrpes2.."
useOcr	String	optional	Use ocr model or not	"True" or "False"
useTsr	String	optional	Use tsr model or not	"True" or "False"
useLmm	String	optional	Use lmm model or not	"True" or "False"
parsingTimeout	Integer	optional		
ragMode	String	optional		agentic, multimodal

Query Parameters 예시

```

1 {
2   "parserName": "FABRIX",
3   "targetObjUrl": "s3://my-preprocessing/parsed",
4   "targetVerifyKey": "kkrpes2..",
5   "useOcr": "True",
6   "useTsr": "True",
7   "useLmm": "True"
8 }

```

Request Body

Field	Type	Required	Description	Value
file	binary	required	document	@file.pdf (탐색기에서 파일 선택)

Response

Field	Type	Description
parsingJobId	String	Id of parsing job
status	String	status of job
fileName	String	document name
targetObjUrl	String	Storage URL where the parsed document will be stored
errorType	String	error type
message	Object	Error message description values

Response 예시

```

1  {
2    "parsingJobId": "202511031...",
3    "status": "QUEUED",
4    "fileName": "string",
5    "targetObjUrl": "",
6    "errorType": "string",
7    "message": {
8      "ko": "",
9      "en": ""
10   }
11 }
```

호출 예시 (python)

```

1  import requests
2  import json
3  from fastapi import UploadFile
4
5  YOUR_CLIENT_KEY = ""
6  YOUR_PASS_KEY = ""
7  ENDPOINT_URL = ""
8  YOUR_EMAIL = ""
9
10 f: UploadFile = open("./sample.png", "rb")
11
12 headers = {
```

```
13     "x-fabrix-client": YOUR_CLIENT_KEY,
14     "x-openapi-token": YOUR_PASS_KEY,
15     "x-generative-ai-user-email": YOUR_EMAIL
16 }
17
18
19 form_data = {
20     "parserName": "FABRIX",
21     "targetObjUrl": "s3://my-preprocessing/parsed",
22     "targetStorageVerifyKey": "kkrpes2..",
23     "useOcr": "True",
24     "useTsr": "True",
25     "useLmm": "True"
26 }
27
28 api_endpoint_url = f"{ENDPOINT_URL}/openapi/parsing/v1/documents/parsing-jobs/files"
29 files = {"file": f}
30 response = requests.post(api_endpoint_url, headers=headers, data=form_data, files=files)
31
32 # Response
33 print(response.json())
```

G

E

T

/openapi/parsing/v1/documents/parsing-jobs/{parsingJobId}

Header

Field	Type	Required	Description	Value
Content-Type	string	optional	Type of content	application/json
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-openapi-token	string	required	OpenAPI Token information	ex) Bearer eyJXXX

Header 예시

```

1 {
2   'x-fabrix-client': 'ey...',
3   'x-openapi-token': 'Bearer ey...'
4 }

```

Query Parameters

Field	Type	Required	Description	Value
parsingJobId	String	required	Id of parsing job	"202511031..."

Response

Field	Type	Description
parsingJobId	String	Id of parsing job
status	String	Status of job · QUEUED · EXECUTING · COMPLETED · FAILED
fileName	String	document name
targetObjUrl	String	Storage URL where the parsed document will be stored
errorType	String	error type
message	Object	Error message description values

Response 예시

```

1 {
2   "parsingJobId": "202511031...",
3   "status": "EXECUTING",
4   "fileName": "string",
5   "targetObjUrl": "string",
6   "errorType": "string",
7   "message": {
8     "ko": "",
9     "en": ""
10  }
11 }

```

| 호출 예시 (python)

```
1 import requests
2
3 YOUR_CLIENT_KEY = ""
4 YOUR_PASS_KEY = ""
5 ENDPOINT_URL = ""
6 YOUR_EMAIL = ""
7 parsingJobId = ""
8
9 headers = {
10     "x-fabrix-client": YOUR_CLIENT_KEY,
11     "x-openapi-token": YOUR_PASS_KEY,
12     "x-generative-ai-user-email": YOUR_EMAIL
13 }
14
15 api_endpoint_url =
f"{ENDPOINT_URL}/openapi/parsing/v1/documents/parsing-jobs/{parsingJobId}"
16
17 response = requests.get(
18     api_endpoint_url,
19     headers=headers,
20 )
21
22 # Response
23 print(response.json())
```

GET /openapi/parsing/v1/documents/parsing-jobs/{parsingJobId}/result

| Header

Field	Type	Required	Description	Value
Content-Type	string	optional	Type of content	application/json
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-openapi-token	string	required	OpenAPI Token	ex) Bearer eyJXXX

Field	Type	Required	Description	Value
			information	

| Header 예시

```

1 {
2   'x-fabrix-client': 'ey...',
3   'x-openapi-token': 'Bearer ey...'
4 }
```

| Query Parameters

Key	Type	Default	Description	Default Value
parsing_job_id	string	required		

| Response

Field	Type	Description
type	string	parsing type - table - title - text - image
anchor	string	reference point
page	string	parsing page number
subtitle	string	subtitle
content	string	content

| Response 예시

```

1 {
2   'original_filename': '(SDS) Background and Work Scope (Italy).docx',
3   'title': '(SDS) Background and Work Scope (Italy)',
4   'contents': [
5     {
6       'type': 'text',
7       'anchor': '',
8       'page': '',
9       'subtitle': '',
```

```

10         'content': 'Background^AAs a general practice, SDS provides IT services, either
proprietary or from a third-party, to its clientele predominantly through its global IT
subsidiaries. This entails partnering with third-party solution providers, such as software or
SaaS vendors like SAP, to perform specific services as outlined in the table below. Given the
restricted presence of IT subsidiaries globally, SDS is contemplating on utilizing its logistic
entities, including subsidiaries and branches established in your jurisdiction, to facilitate
these services with the solution vendor. Please note that SDS has not established a distinct IT
entity in your jurisdiction. Moreover, the logistic entity therein has not secured any separate
licenses, permits, or government approvals specifically for rendering IT services.^AWork
Scope^AIn light of the foregoing, please advise - '
11     }
12 ]
13 }

```

| 호출 예시 (python)

```

1  import requests
2
3  YOUR_CLIENT_KEY = ""
4  YOUR_PASS_KEY = ""
5  ENDPOINT_URL = ""
6
7  headers = {
8      "x-fabrix-client": YOUR_CLIENT_KEY,
9      "x-openapi-token": YOUR_PASS_KEY
10 }
11
12 api_endpoint_url =
f"{ENDPOINT_URL}/openapi/parsing/v1/document/parsing-jobs/{parsingId}/result"
13
14 response = requests.get(
15     api_endpoint_url,
16     headers=headers
17 )
18
19 # 응답 반환
20 print(response.json())

```

공통 Response Code

Http status code	Description
200	Request and response okay. If stream is True, all HTTP statuses except system errors will respond with 200.
400	Requested with incorrect information
401	Authentication information is incorrect
403	Accessing without authorization
404	Request URL information is incorrect.
405	Requested Method not allowed
413	Too long request.
422	Required value missing
500	Unexpected Error
503	Failed to prepare environments. (No user machine available or No kernel available)
504	Code Execution Timeout

11.3 FAQ

sourceVerifyKey, targetVerifyKey 에 어떤 값을 넣어야 하는지 모르겠어요.

Storage 의 접근 정보는 사전에 시스템 담당자와 협의하여 등록할 수 있습니다. 등록 후 Verify Key 정보를 전달드리오니, 해당 값을 사용하시면 됩니다.

Source 와 Target 의 저장 Storage 를 서로 다른 서버로 지정할 수 있나요?

가능합니다. 단, Source, Target의 각 Storage 의 접근 정보는 사전에 시스템 담당자에게 요청하여 사전에 시스템에 등록되어야 합니다.

12.

Chunking APIs

12.1 Chunking APIs

Chunking APIs는 문서에 대하여 chunk를 생성하는 API를 제공합니다.

Chunking APIs 에서 제공하는 API는 다음과 같습니다.

POST /openapi/knowledge/v1/knowledge-embedding/chunking

12.2 API 명세

P O S T
/openapi/knowledge/v1/knowledge-embedding/chunking

Header

Field	Type	Required	Description	Value
Content-Type	string	optional	Type of content	application/json
x-fabrix-client	string	required	Client authentication information	ex) eyJXXX
x-openapi-token	string	required	OpenAPI Token information	ex) Bearer eyJXXX
x-generative-ai-user-email	string	optional	Portal user email	ex) kim.samsung@samsung.com

Header 예시

```
1 {
2   'x-fabrix-client': 'ey...',
3   'x-openapi-token': 'Bearer ey...'
4   'x-generative-ai-user-email': 'mymail...'
5 }
```

Request Body

Field	Type	Required	Description	Value
sourceObjUrl	String	required	Storage URL where the document is stored	"s3://my-preprocessing/raw/HTML_10mb.html"
sourceVerifyKey	String	required	Source Storage Key (pre-registration required)	"ardcwe.."
chunkingAlgorithmID	String	optional	Id of chunking algorithm · default-RecursiveCharacterTextSplitter · default-CharacterTextSplitter · default-MarkdownTextSplitter · default-LatexTextSplitter · default-PythonCodeTextSplitter · default-SemanticChunker	"default-RecursiveCharacterTextSplitter"
title	String	optional	document name	"healthcare-1010.txt"
chunkOption	Object	required	refer to type <ChunkOption>	

Type <ChunkOption>

Field	Type	Required	Description
chunkTextGroupByMinSize	Integer	optional	When the subtitle is different and the content is short, merge it with the content of the subtitle

Field	Type	Required	Description
			above.
chunkSize	Integer	optional	text chunking size
chunkOverlap	String	optional	text chunking overlap size
chunkColSeparator	String	optional	column separator
chunkRowSeparator	String	optional	row Separator
useChunkOneByOneTableToText	Boolean	optional	1x1 cells in table type are chunked as text
useTableChunk	Boolean	optional	Use chunking in table type
useTableChunkByRow	Boolean	optional	Table chunking is based on row units - if enabled, table_chunk_size is disabled.
tableChunkSize	Integer	optional	In table chunking, it is divided into rows and split is processed when the number exceeds the set value.
tableHeaderSimilarityThreshold	Float	optional	Determine the similarity between the first row and up to the 10th row, and if the similarity is below a certain level, it is determined as a Header.
tableOneRowMaxSize	Integer	optional	If the value of one row in the table is greater than this value, it is discarded.
breakpointThresholdType	String	optional	Breakpoint Threshold Type of SemanticChunker
breakpointThresholdAmount	String	optional	Breakpoint Threshold value of SemanticChunker
useGroupBySubtitle	Boolean	optional	Group by Subtitle

| Request Body 예시

```

1  {
2      "sourceObjUrl": "s3://bidp-parsing-rag-sds/e06ef8790fd6/cfe524b3495f/6d0c24aa-1996-4cf4-88eb-e72789bb3364.txt.json",
3      "sourceVerifyKey": "string",
4      "chunkingAlgorithmID": "default-RecursiveCharacterTextSplitter",
5      "title": "healthcare-1010.txt",

```

```

6    "chunkOption": {
7        "chunkTextGroupByMinSize": 100,
8        "chunkSize": 800,
9        "chunkOverlap": 100,
10       "chunkColSeparator": "^B",
11       "chunkRowSeparator": "^A",
12       "useChunkOneByOneTableToText": false,
13       "useTableChunk": true,
14       "useTableChunkByRow": false,
15       "tableChunkSize": 1400,
16       "tableHeaderSimilarityThreshold": 0.8,
17       "tableOneRowMaxSize": 30000,
18       "breakpointThresholdType": "percentile",
19       "breakpointThresholdAmount": 95,
20       "useGroupBySubtitle": true
21   }
22 }

```

Response

Field	Type	Description
result	String	Success or not
dataJson	list[{"contents": Object}]	refer to type <Content>

Type <Content>

Field	Type	Description
title	String	title
subtitle	String	subtitle
content	String	text

Response 예시

```

1  {
2      "result": "success",
3      "dataJson": [
4          {
5              "contents": {
6                  "title": "healthcare-1010.txt",

```

```

7         "subtitle": "",
8         "content": "1. Purpose of Health Insurance    A system to improve national
health and promote social security    It is a social insurance operated for the purpose of
paying insurance benefits for accidents such as illness or injury that occur to the insured or
dependents    2. Standards for Health Insurance    □ Eligibility: All workers in the
workplace"
9     }
10 },
11 {
12     "contents": {
13         "title": "healthcare-1010.txt",
14         "subtitle": "",
15         "content": "- Eligibility for reduction: Employees staying overseas for 3
months or more and staying overseas for 1 month or more but less than 3 months for business
purposes - Reduction amount · Full amount reduction if both the employee and their dependents
are staying overseas · Half amount reduction if the employee is staying overseas and at least 1
of their dependents is in Korea"
16     }
17 }
18 ]
19 }

```

| 호출 예시 (python)

```

1 import requests
2
3 YOUR_CLIENT_KEY = ""
4 YOUR_PASS_KEY = ""
5 ENDPOINT_URL = ""
6 YOUR_EMAIL = ""
7
8 headers = {
9     "x-fabrix-client": YOUR_CLIENT_KEY,
10    "x-openapi-token": YOUR_PASS_KEY,
11
12    "x-generative-ai-user-email": YOUR_EMAIL}
13
14
15 body = {

```



```

16 "source_obj_url": "s3://bucket/0349b307-fbb9-46a3-b1ae-eeef9b4baa93.txt_0.json",
17 "chunking_algorithm_id": "default-CharacterTextSplitter",
18 "sourceVerifyKey": "source verify key",
19 "title": "Health Insurance",
20 "chunkOption": {
21     "chunkSize": 800,
22     "chunkOverlap": 100,
23     "chunkTextAddUnit": "$size",
24     "chunkTextAddSize": 300,
25     "chunkTextAddChunk": [
26         1,
27         1
28     ],
29     "chunkTextGroupByMinSize": 100,
30     "useAllRespContent": false,
31     "chunkColSeparator": "^B",
32     "chunkRowSeparator": "^A",
33     "useChunkOneByOneTableToText": false,
34     "useTableChunk": true,
35     "useTableChunkByRow": false,
36     "tableChunkSize": 1400,
37     "tableHeaderSimilarityThreshold": 0.8,
38     "tableOneRowMaxSize": 30000,
39     "breakpointThresholdType": "percentile",
40     "breakpointThresholdAmount": "95",
41     "useMarkdownFormatHtmlForOneResp": false,
42     "useGroupBySubtitle": true
43 }
44 }
45
46
47 api_endpoint_url = f"{ENDPOINT_URL}/openapi/knowledge/v1/knowledge-embedding/chunking"
48
49 response = requests.post(api_endpoint_url, headers=headers, json=body)
50
51 # Response
52 print(response.json())

```

공통 Response Code

Http status code	Description
200	Request and response okay. If stream is True, all HTTP statuses except system errors will respond with 200.
400	Requested with incorrect information(ex: empty sourceObjUrl, empty sourceVerifyKey, empty chunkingAlgorithmId, etc)
403	Accessing without authorization
422	Required value missing
500	Unexpected Error(ex: URL parsing error, too large file size error, file download error, file decoding error, chunking client creation error, default embed model not existing error, chunk data error, etc)

12.3 FAQ

sourceVerifyKey 에 어떤 값을 넣어야 하는지 모르겠어요.

Storage 의 접근 정보는 사전에 시스템 담당자와 협의하여 등록할 수 있습니다. 등록 후 Verify Key 정보를 전달드리오니, 해당 값을 사용하시면 됩니다.

ChunkOption 에 값을 입력하지 않으면 어떤 설정으로 동작하게 되나요?

ChunkOption Object 내 모든 값은 optional 설정으로 지정하지 않아도 됩니다. 이때는 사전에 정의된 기본 설정을 따르게 되어 있으며, 기본 설정 값은 테넌트에 따라 상이하게 설정할 수 있게 되어 있으므로, 시스템 담당자 또는 테넌트 담당자에게 문의해주세요.